DOT-Design

Version Next, 2026-11-08

Overview

Describe a tool once. Reuse it in different deployments and compositions.

DOT-Design helps you reuse software components in the MARE 6G security ecosystem. A Dynamic Open Trust component (**DOT**) has a reusable definition: what the tool does, which interfaces it exposes or requires, and how it can run. The `dotx` CLI validates the definition and runs the deployment you choose.

Keep a reusable tool in `dot.json`. Put one use case's selected tools, connections and configuration in a separate `composition.yaml`. For example, reuse the same Prometheus DOT in different monitoring compositions instead of maintaining a new product definition for each one.

New to DOT? Follow DOT-Design from Zero to Hero: start with DOT-Design 101 for the core models and ecosystem, then run Prometheus, author its definition, connect Node Exporter and extend the same system with Grafana. The tutorial introduces each model through a working example and includes cleanup at every live stage.

First success: run Prometheus

Follow Getting started to:

1. Install the checksummed Linux-amd64 CLI and its matching catalogue.
2. Deploy the existing Prometheus DOT with Docker Compose.
3. Check the expected version through the official API, then verify cleanup.

You need repository access and Docker. Java, Go, Task and Kubernetes are not required. Use an isolated development host: the example exposes unauthenticated HTTP, and cleanup deletes its metrics volume. The walkthrough explains port selection, persistence and recovery before deployment.



Use one matching CLI/catalogue revision set. The bottom-left selector offers **Next** for current development and immutable stable documentation for released binaries. Published CLI v0.4.0 includes guided authoring, safe editing and composition creation. Follow the authoring guide with its matching catalogue; Next-only instructions are labelled separately.

How the pieces fit together

A DOT describes one reusable tool. Skills state its capabilities. Contracts describe interfaces using OpenAPI, AsyncAPI, Protobuf, GraphQL or the bounded SPARQL query profile. Anchors are named integration points that provide or require interfaces. Wires connect compatible Anchors; deployment bindings map locally provided interfaces to service ports. Choose one supported deployment for the instance you want to run.

A composition describes one use case. It selects exact DOT versions and deployments, connects their Anchors with Wires, and supplies configuration and Secret references. Its reviewed lock pins

the resolved definitions, Contracts and inputs. It references reusable DOTs rather than copying them.

The secure-API walkthrough illustrates this with Keycloak, APISIX, OPA, Oxigraph and a machine client. Realm configuration, routes and policy belong to that composition, not to separate copies of the products.

For Prometheus, a Contract describes the build-information operation, an Anchor names it, and a deployment binds it to a service port. The successful API call comes from the running instance; the declaration alone cannot prove its result.

▼ *Explore the generated model overview*

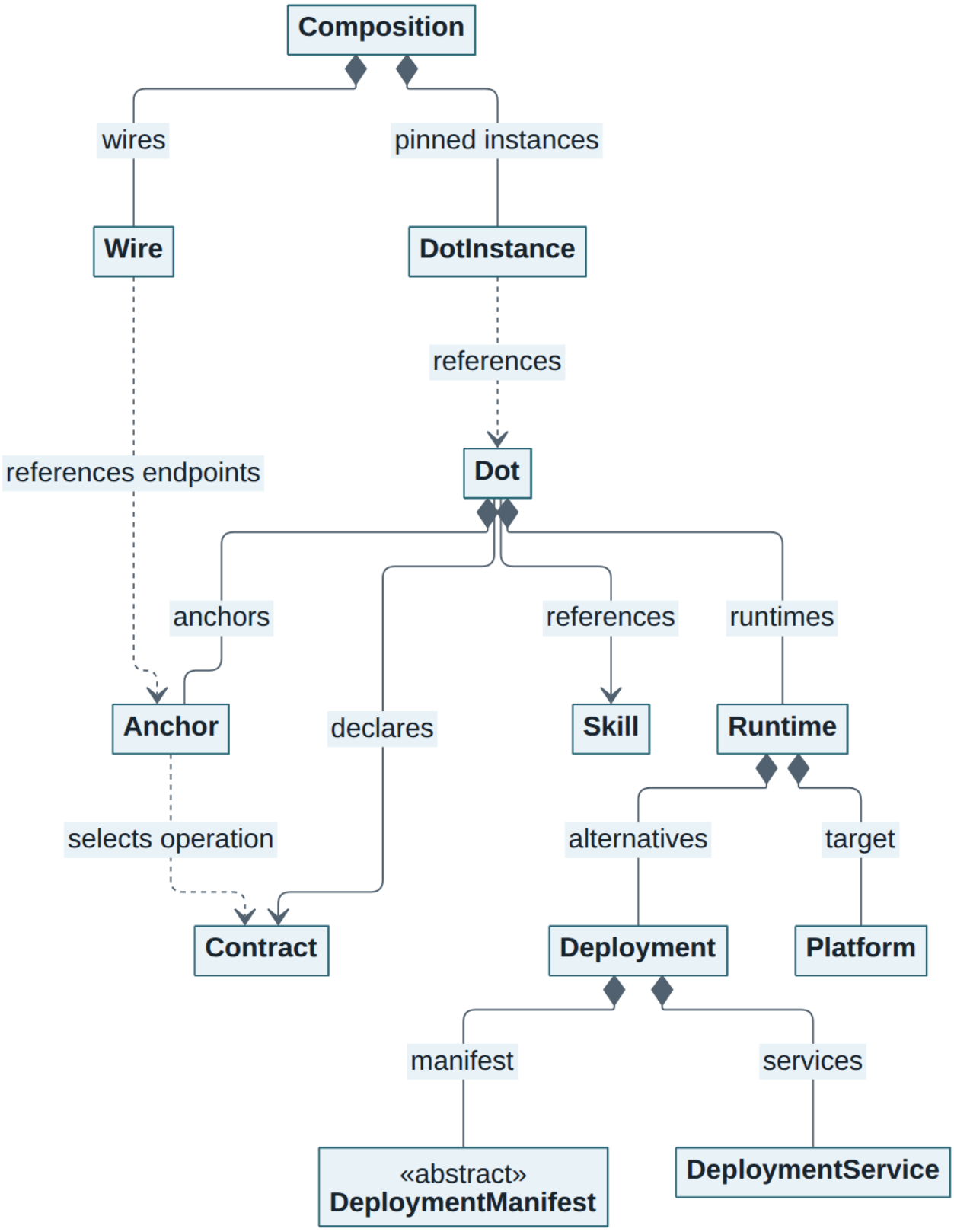


Figure 1. Simplified domain model: reusable DOT definitions and a separate Composition

Diamonds indicate embedded values; arrows indicate declarations or references. The dashed arrow applies to Contract-backed Anchors. Field lists, collection sizes, configuration, ports, binding helpers and audit metadata are omitted for readability; see the full generated class diagram for implementation detail. `Composition` is a separate DOT-Domain aggregate: its instances reference DOTs and its Wires reference their Anchors. Neither is embedded in the reusable Java `Dot` aggregate.

Runtime and deployment branches are alternatives. The complete model, field-level rules and JSON example live in DOT-Domain.



`dot.json` is the deployable DOT definition and the only source inspected by `dotx deploy`. Connections between deployed DOT instances are described in a separate composition document. A wire does not create an API, translate protocols automatically or prove production readiness. Concrete addresses, credentials and live policy results never become Anchor fields.

Where to go next

Read	When you need to
DOT-Design from Zero to Hero	Build a monitoring system gradually: run Prometheus, author its DOT, connect Node Exporter, then add Grafana with configuration and a password reference.
Getting started	Install the released CLI, deploy Prometheus with Docker Compose, verify its API, and clean up.
Contributor guide	Build the project, run shared local/GitHub checks, and generate documentation.
Guided demo	Follow a 10-15-minute live presenter route, a five-minute read-only overview, or a deeper onboarding walkthrough from catalogue validation to live evidence.
Developer demo: reuse and extend	Inspect a real DOT, prove an invalid wire is rejected, and map PoC changes to shared definitions, composition inputs and executable acceptance.
Current-source guided authoring	Create or edit a draft with native forms, then build a composition from exact catalogue entries. Requires the matching source CLI.
Architecture	Understand repository boundaries, sources of truth, runtime/deployment separation, and the composition resolution path.
DOT Anchors and Composition	Author Anchors, bind them to services, connect DOT instances, materialize configuration, handle security inputs, and operate a composition.
Verification and examples	Choose conformance, dry-run, source-adapted, or live evidence and find the executable MARE PoC scenarios.
DOT-Domain	Work with the Java aggregate, value objects, validation, Contracts, Runtimes, and deployment manifests.
DOT-Repository	Find exact tool releases, inspect deployment choices, validate the catalogue and reuse reviewed compositions.
Integration handoff	Implement a catalogue, composer, or SFC adapter without coupling to a specific service, language, framework, or storage technology.

The architecture guide maps the repository components: DOT-Domain owns the Java model, DOT-CLI provides dotx, DOT-Repository holds reusable definitions, and DOT-Testing records verification evidence.

Maturity and scope

The DOT-owned Anchor model, service-port binding, generated schema, repository resolution, conformance fixtures, dot.mare6g.eu/v1 composition document, and dotx compose executor form the stable Composition v1 profile. The Java domain and CLI use aligned release numbers; exact source revisions still matter when trying unreleased features. See release/source compatibility.

The reference executor is intentionally fail-closed. It does not claim to be a general multi-host orchestrator, trust-policy engine, active security scanner, or replacement for Kubernetes, Helm, Docker, Vagrant, or an SFC implementation. See the supported composition profile for the exact boundary.

Validation checks declarations and connections; it does not prove production readiness. Live API/message assertions and verified cleanup are separate evidence. See Verification and examples for the exact distinction between conformance, previews, test fixtures and producer acceptance.

Inspect before deploying

After installing the CLI and checking out the matching catalogue, these commands run from the DOT-Design checkout without deploying anything. The offline catalogue check requires previously verified Contract bytes in the cache; a cold cache fails rather than downloading them.

```
# Validate the catalogue using its verified Contract cache
dotx validate DOT-Repository --schema DOT-Repository/Schema/schema.json --offline

# Inspect the reviewed five-DOT PoC10 resolved composition, not live traffic
composition=DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml
dotx compose validate "$composition" --locked
dotx compose topology "$composition" --locked --color auto
```

Use the guided demo for live execution and cleanup. Build toolchains, Java tests and the full local/GitHub CI route belong in the contributor guide.

Project links

- MARE project
- MARE proof-of-concept catalogue
- DOT-Design source repository

Getting started

Run your first DOT on Linux amd64 using the selected CLI release `v0.4.0` and the existing Prometheus definition. You need Bash, Git, GitHub CLI (`gh`), `curl`, `jq`, SHA-256 utilities, and Docker with Docker Compose. No Java, Go, Kubernetes, or Task is needed for this path.

This example runs Prometheus 3.15.1 on port `18909`. Allow time for initial image and Contract downloads. Use an isolated development host: the Compose manifest publishes unauthenticated HTTP on all host interfaces. Restrict access using your host firewall. Cleanup deletes the example's metrics data.

Install the released CLI

Authenticate with an account granted access to the private MARE repositories and releases. From a workspace directory:

```
set -o pipefail
gh auth login &&
gh auth setup-git &&
mkdir -p dot-quickstart-download &&
cd dot-quickstart-download &&
gh release download v0.4.0 --repo MARE-EU-Project/DOT-CLI \
  --pattern dotx-0.4.0-linux-amd64 --pattern SHA256SUMS &&
mkdir -p "$HOME/.local/bin" &&
sha256sum dotx-0.4.0-linux-amd64 |
  grep -Fx -f - SHA256SUMS &&
  install -m 0755 dotx-0.4.0-linux-amd64 "$HOME/.local/bin/dotx" &&
export PATH="$HOME/.local/bin:$PATH" &&
dotx version &&
cd ..
```

The checksum must match the downloaded binary's exact filename and bytes; installation is skipped if its entry is missing or wrong. Stop on any error. Expected version: `dotx 0.4.0`. Add the `PATH` export to your shell profile for future sessions. The release also provides an SBOM and build/acceptance revision manifest.

Check out the catalogue

Only the independently cloneable catalogue is needed. Pin the acceptance revision recorded in this CLI release's manifest, then check your Docker access:

```
gh repo clone MARE-EU-Project/DOT-Repository &&
cd DOT-Repository &&
git checkout --detach 2b17ee681fbe8eed682283448 4f87149b32fc772 &&
docker info &&
docker compose version
```

Run the following blocks in the same Bash shell from the **DOT-Repository root**. Run each block only after the preceding block succeeds. The selected descriptor embeds its Compose manifest; no neighbouring Compose file is inferred.

Select and validate

The selection block generates and prints a distinct runtime name. Keep that name and the same shell for cleanup. To choose your own name, first run `DOT_NAME=my-prometheus-lab` and ensure it belongs only to your run; a random default is not permission to adopt an existing project. For a different host port, run `export DOTX_HTTP_PORT=18919` before the selection block.

```
DOT_REPOSITORY="${DOT_REPOSITORY:-$PWD}" &&
DOT_DEFINITION="$(dotx repository resolve Prometheus@3.15.1 --repository
"$DOT_REPOSITORY")" &&
DOT_BUNDLE="$(dirname -- "$DOT_DEFINITION")" &&
DOT_NAME="${DOT_NAME:-dot-quickstart-$RANDOM-$RANDOM}" &&
DOTX_HTTP_PORT="${DOTX_HTTP_PORT:-18909}" &&
DOT_PROJECT_FILTER="label=com.docker.compose.project=$DOT_NAME" &&
export DOTX_HTTP_PORT &&
printf 'Selected: Prometheus 3.15.1; instance %s; host port %s. Keep this shell for
cleanup.\n' "$DOT_NAME" "$DOTX_HTTP_PORT"
```

```
(
set -euo pipefail
printf 'Checking the definition and previewing Docker Compose; nothing starts
yet...\n'
dotx validate "$DOT_BUNDLE" --schema "$DOT_REPOSITORY/Schema/schema.json" \
  --profile reusable
dotx inspect "$DOT_BUNDLE"
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" --dry-run
)
```

Validation checks the schema, official HTTPS OpenAPI Contract and SHA-256, Anchor selectors, and reusable deployment bindings. Inspection shows deployment choices. The exact ID `docker-compose` selects only that mechanism. Dry-run prints its commands; it does not prove that a service is running.

Deploy and verify

```
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT"
```

Wait up to 60 attempts (each HTTP attempt has a two-second timeout), then verify the service's build-information API and version:

```
(
set -euo pipefail
ready=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking Prometheus readiness on port %s...\n' "$DOTX_HTTP_PORT"
for ((attempt=0; attempt<60; attempt++)); do
  if curl --fail --silent --max-time 2 \
    "http://127.0.0.1:$DOTX_HTTP_PORT/-/ready"; then
    ready=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for Prometheus: attempt %s/60, %ss elapsed\n' "$((attempt + 1))"
"$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $ready; then
  printf 'Prometheus readiness timed out. Run the cleanup block before retrying.\n'
>&2
  exit 1
fi
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$DOTX_HTTP_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
printf 'Verified: Prometheus 3.15.1 answered its official API. Open
http://127.0.0.1:%s/\n' "$DOTX_HTTP_PORT"
)
```

Expected: `Prometheus Server is Ready.`, `true` from `jq`, then `Verified: Prometheus 3.15.1 answered its official API.` and your browser URL. Check runtime status:

```
dotx ps
```

Expect a running workload. Open `http://localhost:18909` (or your chosen port) to view Prometheus.

The `build-information` Anchor selects the official OpenAPI operation whose server prefix produces `/api/v1/status/buildinfo`. The `promql-query-api` Anchor selects query operations; both bind to the deployed service's `http` port. Its `scrape-target` requirement references Node Exporter's metrics interface; this first example does not connect another DOT as a scrape target.

The pinned image's default configuration starts the service. The descriptor declares a Compose-managed `prometheus-data` volume. Data survives container restarts until the volume is deleted.

Clean up



For Compose, `dotx destroy` runs `down --volumes`: it deletes this instance's containers, network, and named volume, including collected metrics. Use the same `DOT_NAME` as deployment. Never target another user's instance.

```
dotx destroy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" &&
remaining="$(docker ps -aq --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker network ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker volume ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining"
```

The final checks succeed silently. If interrupted, restore your original selection variables and repeat cleanup. Before closing the shell, note the printed instance name, catalogue checkout and selected port. To recover that same run in a new shell, set `DOT_NAME` to its original value and `DOTX_HTTP_PORT` to its original port, change to that same catalogue checkout, then repeat the selection and cleanup blocks. Do not accept a newly generated name for cleanup. Retain the original CLI, cache and Docker context until cleanup is verified. The Zero to Hero tutorial saves these values in a private session file. Downloaded images remain cached.

Next: connect existing DOTs

For the newcomer evaluation route, continue with one matching prebuilt kit and Docker monitoring. It keeps the same binary and catalogue and requires no contributor build. The following Kubernetes example is a separate **source-contributor/operator route**.

For the existing Keycloak/APISIX/OAuth composition, use a full DOT-Design checkout with initialized submodules; see Contributor guide. From the **DOT-Design root**, with `dotx` on `PATH`:

This follow-up uses the source-built development CLI. Install the checked-out CLI using the contributor toolchain before continuing; released binaries do not necessarily implement the current commands or lock format.

```
composition=DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml
task cli:install
dotx compose lock "$composition" --check
dotx compose inspect "$composition" --locked
dotx compose apply "$composition" --locked --dry-run
```

The three wires connect Keycloak discovery to APISIX, Keycloak's token endpoint to the OAuth client, and APISIX's protected operation to that client. The resolved composition resolves endpoints and configuration. Live acceptance additionally proves anonymous rejection and authenticated access; the harness prepares Kubernetes, native Secrets, and workloads. Follow Guided demo and Verification and examples for that path.

To author a component, continue with Authoring Anchors. For development toolchains and local/GitHub CI gates, use Contributor guide.

Troubleshooting

Symptom	Action
Release or clone returns 404/403	Check `gh auth status` and organization repository access, including SSO authorization if required. Run `gh auth setup-git` for HTTPS cloning.
CLI missing or wrong version	Check `command -v dotx`; put `$HOME/.local/bin` first on `PATH`.
Docker permission denied	Check `docker info` as your user. Ask the administrator for the intended Docker access, then log in again. Docker group access grants host-level power.
Compose unavailable	Install the Compose plugin/package for your Docker distribution and verify `docker compose version`. The legacy `docker-compose` executable is insufficient.
Port already allocated	Clean up the failed example, select a free `DOTX_HTTP_PORT`, and deploy again with the same name and explicit `--env` override.
Contract download or digest check fails	Check network/proxy access to the declared URL. Never bypass its digest. Private raw GitHub URLs require `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN`; `gh` login alone does not supply those to `dotx`. Use `--offline` only with matching verified local Contract assets.
Readiness or version fails	Use the instance-scoped logs below and `dotx ps --all`, check the selected image/port, then clean up. No local Compose file is required for this diagnostic.
Interrupted cleanup	Restore the original name and catalogue path, repeat cleanup, and inspect resources carrying `com.docker.compose.project=$DOT_NAME`.

Read only this instance's logs

Keep the original `DOT_NAME`. This block discovers containers by their Compose project label and reads bounded logs. It works from the catalogue checkout: `docker compose -p …` alone would not locate the descriptor's downloaded Compose document. No resources are changed or removed.

```
(
set -euo pipefail
containers="$(docker ps -aq --filter "label=com.docker.compose.project=$DOT_NAME")"
if test -z "$containers"; then
  printf 'No containers found for this instance: %s\n' "$DOT_NAME" >&2
  exit 1
fi
while IFS= read -r container; do
  printf '\nLogs for this instance container: %s\n' "$container"
```

```
  docker logs --tail=100 "$container"
done <<< "$containers"
)
```

If Docker access fails or no matching container is found, the block fails; it does not query another instance. Use the same lesson’s cleanup command and retain the original context until cleanup is verified.

Download the shared command blocks. Contributors rehearse them from DOT-Design with `task testing:quickstart:dry-run` or the opt-in `task testing:quickstart:live`.

DOT-Design from Zero to Hero

Understand the models, then build a monitoring system you can explain, change and run yourself. Begin with DOT-Design 101: learn the core building blocks and how the ecosystem uses them. Then create a reusable tool definition, connect a metrics producer and add Grafana. The practical lessons turn those concepts into files and verified API responses.

Your first visible result is small: **Prometheus answers its real API**. Next, your own DOT passes validation. Then a two-tool query returns 1, and Grafana returns that same sample through a third tool. Each result has a separate check, so you know what worked before adding the next piece.

Start with DOT-Design 101

DOT-Domain defines the portable models used throughout the project. A **DOT** describes one reusable tool release; a **Composition** selects tool instances and connects their **Anchors** through **Wires**. The CLI uses their generated schemas and input types to work with those declarations.

Read DOT-Design 101 first. It explains Skills, Contracts, URL/hash Payloads, Runtime and deployment choices, service configuration, connections, locks, probes and lifecycle safety in simple terms. It also maps the roles of Domain, CLI, Repository, Docs and Testing. No installation is needed.

Build on those concepts

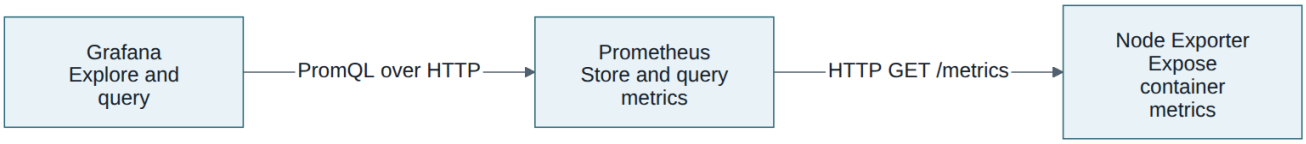


Figure 2. The system you will build: actual HTTP request direction

Grafana sends queries to Prometheus; Prometheus pulls metrics from Node Exporter. The CLI's topology uses a different, logical direction: a Wire points from the Anchor that **provides** an interface to the Anchor that **requires** it. Both views describe the same system. A Wire does not determine which tool sends a request.

The reusable DOT says what a tool can do and how it runs. Your composition says which instances work together for this particular monitoring use case. Scrape targets, datasource configuration and credentials belong to that use case.

Your route

Lesson	What you build	How you know it works
Start here: DOT-Design 101	A mental model of the ecosystem and its main DOT-Domain building blocks	You can trace Contract → Anchor → service port and explain what a composition adds.
1. Run your first DOT	An existing Prometheus deployment	Its real API reports version 3.15.1; cleanup removes its owned resources.

Lesson	What you build	How you know it works
2. Create an interface-aware DOT	Your own Prometheus definition, with a pinned official Contract, named Anchors, configuration and two Docker deployment choices	The reusable profile validates it; inspection explains every declared part.
3. Connect two DOTs	Your Prometheus DOT plus the catalogue's Node Exporter	A labelled PromQL query returns 1 after the real scrape.
4. Grow the system	The same monitoring graph extended with Grafana	Grafana queries the provisioned Prometheus datasource and returns the same sample.
5. Take the pattern further	A route into authenticated APIs, policy, other runtimes and Contract standards	You can identify which additional prerequisites and live checks each extension needs.

Choose your pace. Read DOT-Design 101, then finish Lesson 1 for a first API response. Continue through Lesson 3 for your first authored integration, then Lesson 4 for the complete monitoring chain. Lesson 5 is an optional operator path with additional setup. Every live lesson ends with cleanup; stop there whenever you need a break. Downloads and startup depend on your machine and cache; take the time needed to review each result.

If you want to **watch before authoring**, use the guided demo instead. This tutorial is the hands-on route: you create and inspect the files rather than run an unattended installer.

Before you start

Use **Linux amd64**, Bash, Git, GitHub CLI, `curl`, `jq`, SHA-256 utilities and Docker with Compose. You need authorized access to the MARE repositories and release assets. The main four lessons need **no Java, Go, Task or Kubernetes**.

Use the released CLI `v0.4.0` with its manifest-pinned catalogue `2b17ee681fbe8eed6822834484f87149b32fc772` throughout. Lesson 1 includes installation. Do not switch to floating `main` halfway through: a version string alone does not identify later source changes. The catalogue may use the older single-root layout; identity-based lookups make these commands work without guessing paths.



Use an isolated development machine. The lab exposes disposable HTTP services; Docker publication can listen beyond localhost. Restrict access with your host firewall and use a lab-only Grafana password. This is not a hardened monitoring deployment. Node Exporter observes its container, not your physical host.

Validation establishes declared constraints and resolved interfaces. Only the API checks establish the particular live behavior exercised here. Neither result certifies every catalogue definition or every API of the selected tools.

Follow a repeatable rhythm

1. **Read the goal**, then run one complete Bash block at a time. Copy commands without adding a shell-prompt character. Keep one terminal and the matching CLI/catalogue throughout; variables carry your workspace into later lessons.
2. **Inspect your work.** Open the named file when a lesson asks for an edit. Keep YAML indentation exactly as shown. The longer shell blocks perform checked setup and verification; you do not need to memorize them.
3. **Check the result.** Validation checks declarations; a dry-run previews deployment. Continue to live execution only after those steps succeed.
4. **Clean up before moving on.** Use the lesson's original draft, lock, binary and cache. If a step fails, stop and use the resume and recovery guide.

Know where your work lives

Lesson 1 prints a private workspace path, referred to as `$LAB` throughout. The catalogue is a separate, pinned checkout; exercises do not edit it.

Inside your workspace	Purpose
`session.bash`	Your paths, ports and original CLI identity for resuming. It stores no password.
`base/dot.json`, then `prometheus/dot.json`	The generated scaffold and your enriched reusable tool definition from Lesson 2.
`composition.yaml`, `prometheus.yml`	Your two-tool graph and its scrape configuration from Lesson 3.
`dashboard.yaml`, `grafana/`, `grafana-*.yml`	Your extended graph, private Grafana definition and provisioning inputs from Lesson 4.
Adjacent `*.lock.json` files	Reviewed exact inputs created explicitly for each graph. Keep them for recovery.

The hands-on lessons end with a checkpoint and a short explanation exercise. Use those to test your understanding before moving to the next layer.

Start with DOT-Design 101: understand the models and ecosystem.

Start here: DOT-Design 101

Start here before installing anything. DOT-Design gives different software tools a common description for deployment and integration. A **DOT** is a Dynamic Open Trust component. It keeps the tool's native deployment mechanisms and documented interfaces, so one definition can be reused across use cases. In this guide, you will use Prometheus for monitoring, connect a metrics producer, then add Grafana. First, learn what the documents describe and how the pieces fit together.

No Java knowledge or running service is needed for this introduction. By the end, you should be able to follow one interface from its Contract to a service port, and explain what a composition adds to a reusable tool definition.

Keep four ideas in view

Idea	In the monitoring exercise
DOT: describe a tool once	Prometheus has one release identity and reusable deployment/interface declarations.
Deployment: choose how one instance runs	Start with Docker Compose; choosing it does not launch the other alternatives.
Anchor: name an Integration Point	`build-information` selects one operation from Prometheus's verified Contract.
Composition: connect instances through Wires	Prometheus requires a metrics source; Node Exporter provides it. The Wire supplies the selected endpoint through configuration.

Read the model map below first. The remaining sections explain the names you will meet in the files; return to them as each lesson introduces a field. You do not need to memorize the complete class model before running your first DOT.

Start with the models

DOT-Domain is the source of truth for the portable models. It defines the building blocks and their validation rules in Java. The project generates JSON Schemas and CLI input types from those models. You author JSON or YAML and use the `dotx` binary; Java is not required to use the CLI.

Two documents organize the work:

- `dot.json` **describes one tool release**: its capabilities, interfaces and supported deployment alternatives.
- `composition.yaml` **describes one use case**: selected tool instances, connections and instance-specific configuration. Composition is also a DOT-Domain model.

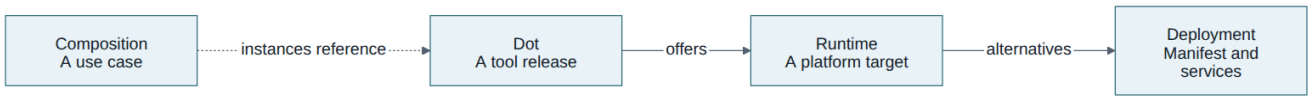


Figure 3. Conceptual map: a use case references reusable tools and selects how they run

`Dot` leads to the reusable product definition; `Composition` leads to instances and Wires that reference those definitions. These arrows describe model relationships, not network packets. Wires connect the selected DOTs' Anchors.

The sections below explain the main families. The complete model reference documents every public type and field, including advanced options.

Describe a tool: identity, capabilities and audit information

A **DOT** is a reusable definition of a component. Here, `Prometheus@3.15.1` identifies the actual Prometheus release, independently of the CLI release. One release definition can offer several deployments. Another supported tool release has its own exact definition.

This is a **JSON fragment**, taken from the compact canonical Prometheus example:

```
{
  "id": "Prometheus@3.15.1",
  "name": "Prometheus",
  "version": "3.15.1",
  "skills": [{"id": "Metrics-Scraping"}]
}
```

Model or field	Meaning
`Dot.id`, `name`, `version`	The product's identity, readable name and actual tool release.
`Skill`	A named capability, such as metrics scraping. A DOT references at least one meaningful Skill by ID; a capability label does not establish API compatibility.
`description`, `groups`	An explanation and open classification labels. `external` records ownership outside MARE; it is not a quality or security certification.
`Metadata`	Definition audit timestamps: `created` and `updated`. These describe the document, not when a container last started.

The fragments on this page explain selected relationships; they are not complete deployable documents. See the complete Prometheus JSON example to read all the required fields together. Lesson 2 creates and validates your own private learning definition without changing the catalogue.

Describe an interface: Contract, Payload and Anchor

A **Contract** documents an interface in a supported machine-readable standard: OpenAPI for HTTP operations, AsyncAPI for messaging operations, Protobuf for service methods, GraphQL for schema fields, or the supported SPARQL query profile. Each standard has a bounded supported profile; selecting its name does not mean every feature of that standard is executable.

The Prometheus Contract has ID `Prometheus@3.15.1::OpenAPI::HTTP`. Its `version` is `3.1.0`, the OpenAPI document version. The tool release remains `3.15.1`. `format: YAML` describes how the specification bytes are encoded.

The shared `Payload` value contains a raw HTTPS `url` and a `sha256` digest. The CLI downloads and verifies those exact bytes, or checks its verified cache. This URL identifies the specification document; it is not the running API address. A digest verifies integrity, not publisher authenticity or live behavior. Prefer release-matched official specifications and retain their provenance.

An **Anchor** is a named **Integration Point**. It gives a useful interface a DOT-local name and declares whether this tool `PROVIDES` or `REQUIRES` it. For a Contract-backed Anchor, the selector identifies the operation inside the verified specification:

```
{
  "id": "build-information",
  "name": "build-information",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

Read this as: “Prometheus provides build information through this selected OpenAPI operation.” `~1` encodes `/` in the JSON Pointer. The selector is separate from the Contract ID; an Anchor contains no running host, port or password.

Root `Dot.contracts` holds complete declarations **owned by that tool release**. An Anchor requiring another tool’s Contract names its exact originating DOT through `contract.source`; the consumer does not copy the producer’s Contract. The model also supports a provider-neutral requirement profile, initially OIDC discovery, and explicit contractless communication boundaries. Those modes do not create a verified Contract where none exists. See Anchor authoring for their distinct validation rules.

Describe execution: Runtime, Platform, Deployment and Service

These models answer different questions:

Model	Question	Prometheus example
`Runtime` and `Platform`	On which engine can it run?	A Docker Runtime has `platform.engine: DOCKER`. Kubernetes is another engine.
`Deployment`	Which supported deployment alternative should this instance use?	Choose the exact ID `docker-compose` or a Docker image deployment.
`DeploymentManifest`	What native specification deploys it?	`DOCKER_COMPOSE` points to the verified Compose document. An image deployment instead names its container image. Other mechanisms include Kubernetes manifests, Helm charts, Vagrantfiles and WASM modules.

Model	Question	Prometheus example
`DeploymentService`	Which logical services does that deployment materialize?	Service `prometheus` declares its ports, interface references and configuration. One deployment can materialize several services.
`ServicePort` and `AnchorBinding`	Where does a provided interface become reachable?	Named port `http`, application protocol `HTTP`, service port `9090`; `build-information` is bound to that named port.

A single Runtime may contain several compatible deployment mechanisms. Selecting one deployment does not launch all the alternatives. The selected `dot.json` contains the references and declarations needed for deployment; a neighbouring Compose file or Dockerfile is not an implicit input.

This service fragment completes the connection between the logical Anchor and its implementation:

```
{
  "id": "prometheus",
  "ports": [{"name": "http", "protocol": "HTTP", "port": 9090}],
  "contracts": [{"id": "Prometheus@3.15.1::OpenAPI::HTTP"}],
  "anchorBindings": [{"anchorId": "build-information", "port": "http"}]
}
```

Follow the chain: **Contract operation → Anchor → service binding → named port**. The deployment supplies the implementation. A published host port such as `18909` is an instance-facing choice; it is not the service's `9090` port. The service's Contract list contains ID-only references; the complete owned declaration stays at the DOT root.

Describe configuration where it belongs

Each service owns a `DeploymentConfig`: supported environment variables, configuration-file paths and volume mounts. A multi-service DOT gives each service its own configuration and bindings.

The reusable definition declares supported inputs and any allowed defaults. For example, Prometheus accepts a scrape configuration file. Your monitoring composition supplies the targets and labels for that use case. A file default uses the same URL/hash `Payload` as a Contract or manifest; ordinary scalar environment defaults remain strings.

Sensitive inputs are marked explicitly and supplied at execution time through references such as `secretRef`. Do not embed passwords in reusable definitions, public file payloads or locks. Required capabilities, host namespaces and CDI devices are declarations that need explicit operator authorization and supported bindings; describing them does not grant access.

Connect tools: Composition, DotInstance and Wire

A **Composition** selects exact definitions and one deployment for each named `DotInstance`. In the tutorial, `prometheus` is an instance name while `Prometheus@3.15.1` is the product identity. You can select the same definition for several distinctly named instances.

A **Wire** connects an Anchor that provides an interface to one that requires it. For the first monitoring connection:

```
node-exporter / node-metrics  ->  prometheus / scrape-target
          PROVIDES                          REQUIRES
```

A **materializer** writes the selected producer's endpoint into a declared consumer input. Here it updates one target in Prometheus's scrape file. The Wire supplies configuration; Prometheus implements the HTTP scrape. No automatic protocol translation or managed networking is implied.

Other Composition building blocks have specific jobs:

Building block	Simple meaning
Parameters and deployment inputs	Values for this use case, service-specific environment/files/volumes and exact native workload bindings when needed. Secret references name credentials; resolved values remain execution concerns.
`dependsOn`	Explicit startup dependencies. This is separate from the Wire's interface relationship.
Exposed Anchors	Named interfaces selected for access, such as `prometheus-query` with `--forward`. An exposure does not add authentication.
Wire probes and response assertions	Bounded checks of supported operations and expected responses. A declaration or successful compatibility result alone does not prove a running interaction.
Broker and external-gateway bindings	Explicit transport endpoints or cross-runtime reachability supplied by the operator. They do not create extra Anchor Wires. Infrastructure remains operator-owned and is not adopted or deleted by the composition.

The composition has a UUID used in ownership management. Its readable name and version describe the use case; they are separate from product identities and tool releases. A new UUID alone does not resolve resource or port collisions. `apiVersion` and `kind` identify the Composition document format; they are separate from its use-case version, the tool releases and the CLI release.

Understand the workflow and its evidence

Step	What it establishes
Author and validate	The document satisfies the applicable generated shape and semantic/resolution checks. Validation does not start a service.

Step	What it establishes
Inspect and view topology	The resolved declarations, deployments and interface relationships you selected. A topology is not live status. Compatibility may be incompatible or unresolved; an unknown result must not be treated as verified.
Create and review a lock	`composition.lock.json` pins resolved definitions, documents and configuration. The lock is a CLI artifact, not another portable domain model or live state. Review intentional input changes before explicitly regenerating it.
Preview, then apply	A dry-run previews execution. Apply performs runtime operations using the selected deployment and supported grants; its success still needs live checks.
Check status and verify	Status reports recorded lifecycle observations. Actual API calls or probes establish only the specific interactions and assertions exercised.
Destroy and check cleanup	Remove the run's owned resources using its original context. Preserve the original files, cache, binary and ownership records until cleanup succeeds. Never adopt unknown resources or broadly prune shared infrastructure.

Linux amd64 and the supported single-host execution profiles are the baseline. Cross-runtime examples need explicitly prepared reachability. Stateful upgrades, recovery, production security and broader interface coverage need their own acceptance; a valid DOT is not a production certification.

Know the project components

Component	Responsibility
DOT-Domain in DOT-Design	The portable Java models, validation rules and generated JSON Schemas.
DOT-CLI	The Go `dotx` binary: author, validate, inspect, resolve, lock, deploy, probe and manage supported lifecycles using the generated input models.
DOT-Repository	The version-controlled catalogue: `DOTs/` contains product/release bundles, `Skills/` capability definitions, `Schema/` generated schemas, and `Compositions/` reusable use-case graphs. An exact release selection never silently chooses another release. External/internal placement records product ownership; testing entries identify synthetic or source-adapted definitions and retain their stated limitations.
DOT-Docs and **DOT-Testing** in DOT-Design	Antora guides/reference and executable conformance/live test harnesses. Test fixtures and source-adapted components are labelled separately from real producer acceptance. Models, schemas, CLI behavior and tests have distinct roles.

You can use the CLI and catalogue without building the development workspace. Follow one matching binary/catalogue set in the lessons. Repository access, network availability and a verified cache are prerequisites, not model fields. No mandatory registry is required for this catalogue-based route.

Check your understanding

Before moving on, locate these relationships in the diagram:

1. The DOT references a Skill, owns Contracts and contains Anchors and Runtime alternatives.
2. A Runtime targets a Platform and contains Deployments; each Deployment has a manifest and logical Services.
3. A provided Anchor selects an operation and is bound to a Service's named port.
4. A separate Composition references DOT instances and connects Anchors through Wires.

▼ *Try explaining the Prometheus example in one sentence*

"This DOT describes Prometheus 3.15.1, identifies its metrics capability and verified API, names a build-information Integration Point, and offers a Docker Compose deployment whose service implements that interface on the `http` port." The composition will later choose its peer tools, connection configuration and instance-specific inputs.

For details, use the model reference, the architecture guide and Anchors and Composition. You can return to them as each exercise introduces a concrete field.

All lessons · **Next: run your first DOT and verify its API.**

1. Run your first DOT

Your first result is a running Prometheus service whose API reports the expected version. Start with reuse; you will author your own definition in Lesson 2. Read DOT-Design 101 first for the models and project components.

Goal: deploy one existing DOT, verify its real API and remove only your instance. **You will learn:** tool identity, explicit deployment selection and the difference between a preview and a running service.

You need the prerequisites and isolated development host described in Zero to Hero. Run each Bash block only after the previous one succeeds. Keep the same terminal throughout the lessons. Parenthesized command blocks stop on errors inside their own shell; they do not close your terminal. A failed block is a stopping point, not permission to continue to deployment.

Install the released CLI

Authenticate with an account granted access to the private MARE repositories and releases. From a workspace directory:

```
set -o pipefail
```

```
gh auth login &&
gh auth setup-git &&
mkdir -p dot-quickstart-download &&
cd dot-quickstart-download &&
gh release download v0.4.0 --repo MARE-EU-Project/DOT-CLI \
  --pattern dotx-0.4.0-linux-amd64 --pattern SHA256SUMS &&
mkdir -p "$HOME/.local/bin" &&
sha256sum dotx-0.4.0-linux-amd64 |
  grep -Fx -f - SHA256SUMS &&
  install -m 0755 dotx-0.4.0-linux-amd64 "$HOME/.local/bin/dotx" &&
export PATH="$HOME/.local/bin:$PATH" &&
dotx version &&
cd ..
```

The checksum must match the downloaded binary's exact filename and bytes; installation is skipped if its entry is missing or wrong. Stop on any error. Expected version: `dotx 0.4.0`. Add the `PATH` export to your shell profile for future sessions. The release also provides an SBOM and build/acceptance revision manifest.

Check out the catalogue

Only the independently cloneable catalogue is needed. Pin the acceptance revision recorded in this CLI release's manifest, then check your Docker access:

```
gh repo clone MARE-EU-Project/DOT-Repository &&
cd DOT-Repository &&
git checkout --detach 2b17ee681fbe8eed6822834484f87149b32fc772 &&
docker info &&
docker compose version
```

Make a private learning workspace

You are now in the **matching DOT-Repository checkout root**. The catalogue stays unchanged: exercises go in a new directory under your home directory. Save the printed path. If ports `18909`, `19041` or `13000` are occupied, stop and select unused ports before the corresponding live lesson. To use different ports throughout, set `DOTX_HTTP_PORT`, `TUTORIAL_PORT` and `DASHBOARD_PORT` to unused values before the workspace block; it saves your choices. Use those chosen ports in browser links instead of the displayed defaults.

```
# Start in the matching DOT-Repository checkout, in Bash.
set -o pipefail
DOT_REPOSITORY="$PWD" &&
PROMETHEUS_JSON="$(dotx repository resolve Prometheus@3.15.1 --repository
"$DOT_REPOSITORY")" &&
DOTX_TUTORIAL_CLI="$(command -v dotx)" &&
DOTX_TUTORIAL_CLI_SHA256="$(sha256sum "$DOTX_TUTORIAL_CLI" | cut -d ' ' -f 1)" &&
LAB="$(mktemp -d "${DOTX_TUTORIAL_WORKSPACE:-$HOME}/dot-zero-to-hero.XXXXXXXX")" &&
DOT_BUNDLE="$(dirname -- "$PROMETHEUS_JSON")" &&
```

```
DOT_NAME="hero-$RANDOM-$RANDOM" &&
DOTX_HTTP_PORT="${DOTX_HTTP_PORT:-18909}" &&
TUTORIAL_PORT="${TUTORIAL_PORT:-19041}" &&
DASHBOARD_PORT="${DASHBOARD_PORT:-13000}" &&
export DOTX_HTTP_PORT XDG_CACHE_HOME="${XDG_CACHE_HOME:-$HOME/.cache}" &&
(umask 077; declare -p DOT_REPOSITORY LAB PROMETHEUS_JSON DOT_BUNDLE DOT_NAME \
  DOTX_HTTP_PORT TUTORIAL_PORT DASHBOARD_PORT XDG_CACHE_HOME \
  DOTX_TUTORIAL_CLI DOTX_TUTORIAL_CLI_SHA256 > "$LAB/session.bash") &&
printf 'Your private workspace: %s\nResume with: source %q\n' "$LAB"
"$LAB/session.bash" &&
dotx version --verbose &&
git -C "$DOT_REPOSITORY" rev-parse HEAD
```

`repository resolve` finds an exact tool release. `LAB` holds your private drafts; it is not another catalogue root. `DOT_NAME` distinguishes this first deployment from someone else's instance. The saved `session.bash` contains paths and names, not credentials. After closing the terminal, source **your own** saved file to restore these variables before cleanup; do not create a new workspace instead.

Inspect before running

```
(
set -euo pipefail
printf 'Checking Prometheus and the selected Docker Compose deployment; nothing starts
yet...\n'
dotx validate "$DOT_BUNDLE" --repository "$DOT_REPOSITORY" --profile reusable
dotx inspect "$DOT_BUNDLE"
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT" --dry-run
printf 'Preview complete. Next: start this one instance and check its real API.\n'
)
```

Expected: validation succeeds, inspection lists deployment choices, and dry-run shows a Docker Compose command. Nothing has started yet.

The tool is Prometheus; `docker-compose` is the exact deployment ID. Choosing a different deployment does not create a different tool identity. Its definition already contains the pinned Compose document and image references; a random Compose file beside `dot.json` is not a deployment source.

Get a real response

If deployment or verification fails, use the cleanup block below before retrying. Keep the original binary, instance name and cache until cleanup is verified.

```
printf 'Starting Prometheus; Docker may download its pinned image on the first
run...\n'
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" \
```

```
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT"
```

Wait for readiness, then check the official build-information operation:

```
(
set -euo pipefail
ready=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking Prometheus readiness on port %s...\n' "$DOTX_HTTP_PORT"
for ((attempt=0; attempt<60; attempt++)); do
  if curl --fail --silent --max-time 2 \
    "http://127.0.0.1:$DOTX_HTTP_PORT/-/ready"; then
    ready=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for Prometheus: attempt %s/60, %ss elapsed\n' "$((attempt + 1))"
"$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $ready; then
  printf 'Prometheus readiness timed out. Run the cleanup block before retrying.\n'
>&2
  exit 1
fi
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$DOTX_HTTP_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
printf 'Verified: Prometheus 3.15.1 answered its official API. Open
http://127.0.0.1:%s/\n' "$DOTX_HTTP_PORT"
)
```

Expected: `Prometheus Server is Ready.`, `true`, then a `Verified:` message confirming Prometheus `3.15.1` and printing its browser URL. The loop is bounded: up to 60 attempts, each with a two-second HTTP timeout and a one-second pause. Open the printed URL (Prometheus with the default port).

You have verified an actual service response, not just a valid definition. The selected Anchor names an operation; its Contract describes that operation; the service binding identifies the listening port.

Stop and verify cleanup



This Compose deployment's destroy operation removes its named metrics volume. The data is disposable. Do not use another person's instance name.

```
dotx destroy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" &&
remaining="$(docker ps -aq --filter "label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
remaining="$(docker network ls -q --filter
"label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
remaining="$(docker volume ls -q --filter
"label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
printf 'Cleanup verified: Prometheus containers, network and volumes are absent.\n'
```

Expected: the instance stops and the three successful leftover queries print `Cleanup verified: Prometheus containers, network and volumes are absent.` Images remain cached. Keep `LAB` and the same terminal for the next lesson.

Resume or retry without losing your work

Paste the exact `source …/session.bash` command printed by the workspace block. Source only your own saved file. Then verify the original binary and restore its PATH before continuing:

```
# Source your own saved session.bash first, using the command printed in Lesson 1.
set -o pipefail
if [[ -n "${DOTX_TUTORIAL_CLI:-}" && -n "${DOTX_TUTORIAL_CLI_SHA256:-}" &&
      -n "${LAB:-}" && -n "${DOT_REPOSITORY:-}" ]] &&
  test "$(sha256sum "$DOTX_TUTORIAL_CLI" | cut -d ' ' -f 1)" =
"$DOTX_TUTORIAL_CLI_SHA256"; then
  DOTX_TUTORIAL_BIN_DIR="$(dirname -- "$DOTX_TUTORIAL_CLI")" &&
  export PATH="$DOTX_TUTORIAL_BIN_DIR:$PATH" &&
  dotx version --verbose &&
  printf 'Resumed existing workspace: %s\n' "$LAB"
else
  printf 'Saved session is missing/incomplete, or its CLI changed. Source your own
session file and recover the recorded binary before managing this workspace.\n' >&2
  false
fi
```

The saved session restores the repository, ports, instance name and cache location. If the binary check fails, stop and recover the original checksummed CLI before managing existing resources. Passwords are never saved: for a running Grafana instance, repeat Lesson 4's password prompt with the original lab password. Choose a new value only after that instance is cleaned up.

What already exists	Safe next action
`base/dot.json` or `prometheus/dot.json`	Validate and inspect the existing file. Do not rerun its creation block over your edits. Complete missing edits in your editor, then validate again.

What already exists	Safe next action
`prometheus.yml`, Grafana inputs or a candidate	Open and review the existing file. Creation blocks refuse replacement; they do not reset an exercise for you. An incomplete, empty directory may be removed with `rmdir` before retrying; never delete a directory containing your work.
`composition.yaml` or `dashboard.yaml`, with its reviewed lock	Use the original file and `dotx compose status` with the restored repository. After an interrupted apply, use that lesson's destroy command before retrying. Do not create another UUID or regenerate a lock to adopt leftover resources.
A successful destroy	Runtime state is gone; `COMPOSITION_STATE_MISSING` is expected. Keep the learning files and continue with the next lesson, rather than rerunning creation blocks.

If cleanup fails or Docker cannot be queried, cleanup is **unverified**. Keep the original files, cache and binary, restore Docker access, and retry the same cleanup command. Do not prune unrelated resources.

If a command failed, consult quickstart troubleshooting for PATH, access, Docker, ports and download failures. Never change a digest just to get past a validation error.

Checkpoint

Continue when you have:

- Seen the official API report Prometheus `3.15.1`.
- Seen the explicit cleanup-verification message.
- Saved your private workspace path and kept the original CLI and cache.

▼ *Explain it back: what did the dry-run prove?*

It checked and previewed the selected deployment without starting the service. The later API call established the running service's response and version. Successful cleanup established that this standalone run's resources were gone. These are three separate observations.

Previous: DOT-Design 101 · **All lessons** · **Next: create a real DOT and understand its model.**

2. Create an interface-aware DOT

Now build the definition yourself. You will keep the real product identity `Prometheus@3.15.1`, use its official OpenAPI document and bind two named Integration Points to a real service port. This is a **private learning copy**, not a second catalogue entry or a renamed fork of Prometheus.

Continue in the same Bash shell from Lesson 1, with `LAB` and `DOT_REPOSITORY` set. No runtime is started in this lesson.

Goal: create `$LAB/prometheus/dot.json` with verified interfaces and two deployment choices. **You will learn:** Contract ownership, provided/required Anchors and service-scoped configuration. Keep the generated scaffold so you can review exactly what the enrichment changes.

Author the provided interfaces

Reuse the catalogue's reviewed image digest and official Contract URL/hash. The CLI creates identifiers and the complete generated-model document:

```
(
set -euo pipefail
IMAGE="$(jq -r '.runtimes[].deployments[] |
  select(.id == "docker") | .manifest.image' "$PROMETHEUS_JSON")"
CONTRACT_URL="$(jq -r '.contracts[0].payload.url' "$PROMETHEUS_JSON")"
CONTRACT_SHA256="$(jq -r '.contracts[0].payload.sha256' "$PROMETHEUS_JSON")"

printf 'Creating a private Prometheus definition from the pinned official OpenAPI
document...\n'
dotx new --from docker-image "$IMAGE" \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract-url "$CONTRACT_URL" --contract-sha256 "$CONTRACT_SHA256" \
  --contract-name HTTP --contract-standard OpenAPI --contract-format YAML \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' \
  --operation 'promql-query-api=#/paths/~1query/get' --bind http \
  --output "$LAB/base/dot.json"

dotx validate "$LAB/base/dot.json" --repository "$DOT_REPOSITORY" --profile reusable
dotx inspect "$LAB/base/dot.json"
printf 'Draft verified: two provided Anchors. Open %s/base/dot.json to follow their
Contract references.\n' "$LAB"
)
```

Expected: an owned interface is verified, the output is saved, and reusable validation succeeds. `Draft verified: two provided Anchors` is your checkpoint. Open `$LAB/base/dot.json` in your editor. You supplied the release and selected operations; the CLI constructed their readable IDs and service bindings.

What you see	What it means here
`id`, `name`, `version`	One real tool release: `Prometheus@3.15.1`. This is not the CLI version.
`skills`	`Metrics-Scraping` is a capability reference, not an API operation.
`contracts`	A complete owned declaration. `payload.url` points to raw official bytes; `sha256` pins them. Contract version `3.1.0` is the OpenAPI standard version.
`anchors`	`build-information` and `promql-query-api` provide selected operations from that Contract. Their selectors are JSON Pointers, separate from Contract IDs.
`runtimes[].platform`	The `DOCKER` engine: where this deployment runs.

What you see	What it means here
`deployments[].id` and `manifest`	`docker-image` selects a `DOCKER_IMAGE` mechanism and exact image bytes.
`services`, `ports`, `anchorBindings`	Service `prometheus` materializes both Anchors on the named `http` port, 9090. The flag `http:9090` supplies a name and number. Native authoring binds TCP listeners; the name `http` alone does not change that protocol label.
`metadata`	Definition audit timestamps. Live endpoints and lifecycle state are elsewhere.

For example, one generated Anchor looks like this:

```
{
  "id": "build-information",
  "name": "build-information",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

`~1` encodes `/` inside a JSON Pointer segment. It selects the operation from verified Contract bytes; a documentation hyperlink would not establish an API match. The image pin and document pin establish identity and integrity, not publisher authenticity by themselves: the catalogue's provenance still matters.

Describe what Prometheus needs

An API provider can also be a consumer. Prometheus needs an exporter endpoint and a scrape configuration file. Its catalogue definition already documents that requirement and the real configuration path. Reuse those release-matched parts instead of inventing another interface. Also add its existing Compose deployment to see how one DOT supports more than one mechanism:

```
(
set -euo pipefail
set -o noclobber
mkdir "$LAB/prometheus"
jq --slurpfile product "$PROMETHEUS_JSON" '
  .anchors += [$product[0].anchors[] | select(.id == "scrape-target")]
  | .runtimes[0].deployments[0].services[0].ports[0].protocol = "HTTP"
  | .runtimes[0].deployments[0].services[0].config =
      ($product[0].runtimes[].deployments[] | select(.id == "docker")
       | .services[] | select(.id == "prometheus") | .config)
  | .runtimes[0].deployments +=
      [$product[0].runtimes[].deployments[] | select(.id == "docker-compose")]
' "$LAB/base/dot.json" > "$LAB/prometheus/dot.json"
```

```
dotx validate "$LAB/prometheus/dot.json" --repository "$DOT_REPOSITORY" --profile
reusable
dotx inspect "$LAB/prometheus/dot.json"
printf 'Definition verified: three Anchors and two Docker deployment choices. Review
the changes below.\n'
diff -u "$LAB/base/dot.json" "$LAB/prometheus/dot.json" || test "$?" -eq 1
)
```

Read the diff. There are four intentional changes:

1. `scrape-target` is a **REQUIRES** Anchor referencing Node Exporter's exact Contract and originating DOT. Prometheus does not copy or own that Contract.
2. The named service port is labelled `HTTP`, the actual application protocol. This matters when deriving an HTTP URL for another tool; the network transport still uses TCP. The initial native scaffold uses a TCP label.
3. The image deployment declares `/etc/prometheus/prometheus.yml` as a required service configuration file. The composition will supply its contents.
4. The same Docker Runtime gains the catalogue's `docker-compose` deployment, including its verified URL payload and declared defaults.

The final `$LAB/prometheus/dot.json` has three Anchors and two deployments. It is complete for the interfaces taught here; it is not a copy of every capability Prometheus could have. `DOCKER_IMAGE` and `DOCKER_COMPOSE` are mechanisms on the same engine. A Kubernetes deployment would belong in another Runtime.

`config` is service-scoped. File defaults, when declared, use `payload.url` and `sha256`; actual instance overrides belong outside the reusable definition. Ordinary environment defaults are scalars, not encoded file documents. Each service in a multi-service deployment owns its own configuration and bindings.

Try the guided editor

This optional exercise changes declarations only. From the same shell:

```
dotx edit "$LAB/prometheus/dot.json" --interactive \
  --output "$LAB/prometheus.candidate.json" --repository "$DOT_REPOSITORY"
```

Choose **Skills**, then **Add catalogue capability**. Filter for `Observability`, select it, then choose **Review and save**. Review the summary and explicitly choose **Write draft**. Arrows browse, typing filters, Enter selects, Ctrl-U clears a text field and Ctrl-C cancels. `ACCESSIBLE=1` selects line menus. The default publication choice is Cancel. In that fallback, `/text` filters, the displayed number selects, and `q` cancels.

Compare the candidate to the original. Unchanged Anchor IDs, manifests and configuration stay intact. The candidate is not automatically promoted; the remaining lessons deliberately use `$LAB/prometheus/dot.json`. Try another edit with an unused output path and cancel: no second

candidate should appear.

The editor handles interfaces and Skills; it is not a manifest/configuration editor. The explicit JSON enrichment above is therefore a reviewed model edit, followed by validation, not an implied editor feature. For a genuinely new tool, use its own release, image and interfaces rather than relabelling these ones.

▼ *Inspect more detail when you need it*

The included commands use concise inspection to keep the next action visible. For the full declared configuration and bindings, run:

```
dotx inspect "$LAB/prometheus/dot.json" --verbose
```

This is still declaration inspection; no runtime is started.

If a creation block says the destination exists, your earlier work is protected. Inspect that existing file and follow the resume/retry guidance; do not remove an edited definition just to make the block run again.

Checkpoint

Continue when reusable validation passes for `$LAB/prometheus/dot.json` and inspection shows three Anchors and both `docker-image` and `docker-compose`. The optional editor candidate is separate; the next lesson uses the enriched `prometheus/dot.json` you just validated.

▼ *Explain it back: where does the future scrape target belong?*

The DOT declares the required interface and the supported configuration path. The composition selects a concrete exporter and supplies the scrape file. Its Wire materializes that exporter's address into the execution copy of the file. Changing the monitoring use case does not require another product identity.

Previous lesson · **All lessons** · **Next: make this authored DOT consume real metrics.**

3. Connect two DOTs

Your Prometheus definition has both provided and required interfaces. Connect its `scrape-target` to the catalogue's Node Exporter, then query the result. Continue in the same shell; Lesson 1's standalone service must already be stopped.

Goal: make `up{job="node-exporter",tutorial="zero-to-hero"}` return `1`. **You will learn:** exact instance selection, a provided-to-required Wire, connection configuration and a reviewed lock. Your final graph has **two DOTs and one Wire**.

Create the use-case configuration

This file belongs to **your monitoring use case**, not to the shared product. The label will let you distinguish your configured scrape from an arbitrary successful Prometheus response:

```
(
set -euo pipefail
set -o noclobber
cat > "$LAB/prometheus.yml" <<'YAML'
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: node-exporter
    static_configs:
      - targets: [node-exporter:9100]
        labels:
          tutorial: zero-to-hero
YAML
chmod 0644 "$LAB/prometheus.yml"
)
```

The file is non-secret. Mode 0644 lets the unprivileged Prometheus container read the mounted file; its enclosing workspace stays private. Do not apply these permissions to secret files. The initial target is a placeholder: the Wire will write the selected instance's real host and port into the execution copy.

Create a fresh graph from the reviewed example

```
dotx compose new \
  --from "$DOT_REPOSITORY/Compositions/node-exporter-prometheus-
monitoring/composition.yaml" \
  --repository "$DOT_REPOSITORY" --name zero-to-hero \
  --output "$LAB/composition.yaml" --yes
```

Expected: a new UUID and a two-DOT, one-Wire preview. The template and its lock stay untouched. No lock is created for your draft and no service is deployed. The template's configuration references are initially shared; redirect them before editing a configuration.

Read the draft before changing it:

Field	Purpose in this graph
`apiVersion`, `kind`	Identify the generated Composition format, not a tool release or runtime.
`id`, `name`, `version`	A fresh ownership UUID, a readable use-case name and this graph's version. The graph's `1.0.0` is separate from each tool's release.
`dots`	Named instances, exact tool identities, selected deployments and instance inputs. `prometheus` is an instance name; `Prometheus@3.15.1` is its tool identity.

Field	Purpose in this graph
`contracts`	Verified interface inputs resolved from their exact owning DOTs, not copied specifications.
`wires`	Connections and the configuration needed to materialize them.
`exposedAnchors`	Names you can select with `--forward`, such as `prometheus-query`. An exposure does not add authentication.

Open `$LAB/composition.yaml` in a text editor and make **two changes**:

1. Replace only the `prometheus` entry under `dots` with:

   Keep the four leading spaces shown before `- name`, matching the other DOT item in the CLI-created draft. YAML indentation is part of the document.

   ```
       - name: prometheus
         source: ./prometheus
         dotId: Prometheus@3.15.1
         dotVersion: 3.15.1
         deployment: docker-image
         dependsOn: [node-exporter]
   ```

   Before, this entry selected `repository://Prometheus@3.15.1` and deployment `docker`. After, `./prometheus` and `docker-image` select your private authored definition. Its identity, tool release and readiness dependency stay the same.

2. In `node-metrics-to-prometheus`, change `materializer.source` to `prometheus.yml`. Keep the other Wire fields unchanged.

   Before, the source points to the template's `runtime/prometheus.yml` (the draft may show its full absolute path). After, it names your own file beside the draft:

   ```
   materializer:
     type: CONFIG_FILE
     source: prometheus.yml
   ```

   This last snippet shows the changed field's location, not the complete materializer: retain its existing `valueFrom` and `target` below these lines.

`./prometheus` selects **your authored definition**, not the catalogue's deployment. The local `docker-image` ID comes from your `dotx new` output. Leave Node Exporter and the composition's Contract inputs at their exact catalogue coordinates. The owned Prometheus Contract has the same verified bytes as that release's catalogue declaration. Relative file paths resolve beside this YAML file.

Understand the connection

The retained Wire names these model relationships:

```
from: {instance: node-exporter, anchorId: node-metrics}
to: {instance: prometheus, anchorId: scrape-target}
materializer:
  type: CONFIG_FILE
  source: prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

`from` provides metrics; `to` requires them. The materializer derives the producer's address and places it at one location in the consumer's configuration. It does not implement the scrape: Prometheus does that. `dependsOn` expresses startup ordering; a Wire expresses an interface connection.

Node Exporter's metrics operation uses a **source-derived OpenAPI projection**. It is not a vendor-published OpenAPI specification or complete OpenMetrics certification. Both sides select that same exact interface. The unprivileged exporter observes its container; this exercise makes no physical-host claim.

Validate, lock and preview

These are deliberate review steps: validate the edited declaration, create its lock, inspect the exact resolved inputs and topology, then preview deployment. The block stops if any step fails; it starts no services.

```
(
set -euo pipefail
dotx compose validate "$LAB/composition.yaml" --repository "$DOT_REPOSITORY"
dotx compose lock "$LAB/composition.yaml" --repository "$DOT_REPOSITORY"
dotx compose inspect "$LAB/composition.yaml" --repository "$DOT_REPOSITORY" --locked
dotx compose topology "$LAB/composition.yaml" --repository "$DOT_REPOSITORY" --locked
dotx compose apply "$LAB/composition.yaml" --repository "$DOT_REPOSITORY" \
  --locked --dry-run --forward "prometheus-query=$TUTORIAL_PORT"
printf 'Graph verified: two DOTs, one Wire. No service has started.\n'
)
```

Expected: two DOTs, one Wire and a deployment preview using your local `docker-image` definition. Review `$LAB/composition.lock.json`: it binds exact definitions, documents and configuration. A lock is reproducibility evidence, not runtime state or a second authored model. Intentional input changes require another explicit lock review; never regenerate locks just to hide an error.

Run and ask a meaningful question

Use an unused `TUTORIAL_PORT` (default `19041`). Before applying, keep the stop command below available. **Run it even if apply or a query fails.** Preserve the original workspace, cache and binary

for recovery.

```
printf 'Starting Node Exporter and your Prometheus; the next check waits for their
labelled scrape...\n'
dotx compose apply "$LAB/composition.yaml" --repository "$DOT_REPOSITORY" \
  --locked --forward "prometheus-query=$TUTORIAL_PORT"
```

```
(
set -euo pipefail
set -o pipefail
scraped=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking the labelled Node Exporter scrape through Prometheus...\n'
for ((attempt=0; attempt<90; attempt++)); do
  if curl --fail --silent --max-time 3 --get \
    --data-urlencode 'query=up{job="node-exporter",tutorial="zero-to-hero"}' \
    "http://127.0.0.1:$TUTORIAL_PORT/api/v1/query" |
    jq -e '.status == "success" and any(.data.result[]?; .value[1] == "1")'
>/dev/null; then
    scraped=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for the scrape: attempt %s/90, %ss elapsed\n' "$((attempt + 1))"
"$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $scraped; then
  printf 'Scrape check timed out. Inspect compose status, then run the Lesson 3
cleanup block before retrying.\n' >&2
  exit 1
fi
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$TUTORIAL_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
dotx compose status "$LAB/composition.yaml" --repository "$DOT_REPOSITORY"
printf 'Verified: your labelled Node Exporter scrape is up (value 1).\n'
printf 'Try the same query in Prometheus: http://127.0.0.1:%s/\n' "$TUTORIAL_PORT"
)
```

Expected: the check reports progress while waiting, build information returns `true`, status lists the selected instances, and a `Verified:` message confirms the labelled scrape. The loop permits up to 90 attempts with three-second HTTP timeouts and one-second pauses. Open the URL printed by the check (Prometheus with the default port) and evaluate:

```
up{job="node-exporter",tutorial="zero-to-hero"}
```

The result `1` establishes your labelled scrape, not merely an HTTP 200. The label came from your configuration; the target address came from the Wire. The original catalogue and its configuration are unchanged.

Stop before extending

```
dotx compose destroy "$LAB/composition.yaml" --repository "$DOT_REPOSITORY"
```

Expect no running resources for this composition after successful destroy. The lab's time-series data is disposable. Do not delete the draft or lock while cleanup is unresolved, and never prune unrelated Docker resources. If ownership is uncertain, stop and inspect rather than deleting by a guessed name. Destroy removes the lifecycle record. A later `compose status` therefore returns `COMPOSITION_STATE_MISSING`; that is expected after successful cleanup.

Try a safe diagnostic exercise: copy the YAML to a new file, change `from.anchorId` to `missing-metrics`, and validate that copy **without** `--locked`. Expect a missing-Anchor rejection. Restore the reference and validate again. Do not modify the reviewed live draft or publish another lock for the bad copy.

If the result differs

Symptom	What to check before retrying
`INPUT_YAML`	Check the edited list item's four leading spaces and the surrounding list. Do not add a second `dots:` key. Validate again before creating a lock.
A missing Anchor or a lock mismatch	Compare the exact instance, Anchor and local definition references with the reviewed edit. If an input changed intentionally, inspect it before explicitly recreating the lock. A new lock cannot repair an invalid reference.
No sample, or a sample with value `0`	`0` means the target's scrape failed; no sample means the selected series is absent. Check lifecycle status, the `node-exporter` job, the tutorial label and the Wire's materializer source. The API build-information check alone is insufficient.
A readiness or query timeout	Stop and use this lesson's destroy command. Keep the original files, cache and binary if cleanup fails; use the recovery guide.

Checkpoint

Continue when the labelled query returns `1`, the topology shows two DOTs and one Wire, and destroy succeeds. Keep the draft, its lock and your scrape file: Lesson 4 extends those inputs into a new graph with a fresh ownership UUID.

▼ *Explain it back: which direction does this connection run?*

The Wire points from Node Exporter's provided Anchor to Prometheus's required Anchor. At runtime, Prometheus sends an HTTP scrape request to Node Exporter. The Wire describes the interface relationship; the application implements the request. The materializer supplies its address through configuration.

Previous lesson · **All lessons** · **Next: extend this same system with Grafana.**

4. Grow the system with Grafana

You now have a useful two-tool system. Add a dashboard without rewriting either tool. This lesson extends your existing graph: **Node Exporter → your Prometheus → Grafana**. It adds one DOT instance, one Wire and a browser-facing exposure.

Continue in the same Bash shell. First complete Lesson 3's destroy step. The new graph gets another ownership UUID; it does not adopt the stopped graph's resources or inherit its live state.

Goal: see the same labelled scrape through Grafana. **You will learn:** extending a graph, provisioning a datasource and supplying a password through a reference. The result is **three DOTs and two Wires**; the original scrape relationship stays intact.

Prepare the next real tool

Reuse Grafana's exact catalogue release in a private exercise copy. Add a service-scoped declaration for its supported admin-password environment option:

```
(
set -euo pipefail
set -o noclobber
GRAFANA_JSON="$(dotx repository resolve Grafana@13.3.0 --repository
"$DOT_REPOSITORY")"
mkdir "$LAB/grafana"
jq '.runtimes[0].deployments[0].services[0].config.environment +=
  [{"name":"GF_SECURITY_ADMIN_PASSWORD","required":true,"sensitive":true}]' \
  "$GRAFANA_JSON" > "$LAB/grafana/dot.json"
dotx validate "$LAB/grafana/dot.json" --repository "$DOT_REPOSITORY" --profile
reusable

cat > "$LAB/grafana-prometheus.yml" <<'YAML'
apiVersion: 1
datasources:
  - name: Prometheus
    uid: hero-prometheus
    type: prometheus
    access: proxy
    url: http://replaced-by-the-wire.invalid
    isDefault: true
YAML
cat > "$LAB/grafana-empty.yml" <<'YAML'
```

```
apiVersion: 1
datasources: []
YAML
chmod 0644 "$LAB/grafana-prometheus.yml" "$LAB/grafana-empty.yml"

dotx compose new --from "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" --name hero-dashboard \
  --output "$LAB/dashboard.yaml" --yes
)
```

Expected: the Grafana candidate validates and `dashboard.yaml` retains the earlier monitoring graph with a fresh UUID. The catalogue remains unchanged. The copied product already carries its official pinned OpenAPI declaration, required Prometheus Anchor, provided dashboard Anchor and service bindings. You extended its **configuration surface**, not its API identity.

The datasource file is an instance input. `uid: hero-prometheus` makes it easy to select and test. `access: proxy` makes Grafana's backend query Prometheus over the composition network; the browser does not need to reach an internal container address. The Wire replaces the `.invalid` URL before deployment.

This catalogue release also requires an Elasticsearch provisioning file. The empty, valid `grafana-empty.yml` deliberately provisions no Elasticsearch datasource: that integration is outside this lab. Do not draw a fictitious Wire or claim that it has been tested.

Extend the existing graph

Open `$LAB/dashboard.yaml`. Before editing, it has two DOTs and one Wire. You will add one item to each of three **existing** lists. Keep the earlier items, Contract inputs and ownership UUID; do not paste another root `dots:` or `wires:` key. Each snippet below is a list item, beginning with `- name`, at the same indentation as the existing items. Keep the four leading spaces shown before each `- name` in the CLI-created draft.

Add Grafana under `dots`

Append this after the existing `prometheus` instance:

```
    - name: grafana
      source: ./grafana
      dotId: Grafana@13.3.0
      dotVersion: 13.3.0
      deployment: docker
      dependsOn: [prometheus]
      deploymentInputs:
        environment:
          - name: GF_SECURITY_ADMIN_PASSWORD
            secretRef: {store: environment, name: HERO_GRAFANA_PASSWORD}
        configFiles:
          - source: grafana-empty.yml
```

```
            target: /etc/grafana/provisioning/datasources/elasticsearch.yml
```

Add the second connection under `wires`

Append this after `node-metrics-to-prometheus`:

```
    - name: prometheus-to-grafana
      from: {instance: prometheus, anchorId: promql-query-api}
      to: {instance: grafana, anchorId: promql-query-api}
      materializer:
        type: CONFIG_FILE
        source: grafana-prometheus.yml
        valueFrom: ENDPOINT_URL
        target:
          instance: grafana
          service: grafana
          path: /etc/grafana/provisioning/datasources/prometheus.yml
          jsonPointer: /datasources/0/url
```

Add the browser endpoint under `exposedAnchors`

Append this after `prometheus-query`:

```
    - name: dashboard
      target: {instance: grafana, anchorId: monitoring-dashboard}
```

The new Wire connects a provided PromQL operation to Grafana's required operation. Its materializer writes the selected Prometheus base URL into `/datasources/0/url`. The two Wires have different jobs: the first configures scraping; the second configures querying.

`secretRef` names a host environment variable; it contains no password. The executor resolves it at apply time. Do not paste a sensitive `value` into the composition. Docker environment delivery is still visible to an authorized Docker administrator; it is not a vault or native Kubernetes Secret.

```
(
set -euo pipefail
dotx compose validate "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY"
dotx compose lock "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY"
dotx compose topology "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY" --locked
printf 'Extended graph verified: three DOTs, two Wires. Next: supply a lab password,
then preview and apply.\n'
)
```

Expected: **three DOTs, two Wires**. Review the new lock separately. Creating or inspecting it does not require the password.

Set a password and start the extended system

Use a new lab-only password. The prompt does not echo it or put it in shell history. Keep this environment and the same binary for the lifecycle:

```
if read -r -s -p 'Choose a lab-only Grafana admin password: ' HERO_GRAFANA_PASSWORD &&
  test -n "$HERO_GRAFANA_PASSWORD"; then
  printf '\n'
  export HERO_GRAFANA_PASSWORD
else
  unset HERO_GRAFANA_PASSWORD
  printf '\nNo password set. Repeat this prompt before applying.\n' >&2
  false
fi
```

Ports 19041 and 13000 must be free. Use only the isolated development host and network described at the start. Keep the cleanup command below available before applying; use it after a failure as well. The next block previews both exposures first; a failed preview prevents its apply command from running. Runtime preflight, including the dry-run, needs the referenced password available.

```
printf 'Previewing, then starting Node Exporter, Prometheus and Grafana; pinned images
may need downloading...\n'
dotx compose apply "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY" \
  --locked --dry-run --forward "prometheus-query=$TUTORIAL_PORT" --forward
"dashboard=$DASHBOARD_PORT" &&
dotx compose apply "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY" \
  --locked --forward "prometheus-query=$TUTORIAL_PORT" --forward
"dashboard=$DASHBOARD_PORT"
```

Verify the whole chain

Grafana health is the first check. The datasource query is the second: it exercises the connection to Prometheus and the labelled scrape from Node Exporter. Complete both before calling the chain successful.

Wait for Grafana's actual health/version response:

```
(
set -euo pipefail
set -o pipefail
dashboard_ready=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking Grafana 13.3.0 health on port %s...\n' "$DASHBOARD_PORT"
for ((attempt=0; attempt<90; attempt++)); do
  if curl --fail --silent --max-time 3 "http://127.0.0.1:$DASHBOARD_PORT/api/health" |
    jq -e '.database == "ok" and .version == "13.3.0"' >/dev/null; then
```

```
    dashboard_ready=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for Grafana: attempt %s/90, %ss elapsed\n' "$((attempt + 1))"
"$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $dashboard_ready; then
  printf 'Grafana health check timed out. Inspect compose status, then run the Lesson
4 cleanup block before retrying.\n' >&2
  exit 1
fi
printf 'Ready: Grafana 13.3.0 at http://127.0.0.1:%s/\n' "$DASHBOARD_PORT"
)
```

Expected: named waiting messages include elapsed time, followed by `Ready: Grafana 13.3.0` and the browser URL. Its loop permits up to 90 attempts, with three-second HTTP timeouts and one-second pauses.

Open Grafana, sign in as `admin` with your chosen password, and open **Explore**. Choose the provisioned **Prometheus** datasource and run:

```
up{job="node-exporter",tutorial="zero-to-hero"}
```

Expect a sample with value 1. Grafana UI labels can vary by release; the backend check below provides a reproducible alternative. You have extended one running use case without putting a use-case-specific datasource or password into the shared product definitions.

▼ *Verify the same query without a browser*

Make Grafana's backend query the provisioned datasource. The temporary curl credentials file is private and removed when the check finishes, including a failed query. Neither the password nor its value enters YAML or the lock.

```
(
set -euo pipefail
set -o pipefail
# curl reads credentials from a private file, not command-line arguments.
credentials="$(mktemp "$LAB/grafana.curl.XXXXXXXX")"
trap 'rm -f -- "$credentials"' EXIT
(umask 077; jq -nr '"user = " + ("admin:" + env.HERO_GRAFANA_PASSWORD | tojson)' \
  > "$credentials")
dashboard_queried=false
started=$SECONDS
next_notice=$SECONDS
printf 'Querying the labelled scrape through Grafana...\n'
```

```
for ((attempt=0; attempt<90; attempt++)); do
  if curl --config "$credentials" --fail --silent --max-time 5 \
    --header 'Content-Type: application/json' \
    --data '{
      "from":"now-5m", "to":"now",
      "queries":[{
        "refId":"A",
        "expr":"up{job=\"node-exporter\",tutorial=\"zero-to-hero\"}",
        "instant":true,
        "datasource":{"type":"prometheus","uid":"hero-prometheus"}
      }]
    }' \
    "http://127.0.0.1:$DASHBOARD_PORT/api/ds/query" |
    jq -e '.results.A.status == 200 and
      any(.results.A.frames[]?.data.values[]?[]?; . == 1)' >/dev/null; then
    dashboard_queried=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for the Grafana query: attempt %s/90, %ss elapsed\n' "$((attempt
+ 1))" "$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $dashboard_queried; then
  printf 'Grafana query timed out. Check the password and datasource; run the Lesson
4 cleanup block before retrying.\n' >&2
  exit 1
fi
printf 'Verified: Grafana queried the labelled scrape (value 1).\n'
)
```

Expected: `Verified: Grafana queried the labelled scrape (value 1).` The bounded loop allows 90 attempts with five-second HTTP timeouts and one-second pauses. The native request selects datasource UID `hero-prometheus` and runs an instant query. This verifies Grafana → Prometheus → Node Exporter; the diagram alone could not establish it.

Clean up the complete graph

```
dotx compose destroy "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY" &&
unset HERO_GRAFANA_PASSWORD &&
{ if test -f "$LAB/grafana.curl"; then rm -- "$LAB/grafana.curl"; fi; } &&
printf 'Cleanup completed: keep the workspace files for reference.\n'
```

Expected: all three owned instances stop. This is disposable storage; downloaded images remain cached. Keep your authored files as a learning reference, but retain the original cache and lock if cleanup did not finish. Do not remove unrelated resources or change a lock to adopt an existing

instance. The success message and password removal happen only after destroy succeeds. If it fails, retain the current environment and use the recovery steps before retrying.

Checkpoint

Finish when Grafana reports version `13.3.0`, its datasource query returns the labelled sample with value `1`, and the complete graph's destroy succeeds. Either Explore or the documented backend query can verify that sample; the health response alone cannot.

If sign-in or the backend query fails, check that you used the lab password prompt, datasource UID `hero-prometheus` and the chosen dashboard port. A resumed running instance needs its original password. Use the recovery guide before retrying an interrupted run.

▼ *Explain it back: what stays reusable when you add a dashboard?*

Prometheus keeps its product identity, Contract and Anchors. Grafana's datasource, the selected Prometheus instance, the password reference and both Wires belong to this composition. The private Grafana copy adds a supported sensitive configuration declaration; it contains no password value. Live addresses and resolved credentials remain execution concerns.

Previous lesson · **All lessons** · **Next: apply the pattern to larger, secured systems.**

5. Larger and secured systems

You have built a real chain: **Node Exporter** → **your Prometheus** → **Grafana**. You authored a DOT, reused two others, connected required and provided interfaces, supplied configuration and a secret reference, verified actual traffic, and cleaned up. That is the reusable pattern behind larger systems.

Before leaving the Docker lab, confirm that both lessons' destroy steps succeeded. Repeat the original cleanup command only for an interrupted or unresolved run; missing lifecycle state is expected after successful cleanup. Keep the private learning files if useful. Do not publish the exercise copies as additional Prometheus or Grafana catalogue roots.

What you can now do

You have practiced the complete small-system loop:

1. **Describe** a release with Skills, verified Contracts and named Anchors.
2. **Select** a deployment and declare its service configuration.
3. **Connect** provided and required Anchors through reviewed Wires.
4. **Review** exact inputs with validation, a lock and a deployment preview.
5. **Verify** an actual API result, then clean up the owned resources.

Keep your final topology and the two successful query results as a learning record. This lab established those particular interactions. Production work also needs the security, persistence, operations and interface coverage in the production integration checklist.

Grow deliberately

Your next goal	What changes	What must be verified
Observe more targets	Select more instances and configure additional targets; keep reusable product definitions shared. Give each materialization its own reviewed configuration location.	Each intended target is actually scraped; labels distinguish producers; interruption and recovery affect only the selected instance.
Use another runtime	Choose a matching deployment from the same tool release. Review its service configuration, native workload selection and required permissions.	Readiness, real API behavior and cleanup on that adapter. The Docker/Kubernetes example is deployment portability, not proof of a cross-runtime Wire.
Cross a runtime boundary	Supply an explicitly provisioned external-gateway host/port using the existing execution profile. The operator retains gateway ownership.	Reachability from the actual consumer and attribution to the exact producer. There is no automatic infrastructure or protocol translation.
Package a multi-service tool	Declare each logical service's ports, config and Anchor bindings. Bind services to exact native workloads at composition level when needed.	Separate configuration and secrets, same-Pod/container selection, peer isolation and ownership-safe cleanup. See services and inputs.
Use messaging or another API style	Select verified AsyncAPI, GraphQL, Protobuf or SPARQL operations through guided authoring. Broker transport endpoints are separate inputs.	The supported operation/transport profile, authentication and actual exchange. Four-standard test fixtures are not production-producer certification.

Operator capstone: authentication and policy

The next difficulty is controlling who can use an API. The existing Keycloak/APISIX/OPA/SPARQL composition provides a five-DOT exercise:

- Keycloak provides identity and token operations.
- APISIX routes a protected capability.
- OPA decides whether a request is allowed.
- Oxigraph provides the SPARQL query service.
- A clearly labelled demo client exercises the authenticated path.

This is a **separate advanced graph**, not a claim that the preceding Grafana lab has already been secured or connected to it. Keep your tested monitoring system as the small reference while learning native Secret references, policy, multiple Wires and service startup. Only combine graphs after defining and testing the actual additional interfaces and ownership boundaries.

This capstone requires a qualified operator environment, Kubernetes, native Secrets and the

matching source toolchain/artifact set. It is not a hidden prerequisite for the Docker tutorial. From the **DOT-Design checkout root**, after following the contributor setup and the secure-API prerequisites, the existing presenter route is:

```
task testing:pocs:present:secure-api
```

Expected: anonymous rejection, authenticated success, the expected RDF result and policy checks, followed by an inspection pause and owned cleanup. The full qualification route also exercises failure and interruption cleanup. Local HTTP defaults, synthetic clients and explicit-interface boundaries remain documented; this is not complete MARE producer acceptance.

Turn a learning result into a contribution

1. For an existing product release, propose changes to its one canonical definition. For another supported release, use its exact release folder and release-matched artifacts and Contracts; see catalogue layout.
2. Keep use-case files in a Composition. Document its purpose, prerequisites, selected deployments, topology, expected API results, cleanup and limitations.
3. Validate against DOT-Domain's generated schemas, inspect and review changed locks, then rehearse the affected real interactions and failure cleanup.
4. Keep credentials and runtime state out of Git. Treat official and source-derived specifications honestly; a hash does not establish origin.

For exact CLI options use `dotx <command> --help`. For deeper explanations use DOT authoring, composition authoring, the complete model reference and production integration limits.

Contributors can rehearse the same included command blocks from DOT-Design with `task testing:tutorial` (declarations/previews) and `task testing:tutorial:live` (actual Docker services and cleanup). The test driver applies the documented editor changes, rather than maintaining a second tutorial definition. These are maintainer rehearsals, not independent newcomer-trial results.

Previous lesson · **All lessons**

Adoption, compatibility and editor setup

DOT connects reusable component descriptions, authoritative API Contracts, deployment choices and explicit composition wires. It does not replace OpenAPI, AsyncAPI, Protobuf, GraphQL, Docker Compose, Helm or Kubernetes. Start by reusing a complete catalogue bundle, not by inventing a new descriptor.

Choose a compatible set

For an authorized evaluation without contributor toolchains, use the matching prebuilt evaluation kit. Its first composition stays on Docker and retains the same binary/catalogue throughout. Availability and sharing require an authorized issuer; no new public release is implied.

The site opens **Next**: documentation for the current source checkout, not a promise that every command exists in the latest published binary. Use the version selector for historical release documentation.

Selected release baseline: CLI `v0.4.0` includes `dotx compose new` and the current model conventions for the supported Linux-amd64 profile. It does not certify every producer or catalogue entry. The original released CLI `v0.2.5` predates these features. For the official demo, use the submodules pinned by one clean DOT-Design commit, run `task testing:demo`, and retain its exact checkout revisions and binary SHA-256. Do not mix the latest release binary with the current catalogue. See the source-qualified demo route.

Artifact	Compatibility rule
Published CLI binary	Use its immutable release tag, checksum, acceptance manifest and matching catalogue revision. The released quickstart pins that catalogue explicitly; do not substitute current `main`.
Source-built CLI	Use the exact DOT-CLI and DOT-Repository submodules pinned by the DOT-Design checkout. `dotx version --verbose` identifies the revision and modified state. The source version label alone does not identify its capabilities.
DOT definition and Contract versions	These describe the component and interface, not the CLI release. Keep stable IDs, select exact DOT versions and review changed payload hashes and locks.
Composition v1	`apiVersion: dot.mare6g.eu/v1` selects the portable Composition input model. It does not imply support for every adapter, probe or future extension.

Current source supports textual Skill/Contract IDs and exact Contract standard spellings `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`, plus `SPARQL` in 0.4.0; `format` retains its separate uppercase vocabulary. The original published CLI `v0.2.5` predates these changes and the bundle-aware prerequisite check below. Do not silently rewrite IDs, spelling, manifests or reviewed locks to make mismatched artifacts appear compatible. Follow the pinned source route in the contributor guide for current catalogue work. See the compatibility and migration policy for intentional spelling/identity boundaries, release-version rules and preservation of existing files, Anchor IDs and reviewed locks.

The native SPARQL addition uses a pinned W3C Service Description in `N_TRIPLES` with standard version `1.1`; OpenMetrics remains an OpenAPI response format. See the native profiles for query, media, provenance and acceptance limits. The published v0.3.0 binary remains unchanged.

Current source also distinguishes owned Contract declarations, explicit external Anchor references and provider-neutral OIDC requirements. Consumer bundles no longer carry producer Contract copies. An external reference requires its exact originating DOT in the selected catalogue; missing dependencies fail validation, including offline validation. See Anchors and composition for syntax, resolution and the deliberately narrow OIDC profile.

Edit with the actual model schema

Opening the DOT-Design root in VS Code associates `dot.json` with `DOT-Repository/Schema/schema.json`; opening the standalone DOT-Repository root uses `Schema/schema.json`. Both checkouts include workspace associations. JSON editing uses the built-in language service. For YAML compositions, enable the recommended Red Hat YAML extension; its workspace association uses the generated `composition-v1.schema.json`. No DOT-specific editor plugin is needed.

For other editors, associate those same **local** files with `dot.json` and `composition.yaml`, respectively. Use a matching pinned catalogue rather than a mutable remote schema. The generated schemas provide field descriptions, allowed values and structural validation without a Java runtime. Do not add an editor-only `$schema` property to DOT definitions: use editor configuration instead, preserving the portable document shape.

Editor validation is only the first check. It cannot replace bundle-local Contract integrity, Anchor selector resolution, catalogue identity checks or locked composition validation. Run `dotx validate` and the applicable local CI Tasks before committing. There is no automatic migration or lock repair.

For guided creation, author an owned OpenAPI interface with the same generated Contract model and existing verifier. The actual tool release is explicit; the wizard can generate provided Anchors and service-port bindings without copying a dependency Contract or inferring live acceptance. Raw-URL import can construct that declaration without hand-writing Contract JSON; its computed digest pins downloaded bytes, not upstream authority.

Check the selected deployment without starting it

Selected release or matching source. From the pinned DOT-Design checkout with the selected release or its source CLI installed:

```
dotx version --verbose
PROMETHEUS_JSON="$(dotx repository resolve Prometheus@3.15.1 --repository DOT-
Repository)"
PROMETHEUS_BUNDLE="$(dirname -- "$PROMETHEUS_JSON")"
dotx inspect "$PROMETHEUS_BUNDLE"
dotx tool check --dot "$PROMETHEUS_BUNDLE" --deployment docker-compose
dotx tool check --dot "$PROMETHEUS_BUNDLE" --deployment docker-compose --json
```

From a standalone catalogue checkout resolve with `--repository .` instead. Exact identity resolution works with the released and product/version layouts. For a standalone bundle outside a catalogue add `--schema /path/to/matching/Schema/schema.json`.

The command requires an **exact deployment ID**, validates the bundle offline with the ordinary validator, and checks only that adapter's prerequisites:

Manifest	Prerequisites checked
`DOCKER_IMAGE`	Docker CLI and daemon
`DOCKER_COMPOSE`	Docker CLI/daemon and Compose plugin
`KUBERNETES_MANIFEST`	kubectl and selected cluster
`HELM_CHART`	kubectl/cluster and Helm cluster access
`VAGRANTFILE`	Existing Vagrant/libvirt/KVM readiness checks
`WASM_MODULE`	Wasmtime installation

Expected Compose output includes `Definition: validated offline`, `docker: ready=true` and `compose: ready=true`. Every applicable tool is checked even when another fails. No downloads, installation, cluster startup, workload mutation or cleanup occur. Missing or tampered local Contracts fail before tool checks. Warm a bundle's verified Contract assets through the ordinary online validation path separately if necessary; do not bypass a digest mismatch.

JSON has `formatVersion: 1`, `ready`, `validated`, selected `deployment` and `manifestType`, ordered `checks`, shared validator `diagnostics` and an explicit `evidence` scope. Check exit status as well as the report: 0 means checks passed, 1 means validation/tool failure, 2 means usage error. Ordinary JSON failures leave stderr empty; raw subprocess output and configuration values are withheld.

`ready` applies **only to these checks**. Instance inputs and permissions, available images, free ports, artifact reachability, storage and live API behavior still need their own checks. This is neither deployment authorization nor production certification. Continue with the first DOT or composition demo using that walkthrough's compatible artifact set.

Integrate without duplicating the model

All portable inputs are owned by DOT-Domain: consume its generated `Schema/schema.json`, `Schema/composition-v1.schema.json` and `Schema/models/*.schema.json`. Do not maintain a competing schema, struct model or copied Contract operation catalogue. Executor resolved compositions, locks and runtime state are CLI concerns rather than portable domain inputs.

DOT, Skill and Contract identities are case-sensitive textual IDs; legacy UUID-shaped strings remain accepted. Catalogue DOT IDs use `<name>@<tool-release>`; Contract IDs use `<origin-DOT.id>::<standard>::<name>` with canonical standard spelling. Every DOT must reference at least one meaningful Skill. Anchors use exact DOT-local kebab-case IDs (explicit legacy UUID strings remain readable); Composition ownership identities remain UUIDs. A DOT name or version change therefore changes its catalogue identity; update composition pins and regenerate reviewed locks. IDs are not a replacement for the raw Contract URL plus SHA-256. The integration handoff and executable acceptance profiles document cross-reference, conservative compatibility and runtime

boundaries.

Independent onboarding trial

Use this checklist with three developers who did not implement DOT. The targets below are proposed usability goals, not measured results or acceptance claims.

Download the unfilled trial record for each participant. Keep release and current-source attempts separate; record pre-cached assets, maintainer setup, cold-download time, every intervention and cleanup. The template deliberately contains no enrolled participants or measured results. A maintainer's automated rehearsal does not fill these blanks.

1. Give each participant only repository access, a compatible Linux-amd64 host, Docker/Compose and the documentation entry point. Record exact revisions and whether images/Contracts were already cached.
2. Ask them to follow the released quickstart: install, find the DOT, validate, deploy, obtain the expected API response and clean up. Target: ten minutes of hands-on work, recording cold-download time separately.
3. Using the current-source route, ask them to inspect another deployment, explain one Contract/Anchor/wire, introduce an invalid reference, interpret the diagnostic and repair it. Record every undocumented intervention.
4. Check cleanup removed only their test-owned resources. Never remove another participant's workloads, volumes or forwards.
5. Keep redacted receipts, timings, failed commands and documentation gaps. Turn reproducible defects into regression tests; do not count coaching as independent success. No participants have been enrolled or trials performed by adding this checklist.

An independent implementation/conformance exercise is a separate next step: internal test coverage and a successful CLI demo do not establish adoption by another team. Licensing, governance and security-response decisions also need the project owners' approval; this guide does not grant new reuse rights.

Evaluate with one matching kit

An evaluation kit keeps one source-qualified Linux-amd64 CLI, catalogue, schemas and instructions together. It is an evaluation build, not a published release. Its source version label can equal an older release; exact commits and hashes in the kit receipt identify its capabilities.

Participant route

Obtain the kit and its independently recorded outer SHA-256 from your authorized project contact. No public download or redistribution approval is implied. Follow the included `START-HERE.md` from a fresh private folder. It verifies the complete inventory, installs the binary in that private workspace and extracts the matching local catalogue. Use that binary for the entire attempt. Keep its scoped extraction mask: non-secret example configurations must be readable by unprivileged containers even when your shell uses `umask 077`. The enclosing trial folder stays private; operator secret-file

permissions are not relaxed.

The first graph is **Node Exporter → Prometheus on Docker**: inspect the DOTs, create a fresh draft, review topology and lock, deploy, query the real scrape and clean up. Linux amd64, Bash, Docker/Compose, curl and jq are needed. Java, Go, Task and Kubernetes are not participant prerequisites for this graph.

The exporter observes its container rather than the physical host. Its metrics Wire selects an operation from a release-scoped, source-derived OpenAPI Contract for the HTTP metrics endpoint. This is not a vendor-published specification or complete OpenMetrics validation. First validation may download pinned documents; images are a separate runtime download. A warm document cache does not establish offline runtime readiness.

Reserve an exclusive session with the operator. Draft UUIDs alone do not isolate ports, namespaces, inputs or external gateways. Keep the original binary/cache until cleanup is verified; stop on unknown ownership. Never prune unrelated resources. Advanced Contract authoring, safe edit/cancel, graph creation and lock review remain part of the trial protocol.

Operator and maintainer route

From the **DOT-Design checkout root**, check the private-kit infrastructure:

```
task testing:adoption:checks
```

Assembly uses a clean committed source set, existing model packaging and complete asset checks. The kit records pending distribution rights and excludes resolved credentials, caches, runtime state and images. Keep it in a restricted persistent archive outside build directories and retain the independent issuer hash. Standalone checking is data-only: it never executes a supplied binary.

The secure-API walkthrough is a separate Kubernetes operator chapter. The Prometheus portability example has zero Wires and demonstrates independent deployments, not cross-runtime data flow. Actual cross-runtime qualification must identify the intended producer from the consumer context and prove interruption/recovery.

Evidence boundaries

The initial neutral-core inventory maps 77 exported models through existing generated-schema discovery and records reviewed semantic rule families. Its consistency guard does not validate arbitrary DOTs or certify a reader. Known unresolved issues include ownership-group policy disagreement, the narrower raw UUID restriction, raw parsing/presence coverage and complete semantic inventory. Affected neutral-profile publication is blocked until those are resolved.

Contributors can run `task testing:adoption:raw` from the DOT-Design checkout root to compare original raw inputs, Java decoding and object validation. It also enumerates compiled constraint groups, cascades, container elements and schema annotations. It requires the contributor toolchain; it is separate from the participant's prebuilt-binary route. The bootstrap may download pinned test tools; the gate fetches no DOT payloads and starts no workloads.

The generated DOT schema rejects the combination of exact `internal` and `external` group labels. The generated Composition schema restricts UUID versions/variants. Java object validation is broader in these contexts. Missing raw fields can also be supplied by Java construction defaults. These differences are now regression-tested, not silently unified. Nested ID-only references and complete owned declarations use different validation groups; catalogue publication is stricter than model readability. Parser-characterization results and descriptor fingerprints do not establish a neutral parser policy or complete semantic coverage.

No human trials, independent implementation or owner approvals are supplied by these checks. Read the remaining release gates before making release or production claims. Stable-tag documentation remains unchanged.

Catalogue products and releases

Select a tool release explicitly. A product folder groups its supported releases; a release folder contains one complete DOT definition and its quickstart.

```
DOTs/external/Prometheus-DOT/
  README.md
  3.15.1/
    dot.json
    README.md
```

The product README lists available releases. Each release's `dot.json` contains all its supported deployment mechanisms; there is no parent `dot.json`, `versions/` wrapper or mutable `latest` copy. The initial migration moves the existing 67 definitions without qualifying any additional historical releases.

Find and select a release

From the matching **DOT-Repository checkout root**, with its matching CLI installed:

```
dotx repository list --repository .
dotx repository inspect Prometheus@3.15.1 --repository .
dotx repository resolve Prometheus@3.15.1 --repository .
```

Expected: the exact selected definition, its deployments and its `dot.json` path. These commands create no workloads; no runtime cleanup is needed. Current-source menus group product releases in descending SemVer order. Ordering is navigation, not a support recommendation or automatic selection.

Compositions keep logical coordinates, independent of folder spelling:

```
source: repository://Prometheus@3.15.1
dotId: Prometheus@3.15.1
dotVersion: 3.15.1
```

```
deployment: docker-compose
```

A missing release or a mismatched coordinate fails. Choose a deployment from that exact release. Contract IDs retain the provider's release, so a consumer for one release is not automatically compatible with another. Fixed Kubernetes workload names or WASI HTTP ports may also prevent two releases from sharing an execution context; the existing isolation checks still apply.

Keep the version meanings separate

Identity	Meaning
DOT version	Actual upstream tool/component release, used in folder and DOT ID.
Definition revision	Git commit and definition digest; corrections do not invent a new tool release.
Contract version	The specification standard version, such as OpenAPI `3.1.0`; GraphQL omits it.
CLI/format compatibility	The matching binary, generated model package and catalogue snapshot.

Add or update a release

Add an exact release leaf in the existing product folder. Include every supported deployment for that release, release-matched official Contracts where available, pinned artifact bytes and a readiness/cleanup README. Validate the catalogue and rehearse affected deployments before claiming support. Keep composition-specific inputs outside product definitions. Generated product navigation is maintained by the existing Contract-inventory documentation task.

Updating a composition's release selection requires explicit review and a new reviewed lock. The folder-only migration preserves existing logical references and locks; do not regenerate unchanged locks. Operate existing state using its original binary and context before changing checkouts; no adoption is implied.

Released walkthroughs use their manifest-pinned catalogue, which may retain the historical single-root layout. Generic readers retain that support. Current catalogue publication requires exact release leaves. Stable tags and their docs remain unchanged. See Compatibility and migration and Production-readiness limits.

Compatibility and migration policy

Select one tested artifact set

A product version alone is insufficient to identify the source-built ecosystem. Use either a released CLI with its release manifest and pinned catalogue, or a DOT-Design commit with its exact CLI/catalogue gitlinks. Record `dotx version --verbose` when reporting a problem. The CLI's release acceptance manifest identifies its test fixtures; it is not a floating recommendation for current `main`.

DOT-Domain is the authority for portable models. Its generated schemas and Go bindings must

agree. Composition `apiVersion: dot.mare6g.eu/v1` identifies the input profile, not every runtime capability or CLI command. Locks and runtime state are executor artifacts, not portable domain models.

Current intentional boundaries

Area	Current-source behaviour	Upgrade requirement
Contract standard spelling	Exact values are `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`, and `SPARQL` in 0.4.0. OpenMetrics remains an OpenAPI response format.	Released uppercase values such as `OPENAPI` are rejected, not normalised. Review an explicit document migration and affected locks.
Identifiers	Legacy textual/UUID DOT, Skill and Contract keys remain readable. Current catalogue coordinates use `name@actual-tool-release` and `origin-DOT.id::standard::name`; Anchors use DOT-local keys; Composition UUIDs retain their ownership role.	Renaming a catalogue identity changes references and lock inputs. It does not adopt existing runtime state or establish API compatibility.
Required capabilities	A DOT needs at least one meaningful Skill reference.	Older empty scaffolds need reviewed capability declarations, not guessed IDs.
Anchor identifiers	DOT-local kebab-case IDs and exact legacy UUID strings are accepted. Composition references still pair an instance with its Anchor ID; Composition ownership stays UUID-based.	Java Anchor IDs and references change from `UUID` to `String`; recompile callers. Migrate declarations, bindings, wires and exposed interfaces together, then review new locks. Old UUIDs and display names are never lookup aliases.
Anchor operation references	Contract-backed Anchors select operations from verified Contracts. The retired `semanticTypes` field is absent; Java decoding and generated Anchor schemas reject undeclared properties, including empty or null retired values.	Remove the retired field explicitly and preserve unrelated Anchor IDs/bindings. Choose an actual supported operation, not a schema or documentation URL; keep contractless gaps explicit. Review affected locks and clean old workloads with the original CLI and inputs before switching. No alias or state migration exists.
Contract ownership	Dependency interfaces resolve an explicit originating DOT; owned declarations cannot borrow a different release's namespace.	Follow the catalogue's `IDENTIFIER-MIGRATION.md` and `CONTRACT-AUDIT.md`. Do not copy a dependency Contract or relabel it as consumer-owned.

Area	Current-source behaviour	Upgrade requirement
CLI commands	`dotx compose inspect` displays the resolved composition; no separate command or authored document is needed to represent that derived view.	Rebuild the CLI and regenerate reviewed locks using the `resolutionDigest` field. Clean up running compositions with their original binary before upgrading; saved state is not migrated or adopted.
Java Composition API	Probe protocols/transports/response matching and materializer choices use typed enums instead of Strings; exclusive choices receive Bean Validation.	Recompile Java clients and use, for example, `AsyncAPIProtocol.MQTT` instead of `"MQTT"` in builders/setters. Valid v1 JSON spellings and reviewed lock inputs remain unchanged. Validate required-property presence before deserialization.
Reference and collection validation	ID-only views validate identity, root declarations cascade payload validation, and dangling service Contracts/ambiguous logical keys are rejected. Root ID-only Contract JSON remains ID-only on a Java round trip.	Repair invalid references and duplicate keys explicitly; no aliases, deduplication or lock repair is performed. Recompile Java callers of the old `setServices(Set)` signature for `setServices(Collection)`.
Lifecycle safety	Unsupported stateful and changed Helm/Vagrant upgrades fail before mutation.	A valid historical deployment does not guarantee admission to an upgrade. Never weaken safety checks merely to reproduce an older invocation.
Service configuration keys	Environment/CDI names, config-file paths, and volume names/paths are unique within each service. Java and CLI semantic validation reject ambiguous keys; the generated JSON shapes are unchanged.	Repair duplicates explicitly rather than relying on aggregation to select a winner. The same key in different services remains valid. Schema-only `uniqueItems` is not property-key uniqueness; use full DOT validation.

The original published CLI `v0.2.5` predates these current-source model and authoring changes. These are not a claim of patch-level backward compatibility. The coordinated DOT-Design, DOT-Domain and DOT-CLI baseline is `0.3.0`; it promotes the reviewed `0.3.0-rc.1` portable model surface without changing its generated schemas or reviewed compatibility corpora. The 0.3.0 documentation retains that release's exact artifacts. The current quickstart selects `v0.4.0` and its manifest-pinned catalogue; the next section describes the deliberate 0.4.0 changes. This is a new minor version on the pre-1.0 line, not another `0.2.x` patch. Future breaking changes must likewise use a new minor version, rather than silently replacing an existing artifact. Publication does not make arbitrary CLI/catalogue combinations compatible.

From 0.3.0 to 0.4.0

The **0.4.0 release baseline** requires a matching CLI and catalogue selected by its release manifest. Use the 0.4.0 binary with those generated schemas and exact catalogue revision. Source builds also need recorded revisions, modified state and binary SHA-256; a version label alone is not full

provenance. Historical 0.3.0 binaries retain their original catalogue and behavior. See release qualification and remaining follow-up.

0.4.0 also adds canonical `SPARQL` with `N_TRIPLES` and standard version `1.1`. Existing standard spellings remain unchanged. Regenerate downstream schemas/bindings together and review exact origin IDs, selectors, payload pins and affected locks; there is no automatic state adoption. Native metrics validation adds no `OpenMetrics` enum value. See the bounded native profiles.

The approved file-payload migration is an intentional input-format break, not a transparent 0.3.0 update. It changes file-document storage, not the selected tool releases or the meaning of Anchors and Wires.

Document field	Earlier input	0.4.0 input
Compose/Kubernetes manifest and Vagrantfile `payload`	Plain-text or Base64 document string	Shared `Payload` object: HTTPS `url` and exact lower-case `sha256`
Helm `values[].payload` and Docker `envFiles[].payload`	Embedded file document	The same verified URL/hash object; ordered overrides are retained
Service `config.configFiles[]` default	`defaultContent`	Optional `payload` URL/hash object; `defaultContent` is not an alias
Contract `payload`	URL/hash object	Unchanged JSON shape, now using the shared Java `Payload` type

Java clients must recompile: replace `eu.mare6g.dot.domain.contract.ContractPayload` with `eu.mare6g.dot.domain.Payload`, supply `Payload` values instead of embedded document Strings, and use `ConfigFile.payload` rather than `defaultContent`. Regenerate downstream bindings from the matching generated schemas; do not maintain a second hand-written input model.

Sensitive files must omit a default payload. Explicit deployment-instance inputs still override non-sensitive defaults. Scalar environment defaults, descriptions, image/chart references and WASM's existing module/hash fields are not converted to file-document payloads. Composition instance inputs remain instance inputs; do not rewrite every string into a URL.

For a concrete current definition, inspect the pinned Prometheus DOT. The catalogue's payload audit records the byte-preserving migration and hosting provenance. Prefer an exact, release-matched official raw file. Approved fallback Gists pin the full revision and filename; **secret means unlisted, not confidential**. Never upload credentials or private configuration, and obtain the required redistribution/hosting authority.

For a custom DOT, preserve the exact original document bytes (decode Base64 only where the old field actually used that encoding), publish only reviewed non-sensitive bytes, and hash those bytes without normalizing whitespace or line endings. Replace the file field with its URL/hash declaration; move a configuration default to `payload` and remove `defaultContent`. Use the generated schema and full validator, not a blanket text replacement. There is no automatic migration command or fallback to neighboring bundle files.

Before switching binaries, stop and destroy old instances with their original CLI, checkout and inputs. Retain application backups separately. Validate the target online to warm verified bytes, then offline; all selected deployment documents, not only Contracts, must be available and rehashed before mutation. A missing entry or digest mismatch fails closed. Do not repair the digest merely to accept changed bytes. Review affected locks explicitly and deploy a fresh test-owned instance; do not copy or edit saved state to adopt old workloads.

Safe migration procedure

1. Record the current binary, repository revisions, compositions, reviewed locks and application-data recovery instructions. Preserve the original files.
2. Use a separate checkout for the target artifact set. Read its migration and Contract-ownership notes before changing IDs, enum spellings or dependencies.
3. Retain stable Anchor IDs for unchanged logical interfaces. `dotx new --force` creates a replacement scaffold; it is not an incremental editor.
4. Run online `dotx validate` against the pinned catalogue once to populate the verified Contract cache outside the DOT bundles, also used for deployment/file payloads in current source. Then use `--offline` for definition validation, `dotx compose validate` and `dotx compose inspect` on the reviewed target inputs. A cold offline cache fails closed; it does not download Contracts or search for a local `contracts/` folder. Cached bytes are rehashed, and a digest mismatch requires investigation, not replacing the pin. Generate a new lock separately and review its differences; never repair the old lock to make it appear compatible with a changed graph.
5. Rehearse the supported lifecycle and actual API behaviour in an isolated, test-owned environment. Do not adopt live resources by renaming identities or editing private state. Stateful application migration remains application-owned.

`dotx compose new` already creates a template-based draft from a reviewed, adjacent-lock-verified composition, with a fresh ownership UUID. It preserves the selected DOT releases and wires; it does not infer a new graph, isolate fixed ports or shared inputs, migrate state, or create a lock automatically. See Create a composition from a reviewed example for preview, offline-cache requirements and explicit lock review.

Current-source `dotx edit` is a guided interface editor that writes a separate, validated candidate; it does not replace the canonical DOT, migrate state or repair locks. Current-source `dotx compose new --from-dots` selects exact DOTs, deployments and Anchors for a fresh draft, with conservative compatibility checks. Neither is present in the released v0.3.0 binary. See Edit an existing DOT and Build from selected DOTs. There is **no automatic migration command** or schema-version conversion. Retain manual review of interfaces, inputs, publication and locks.

Compatibility gates

`task testing:models` is the deployment-free local coding gate: Java validation, serialization and coverage; the all-public-model compatibility corpus; generated schema/Go agreement; CLI race tests and coverage; offline catalogue validation and reviewed locks. It starts no runtimes and dispatches no GitHub workflows. Initial tool bootstrap may require network access. Live adapter and presentation qualification remain separate through `task ci:live` and `task testing:demo`; coding-only checks do not establish runtime acceptance or publish Antora. Existing workflows remain

configured for later verification.

`task testing:compatibility` runs the historical Contract/identity boundary tests and the existing generated Composition-model compatibility checks. The frozen Contract declaration comes from the exact catalogue pinned by CLI `v0.2.5`: it is expected to fail the new spelling constraint until an explicit change. Tests preserve its original UUID and payload pin. This is not a claim that the whole historical catalogue is accepted unchanged.

`task testing:compose:acceptance` additionally exercises current compositions, reviewed locks and tamper rejection. Authoring serialization oracles protect field ordering, omission and payload bytes. None of these gates starts a production producer or proves independent-adopter acceptance.

Before a release, run the full applicable local workflow equivalents, require all mandatory remote jobs to succeed, verify reproducible packages and downloaded binaries, and retain the exact accepted artifact set. Queued, cancelled, skipped or running checks are not passing required gates.

After 1.0, backwards-incompatible portable document changes require a new public model/profile version and a migration path. Before 1.0, breaking product changes require a minor-version increment and explicit compatibility notes; patch releases remain backwards-compatible except documented fail-closed security/correctness fixes. Deprecation notes must identify the old behaviour, replacement and removal release before removal, unless an immediate safety fix requires rejecting the operation.

Guided demo

This guided demo explains how a reusable DOT becomes part of a checked composition. Start with the one-command guided experience, then explore the official ecosystem demo, choose the shorter 10-15-minute standards route, or the five-minute read-only stages later on this page. These paths connect authoritative Contracts, Anchors, deployment choices and explicit wires; they prove different things.

The read-only core path takes about five minutes on an existing checkout. It is read-only apart from temporary files created and removed by the CLI. The live presenter route uses Docker and Kubernetes and performs deployment, protocol probes, inspection and cleanup.



Run the commands from the DOT-Design repository root. Keep evidence levels separate: a valid resolved composition proves deterministic resolution, a dry-run proves that the executor can render supported runtime actions, and only a live probe proves the observed protocol exchange.

Choose a path

For MARE use cases, start with the PoC composition walkthroughs: real PoC4 monitoring, PoC10 authenticated policy enforcement, then a labelled PoC6 CACAO/OpenC2 rehearsal using an NWDAF double. Its presenter tasks add a bounded post-verification inspection pause and automatic cleanup, with the same live assertions used by acceptance.

For a fully executable local presentation spanning Docker, Kubernetes, and OpenAPI/GraphQL/Protobuf/AsyncAPI wires, follow Docker/Kubernetes four-Contract composition. It includes the rehearsal, real probe outcomes, cleanup, and evidence boundaries.

Path	Time	Demonstrates	Runtime mutation
One-command guided experience	At your pace; actual startup time depends on the host	A real Prometheus API response first, then optional authenticated five-DOT composition	Disposable Docker resources; Kubernetes only if you continue
Official ecosystem demo	15-20 minutes of explanation, plus runtime waits	One DOT on two runtimes, secure policy enforcement and stateless recovery; optional four-standard deep dive	Disposable Docker and Kubernetes resources
Read-only overview	About 5 minutes	Repository validation, a reusable external DOT, 21 cross-standard wires, and a Kubernetes lifecycle rendering	None
Live presenter route	About 10-15 minutes after preparation	Six core concepts, one traced wire, four Contract standards across Docker and Kubernetes, and a real MARE use-case graph	Disposable Docker and Kubernetes resources

Path	Time	Demonstrates	Runtime mutation
New developer	About 15 minutes	The read-only overview plus selected resolved composition details, authoritative Contract facts, endpoint materialization, and evidence boundaries	None
Live acceptance	Depends on image downloads and host capacity	GraphQL, all four finite gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery against standards-native test producers and real brokers	Disposable Docker resources

One command, immediate explanation, real feedback

Prepare once offstage from the matching checkout, then check readiness:

```
task demo:prepare
task demo:check
```

These short commands default to the Prometheus Docker-only chapter: Kubernetes is not required. Preparation validates the canonical DOT and warms its pinned image/document cache; it never installs tools or deploys workloads. The check is read-only and never repairs a cold or corrupt cache. Prerequisites such as the matching installed CLI, Docker and Compose must already be available.

For the optional five-DOT chapter, use `task demo:prepare DEMO=secure-api` and `task demo:check DEMO=secure-api` with an already Ready Minikube node. Neither command starts Minikube.

Preparation/check success is not live API acceptance. The long `task testing:demo` qualification suite remains separate.

On stage, run:

```
task demo
```

Run from this checkout in an interactive Linux-amd64 terminal. This is the on-stage experience, not another qualification suite. This wrapper is available in the current source checkout. Follow its preparation instructions to install the exact pinned CLI source revision; the released quickstart is a separate, manifest-matched route. The wrapper adds no new `dotx` command or portable model. It immediately explains what will happen, checks prerequisites without deployment, and reuses the exact Prometheus quickstart commands from Getting started. It then shows the verified build-information version and a clickable localhost URL while Prometheus is running. Open the URL, explain the result, and press Enter to clean up. Type `d` then Enter at the inspection pause for the underlying deployment command and Contract ID; detailed tool logs stay private rather than flooding the screen. Green result headings respect `NO_COLOR` and plain terminals.

Startup is not a silent wait. The display shows elapsed `mm:ss` time and the current phase: prerequisites, startup, API verification, inspection and cleanup. The composition lists the exact DOT releases from native locked resolution, then reports each DOT as `QUEUED`, `STARTING` and `DEPLOYED`, with `n/total` deployment counts and per-DOT elapsed time. While waiting, it prints a status line every 10 seconds; native rollout changes can appear sooner. Counts describe deployment progress, **not** an estimated percentage, ETA or production readiness. `DEPLOYED` means the deployment command completed; only the separate verified API heading means the live assertions passed. Timer updates pause during inspection so they do not interrupt your input; cleanup resumes visible progress. Raw tool logs, tokens, URLs from private errors and API bodies are not forwarded to the public progress display. Colors are optional; all statuses remain readable in plain text.

After Docker cleanup passes, press `q` then Enter to finish, or simply press Enter to continue with Keycloak, APISIX, OPA, Oxigraph and the OAuth test client. Kubernetes is needed only for this optional chapter. A separate read-only preflight checks Ready Docker-driver Minikube, exact cached images, namespace and port conflicts before it deploys anything. The node may use either Docker or containerd; the Docker **driver** and the node's **container runtime** are separate choices. Preflight detects the runtime from the validated Linux-amd64 node and inspects its actual image cache, not a substitute host cache. Unknown or missing runtime metadata fails closed. The short results display explains anonymous/invalid-token rejection, authenticated reads, forbidden writes and the controlled `200 → 403 → 200` policy transition. Type `d` then Enter for the native `dotx compose topology` graph and status command. Tokens and raw API payloads are never printed by the guided display. The protected localhost URL must reject anonymous browser access; it is not a broken demo or a public unauthenticated UI.

No long command needs to be copied. Enter requests cleanup at each inspection pause; Ctrl-C, EOF and the bounded timeout also trigger the original harness's ownership-safe cleanup. Inspection defaults to 600 seconds, configured by the existing `DOTX_POC_PRESENT_TIMEOUT` (1–1800 seconds); the optional chapter choice also finishes after 600 seconds or EOF, when no demo workloads remain. The command stops on prerequisite, API or cleanup failure. It does not claim success from a compatible graph, a running container, or an earlier receipt. Readiness failures name the short chapter-specific preparation/check commands. They do not automatically retry or mutate the runtime. For a deployment failure, inspect the original harness's cleanup receipt before retrying: a failed exit alone cannot establish that cleanup succeeded. Private diagnostic logs are retained under `DOT-Testing/e2e/target/guided-demo/run-*/`; do not publish raw logs. The terminal-driven CI rehearsal retains its public console transcript separately from those private tool logs. CI prepares the optional secure-API image cache offstage before exercising the complete interactive route; `task demo` itself still checks prerequisites without installing tools or building images on stage. Explicit cache preparation pulls only the selected digest-pinned images into their target runtime (Docker or containerd's CRI image service), then rechecks the cached identity and Linux-amd64 platform. Containerd verification also requires the requested manifest/index digest in its repository digests; that digest is not the image configuration ID. Read-only preflight never pulls or silently falls back to another runtime when an image or its metadata is missing.

The quickstart's port lease coordinates our test harnesses, not unrelated host processes. If another socket takes the selected port before Docker binds it, the run fails closed and cleans up its own resources. The guided display reports a host-port conflict without exposing the raw error; check listeners and retry, or set `DOTX_HTTP_PORT` to a free port. Never stop an unrelated listener to make the demo pass. A failed control run must be repeated, not counted as a successful signal or API test.

Prepare once offstage

For the complete route, use the existing qualification and preparation instructions and run `task testing:demo` before the presentation. That installs the aligned source CLI, prepares caches, qualifies the full matrix and cleans up. `task demo` does none of that installation, building or full-matrix testing on stage. It is **not** an offline guarantee: real deployment/probe paths may still require registry or Contract-server access. Keep the qualified source set, CLI and runtime contexts unchanged.

For a separate resilience rehearsal, use the OPA outage/restart qualification. It verifies fail-closed access, restart recovery and injected-failure cleanup offstage; the short interactive route never stops a dependency unexpectedly.

For the Docker-only first chapter, only Bash, Python 3, Task, the pinned source `dotx`, Git/Go (for provenance), jq, curl, Docker/Compose, timeout, awk, sha256sum, ss and flock are needed, with the selected Prometheus image and verified Contract already cached. Java, Kubernetes, Helm and the four-standard fixture builds are not prerequisites. The small display intentionally leaves deeper concepts until there is a working service to explain.

This first chapter has no wires. The optional composition is the existing source-adapted PoC10 slice, including an Oxigraph substitution and synthetic OAuth client—not acceptance of the whole producer PoC or every catalogue DOT.

Official ecosystem demo: reuse, compose, enforce, recover

For the simplest first experience, use `task demo` above; it starts with Docker only and offers composition after the first verified API. Use this route for the deeper ecosystem presentation. Budget 15-20 minutes of explanation for the first three chapters; provisioning, readiness and the intentional failed rollout add runtime-dependent waiting. The fourth chapter is an optional standards deep dive. For a short slot, show chapters 1 and 2. The shorter standards-focused route remains available below.

The story is: **one authoritative definition → explicit composition → observed behaviour → controlled recovery**. DOT-Domain and its generated Schemas define the portable models; `dotx` resolves and executes them. Docker/Kubernetes remain the workload engines. A wire connects declared Anchors; it does not magically translate protocols or implement missing producer functionality.

For questions about the rest of the catalogue, use the catalogue readiness guide and the generated report for the selected revision. Definition validity, declared interface completeness and live producer acceptance are separate; the rehearsed chapters do not qualify every catalogue entry.

Prepare and qualify one exact checkout

Run from the DOT-Design checkout root on Linux amd64, with repository access, Git, Go, Task, Python 3, Bash, jq, curl, `ss`, Docker/Compose, kubectl and a Ready local Docker-runtime Minikube node. Select context `minikube`. This advanced developer rehearsal installs the pinned source CLI.

The selected release `v0.4.0` includes these model and authoring features, unlike the original `v0.2.5`. Nevertheless, do not replace the source-built binary after qualification, even with the same version: the exact checkout revisions and binary SHA-256 identify the frozen, qualified source set. This is the supported Linux-amd64 profile, not certification of every MARE producer or deployment. Put Go's executable install directory (`GOBIN`, or `$(go env GOPATH)/bin` when unset) on `PATH`; the installation phase verifies that plain `dotx` resolves to the aligned version and records any PATH warning in its printed install log. The simpler Getting started path still needs neither Go, Task nor Kubernetes.

Commit reviewed source changes before using these qualification commands. Choose unused namespaces and ports; keep the same overrides for preflight, rehearsal and presentation:

```
export DOTX_PORTABILITY_NAMESPACE=dotx-demo-portability
export DOTX_PORTABILITY_KUBERNETES_PORT=19083
export DOTX_POC10_NAMESPACE=dotx-demo-secure-api
export DOTX_POC10_FORWARD_PORT=19010
export DOTX_UPGRADE_NAMESPACE=dotx-demo-upgrade
export DOTX_UPGRADE_PORT=19081
task testing:demo
```

`task testing:demo` is the complete off-stage readiness command. Sequentially, it installs and verifies the aligned source CLI, runs the existing demo guards and locked validation/rendering, prepares all image caches with preflight, qualifies all seven live paths twice plus the four official presenter chapters, and exports sanitized fallback evidence from that exact successful receipt. It stops at the first failure; an older successful receipt is never substituted. Every lane is included regardless of a focused `DEMO` selector; namespace and port overrides still apply. Allow substantial runtime for builds, rollouts and cleanup; this is not a command to start during the talk. It needs the tools and already Ready Minikube listed above; it does not install system prerequisites. Successful qualification cleans up its test-owned workloads rather than leaving the presentation running. Start the chapter-specific presenter commands below when ready to show them. Keep sources and `dotx` unchanged throughout the run. The console explains five numbered phases and each live lane's purpose, reports elapsed time and outcome, and finishes with a results table, exact evidence and fallback paths, and the next presenter commands. Detailed tool output stays in per-step logs rather than flooding the terminal. The umbrella's own report and table live under `DOT-Testing/e2e/target/demo/run-*/`; preparation and live qualification retain their separate evidence scopes below. A failed or interrupted phase stops the run and names its logs; it never prints demo-ready success or exports a fallback from an older run.

After qualification, use `dotx version --verbose` and the exact root `summary.json` printed by this run to show the three checkout SHAs and binary SHA-256. Expect `Modified worktree: false`. Do not replace this identity evidence with the version label alone, an older receipt, or a newer unrehearsed checkout. Changes after qualification require a fresh rehearsal and fallback export.

The granular commands remain available for investigation:

`prepare` explicitly pulls only selected digest-pinned official images, builds and tests the standards fixtures from this checkout, and imports their provider image. It may start disposable image-test containers, but never applies a composition or provisions credentials. Fixture preparation shares the live harness's concurrency guard and label-scoped failure cleanup. Image caches and build

cache intentionally persist. Registry access is needed; preparation does not promise offline operation. The command ends with read-only preflight. Use `DEMO=portability`, `upgrade`, `secure-api`, `monitoring` or `four-contracts` with either preparation or preflight for a focused lane; the default is all five.

`preflight` does not pull, build, deploy, adopt or delete resources. It checks the installed CLI provenance, local Contracts/locks, Ready Minikube, selected image caches and relevant namespace/port/runtime conflicts. It reserves nothing. The Compose comparison also needs the descriptor-declared port `18919` free.

`rehearse` runs the quickstart, monitoring, portability, secure API, upgrade/rollback, OAuth gateway and four-Contract paths **twice each**, followed by injected-failure and signal cleanup controls. It preserves every existing live assertion rather than replacing tests with a summary. All three checkouts must be clean; before/after revisions, tracked/untracked fingerprints and the binary SHA-256 must remain identical. Do not edit sources or reinstall `dotx` during it. This is qualification before the talk, not a command to start on stage.

`qualify` runs that same complete live rehearsal, then drives all four official chapters through a real terminal: wait for the verified inspection pause, inspect native status and topology, press Enter, and verify cleanup. It reuses the existing live harnesses and terminal driver rather than another deployment implementation. Application-result expectations are shared by both paths. Interrupted, timed-out and rejected-evidence steps remain visible as failures, even when the child command itself exited zero. The existing standalone `rehearse` Task remains useful when only noninteractive acceptance is wanted.

For the optional four-Contract chapter, use the prepared fixture path on stage:

```
export DOTX_DEMO_PREPARED=1
```

Preparation records source fingerprints, the installed binary hash, both Docker fixture image IDs and the imported Minikube provider image ID. `qualify` enables this mode automatically. With this variable set, the four-Contract live/presenter harness verifies that receipt before deploying and does **not** build, pull, import images or start Minikube. Missing preparation or changed sources/binary/images fail closed: re-run explicit preparation off stage and qualify again. Official digest-pinned images and probe-runner caches must still be available; this is not an air-gapped operation guarantee or a promise against registry outages. Do not retag images or switch Docker/Minikube contexts during the talk. Without this opt-in, the existing source-building live test path remains available; builds now have bounded five-minute timeouts.

The printed `summary.json` and adjacent `results.txt` consolidate step exits, logs and hash-linked child receipts/results. Receipt-backed live lanes require successful assertions, exact CLI identity and `cleanup=PASSED`; prerequisite, cleanup-control and log-only quickstart evidence stay separately labelled. Preparation has its own report under `DOT-Testing/e2e/target/preparation/` and explicitly claims **no live API acceptance**. Qualification reports live under `DOT-Testing/e2e/target/rehearsal/`. A missing/failed receipt or source change fails the complete rehearsal. These local evidence directories are not published as though they were current online service health.

Chapter 1: reuse one DOT on two runtimes

```
task testing:demo:portability:present
```

Say: both nodes use the same Prometheus release ID, official OpenAPI Contract and immutable image, with two selected deployment mechanisms. There are **zero wires** between these independent servers: this proves reuse, not replication/migration. **Show:** both build-information API versions match the selected image, `GOGC=75`, the declared default configuration and a real self-scrape `up=1`. At the verified pause, compare the runtime observations and the localhost endpoints in the table. Press Enter and require cleanup to pass before chapter 2. See the complete comparison and storage boundaries.

Chapter 2: compose and enforce a real API boundary

```
task testing:pocs:present:secure-api
```

Say: identity, gateway, policy engine, RDF service and client are independently reusable DOTs, connected by reviewed Anchors/wires and deployment-instance inputs. Existing Contracts remain authoritative where present; explicit interfaces are not misrepresented as vendor-official OpenAPI documents. **Show:** anonymous/invalid-bearer rejection, token acquisition and authenticated SPARQL access to stored RDF. The controlled policy change gives `200 → 403 → 200` and restores the original policy without changing the stored RDF data. Tokens stay out of printed results; native Secret references are inspected by name only. This is the source-adapted portable PoC10 slice, not acceptance of every producer or of the complete MARE security architecture. Enter verifies cleanup. See PoC walkthroughs and their remaining gaps.

Chapter 3: inspect a change and recover safely

```
task testing:demo:upgrade:present
```

Say: reuse the canonical OPA DOT; the change belongs to stateless composition configuration. **Show:** the native upgrade diff, successful stricter policy, deliberately invalid Rego, failed rollout and automatic last-known-good recovery. The recovered version is `1.1.0`, not the original `1.0.0`; a fresh API decision must pass with no pending transaction before the presenter pause appears. This does not demonstrate an OPA software-version upgrade, database migration or universal stateful rollback. Enter verifies cleanup. See the native lifecycle safety boundary and recovery.

Optional chapter 4: cross-runtime, four-standard interoperability

```
task testing:demo:docker-kubernetes-four-contracts:present
```

Show: real OpenAPI/HTTP, GraphQL/HTTP, Protobuf/gRPC and AsyncAPI/MQTT exchanges from the actual Docker consumer's network boundary to Kubernetes. A controlled HTTP-200 value mismatch is rejected, changes the consumer decision, and is then restored and freshly verified. Mosquitto is

real; the provider and consumer are labelled standards-native fixtures, **not** production producer acceptance or MARE analytics. See the protocol and network details.

Close with evidence, not a blanket readiness claim

At every pause, the Mermaid graph is the **locked composition**. The new runtime table is a timestamped observation; a running container or available controller is not API acceptance. Historical probe records keep their own timestamps and age. The final application tables come from the separately completed live assertions. Use the printed cache/Kubernetes context for further inspection. Never substitute a fresh cache when recovering an interrupted run. Enter, Ctrl-C, EOF and timeout take the ownership-safe cleanup path; never delete unrelated resources or force namespace finalizers. Require the cleanup receipt, not just an empty terminal.

Keep matching rehearsal reports as a clearly labelled fallback. The umbrella `task testing:demo` automatically exports the sanitized audience summary and prints its path. When using the granular `testing:demo:qualify` task instead, export it manually using the exact printed receipt path:

```
task testing:demo:fallback RECEIPT=/absolute/path/to/summary.json
```

The exporter requires complete, clean-source live and presenter qualification and checks linked artifact hashes. It writes only `fallback/summary.json` and `fallback/README.md` beside that receipt: fixed step outcomes, source revisions and binary/receipt hashes. It does not copy logs, arbitrary API fields, host paths, runtime state, tokens or kubeconfig. Keep the parent run directory private. The exported result is historical rehearsal evidence, not live service health or an independent adopter/producer certification. Do not replay it as a claimed live result. For the catalogue-wide appendix, run `task testing:contracts:readiness` and consult the verification boundaries. Interface declarations, source-derived Contracts, runtime-only brokers, validation-only fixtures, stubs and producer doubles do not become production-certified through this demo. DOT-Registry and partner/SFC acceptance remain separate integration work.

The existing **Kubernetes and Helm runtime E2E** workflow accepts the optional `full_demo_readiness=true` manual input to run the same `task testing:demo` entry point. Ordinary pushes retain their focused runtime matrix; the long full-demo qualification is explicit. Uploaded reports are redacted evidence, not automatically a public live-service dashboard.

10-15-minute live presenter route

Use this route for an audience new to DOTs. The detailed stages below remain the reference walkthrough; this section is the talk track, not another harness. For a ten-minute slot, shorten the wire inspection and show the MARE graph as the closing example rather than starting a second deployment.

The three messages to leave with the audience:

- Reuse a product definition; put PoC-specific configuration in the composition.
- Keep existing interface standards authoritative; wires are not protocol translators.
- Distinguish a compatible resolved composition from a verified live exchange and production

acceptance.

For technical Q&A, add the five-minute developer extension: inspect a real product's deployment choices and official Contract provenance, demonstrate rejection of an invalid wire, and show where a PoC contribution belongs.

Before the audience arrives

Follow the mixed-runtime prerequisites and rehearse on the same Linux-amd64 checkout, installed CLI, Docker and local Minikube context that you will present. Install this checkout's CLI for the exact pinned acceptance suite. Published `v0.4.0` includes native topology and the CLI operations used by the presenter harnesses; a release binary does not include later source fixes, and the version string alone does not identify the tested binary. Record the three checkout revisions as shown in that walkthrough.

For the Keycloak/APISIX/OPA/SPARQL follow-up, use `task testing:demo:preflight DEMO=secure-api` before deployment. This read-only gate checks the exact installed CLI revision, local Contracts, cluster readiness, cached images and namespace/port conflicts. Follow the secure-API preflight instructions to keep the same overrides in the presentation terminal. Select `DEMO=four-contracts` for the mixed-runtime fixture, or `DEMO=monitoring` for node-exporter/Prometheus. Select `DEMO=portability` or `DEMO=upgrade` for the additional comparison/recovery lanes. Without a selector, all five are checked. These are point-in-time prerequisite checks, not live acceptance or resource reservations. In particular, four-Contract preflight checks cached images, Docker resource conflicts and selected NodePorts, not end-to-end gateway routing.

After deployment, the mixed-runtime harness waits for actual TCP reachability from its owned Docker consumer, with bounded socket attempts and an overall deadline. Pod readiness alone does not establish NodePort reachability. Connection establishment may be repeated during this readiness wait; Contract operations are not retried. A persistent failure stops the run and triggers owned cleanup, with allowlisted connectivity diagnostics retained separately.

Pre-download/build the images by completing the existing live acceptance task before the talk, and require successful cleanup:

```
task testing:demo:docker-kubernetes-four-contracts:live
```

The timing below is a speaking budget, not a deployment-time guarantee. Cold downloads, image builds and a slow cluster can exceed it. Keep the previously recorded, revision-matched rehearsal evidence available and label it as such if the live run fails; never describe a resolved composition or saved result as a new live pass. Use an isolated demo network: the example's plaintext NodePorts and anonymous MQTT are not safe public production defaults. Do not run competing acceptance or cleanup tasks during the presentation.

Minutes 0-2: the problem and the six concepts

Say: "We want to describe a component once and reuse it across deployments and MARE use cases. DOT-Design connects existing interface standards to explicit deployment and composition choices; it does not replace those standards."

Concept	Audience meaning	In this demonstration
DOT	Reusable software component description	Provider, consumer and Mosquitto use canonical catalogue definitions.
Contract	Authoritative description of an interface	OpenAPI, GraphQL SDL, Protobuf and AsyncAPI describe different interactions.
Anchor	Named provided or required interaction	The provider and consumer select the REST alerts operation from its Contract.
Wire	Connection between compatible Anchors	`rest-alerts` connects the provider's Anchor to the consumer's Anchor.
Deployment	How a DOT runs on a selected platform	Provider and broker run on Kubernetes; consumer runs on Docker.
Composition	Participants, connections and deployment-instance inputs	One locked graph selects those DOTs, deployments, wires and configuration.

Point out the ownership: `DOT-Repository` stores reusable DOT bundles and compositions; `dotx` in `DOT-CLI` validates, resolves and executes them; `DOT-Domain` defines the shared model. `DOT-Design` brings together documentation and executable acceptance harnesses. The synthetic demo graph stays in `DOT-Testing`; it is not a new production product.

Minutes 2-4: read the topology before deploying

```
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --color always
```

Show: three DOTs and four logical wires. Match the diagram's `N` and `W` identifiers to the aligned DOT and connection tables. `CONTRACT` names the interface standard; the separate `PROTOCOL` column names the transport protocol. Explain provider versus consumer and the selected runtime, rather than reading UUIDs aloud. The separate `T4` connection identifies MQTT's broker transport endpoint; it is not a fifth Anchor wire. Arrows mean logical PROVIDES-to-REQUIRES, not packet direction: an HTTP consumer may initiate the request.

Expected observation: the graph is available without deploying anything. Parallel wires may share one arrow, but every wire remains in the table. This is a locked composition view, not a live health display.

Minutes 4-6: trace one wire from Contract to configuration

```
dotx compose inspect DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --json |
  jq '.wires[] | select(.name == "rest-alerts") |
      {name, from, to, contractStandard, selector, operationPath,
       compatibility, materializer}'
```

Say: "The OpenAPI Contract owns `GET /alerts`. The Anchors select that operation; the wire names

the two participants. The materializer puts the resolved operation URL into the consumer's declared `OPENAPI_YAML_URL` environment variable. We are not copying the API definition into the wire."

Expected observation: `OpenAPI`, the selected operation, provider/consumer Anchor references, `COMPATIBLE` and the environment-variable materializer. The template's `192.0.2.1` address is deliberately not a live target. The existing harness substitutes the actual Minikube address and locks that run-specific graph before applying it. Do not manually apply the template.

Limit: compatibility is checked against declared interfaces. It is not proof of reachability, delivery or application correctness. There is no automatic OpenAPI-to-gRPC-to-MQTT protocol translation.

Minutes 6-11: deploy, verify and inspect the live composition

Prepare offstage once before the talk. This builds/imports the fixture images and prepares the exact pinned probe-helper images; it does not deploy the composition:

```
task testing:demo:prepare DEMO=four-contracts
```

Then run the presenter. It verifies the prepared caches without building or pulling images on stage:

```
task testing:demo:docker-kubernetes-four-contracts:present
```

This is the existing acceptance path with an inspection pause, not a parallel demo implementation. It deploys through `dotx`, checks readiness and executes probes from the actual Docker consumer's network namespace to Kubernetes. Presenter mode keeps build and deployment chatter in the printed, owner-only `details.log`. The terminal shows the numbered resolved composition, deployment, probe, inspection and cleanup stages; a failure still reports its stage, exit code and diagnostic-log location. Ordinary noninteractive acceptance remains verbose.

Standard	Explain the expected live result
OpenAPI	HTTP 200 plus the expected JSON alert; the running consumer derives `REVIEW_REQUIRED` from its wired endpoint.
GraphQL	`Query.health` returns `true`.
Protobuf / gRPC	The selected health RPC returns `serving: true`.
AsyncAPI / MQTT	The broker delivers the exact uniquely correlated, schema-valid message after subscription acknowledgement.

Show: the verified pause appears only after all four live assertions and recorded status checks pass. Keep the first terminal at this pause. In a second terminal, copy the exact recovery/status command printed by the harness; it selects this run's private cache and Kubernetes configuration. Inspect the per-wire outcomes and redacted `result.json`, not access tokens or Secrets. The final live-results table separates `CONTRACT`, `PROTOCOL`, `OBSERVED` and `OUTCOME`. It is generated only after each successful probe is also present in recorded lifecycle status; `COMPATIBLE` and running workloads do not populate it. The observations summarize checked assertions, not raw API responses. The same four

rows are saved in `result.json` and the plain-text `results.txt`. Before the pause, point out the controlled negative step: the owned fixture temporarily returns HTTP 200 with severity `low`, the native assertion records `RESPONSE_MISMATCH`, and the running consumer changes its decision. The harness then restores the critical alert and requires a newer `REVIEW_REQUIRED` receipt and successful native assertion. This distinguishes endpoint health from correct application data; it is not a claim of complete schema or policy validation. The graph remains the locked composition, not an inferred network map. Presenter lanes using the shared lifecycle helper, including the four-Contract lane, append timestamped runtime observations with `compose topology --locked --status`. Their results tables still provide the separate application assertions.

To refresh this run's topology in the second terminal, start from its printed recovery/status command. Keep its complete environment assignments and exact composition path; replace only `compose status` with `compose topology` and append `--locked --status`. Do not substitute a catalogue graph: the four-Contract fixture and the Keycloak/APISIX composition have different ownership identities and state, even when they use the same cluster.

Explain the layers separately: `APPLIED` is a recorded lifecycle phase; `RUNNING` describes a process, container or VM; `CONTROLLERS_AVAILABLE` describes selected Kubernetes controller status, not rollout or API acceptance. An empty selection is `NO_CONTROLLERS`; query failures are `UNKNOWN` and return nonzero. Historical wire probes show their original timestamps. `RECENT_RECORD` means recently recorded, never a fresh API call; `STALE_RECORD` requires a fresh probe before claiming current acceptance. The default record age boundary is five minutes, adjustable with `--probe-max-age 2m` (positive, at most 24 hours). This observation mode does not deploy, mutate configuration or run probes; it uses the lifecycle coordination lock. Mermaid export remains declaration-only.

State the boundary: Mosquitto is a real external broker, but provider and consumer are test-only DOTs with standards-native fixture Contracts, not vendor-official or MARE producer APIs. The consumer now executes the REST exchange and derives a synthetic review decision; explicit probes exercise GraphQL, gRPC and MQTT from its network namespace. The MQTT probe subscribes and publishes to prove broker delivery, not a provider application's business-event publication. This verifies interoperability mechanics across runtimes, not a production application or the whole MARE stack.

Minutes 11-13: relate the mechanism to a real MARE use case

Keep the live demo paused and show this separate, read-only catalogue graph:

```
dotx compose topology \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --color always
```

Say: "The same model expresses a PoC10-derived protected data service: Keycloak supplies identity, APISIX gates access, OPA decides policy, Oxigraph serves SPARQL and a client obtains a token and requests the protected capability." Point to the five wires, then explain that this graph has not been deployed by the four-Contract demo.

For a longer session, use the existing `task testing:pocs:present:secure-api` in a separate, rehearsed run. Its live checks require HTTP `401` for missing and malformed bearer tokens, authenticated POST

403, and an authorized GET reading the stored mare-poc10 value. See MARE PoC composition walkthroughs for prerequisites and commands. This is source-adapted evidence: Oxigraph substitutes for GraphDB, the client is synthetic, and SPARQL/OPA use explicit protocol interfaces rather than fabricated official OpenAPI Contracts. It does not accept the complete producer deployment, LDAP, telemetry or automated response.

Minutes 13-15: clean up and leave a reproducible next step

For deployment reuse, the Prometheus portability lane runs one canonical DOT and official Contract on Docker Compose and Kubernetes, with the same pinned image and fresh API/self-scrape checks. It compares two independent servers; it does not claim a wire, replication or data migration.

For a separate technical follow-up, the stateless upgrade lane reuses the canonical OPA DOT, changes a policy configuration through native compose upgrade, and verifies automatic last-known-good recovery after a deliberately broken revision. Rehearse it separately; do not fit its intentional rollout timeout into this 10–15-minute route or imply application-data rollback.

Return to the paused demo terminal and press Enter. Expect a receipt with cleanup=PASSED and exit=0; the harness checks its owned resources for leftovers. Ctrl-C also invokes cleanup and exits 130. EOF, timeout and SIGTERM are non-success exits with cleanup, not successful acceptance. The inspection pause defaults to ten minutes; see the mixed-runtime walkthrough for its bound. If cleanup fails, retain the receipt and use that walkthrough's recovery instructions. Never delete unrelated namespaces, containers or volumes.

Close with: "A valid definition is not a running workload; a compatible resolved composition is not a successful exchange. We have shown the live results and their limits."

Direct attendees to Getting started to deploy real Prometheus, the reusable PoC walkthroughs to explore compositions, and Anchors and Composition to author their own integration. Keep the verification matrix available for questions; do not equate catalogue validity with production readiness or claim DOT-Registry REST API or joint SFC acceptance.

Audience FAQ

Why not just use Docker Compose, Kubernetes or Helm?

Those tools remain the native deployment mechanisms. DOT adds a reusable component description, authoritative interface references, named interactions, explicit cross-component wiring and a checked resolved composition. dotx delegates supported deployment actions rather than replacing those platforms. If an existing Compose file is all a use case needs, DOT's additional value is the shared catalogue and interoperability description, not a different container scheduler.

What is reused, and what changes between PoCs?

The product's canonical dot.json, Contracts and Anchor identities are reused. A composition selects participants and deployments and supplies use-case-specific configuration: routes, policies, scrape targets and credential references. There is no need to create another Keycloak DOT for every PoC.

Runtime platform means *where* it runs; deployment mechanism means *how* it is deployed.

Does connecting two Anchors make incompatible APIs work together?

No. Wires connect declared compatible interactions; they do not translate OpenAPI, GraphQL, gRPC or MQTT automatically. An application-specific adapter, if required, must be implemented and described explicitly. The four-Contract demo verifies four separate standards-native exchanges, not one message automatically converted through four protocols.

Is a Contract the same as a protocol? Are all Contracts official?

No. OpenAPI is an interface-description standard; HTTP is a protocol used by the selected operation. Protobuf describes messages/services used by the gRPC profile; AsyncAPI describes messaging interactions and transport bindings. The topology table separates `CONTRACT` from `PROTOCOL` for that reason. Use vendor/project-official documents when available. Source-derived Contracts and test-fixture Contracts must be labelled, not advertised as official. An explicit protocol interface is not an invented official Contract; the topology labels that case `explicit interface`. A `-` on a transport-only `T` link does not create another Contract-backed Anchor wire.

Does validation mean a DOT is production-ready?

No. Base validation checks definitions and declared references; stricter interface checks assess declaration completeness. A locked compatible resolved composition checks the selected graph. A successful deployment establishes a runtime state. A live probe verifies a particular observed exchange. None alone proves full business functionality, resilience, security hardening, scale or acceptance of an unmodified PoC producer. Read the declared scope with each result.

What does the lock file guarantee?

It pins the composition's portable definition inputs and resolved composition so that unreviewed definition changes are detected. It does not freeze runtime health, network addresses, credentials or upstream service behavior. Contract payload digests verify the referenced bytes; a downloadable URL is not itself evidence that the live implementation conforms to the whole Contract.

Where do credentials and configuration belong?

DOTs declare supported service configuration. Compositions provide deployment inputs and secret references; secret values and concrete runtime state are not part of a reusable public interface description. The PoC10 rehearsal uses native Kubernetes Secret references. Do not display Secret contents, tokens, raw deployment state or private diagnostic logs during the talk. The four-Contract fixture uses plaintext/anonymous demo settings and is not an example of production security hardening.

What did this presentation actually demonstrate?

The official route verifies one Prometheus DOT on two runtimes, a source-adapted secure API with policy denial/restoration and unchanged stored RDF, and native stateless OPA upgrade/recovery. The optional standards chapter uses test-only provider/consumer components and real Mosquitto

to verify four exchanges from the consumer's network namespace. The shorter standards-focused route shows the PoC10 graph only unless its separate live task is run; a topology alone never accepts it. Producer doubles, stubs, validation-only fixtures and source adaptations remain labelled. DOT-Registry REST API, complete producer/PoC acceptance and joint SFC/partner acceptance are not claimed by these demonstrations.

How can an attendee try this afterwards?

Start with the real Prometheus quickstart, which needs neither a Java/Go development toolchain nor Kubernetes when using the supported released binary. Then explore reusable compositions and the authoring guide. The new topology and presenter features require the installed development checkout described above; do not assume an older release contains them.

The story in one diagram

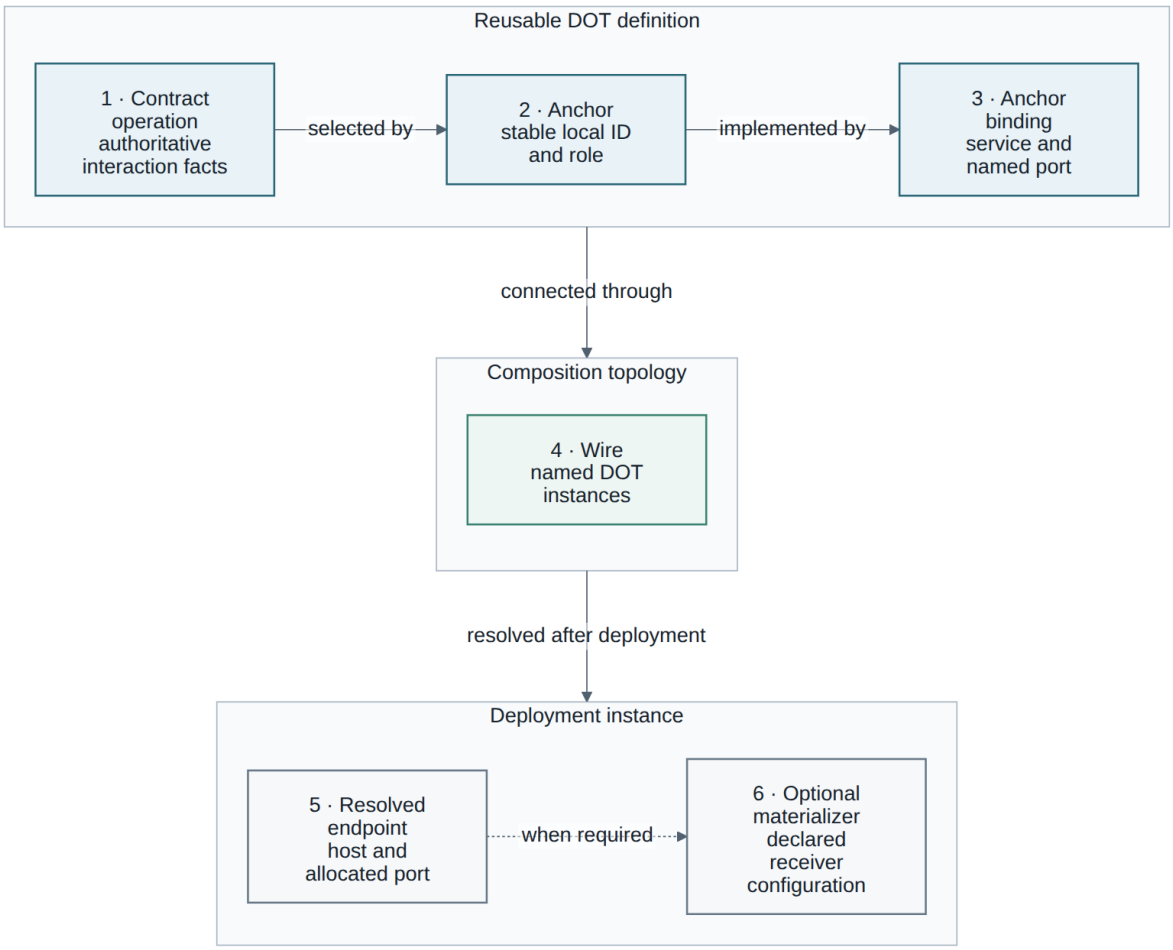


Figure 4. From an authoritative Contract operation to a deployed connection

The Contract owns protocol semantics. An Anchor gives one selected Contract operation a stable logical identity. A deployment-service binding identifies the service port that implements the Anchor. A composition wire connects two named DOT instances and may materialize the resolved endpoint into configuration already declared by the consumer.

Prepare the checkout

For a first deployment with the released CLI, follow Getting started. This presenter path uses a full checkout from Contributor guide with initialized `DOT-CLI` and `DOT-Repository` submodules. The demo needs `dotx`, `jq`, and network access when URL-backed Contracts are not already cached.

```
git submodule update --init --recursive
task cli:install
dotx version
```

Source builds use `DOT-CLI/VERSION`, which may advance independently of the published release. The published release reports:

```
dotx 0.4.0
```

The independently released Linux-amd64 binary, its SBOM, release manifest, and checksums are also available from the DOT-CLI v0.4.0 release. Follow the binary-specific checksum verification before installing or executing a downloaded release asset.

Stage 1: validate the authoritative catalogue

```
dotx validate DOT-Repository
```

This recursively discovers every `dot.json`; there is no separate inventory. The command validates schema shape, identities, Skills, Contract payload integrity, selectors, service ports, Anchor bindings, repository placement, and catalogue-wide uniqueness. Contract-backed external and internal bundles also receive the strict reusable profile.

What this proves

The checked-out catalogue is internally consistent and every declared reference resolves. It does not deploy a workload or claim that a third-party service is reachable.

Stage 2: inspect a reusable external DOT

Keycloak is a useful example because its canonical DOT uses official, digest-pinned Contracts while offering multiple deployment mechanisms.

```
KEYCLOAK=DOT-Repository/DOTs/external/Keycloak-DOT/26.8.1
dotx inspect "$KEYCLOAK" --json |
  jq '{name, version,
       skills: (.skills | length),
       contracts: (.contracts | length),
```

```
       anchors: (.anchors | length),
       deployments: (.deployments | length),
       supportedDeploymentSystems}'
```

The current source catalogue produces this compact projection:

```
{
  "name": "Keycloak",
  "version": "26.8.1",
  "skills": 2,
  "contracts": 3,
  "anchors": 4,
  "deployments": 5,
  "supportedDeploymentSystems": [
    "DOCKER_COMPOSE",
    "DOCKER_IMAGE",
    "HELM_CHART",
    "KUBERNETES_MANIFEST"
  ]
}
```

What this proves

One reusable DOT root can describe several supported deployment choices without forking its identity for each PoC. Deployment-instance realms, routes, credentials, and endpoints remain composition inputs rather than copied DOTs. All five deployment choices target the same Keycloak 26.8.1 release in this one release definition. Additional supported tool releases have separate product/release folders; the exact selection never falls back to another release.

Stage 3: resolve the cross-standard composition

The canonical Contract matrix connects six DOT instances with 21 compatible wires. Its committed lock pins every DOT, Contract, configuration input, and portable resolved composition.

```
dotx compose lock \
  DOT-Testing/composition/contract-standards/scenario.yaml --check

dotx compose validate \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked

dotx compose inspect \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '{apiVersion, name, version,
       dotInstances: (.dots | length),
       wires: (.wires | length),
       compatibleWires: ([.wires[] |
```

```
        select(.compatibility == "COMPATIBLE")] | length),
      standards: ([.wires[].contractStandard] | unique),
      interactions: ([.wires[].providerInteraction] | unique)}'
```

Expected summary:

```
{
  "apiVersion": "dot.mare6g.eu/v1",
  "name": "contract-standards-matrix",
  "version": "1.0.0",
  "dotInstances": 6,
  "wires": 21,
  "compatibleWires": 21,
  "standards": ["AsyncAPI", "GraphQL", "OpenAPI", "Protobuf"],
  "interactions": [
    "BIDIRECTIONAL_STREAM", "CLIENT_STREAM", "MUTATION", "QUERY",
    "REQUEST_RESPONSE", "SEND", "SERVER_STREAM", "SUBSCRIPTION"
  ]
}
```

The project gate asserts the complete result rather than only printing a summary:

```
task testing:compose:contract-standards
```

What this proves

Every provider and consumer selector resolves against its authoritative Contract; all 21 pairs are compatible; endpoints, interactions, probes, and materializers can be derived without copying standard-specific semantics into Anchors.

Follow one AsyncAPI wire

Inspect the Kafka wire to see how standard-derived facts and deployment facts remain separate:

```
dotx compose inspect \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '.wires[] | select(.name == "asyncapi-kafka") |
      {name, compatibility, contractStandard, contractFormat,
       providerInteraction, consumerInteraction, channelAddress,
       connectionEndpoint, materializers, probe}'
```

The AsyncAPI Contract supplies the complementary send/receive operations, channel, message schema, Kafka binding, and correlation field. The composition selects the broker instance and materializes its resolved address and topic for the consumer. A resolved composition still does not claim that a message was delivered.

Stage 4: render a lifecycle without mutating it

The APISIX, Keycloak, and OAuth-client scenario demonstrates dependency order, configuration materialization, native Kubernetes Secret references, rollout readiness, and an exposed Anchor:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml \
  --locked \
  --namespace dot-guided-demo \
  --dry-run \
  --forward protected-metadata-through-gateway=19080
```

The rendering ends with:

```
Dry run complete; composition state was not written.
```

The command may create bounded temporary rendering files and removes them when it exits. It does not create Kubernetes resources, start a port forward, or write composition state. The rendered order is Keycloak, APISIX, then the OAuth client, matching the declared readiness dependencies.

What this proves

The reference executor accepts the locked inputs and can derive the exact platform-native actions. It does not prove rollout success, credentials, or HTTP behavior; those require the live acceptance layer.

Stage 5: connect claims to evidence

Claim	Executable evidence	Boundary
Catalogue correctness	`dotx validate DOT-Repository`	Static and Contract resolution evidence; no deployment
Cross-standard interoperability	`task testing:compose:contract-standards`	21 deterministically compatible wires; no live traffic
Live Contract probes	`task testing:compose:contract-probes:live`	Standards-native test producer and real brokers on disposable Docker resources
Runtime configuration portability	`task testing:compose:runtime-configuration`	Dry-run rendering across six adapters

Claim	Executable evidence	Boundary
Workload isolation	`task testing:compose:service-workloads:live`	Live repeated Compose, Kubernetes, and Helm instances
Portable-runtime isolation	`task testing:compose:runtime-isolation:live`	Live Vagrant/libvirt and WASI CLI instances
Released-product acceptance	0.2.4 downloaded-binary acceptance	Linux amd64, single-host supported profile

See Verification and examples for the complete test matrix and the distinction between model, resolved composition, dry-run, source-adapted, and live production evidence.

Optional: run the live Contract probes



This path builds images, starts disposable Docker workloads, and downloads pinned images when absent. Use the presenter path when time, network, or host capacity is uncertain.

```
task testing:compose:contract-probes:live
```

The harness builds a standards-native GraphQL/gRPC producer and an independent consumer, then deploys the canonical Mosquitto, NATS, Kafka, and RabbitMQ DOTs. It proves GraphQL query, mutation verification/cleanup, and GraphQL-over-SSE; all four finite Proto3 `NO_SIDE_EFFECTS` gRPC shapes through reflection; and correlated MQTT, NATS, Kafka, and AMQP 0-9-1 delivery. It records redacted evidence and destroys composition-owned resources on success, failure, or interruption.

Five-minute presenter script

1. Show the Contract-to-Anchor-to-wire diagram and explain the four ownership boundaries.
2. Run `dotx validate DOT-Repository` to establish that the catalogue is the checked source of truth.
3. Inspect Keycloak to demonstrate one reusable product identity with several deployment choices.
4. Print the 21-wire resolved composition summary and highlight the four Contract standards and eight interaction forms.
5. Inspect the Kafka wire to distinguish Contract semantics from its resolved broker endpoint.
6. Render the APISIX/Keycloak/OAuth lifecycle with `--dry-run`.
7. Finish on the evidence table and state the supported boundary explicitly.

Keep the published release notes and verification matrix open as fallback evidence if a local network or runtime is unavailable.

Supported boundary

The stable profile supports Linux amd64 and private single-host composition state. Multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, AMQPS, AMQP 1.0, native arm64 and non-Linux executables, the DOT-Registry REST API, and joint SFC or partner acceptance are separate activities. Unsupported combinations fail closed rather than being silently approximated.

All eleven MARE PoCs have resolution or source-adapted evidence. That evidence is not presented as acceptance of every unmodified partner workload. Continue with Architecture for system boundaries or DOT Anchors and Composition for authoring and lifecycle details.

Developer demo: reuse and extend

Use this optional five-minute extension after the 10-15-minute presenter route, or during technical Q&A. It adds no new live deployment. Run from the DOT-Design checkout root with its installed `dotx`, initialized catalogue, Bash and `jq`.

The developer takeaway is: **reuse the product definition, describe the real interface, configure a composition, then prove the intended exchange.**

1. One real product, several deployment choices

Ask the audience: "If two PoCs use Keycloak with different realms and deployment platforms, which parts should they share?"

```
KEYCLOAK=DOT-Repository/DOTs/external/Keycloak-DOT/26.8.1
dotx validate "$KEYCLOAK" \
  --schema DOT-Repository/Schema/schema.json --profile reusable --offline

dotx inspect "$KEYCLOAK" --json |
  jq -e '{name, version,
          deployments: [.deployments[] |
            {id, platformEngine, type, supported}],
          anchor: (.anchors[] | select(.name == "admin-realm-read") |
            {name, role, contract})} |
         select([.deployments[].platformEngine] | unique) ==
           ["DOCKER", "KUBERNETES"])'

jq -e '.contracts[] | select(.name == "Admin") |
       {id, name, version, standard, format, payload}' "$KEYCLOAK/dot.json"
```

Show: one `Keycloak@26.8.1` identity with Docker Compose and Kubernetes/Helm deployment choices in one release `dot.json`. Supported tool releases have separate exact release folders in the same product folder. All deployment artifacts must match 26.8.1. PoC-specific configuration stays in composition inputs or named deployments, not a cloned product identity. The selected `admin-realm-read` Anchor refers to the Admin REST Contract and its `GET` selector. The root Contract declaration identifies the release-specific official raw document and SHA-256; source/provenance boundaries

remain in the audit. No API definition is copied into the Anchor.

This bundle declares every owned Contract completely in `dot.json`, using `payload.url` for raw HTTPS bytes and `payload.sha256` for integrity. Contracts do not need a local directory. Validate online once before using `--offline`; the CLI then rechecks cached bytes without downloads. Composition inputs name the exact owner, not a copied wrapper. Official documents remain preferred; derived specifications and their acceptance limitations remain labelled.

Explain the boundary: an inspector reporting a supported deployment is not live acceptance of every deployment variant. The PoC10 rehearsal selects pinned native Kubernetes deployments `Keycloak@26.8.1` and `APISIX@3.19.0`, with composition-owned realms, routes and policy. It does not certify every variant or every Admin REST operation. Compare each resolved composition's `dotId`, `dotVersion` and `deploymentId` when explaining provenance.

2. Show the guardrail, not just the happy path

```
task testing:demo:developer-tour
```

The task repeats the short inspection, validates the existing two-DOT PoC7 validation-only graph, then attempts its existing deliberately rejected wire. Expect:

```
EXPECTED REJECTION (exit=1): scoring-api-to-collector mixes Contract-backed and
contractless Anchors.
The valid two-DOT graph has zero wires; no runtime was deployed by either validation
command.
```

The task itself exits zero only when the positive graph resolves and the negative graph returns that specific compatibility diagnostic. Missing credentials, a broken checkout, a missing CLI or an unrelated validation error are not counted as a successful negative test.

Say: "Putting two boxes on a diagram is not an implemented integration. We cannot make an undeclared collector interface compatible by drawing a wire." At the pinned source boundary, the collector prints flow data rather than calling the scorer. This control demonstrates interface admission, not a live trust-scoring result or a runtime security test.

The first current rejection is Contract-backed/contractless incompatibility; it does not prove that a later undeclared-configuration check ran. After implementing a real consumer, declare its supported endpoint configuration, Anchor and binding, then validate the resulting wire and test the actual call.

3. What a PoC developer changes, and what stays shared

Concern	Owning artifact	Developer action
Product capability and interfaces	The existing DOT bundle, root Skills/Contracts, Anchors and service bindings	Reuse it. Extend the shared definition only when the software genuinely supports an additional capability or interface.
API or messaging semantics	Official Contract, or an explicitly labelled source-derived Contract	Select an existing operation. Do not redefine request/response schemas or invent a consumer implementation in a wire.
Deployment choice	A selected runtime/deployment already declared by the DOT	Choose a supported deployment ID. Add a reusable variant to the product root if a new mechanism is genuinely needed; test that variant.
Realm, route, policy, scrape target or configuration file	Composition inputs and endpoint materializers	Configure the use case without copying or hard-coding the product definition.
Credentials and concrete runtime state	Secret references and the deployment instance	Keep secret values outside reusable public descriptors and presentation output.
Reproducibility and proof	Reviewed composition lock and executable acceptance harness	Inspect the resolved composition, review changed lock inputs, run dry-run checks, then verify the real workload and its cleanup. Commit definitions, not local runtime state.

The practical contribution sequence is:

1. Find the product under `DOT-Repository/DOTs/external/` or `DOTs/internal/`; check its README, source revision and declared gaps.
2. Use official interface documents where available. For MARE-owned code, export or derive the Contract from the implementation and review its limits.
3. Declare the provided/required Anchor and the selected deployment's service binding. Ensure the application really reads the declared configuration.
4. Add or update a reusable graph under `DOT-Repository/Compositions/`, with its README, non-secret inputs and reviewed lock. Put synthetic fixtures and live test drivers under `DOT-Testing`, not beside a new duplicate product.
5. Run `dotx validate DOT-Repository`, inspect the locked composition and deployment dry-run, then add a far-end behavioral assertion and success/failure cleanup.

`Skill` is a required high-level capability reference; it is not an API operation or a command executor. Anchors and authoritative Contracts describe the concrete interoperability boundary.

4. Show the broader potential without extending the live claim

Capability	Existing executable evidence	What to say
Four Contract standards, 21 selected wires	`task testing:compose:contract-standards`	Broader deterministic interface coverage than the four-wire live demo; a compatible resolved composition is not traffic.
GraphQL query/mutation/SSE; finite unary and streaming gRPC; MQTT, NATS, Kafka and AMQP 0-9-1	`task testing:compose:contract-probes:live`	A separate, longer test-producer/broker acceptance suite. Do not start it unexpectedly during a short presentation or claim partner-producer acceptance.
Configuration defaults and overrides across six adapters	`task testing:compose:runtime-configuration`	Docker image/Compose, Kubernetes/Helm, Vagrant and WASI rendering; not proof that every possible DOT runs on every adapter.
Repeated multi-service instances and selective cleanup	`task testing:compose:service-workloads:live`	A separate Compose/Kubernetes/Helm isolation test, not multi-host orchestration.

Choose one row in response to a technical question rather than reading the entire test matrix aloud. The examples demonstrate extension points and tested profiles, not automatic protocol conversion, arbitrary networking, complete PoC production readiness, a DOT-Registry REST API or joint SFC acceptance.

Continue with Anchors and Composition for authoring, Verification and examples for the evidence matrix, and the audience FAQ for short answers.

MARE PoC composition walkthroughs

Start here when presenting DOT composition against MARE use cases. Run commands from DOT-Design with its pinned submodules. These are integration slices, not complete official PoC outcomes. For generic cross-runtime/four-Contract mechanics, use the Docker/Kubernetes demo.

Presenter route: inspect, run, prove, clean up

For a 15-20 minute presentation, show PoC4 first, PoC10 second, and PoC6 only with its adapter/double boundary visible. Avoid new feature changes on the day. Each presenter command uses the same locked composition and live assertions as acceptance; there is no second set of DOT definitions or demo-only success path.

```
# Run from a real interactive terminal, one example at a time.
task testing:pocs:present:monitoring
task testing:pocs:present:secure-api
```

```
task testing:pocs:present:remediation
```

Each follows five stages: **inspect topology → deploy → verify → inspect the running result → clean up**. The terminal prints resolved DOT versions, deployments, wires and evidence boundaries. It pauses only **after** every API assertion passes. While paused, open the printed Prometheus targets page or inspect the redacted `result.json`; the secure API must still deny anonymous requests. Do not paste access tokens or Secret values into the presentation.

The topology is generated from the locked composition by `dotx compose topology`, using the MIT-licensed `mermaid-ascii` 1.6.1 Go library embedded in the binary. There is no Python renderer or separate executable to install. DOTs appear once, with connected Unicode boxes. Parallel wires share a connection labelled `W1,W2,⋯`; the connection legend lists every wire, Contract standard and protocol separately. `T` links show broker transport endpoints, not extra Anchor wires or invented application publishers. Aligned tables show full instance names, platforms, deployments and versions, plus each connection's provider, consumer, Contract standard, protocol and name. Long values wrap inside their columns without truncation, using the same width as the graph. Arrows mean logical PROVIDES-to-REQUIRES relationships, not packet direction. The graph reappears after successful live verification.

Interactive diagrams use cyan Kubernetes, magenta Docker and amber connection IDs. Set `NO_COLOR=1` or `DOTX_TOPOLOGY_COLOR=never` for plain output; `always` forces colour when capturing a demonstration (`NO_COLOR` and `TERM=dumb` still disable it). Logs are plain by default. `DOTX_TOPOLOGY_WIDTH=80` controls width (40-240 columns). The renderer tries vertical/compact layouts for narrow screens and fails clearly if it cannot show every connection; it never silently clips.

For all four Contract standards across Docker and Kubernetes, use `task testing:demo:docker-kubernetes-four-contracts:present`; see the mixed-runtime presenter walkthrough.

Press Enter to destroy owned resources and verify cleanup. Ctrl-C also cleans up and returns 130; SIGTERM returns 143. The default inspection limit is ten minutes (`DOTX_POC_PRESENT_TIMEOUT`, integer 1-1800 seconds). Timeout returns 124 and EOF returns 1, both after cleanup: neither is reported as a successful presentation. Headless launches are rejected before deployment, and `DOTX_E2E_KEEP=1` is forbidden in presenter mode.

```
# Safe headless previews: print the real graph and render without deployment.
task testing:pocs:present:monitoring -- --dry-run
task testing:pocs:present:secure-api -- --dry-run
task testing:pocs:present:remediation -- --dry-run
```

The noninteractive `testing:pocs:*:live` tasks remain unpaused. Use the presenter tasks above, not the full failure-injection suite, in front of an audience. For preparation, `task testing:pocs:present:controls` exercises Enter, EOF, timeout and real signals with live Prometheus on overridden port 19041. `task testing:pocs:present:rehearse` requires clean commits, drives all three presenter flows through a real terminal, inspects live status during each pause, then verifies cleanup. It preserves checkout/CLI fingerprints and redacted results under ignored `DOT-Testing/e2e/target/presenter/` and `poc-rehearsals/`. Keep those receipts and the exported PDF as an

explicitly labelled fallback, not evidence that a new live exchange occurred during the talk.

Prerequisites and fixed inputs

Use Linux amd64, Docker, curl, jq and Git. Acceptance harnesses also require Bash, flock, Python 3 and Task; ShellCheck checks their regressions. PoC10 needs kubectl and a Ready local Docker-driver Minikube node with context `minikube`. PoC6 additionally needs Docker BuildKit and read access to the pinned private producer through Git/`gh`, or `POC6_SOURCE_DIR` at the exact revision below. Automatically acquired PoC6 source uses the private run directory's filesystem, not shared `/tmp`, and is removed after the run. A caller-supplied source checkout remains untouched.

```
git submodule update --init --recursive
task cli:install
dotx version
git rev-parse HEAD
git -C DOT-CLI rev-parse HEAD
git -C DOT-Repository rev-parse HEAD
task testing:pocs:dry-run
```

To display just the graph without running the acceptance harness:

```
dotx compose topology \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --color always
```

Install from the pinned CLI checkout: released 0.2.4 predates repository-wide composition validation. Dry-runs must pass before deployment. The three paths contain respectively 1, 5 and 2 compatible wires. Inspect their exact Anchor IDs, selected deployments and materializers; `COMPATIBLE` is not a live result.

1. PoC4: Node Exporter to Prometheus

```
Node Exporter -- metrics scrape wire --> Prometheus -- query API --> presenter
```

Graph: `DOT-Repository/Compositions/node-exporter-prometheus-monitoring/composition.yaml`. Producer context: PoC4 `c03eca71a9d87358603a1d89fa0eca58a9dabe03`. Node Exporter provides metrics; Prometheus requires the scrape endpoint. A `CONFIG_FILE` materializer inserts `ENDPOINT_HOST_PORT` at `/scrape_configs/0/static_configs/0/targets/0`. Prometheus's query Anchor uses its DOT-local Contract. OpenMetrics is not a fabricated OpenAPI operation.

```
dotx compose inspect DOT-Repository/Compositions/node-exporter-prometheus-
monitoring/composition.yaml --locked --json
task testing:pocs:monitoring:live
```

The task waits for readiness, verifies the exact resolved scrape target is healthy, checks

`up{job="node-exporter"}` equals `1`, and verifies the exporter version through `node_exporter_build_info`. Expect `PASS`, then a receipt with `exit=0, cleanup=PASSED`. Override `DOTX_POC4_QUERY_PORT` if 19040 is busy.

This grants no host namespaces or host root filesystem mounts. It observes the exporter's container, not the testbed host. Full log/dashboard flow, MITM detection and mitigation remain outside this slice.

2. PoC10: authenticated, policy-controlled SPARQL

Presentation preflight

After preparing the local cluster and rehearsing once to cache images, run this read-only gate before presenting. It is scoped to this secure-API composition, not the four-Contract fixture or all catalogue deployments:

```
# Keep these overrides in the same terminal for preflight and presentation.
export DOTX_POC10_NAMESPACE=dotx-poc10-presentation
export DOTX_POC10_FORWARD_PORT=19010
task testing:demo:preflight DEMO=secure-api
task testing:pocs:present:secure-api
```

The preflight requires local Go only to inspect the installed binary's embedded build information. It requires clean CLI/catalogue checkouts and a binary built from the CLI's exact revision, rather than trusting the `0.2.5` version label. If mismatched, review/commit CLI changes and run `task cli:install`. It prints the CLI SHA-256, Design/catalogue revisions, verifies local Contract assets through offline catalogue validation, and resolves the reviewed composition lock.

It then checks Docker/Compose access, the `minikube` context, one Ready Linux-amd64 node without resource pressure, an unused namespace and TCP port, and every selected manifest image in the Docker-driver Minikube cache. All five selected images are digest-pinned at their rehearsed versions; tags remain in the reference for readability. Preflight rejects tag-only references and cached images whose reported platform is not Linux amd64. It prints the resolved image IDs without dumping image configuration. This is cache availability, not a remote registry/Contract URL check, image vulnerability scan, or API acceptance. The check is bounded and does not start tools, pull images, create Secrets, deploy, adopt resources or clean anything up. Read-only CLI resolution can use temporary files/cache; no composition lifecycle state is written.

If a prerequisite fails, correct only that prerequisite and rerun. Prepare the cluster explicitly with `dotx tool check kubernetes --start` if needed. To warm images, run the existing `task testing:pocs:secure-api:live` acceptance task before the talk; it may download images and must finish with verified cleanup. For an occupied port choose another `DOTX_POC10_FORWARD_PORT`. Never delete a foreign namespace to make preflight pass: choose a new name, or recover only a previous run whose ownership is recorded in its receipt. Preflight is a point in time check, not a reservation; the live harness repeats ownership checks.

What the live result proves

The harness now seeds a known RDF triple into the disposable backend, then reads that stored value through the authenticated gateway. Acceptance requires exactly one expected literal, rather than a constant `SELECT` expression that would work with an empty database. An authenticated INSERT attempt must be rejected with 403. Seeding is explicit out-of-band test setup in the owned namespace; it is not a claim that the public read-only interface authorizes writes.

The same live task also requires a reversible authorization transition:

1. Authenticated query of the seeded data: HTTP `200` and the expected literal.
2. Temporarily deny reads in this run's OPA module: the same authenticated query must complete with HTTP `403`, not a redirect or transport failure.
3. Restore the original module, read it back, then issue a fresh authenticated query: HTTP `200` with the same stored value.

The console prints these three stages. The redacted `result.json` contains `policyTransition.httpStatuses: [200, 403, 200]`, `restored: true` and `storedDataUnchanged: true` only after every assertion succeeds. The inspection pause happens after restoration, so the displayed composition remains usable.

Policy administration is an explicit operator-side demonstration action using OPA's Policy API, not a sixth Anchor wire, automatic remediation, or a production policy-management design. The harness requires ownership of the applied namespace and an exact match to the reviewed policy before mutation. A failed or ambiguous update triggers a restoration attempt; any failure still invokes the parent harness's owned-resource cleanup. It never changes another namespace's policy. In production, OPA recommends bundles for policy distribution; this local demonstration uses the API to make one bounded, observable transition.

Offstage dependency-outage qualification

```
task testing:pocs:secure-api:resilience:live
```

Run this separate preparation task offstage, not during the short `task demo`. It repeats a real OPA process outage twice, then injects a failure after OPA has stopped to verify restoration and cleanup. The existing canonical composition is reused; no duplicate DOT or deployment descriptor is introduced.

Each successful run verifies an authenticated stored-data query (HTTP `200`), zero OPA pods and an authenticated rejection (`403`), then a new OPA pod and the same stored value (`200`). The pinned APISIX OPA plugin fails closed with `403` when its decision service is unavailable; a redirect, `5xx` or transport failure does not pass. Pod-drain polling uses a 90-second deadline with individually bounded API requests; an in-flight request can extend the drain's wall-clock duration. Rollout has a 180-second limit. This process outage is distinct from changing a Rego rule.

The operator-side control verifies the namespace UID, Deployment UID, exact pinned image, replica count and resource version before conditional patches. It attempts restoration after ambiguous responses or injected failure, never adopts a replacement Deployment, and leaves final ownership-

safe teardown to the existing harness. Private per-run `resilience-control.json`, `resilience-restoration.json` and successful `resilience.json` receipts retain only redacted assertions. The aggregate receipt under `DOT-Testing/e2e/target/poc-wave/` requires unchanged source/CLI fingerprints, two recoveries and verified failure cleanup.

This does not qualify Keycloak outage/token-cache behaviour, gateway failover, resource pressure, HA, durable backup/restore, stateful upgrades or the complete producer PoC. Those remain separate operational acceptance work.

Composition and ordinary live route

These arrows show logical PROVIDES-to-REQUIRES wires, not HTTP packet direction:

```
Keycloak -- discovery --> APISIX <-- SPARQL -- Oxigraph
OPA ----- decisions ---> APISIX -- protected capability --> Client
Keycloak ---------------- token ------------------------> Client
```

Graph: `DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml`. Producer context: PoC10 `e6a755145bbb71551916163209c0be6e9e996f9c`. Five instances exchange through five wires:

- Keycloak discovery to APISIX;
- Oxigraph SPARQL to APISIX;
- OPA policy decisions to APISIX;
- OAuth token endpoint to the client (`TOKEN_URL`);
- protected SPARQL capability to the client (`CAPABILITY_URL`).

Native Secret references supply credentials. Routes/policy are explicit, composition-owned inputs, not automatic translation. OIDC/OAuth Contracts come from canonical bundles. SPARQL and the legacy OPA workflow remain explicit protocol interfaces, not invented official vendor OpenAPI documents.

```
kubectl config current-context
kubectl get nodes
dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --json
task testing:pocs:secure-api:live
```

Expect a fresh namespace and native test Secrets, HTTP `401` for missing and malformed bearer tokens, authenticated POST `403`, and an authenticated GET returning `mare-poc10` from the real SPARQL engine. Every required client input is checked. The harness destroys its namespace and forward; the receipt must say `cleanup=PASSED`. Override `DOTX_POC10_FORWARD_PORT` if 19010 is busy.

Observation	Expected result
Anonymous request to the gateway	HTTP `401`
Deliberately malformed public test token	HTTP `401`; redirects/transport errors are not acceptance
OAuth client-credentials token	Acquired inside the client pod; never printed
Authenticated POST/write attempt	HTTP `403` from the protected route
Authenticated stored-data GET	Exactly one literal `mare-poc10`
Lifecycle	Recorded status, destroyed owned namespace/forward, `cleanup=PASSED`

During the presenter pause, inspect the printed private-cache `compose status` command and `result.json`. The five compatible wires are composition/configuration evidence; the four HTTP assertions above are the live end-to-end evidence. They are not five independent native Contract probes or full response-schema validation. Press Enter to clean up and retain the receipt. Re-run from the cleaned state to check repeatability; never treat a previous receipt as a new live exchange.

The invalid token is fixed public test data; real access tokens are never printed. `invalidBearerHttpStatus` in `result.json` records the checked 401. The bounded check ignores local curl configuration and does not follow redirects. It proves rejection of this malformed token, not expiry/revocation handling, issuer/audience isolation, signing-key rotation, or a security certification.

Oxigraph substitutes for GraphDB; a synthetic machine client uses client credentials instead of a password grant. This does not accept the full producer GraphDB/LDAP/Data Security Service stack, telemetry/events or automated response. HTTP, bootstrap defaults and the read-only policy are demo settings, not production security hardening.

3. PoC6: explicitly adapted remediation rehearsal

```
CACAO interpreter [adapter] -- MQTT/OpenC2 via Mosquitto --> actuator
actuator -------------------- remediation call ---------> NWDAF [DOUBLE]
```

Graph: `DOT-Testing/e2e/poc6-cacao-openc2/scenario.yaml`, not a catalogue example. Producer revision: `cd359b25df1b010324869d7e1c755067070e42dd`.

```
task testing:e2e:poc6-cacao-openc2:dry-run
task testing:pocs:remediation:live
```

A guarded interpreter adapter resolves the CACAO external broker URI. The unchanged pinned actuator receives OpenC2 over real Mosquitto/MQTT through the AsyncAPI wire, then calls the OpenAPI remediation endpoint. The exact job UUID must reach the far end. That endpoint is an **NWDAF double**, not real NWDAF remediation. Telemetry, AI and playbook selection remain unaccepted. Override `DOTX_POC6_INTERPRETER_PORT`/`DOTX_POC6_NWDAF_PORT` if 18261/18262 are busy.

Repeatability, cleanup and recovery

```
task testing:pocs:rehearse
```

This runs each slice twice, followed by injected failure and real SIGINT/SIGTERM cleanup controls. It records the CLI hash, three checkout revisions, tracked/untracked fingerprints, runtime image identities and per-run receipts under `DOT-Testing/e2e/target/`. Changed inputs invalidate the aggregate receipt. Dirty-worktree evidence does not claim those changes were already published.

The manual **Anchor live E2E** workflow's `first-wave` selection runs the same checks and rehearsal driver. Remote infrastructure and queue failures remain separate from local acceptance; no workflow success is claimed before it runs.

Harnesses refuse existing resources, lock fixed identities, use private caches and check owned leftovers. Anonymous Docker volumes are removed; unrelated resources are untouched. PoC6 test images remain cached. `DOTX_E2E_KEEP=1` records `SKIPPED` cleanup and cannot pass acceptance.

Kubernetes namespace creation records the UID from its create response. Cleanup rejects unknown or replaced ownership and uses a server-side UID precondition for namespace deletion. Lost create confirmation is a failed cleanup requiring review, not permission to adopt a same-named namespace.

Current-source named Helm restart with explicit `--forward` also reconciles a previously recorded forward when its custom release name differs from the DOT product name. Replacement requires an exact namespace/service/port mapping in the private forward registry; unrelated namespaces, services, mappings and unrecorded listeners are not adopted. This is a Next runtime correction, not a change retroactively added to the published `0.3.0` CLI binary. Keep the rehearsed source/binary set together.

If cleanup fails, preserve the printed run directory. Set `XDG_CACHE_HOME` to its `cache` directory and, for Kubernetes runs, set `KUBECONFIG` to that run's private `kubeconfig`. Do not follow a later global context change or share the saved kubeconfig. Stop if the original run files are missing; a new run's state does not authorize recovery. Confirm the namespace still has its recorded UID; stop and review a mismatch or an unconfirmed identity. Inspect `dotx compose status <composition>`, then retry `destroy` on the same graph only while the original ownership still matches. Inspect only recorded namespaces and resolved identities. Never strip finalizers or delete unrelated workloads. Select an unused port instead of killing another service. Review missing Contracts, inputs or stale locks before intentionally regenerating a lock.

Full-catalogue appendix

`DOT-Testing/interoperability/FIRST-WAVE.md` and `pocs.yaml` retain all eleven PoCs and remaining gaps. `DOT-Repository/CONTRACT-AUDIT.md` separates official and source-derived Contracts, protocol-only interfaces and runtime-only brokers. Validation-only fixtures, stubs, adapters and doubles remain labelled. Declaration completeness is not runtime acceptance or production certification.

This page publishes through `next`; stable-tag documentation is unchanged. Rehearsal evidence is a presentation fallback, not a substitute for a claimed current live result or a green remote workflow.

Docker/Kubernetes four-Contract demo

This `next` development walkthrough reuses three canonical catalogue definitions: the standards-native provider, consumer, and official Mosquitto broker. Named deployment variants select Kubernetes for provider/broker and Docker for consumer. No product descriptor is duplicated.

The composition is named `docker-kubernetes-four-contracts`. Its template lives at `DOT-Testing/composition/docker-kubernetes-four-contracts/scenario.yaml`: this is a synthetic test graph, unlike the reusable catalogue examples `keycloak-apisix-oauth` and `keycloak-prometheus-openapi-validation`.



Provider and consumer are explicitly test-only components, not MARE analytics or vendor products. Real HTTP, GraphQL.js, gRPC-js and Mosquitto exchanges demonstrate interoperability mechanics, not producer acceptance, production security, or automatic protocol conversion. Probe clients use the actual consumer's Docker network namespace; its application process consumes REST data and derives a synthetic review decision. MQTT traffic is generated by the correlated delivery probe, not the GraphQL application. The MQTT runner uses the pinned official `mosquitto_rr` client in MQTT 3.1.1 mode, subscribing and publishing on the same Contract-selected channel. It waits for SUBACK before publishing, ignores retained messages, uses a clean session, and requires the exact uniquely correlated payload. This proves broker delivery, not an application-level request/response implementation.

What runs where

Wire	Authoritative Contract	Kubernetes endpoint	Docker-side assertion
`rest-alerts`	OpenAPI 3.1, `GET /alerts`	Provider, NodePort `31080`	HTTP 200 plus the expected fixture alert UUID and `critical` severity
`graphql-health`	GraphQL SDL, `Query.health`	Provider, NodePort `31400`	`/data/health` equals `true`
`grpc-health`	Protobuf, side-effect-free `AlertService.Check`	Provider, NodePort `31551`	Reflected response has `serving: true`
`mqtt-alert-delivery`	AsyncAPI 3.0, MQTT-bound alert channel	Mosquitto, NodePort `31883`	Schema-valid, uniquely correlated QoS 1 delivery

Anchors select Contract operations. Wires select participants and materialize resolved endpoints into the consumer's declared environment. Each wire declares `EXTERNAL_GATEWAY`: the selected DOT declares the NodePort Service, and Minikube provides the reachable node address. Docker normally isolates its bridge networks: the harness explicitly attaches only its disposable consumer to the existing `minikube` Docker network. Cross-runtime probes use `--network container:<consumer>` so they exercise that actual consumer route, not a different runner's network. The attachment is external to `dotx` state, and consumer destruction removes it; the harness verifies its removal. No host listener, firewall change, tunnel, or Minikube-container modification is made.

The REST provider returns one deterministic, read-only synthetic alert: `a4a70000-0000-4000-8000-`

000000000099, severity critical. The native probe.openapi.expect assertion compares the complete JSON array, requiring a declared successful status, JSON media type, and the expected identity/severity. Use the installed development CLI; the released v0.2.5 binary predates this feature. A supplementary independent application assertion reads OPENAPI_YAML_URL from the deployed consumer and fetches it from that consumer's actual network namespace. It requires HTTP 200, JSON content type, exactly one alert, and the expected identity/severity. Redirects, wrong data, oversized bodies and a stalled response fail acceptance. The 10-second, 1-MiB-bounded Node.js client uses the already-built provider image; it is a test client, not a new DOT or an implicit GraphQL-to-REST translator. Only its fixed redacted summary is saved as rest-body.json; arbitrary response fields are not printed. The independent check also verifies the actual materialized consumer configuration; it is not needed to enable the native CLI response assertion.

Show application behaviour and a controlled failure

The Docker consumer now runs a small synthetic application, not sleep. It repeatedly fetches the wire-materialized OPENAPI_YAML_URL, validates the bounded response, and derives REVIEW_REQUIRED when any alert is critical; otherwise it derives NO_CRITICAL_ALERTS. It does not hard-code the demo UUID or emit a canned decision. Its owner-only, atomically replaced receipt lives inside the disposable consumer at /tmp/dotx-consumer-result.json. No new public endpoint or duplicate DOT definition is introduced.

The normal live and presenter commands automatically rehearse this sequence:

1. Observe a fresh consumer receipt: one critical alert, REVIEW_REQUIRED.
2. Change only the owned synthetic provider's test-control file to serve severity low. Its API still returns HTTP 200 and schema-valid JSON, so a readiness check alone would not expose the changed application behaviour.
3. Require dotx compose probe to exit 1 with recorded HTTP 200 and RESPONSE_MISMATCH, not a timeout, transport error, or arbitrary failure. Independently observe the running consumer change to NO_CRITICAL_ALERTS.
4. Restore the critical fixture; require a successful native assertion and a newer consumer receipt with REVIEW_REQUIRED before presenting results.

These steps run automatically through the existing live and presenter commands below; no separate control command or second demonstration is required.

The file /tmp/dotx-rest-severity is an explicitly test-only fault switch in the disposable provider container. The harness controls it only after owning the run's namespace, restores it before accepting the negative control, and destroys the owned composition on any unexpected failure. It never changes a real vendor DOT, Kubernetes context, or unrelated workload. low is valid for the Contract's string field; the rejection proves the reviewed composition's value assertion, not full response-schema validation or production policy.

Evidence includes consumer-before.json, consumer-negative.json, consumer.json, rest-negative-status.txt and response-recovery.json. Increasing receipt sequence numbers prove observation after each transition. result.json includes the recovered consumer decision and negative-control summary. Interrupted or failed checks cannot publish a successful result. The application and fixtures are synthetic: REVIEW_REQUIRED is a demonstration decision, not real security analytics,

mitigation, or producer acceptance.

Prepare before presenting

Use the full checkout in Contributor guide, and run commands from its root. Requirements: Linux amd64, Go and Task to install this checkout's CLI, Git, Bash, jq, curl, GNU coreutils, `flock` (util-linux), Docker with Compose, kubectl, and a running local Minikube cluster using its Docker driver and `minikube` Docker network. The Kubernetes context must be `minikube`. The deterministic demo checks also require ShellCheck and Python 3. On Ubuntu, install the `shellcheck` package before running `task testing:demo:checks`. The introductory Prometheus quickstart still needs no Go or Kubernetes.

```
git submodule update --init --recursive
task ci:prepare
git rev-parse HEAD
git -C DOT-CLI rev-parse HEAD
git -C DOT-Repository rev-parse HEAD
go version -m "$(command -v dotx)"
docker info --format '{{.ServerVersion}}'
docker compose version
kubectl config current-context
kubectl get nodes --request-timeout=10s
minikube ip
```

`dotx 0.2.4` alone is insufficient: this example uses post-release development changes. Install the pinned submodule and keep the checkout fixed. Evidence records revisions, CLI binary SHA-256, root/CLI/catalogue working-tree patch hashes, and provider/consumer image IDs. Non-empty patches mean the commit alone does not describe the tested checkout. Broker/probe images use digest references; the test images are locally built from pinned Dockerfiles/dependency locks, not claimed published immutable images. The provider uses `imagePullPolicy: Never` after explicit loading; the harness verifies actual running image IDs. Rehearse with network access for metadata/dependency/image downloads; caching does not promise fully offline execution.

After a successful live rehearsal has built/loaded the fixture images and cleaned up, run the read-only check for this route:

```
task testing:demo:preflight DEMO=four-contracts
```

It checks the exact installed CLI revision, clean CLI/catalogue checkouts, offline Contract validation and locked composition, the Ready Linux-amd64 Minikube node, selected cached images, Docker identity/network conflicts, and existing Services occupying the four selected NodePorts. It does not build/pull images, reserve ports, attach networks, deploy anything or prove gateway reachability. The live acceptance below remains necessary. Use `DEMO=monitoring` for the node-exporter/Prometheus route, `DEMO=portability` or `DEMO=upgrade` for those lanes, or omit `DEMO` to check all five presenter routes.

For a stage run without fixture rebuilds, first run explicit `task testing:demo:prepare DEMO=four-contracts`, then export `DOTX_DEMO_PREPARED=1` before the live/presenter command. This checks the

prepared source/binary fingerprints and Docker/Minikube fixture IDs and refuses drift. It performs no build, pull, image import or cluster start in that mode. It does not guarantee offline probe execution; see the qualification and sanitized fallback route.



Use an isolated demo machine/network. NodePorts are plaintext; MQTT permits anonymous access and disables persistence. Do not expose them to untrusted networks. These are demo defaults, not production authentication, TLS, retention, or availability settings. Never delete an unrelated namespace or Service.

Inspect without deploying

```
exec bash DOT-Testing/e2e/multi-runtime-demo/test.sh --dry-run
```

This resolves three DOTs/four wires, writes and verifies a lock, checks both platforms and all four standards, and renders deployment actions in a private directory under `DOT-Testing/e2e/target/multi-runtime-demo/`. The template uses documentation-only address `192.0.2.1`; never manually apply that template.

Run the working example

Presenter mode: inspect before cleanup

```
task testing:demo:docker-kubernetes-four-contracts:present
```

Run in a real interactive terminal. The composition-derived Mermaid graph is laid out by the Go `mermaid-ascii` library embedded in `dotx`: Unicode boxes, cyan Kubernetes labels, magenta Docker labels and amber connection IDs. Parallel wires share a labelled arrow and each retains its own legend entry. The node inventory shows selected runtimes/deployments/versions; a `T4` link identifies the MQTT transport endpoint, not another Anchor wire or an application publisher. Arrows mean logical PROVIDES-to-REQUIRES relationships, not packet direction. The graph is printed again after all four real probes and recorded status checks pass.

Only then does the harness pause. Use the printed, shell-quoted recovery/status command in a second terminal to inspect the exact live composition. `resolved-composition.json`, `topology.txt`, `topology.mmd`, probe logs, `status.txt`, `results.txt` and `result.json` remain in the printed private run directory. The result summarizes checked fixture assertions, not real MARE analytics. `rest-body.json` holds the separate REST data assertion. Presenter mode shows concise numbered progress and an aligned results table with separate Contract, protocol, observed assertion and outcome columns. Each row requires both completed probe and recorded status evidence. Detailed build/deployment/cleanup output remains in owner-only `details.log`; a failure prints the original failing stage, exit code and its log location. Do not share raw private logs. Normal live acceptance keeps its verbose output. Both normal and presenter runs save `result.json` and `results.txt`; summaries are published only after all probes, the REST data assertion and recorded status checks pass. A failed recheck removes earlier result summaries rather than leaving stale green output. Cleanup is reported separately in `evidence.json`; an assertion summary alone never proves successful cleanup. See the audience FAQ for the model's value, reuse, standards and evidence

boundaries. The graph deliberately omits endpoint hosts, configuration and credential values. `topology.mmd` can also be rendered in Antora. `NO_COLOR=1` disables colours; `DOTX_TOPOLOGY_COLOR=auto|always|never` controls them (`NO_COLOR` or `TERM=dumb` takes precedence). `DOTX_TOPOLOGY_WIDTH=80` sets a 40-240-column target; narrower screens use vertical/compact layouts. No connection may disappear to fit. The saved `topology.txt` is always plain text. Rendering needs only the `dotx` binary: no Python, Node, browser or separate `mermaid-ascii` executable. The Go module pins the upstream 1.6.1 source revision and verifies dependencies through `go.sum`; no renderer is downloaded at runtime.

```
# Read-only topology preview; Docker/Kubernetes need not be running.
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --color always --width 80
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --format mermaid
```

Press Enter to clean up normally. Ctrl-C exits 130, SIGTERM 143, EOF 1 and timeout 124, all through the existing ownership-checked cleanup path. Inspection defaults to 600 seconds; set `DOTX_POC_PRESENT_TIMEOUT` to an integer from 1 to 1800 to change it. Headless presenter launches and `DOTX_E2E_KEEP=1` are rejected before runtime mutation. CI and the normal live task never pause, even when presenter environment variables are inherited.

`task testing:demo:present:rehearse` drives all five terminal outcomes against the real composition, verifies status while resources are still running, and requires verified cleanup. It is an acceptance preparation task, not the talk.

Noninteractive acceptance

```
exec bash DOT-Testing/e2e/multi-runtime-demo/test.sh --live
```

Equivalent Task: `task testing:demo:docker-kubernetes-four-contracts:live`. The harness locks the real Minikube address, builds/loads images, creates an isolated namespace and state cache, deploys through `dotx compose apply --locked`, checks readiness, and probes each wire with a 45-second timeout. It verifies recorded status, destroys the composition, removes its namespace, and checks for owned Docker leftovers.

Expect `PASSED` lines for `rest-alerts`, `graphql-health`, `grpc-health`, and `mqtt-alert-delivery`, plus exit zero and `cleanup=PASSED`. Inspect the printed run directory's `resolved-composition.json`, `render.txt`, `status.txt`, and `evidence.json` to explain the locked composition, endpoint materialization and observations. Evidence JSON contains no passwords/tokens or response bodies. Each run retains its evidence; the top-level `evidence.json` is only the latest result, possibly an injected failure.

Rehearse the complete presentation

Present Prometheus first, authenticated Keycloak/APISIX/OAuth second, then the cross-runtime standards example:

```
task testing:demo:dry-run
task testing:demo:mqtt-suback:live
task testing:demo:prepare
task testing:demo:qualify
```

The full rehearsal now repeats seven live paths twice: quickstart, monitoring, Prometheus portability, secure API, OPA upgrade/rollback, OAuth and this fixture. Use the integrated ecosystem route for explicit preparation, preflight and evidence interpretation. `qualify` additionally rehearses the four official terminal inspection pauses and Enter cleanup. The narrower `rehearse` Task retains noninteractive coverage. The quickstart covers default/overridden ports and failure cleanup. OAuth verifies HTTP 401 without a token, token acquisition, protected access, native Secret references, status, and cleanup. The separate MQTT regression uses the actual pinned client and a loopback-only fault-injection peer: delayed, denied, or absent SUBACK, stale retained messages, and incorrect correlated payloads must behave correctly. This peer is not a replacement for the full composition's real Mosquitto acceptance. Its ephemeral credentials and Keycloak development mode are not production defaults. The final mixed-runtime run deliberately fails after deployment: exit 42 and successful cleanup are required. Two further live runs receive actual SIGINT and SIGTERM after deployment and network attachment. They must exit 130/143, record cleanup success, and pass independent namespace/container/network checks. Run those checks separately with `task testing:demo:interruption-cleanup`.

The rehearsal prints one per-run `summary.json` under `DOT-Testing/e2e/target/rehearsal/run-*/`, with a separate log for every step, exit codes, durations, and before/after checkout and CLI fingerprints. Tracked patches and untracked-file content fingerprints are recorded for all three checkouts; no source or credential contents are copied into the receipt. Changed inputs during rehearsal fail acceptance. The convenience `latest.json` points to that per-run receipt, including failures; it is not a green badge. A dirty checkout remains explicitly labelled by its recorded status.

This is pre-presentation acceptance, not the short live talk itself. A per-user live-run lock rejects overlapping instances before deployment, including across checkouts using the same runtime/cache environment. `DOTX_DEMO_LOCK_DIR` is an override for isolated lock tests only; do not override it to run concurrent copies of this composition.

Recover safely

Normal failure, Ctrl-C and termination attempt cleanup. Do not use `kill -9` or delete state while cleanup runs. After interruption, use the exact printed path:

```
RUN_DIR=/absolute/path/printed/by/the/failed/run
(
  test -f "$RUN_DIR/kubeconfig" && test -f "$RUN_DIR/scenario.yaml" || exit 1
  XDG_CACHE_HOME="$RUN_DIR/cache" KUBECONFIG="$RUN_DIR/kubeconfig" \
    dotx compose status "$RUN_DIR/scenario.yaml"
)
```

Keep the saved kubeconfig private; it contains credentials. These commands use the run's frozen Kubernetes context, not whichever context is currently selected in another terminal. If either

required file is missing, stop: do not substitute the default cache/context or guess a namespace. Execute the commands one at a time and stop on an unexpected error.

Only after the original harness has stopped and ownership is confirmed, request cleanup separately (using the same `RUN_DIR`):

```
(
  test -f "$RUN_DIR/kubeconfig" && test -f "$RUN_DIR/scenario.yaml" || exit 1
  XDG_CACHE_HOME="$RUN_DIR/cache" KUBECONFIG="$RUN_DIR/kubeconfig" \
    dotx compose destroy "$RUN_DIR/scenario.yaml"
)
```

Inspect the namespace in that run's `render.txt`; delete only that test-owned namespace if it remains. An apply failure may already have rolled back with no applied state; independently verify resource absence. Never prune Docker, remove Minikube, or delete unrelated volumes. Find the owner of an occupied NodePort before proceeding. Fix Docker user permissions before rerunning. Gateway timeouts require checking node IP, routing/firewall, readiness and Services; resolution does not prove reachability. For Contract download failures, restore authorized access to the declared payloads and exact originating DOTs, or warm the matching verified digest cache online before retrying offline. Bundle-local Contract directories are not a fallback; never bypass SHA-256 checks.

After a manually cancelled GitHub job

The live workflow queues subsequent pushes rather than cancelling a rehearsal mid-cleanup. Manual cancellation or loss of the runner can still interrupt teardown. An occupied-name/network preflight failure is not a request to delete whatever was found, and rerunning the job alone cannot repair those leftovers.

First confirm that no job or local demo still owns the resources. Correlate the cancelled attempt's private run path and timestamps with the container's exact name, image and `dotx.runtime.identity` label, the dedicated network's `dotx.composition.id` label and attached containers, and the namespace's workloads. Inspect mounts before considering removal. If the runner checkout no longer contains the run's state, do not use a new run's cache for recovery: an operator must verify each exact leftover independently. Remove only confirmed test-owned resources, verify their absence, and rerun the failed job on the fixed revision. Leave the shared Minikube network/container, cached images, unrelated volumes and namespace finalizers untouched. Successful recovery is not successful live acceptance; the subsequent workflow must still finish its probes and cleanup.

Catalogue readiness appendix

```
task testing:contracts:readiness
jq '.dots | group_by(.status) | map({status: .[0].status, count: length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

Audited release-qualified snapshot: 74 valid definitions, 29 strict interface-profile passes and 45

gaps. Splitting mixed tool releases changes bundle counts, not product readiness; it does not resolve interface gaps. Use the generated report for the actual checkout. Broker-only and negative fixtures do not necessarily need a business API. Official/source-derived Contracts, source-adapted PoCs, validation-only fixtures, stubs and producer doubles remain distinct evidence levels. See catalogue `CONTRACT-AUDIT.md` and Verification and examples. This demonstration does not promote the entire catalogue to production-ready.

Stateless upgrades and recovery

This optional technical-demo lane demonstrates native `dotx compose upgrade` with the existing external OPA DOT's Kubernetes deployment. The OPA image and DOT ID/version stay unchanged; the **composition version and policy input** change. This is a configuration revision, not an OPA software-version upgrade. There are no copied product descriptors, persistent volumes or application-data migrations in this example.

The focused test graph lives under `DOT-Testing/composition/opa-policy-upgrade/`, not the reusable production catalogue: one revision deliberately contains invalid Rego. Its three composition documents and locks resolve the same canonical `DOT-Repository/DOTs/external/OPA-DOT/1.21.4` bundle. This lane has one DOT and an exposed Anchor, not multiple wired DOTs; use the four-Contract and secure-API lanes for those claims.

Prerequisites and preview

Use the DOT-Design checkout root on Linux amd64, with its exact pinned CLI installed (`task cli:install`), Task, Bash, jq, curl, GNU coreutils, `ss`, Docker, kubectl and a Ready local Linux-amd64 Minikube node. Select context `minikube`. Building that pinned CLI requires Go; follow the contributor installation instructions. Go is not needed by the installed binary during this live lane. The canonical OPA image is digest-pinned; Kubernetes must have it cached or be able to obtain it. Use an isolated demo cluster: this OPA API intentionally has no production authentication policy and must not be exposed publicly.

```
task testing:demo:upgrade:dry-run
```

This checks all three locks and renders their native deployment commands without deploying anything. It does not parse Rego or prove upgrade/rollback acceptance: the invalid policy is intentionally a runtime-negative fixture. The native upgrade diff requires an applied baseline, so the live lane runs it after apply.

Run, inspect, and clean up

```
# Optional overrides; keep them in the same terminal.
export DOTX_UPGRADE_PORT=19081
export DOTX_UPGRADE_NAMESPACE=dotx-opa-upgrade-presentation
task testing:demo:upgrade:present
```

Choose a fresh namespace and a free port; the harness refuses existing resources and does not

adopt them. `DOTX_UPGRADE_ROLLOUT_TIMEOUT` defaults to `90s` per Kubernetes rollout. Allow a few minutes: the broken revision intentionally waits for a rollout failure before native recovery.

The task automatically executes this sequence:

Stage	Native lifecycle operation	Required observation
Baseline `1.0.0`	Locked `compose apply`	A viewer's GET authorization decision is HTTP `200`, `result: true`.
Review	`compose upgrade --dry-run --json`	Exactly one `UPDATE` for the existing OPA instance, with different input digests.
Target `1.1.0`	Locked `compose upgrade` from `1.0.0` and its exact lock	Viewer decision becomes `false`; operator decision remains `true`.
Broken target `1.2.0`	Locked `compose upgrade` from `1.1.0`	The exact negative fixture fails parsing with the running OPA binary; its deployment fails rollout and the native command reports automatic rollback.
Recovered `1.1.0`	Native status plus fresh API requests	Last-known-good version remains `1.1.0`, no upgrade transaction is pending, viewer remains denied and operator remains allowed.

OPA's decision API returns HTTP `200` for both boolean decisions; `false` is an authorization decision, not HTTP `403`. In the separate secure-API composition, APISIX enforces that decision and returns HTTP `403` to its caller.

Say: "A valid DOT and lock do not guarantee valid application configuration. The runtime rejected this policy, and the CLI restored the last verified configuration before reporting failure. We then tested the recovered service."

The normal lane calls native CLI commands directly. It does not implement a second upgrade or rollback engine. Recovery is to the last successful `1.1.0` revision, not an undo of the previously successful promotion from `1.0.0`. The baseline and target locks are required for both changes.

After successful recovery, presenter mode holds the running OPA instance for inspection. It prints the private run directory and a status command using that run's cache and frozen kubeconfig. Inspect `upgrade-diff.json`, redacted `result.json`, API decision receipts and status snapshots. The saved `failed-upgrade.log` contains the expected failed rollout and native recovery; that failure is not silently treated as a successful upgrade. Routine native apply/upgrade output stays in private `apply.log`/`upgrade.log`; the terminal shows ordered progress stages and a final observed-decision table.

Press Enter to clean up. EOF, Ctrl-C and the bounded presenter timeout also invoke owned-resource cleanup. Successful acceptance requires a final `cleanup=PASSED` receipt. For noninteractive rehearsal use:

```
task testing:demo:upgrade:live
```

With Python 3 installed, `task testing:demo:upgrade:rehearse` uses the existing terminal-driven acceptance driver to verify the pause, inspect native status, send Enter and require verified cleanup. It records a summary and before/after checkout fingerprints; it fails if inputs change during rehearsal.

Boundaries and recovery

This proves one stateless native-Kubernetes configuration update and automatic recovery, not database rollback, zero downtime, all deployment adapters, policy distribution security or production certification. The exposed API Contract is the bundle's ecosystem-derived PoC1 normalization, not an official comprehensive OPA OpenAPI document. The live checks verify the actual boolean decisions.

Current upgrade admission rejects changed stateful deployments and changed Helm/Vagrant deployments whose storage safety cannot be established. A pending transaction after interruption requires the native `--resume` or `--rollback` route documented in Recover an interrupted upgrade. Do not rewrite its state file, delete unrelated resources or bypass these checks.

If cleanup is interrupted, inspect the printed run's evidence and use **both** its `XDG_CACHE_HOME` and `KUBECONFIG` for status/destroy. Never substitute a new cache or a changed global cluster context. Do not force namespace finalizers.

The existing **Anchor live E2E** workflow offers an explicit `opa-policy-upgrade` selection with the same harness and private evidence upload. The default PoC `all` matrix remains the established twelve scenarios; this optional technical lane is selected separately. Dry-run CI checks the fixtures through the existing presentation-validation job.

One DOT, two runtimes

This optional presenter lane answers: "Can we reuse the same DOT without copying its product definition when the deployment runtime changes?"

The catalogue composition `prometheus-docker-kubernetes-portability` selects the existing `configuration-docker-compose` and `configuration-kubernetes-manifest` profiles of the **same** Prometheus DOT: ID `Prometheus@3.15.1`, upstream release `3.15.1`. Both resolve its official OpenAPI Contract, use the same digest-pinned upstream image, and declare the same default self-scrape configuration and `GOGC=75`. No product descriptors, Contracts or default configurations are copied for this demonstration.



These are two independent servers with **zero wires**. This demonstrates deployment and configuration portability, not replication, data migration, or cross-runtime communication. Use the four-Contract demo for actual Docker-to-Kubernetes exchanges, and the secure-API composition for authentication and authorization between several products.

Prepare and preview

Run from the DOT-Design checkout root on Linux amd64. Install its exact pinned CLI using `task`

`cli:install`; Go is needed for installation, not by the running binary. Follow the contributor checkout/tool guidance. The released `v0.2.5` binary predates this lane's cross-runtime repetition and restrictive-umask config-mode fixes; the version string alone is insufficient. Requirements: Task, Git, Bash, jq, curl, `ss`, GNU coreutils, Docker with Compose, kubectl and a Ready Linux-amd64 Minikube node with Docker runtime. Select context `minikube`. No Java, Kubernetes cluster provisioning or credentials are needed once those local prerequisites are installed and ready.

```
task testing:demo:portability:dry-run
task testing:demo:portability:prepare
```

The first command validates the reviewed lock and renders the two selected deployments without deploying anything. Temporary default-config files are materialized in an owned preview cache and removed on exit. The second command is **explicit preparation**, not read-only preflight: it pulls only the selected digest-pinned image into Docker and the Minikube node cache. It does not deploy the composition. Preparation requires registry access and does not imply live API acceptance or universally offline operation.

Present the comparison

```
# Optional: choose a fresh namespace and unused Kubernetes forwarding port.
export DOTX_PORTABILITY_NAMESPACE=dotx-prometheus-portability-presentation
export DOTX_PORTABILITY_KUBERNETES_PORT=19083
task testing:demo:portability:present
```

The Compose profile's declared localhost port is `18919`. Both host ports must be unused. Never delete an unknown workload to free them. The task refuses existing namespaces and Compose project resources; it does not adopt them.

The task then:

1. Resolves the same canonical DOT/Contract for both instances and requires the same immutable image pin. It reads the image's own `--version` output in a disposable, network-disabled container to establish the expected API version.
2. Applies both native profiles through `dotx compose apply` and records status.
3. Verifies the actual container/pod image identities, bounded readiness, official build-information version, runtime `GOGC`, default configuration and a real healthy self-scrape on **both** servers.
4. Displays the locked topology, a live comparison table and inspection URLs, then holds the verified services for bounded inspection.
5. Destroys only this run's resources and verifies no Compose containers, networks, volumes, namespace, forwards or composition state remain.

The default selected image currently reports Prometheus `3.15.1`; the assertion derives the version from its pinned bytes rather than assuming that a version label or two matching server responses prove provenance. The runtime receipts record actual image IDs and observation timestamps. The terminal table reports `GOGC=75` and `up=1` for each independently verified server.

Say: "The DOT identity and API Contract stay the same. The selected deployment mechanism, resource identities and endpoint change. That is reuse of one product definition, not two products that happen to have similar names."

While paused, open the displayed localhost URLs at `/targets`, or inspect the redacted `result.json` and each runtime's receipt in the printed private run directory. The printed native status command includes the run's cache and frozen kubeconfig; keep both when inspecting or recovering that run.

Press Enter to clean up. EOF, Ctrl-C and presenter timeout also invoke cleanup, but return a nonzero status. A successful demonstration requires the final `cleanup=PASSED` receipt. Native deployment details stay in `apply.log`; early Compose process failures retain bounded startup diagnostics before cleanup.

For noninteractive acceptance and an automated inspection/Enter rehearsal:

```
task testing:demo:portability:live
# Requires Python 3; also exercises a Kubernetes port override.
task testing:demo:portability:rehearse
```

Storage and acceptance boundaries

Data is disposable. Native Compose destroy removes this test project's named TSDB volume. The selected Kubernetes profile has no persistent volume. This comparison does not move history between servers, prove persistence/backup semantics or certify every Prometheus API and deployment variant.

The CLI admits one native Kubernetes instance of this DOT alongside isolated Docker instances. It still rejects two native Kubernetes instances with fixed object names, including when a Docker instance separates them in the document. Do not bypass that guard or infer arbitrary same-namespace object isolation.

If interrupted, inspect the receipt and use the recorded cache/kubeconfig for native status/destroy on the same composition. Never substitute a new run's cache, delete unrelated resources or strip namespace finalizers. The **Anchor live E2E** workflow offers this lane explicitly as `prometheus-portability`; dry-run CI exercises the same script. Runtime and fixture acceptance remain distinct from production certification.

Contributor guide

This page covers building DOT-Design and running its shared local/GitHub gates. To use an existing DOT without installing development toolchains, start with Getting started.

Prerequisites

Tool	Required for	Project baseline
Java and Maven	`DOT-Domain`, schema generation, and Java tests	Java 27 (`27.0.0+35-open` in `.sdkmanrc`); Maven 3.10.0 from `PATH`
Go	Building and testing `dotx`	Go 1.27.2
Graphviz and fonts	Reproducing the legacy full-Maven-build SVG export, not running services	Local baseline: Graphviz 14.1.2 and Ubuntu `fonts-noto-core` / `fonts-croscore` 20201225-2build1. Missing tools or different font fallbacks can change the SVG even when the generated model source is identical.
Task	Repository command aliases	Optional but recommended
ShellCheck, Python 3, jq, Bash, and Git	Deterministic authoring, conformance, and demo-harness checks	Commands must be available on `PATH`, including to the self-hosted runner service user. ShellCheck is required; lint checks are not silently skipped.
Docker, Kubernetes tools, Helm, or Vagrant with libvirt/KVM	Only the matching deployment adapters and live scenarios	Install only what you intend to execute. The Vagrant workflow uses the runner service user's existing KVM/libvirt permissions, without `sg` or `sudo`.
Antora, the Antora PDF extension, Asciidoctor PDF, and Mermaid CLI	Building the HTML/PDF documentation site, Mermaid diagrams and Ruby script syntax checks	CI uses Node.js 26 and Ruby 4.0; `task docs:check` reports the locally installed tool versions; see Build the documentation

Temporary storage

Keep the checkout and generated build outputs on a disk with sufficient free space. Full Go race/coverage builds and Contract-tool installation can exhaust a small memory-backed `/tmp`, even when individual tests need no deployment.

If a gate reports `disk quota exceeded` or `no space left on device`, check both the checkout filesystem and temporary storage. Set `TMPDIR` to a private, disk-backed directory **outside every Git checkout**, then rerun the failed gate. Negative Git-discovery tests deliberately remove fixture metadata; a parent checkout must not become an accidental fallback. Changing `TMPDIR` alone does not relocate generated files under the checkout's `target` directories.

Retain failed logs and verify the rerun; a storage failure is not a passed test. Do not delete shared temporary files, other runs' resources or user caches.

Check out the repository

```
git clone https://github.com/MARE-EU-Project/DOT-Design.git
cd DOT-Design
git submodule update --init --recursive
```

The source repository and its two submodules may require access granted by the MARE-EU-Project organization. `DOT-CLI` and `DOT-Repository` remain separately cloneable; the generated Schema and high-level Skill classifications are tracked inside `DOT-Repository` and require no additional checkout.

When updating submodules, commit and push the child repositories first, then push the parent branch with a missing-submodule-commit check:

```
git -c push.recurseSubmodules=check -c push.followTags=false push
```

This refuses to publish a parent whose child commits are unavailable remotely. Release tags are published separately after qualification; a branch push must not publish prepared tags implicitly.

Verify the domain model

The domain module uses plain JUnit tests and does not start a service:

```
mvn -pl DOT-Domain test
```

Run the complete Java build with:

```
mvn clean install
# equivalent Task alias
task mci
```

The build generates CycloneDX JSON and XML software bills of materials during the Maven `verify` phase.

The full Maven build also regenerates diagram exports. Release packaging requires clean committed sources, including those exports. If rendering changes a tracked SVG, diagnose the source, renderer and fonts; do not discard the difference blindly or weaken the provenance guard to make packaging pass.

Install and test dotx

`DOT-CLI` is an independent Go module and is not part of the Maven reactor:

DOT-Design `VERSION`, its Maven artifacts and the pinned `DOT-CLI/VERSION` share the same version number. Run `task testing:version:alignment` to verify the checkout. Independent repositories and publication jobs do not imply independent version numbers; source revisions still identify unreleased changes. Antora keeps current development content under `Next`, preserving stable tags.

Install the standalone release with Go 1.27 or later. For the private module, authenticate Git first as described in Getting started and bypass the public Go proxy for this organization in this command only:

```
GOPRIVATE='github.com/MARE-EU-Project/*' go install github.com/MARE-EU-Project/DOT-CLI/cmd/dotx@v0.4.0
dotx version
```

The GitHub release also provides a checksummed Linux-amd64 binary, build metadata, and a manifest that pins the exact DOT-CLI source plus the DOT-Design and DOT-Repository acceptance revisions. For source development or the complete cross-repository suite, build the checked-out submodule instead:

```
task cli:build
task cli:test
task cli:install
dotx --version
```

`task cli:install` installs the pinned checkout into `GOBIN` or `GOPATH/bin` and installs Bash completion. Keep that directory on `PATH`; project tasks, test harnesses, examples, and CI all invoke the same `dotx` command rather than a repository-local binary.

Run GitHub workflow gates locally

For quick feedback while editing, from the **DOT-Design checkout root**:

```
task check
```

This source-read-only check uses existing authorities for version alignment, generated Go/schema drift, Go formatting and `go vet`, and syntax-checks maintained Python, Bash, Node and Ruby scripts without executing their bodies. It includes staged, unstaged and untracked sources, but excludes generated outputs, negative fixture sources and user presentation files. Have the documented toolchains and pinned quicktype/dependency caches prepared first: offline mode fails on a cold cache instead of downloading or installing tools. Temporary files and compiler caches may be used; tracked files are not repaired. This is **not** Java/Go test acceptance, full CI, live rehearsal or release proof.

For focused model/code iterations, start with the deployment-free gate:

```
task testing:models
```

It checks Java validation/serialization and coverage, all public-model corpus examples against generated schemas, Go bindings, CLI race tests and coverage, offline catalogue validation and reviewed locks. It starts no workloads or GitHub workflows. First-use setup may download pinned test tools. A coding-only push does not qualify a release or necessarily publish Antora; `[skip ci]` remains an explicit option for an authorized, locally tested batch, not a replacement for required release checks.

GitHub Actions and local pre-push validation share the root Taskfile test bodies. Actions still provisions clean toolchains and publishes artifacts, but the build, validation, conformance, coverage, runtime, and documentation commands are not CI-only shell fragments.

Ordinary test/report archives are best-effort. If an optional upload fails (including exhausted GitHub storage quota), the workflow emits a warning and job summary; tests, security checks and cleanup retain their original results. This covers any optional upload failure, not just quota errors. A green test run does not guarantee a downloadable archive or release qualification. Release evidence and artifacts consumed by another job or Pages remain mandatory. The optional `dot-docs-site` archive is not used for publication: the documentation workflow publishes the locally verified site directly to the publication branch, and publication failures still fail the job. Run `task testing:ci:artifacts` to check this boundary, including negative regressions. The check also runs in the deterministic workflow gate.

After the Contract-tool bootstrap, inspect the one source/acceptance pin report:

```
task testing:acceptance:pins
# Before pushing repositories that depend on unpublished acceptance commits:
task testing:acceptance:pins -- --remote
```

The report checks bootstrap copies against `DOT-CLI/release/acceptance.json` and the catalogue workflow's CLI authority. It distinguishes current source revisions and dirty state from acceptance snapshot revisions. The optional read-only `gh` check requires repository access and fails if any dependency commit is unavailable on GitHub; it does not publish it. Publish acceptance foundation commits on explicit branches first when necessary, then CLI, catalogue and Design. Before each dependent push, rerun the remote check and retain the missing-submodule-commit push guard above. Equal version labels, available commits and successful declaration checks do not qualify live

APIs.

`task testing:ci:tools` audits existing workflow action pins and tool-version authorities, including missing/late Node setup regressions. It uses the pinned Contract-tool Python environment; bootstrap it through the ordinary conformance gate, not an implicit runner-global package. These checks are included in `task ci:contract-conformance` and the full deterministic gate.

```
task ci:prepare
task ci:deterministic
task ci:compatibility-matrix
```

`ci:prepare` installs the pinned DOT-CLI checkout and verifies that `dotx version` matches the root `VERSION`. `ci:deterministic` runs every deterministic non-live release workflow gate, including Java and Go tests, schema drift, official Contract validation, repository and Anchor conformance, Composition v1, lifecycle recovery, upgrade, all PoC dry-runs, external-DOT rendering, coverage, Antora, and PDF generation. Individual equivalents are available as `ci:build`, `ci:contract-conformance`, `ci:repository-conformance`, `ci:coverage`, and `ci:docs`.

Release package checks require the complete expected set of regular files and exactly one checksum per asset. Extra entries, symlinks, missing or duplicate digests, and changed bytes fail before execution. The Design and CLI checkers share this guard; downloaded Design releases use the same package checker before installation. `task testing:release:channel` exercises the Design package and download-step regressions without network access or deployment; CLI `task ci` also exercises the shared guard. Checksums establish integrity, not publisher trust. The documentation guard also requires a versioned component's installation commands to select that same CLI release. Only Next may retain the selected stable binary while documenting unreleased source features.

Publication regressions can also run alone:

```
bash DOT-Docs/scripts/test-publication.sh
```

These tests create their own release-history fixtures and verify a shallow, tag-free checkout, including concurrent site/badge updates against disposable local Git remotes. Actual versioned-site publication still requires complete release-tag history; its audit rejects missing history rather than silently omitting stable documentation. No runner reconfiguration is needed for these tests.

On push and pull-request events, the live workflows compare the exact Design and pinned child-repository Git trees. Only allowlisted, non-executable documentation changes can deliberately skip their live suites; these are reported as **skipped**, never as successful runtime acceptance. Documentation and conformance remain required. Missing history, changed definitions, schemas, locks, code, workflows, tool configuration, or unknown paths keep full live gates. Manual and scheduled executions always keep those gates. The classifier does not change which local checks you must run for a code change.

Run its deployment-free regressions with:

```
python3 scripts/test-ci-change-scope.py
```

```
ruby DOT-Docs/scripts/test-workflow-impact.rb
```

On a Linux host with all runtime prerequisites, run the mutable workflows too:

```
task ci:live
task ci:all
```

The live gate requires Docker, Minikube, Kubernetes, Helm, Wasmtime, Vagrant/libvirt, writable KVM access, Rust 1.98.1 with the `wasm32-wasip2` target, and network access for the first official WASI HTTP sample build. The shared task fetches the exact pinned `sample-wasi-http-rust` revision into an ignored test cache, builds it with Cargo, and passes it to the runtime test, so local and GitHub executions cannot silently omit the WASI HTTP assertions. Set `WASI_HTTP_COMPONENT` only to reuse an already built component. The live gate performs runtime mutation and cleanup; `ci:deterministic` is the normal pre-push default.

Each self-hosted runner account needs its own working local Minikube profile and tool permissions. The multi-service rehearsal can initialize/select Minikube when no context is selected; it rejects a foreign context or unreadable configuration before starting anything. Missing tools and unavailable profiles remain failures, not skipped acceptance. Use the documented runner-selection variables to route runtime jobs to a properly provisioned account; do not copy another user's kubeconfig or silently adopt its workloads.

For a Vagrant runtime on Ubuntu, install or reconcile the complete provider boundary and then open a new login session so the `libvirt` and `kvm` groups are active:

```
dotx tool install vagrant
dotx tool check vagrant
```

The installer selects `qemu-kvm` when the distribution provides it and falls back to `qemu-system-x86` on releases such as Ubuntu 26.04. The readiness check also verifies the pinned `vagrant-libvirt` plugin, the `qemu:///system` connection, the default libvirt network, and `/dev/kvm` access.

Select GitHub Actions runners

The repository variable `DOT_DESIGN_RUNNER` defaults to `self-hosted` for build, validation, documentation and WebAssembly jobs. Set it to `ubuntu-latest` to use GitHub-hosted runners for those jobs.

- `DOT_DESIGN_CONTAINER_RUNNER` optionally selects a Docker/Compose/Minikube runner; otherwise these jobs inherit `DOT_DESIGN_RUNNER`.
- `DOT_DESIGN_VM_RUNNER` selects the Vagrant/libvirt runner. Configure it only for a label with verified KVM/libvirt access. When unset, the Vagrant job is skipped, not passed; required release acceptance still needs a qualified run.

Restrict access to self-hosted runners, keep their software current, and never expose privileged runners to untrusted forks. Runner administration and service permissions are infrastructure

concerns, not a reason to weaken test assertions.

Maintain dependencies

Dependency versions are managed by the root `pom.xml`. GitHub Actions use immutable commit pins with release comments. Dependabot proposes Maven/Actions updates weekly; updates retain the applicable build, conformance and live gates.

```
task mci       # complete Maven build
task update    # propose latest non-prerelease dependencies
```

Use `task --list` for the available aliases. Dependency updates are intentional source changes, not prerequisites for running an existing DOT.

Clean-code boundaries

Prefer small, behavior-preserving refactorings over a new framework or a repository-wide rewrite. Split by responsibility, not by arbitrary file length. Create a package or interface only when it establishes a real dependency boundary; cohesive files in an existing package are often enough.

Responsibility	Authority and boundary
Portable models	DOT-Domain Java classes, generated JSON Schemas and generated Go inputs. No hand-maintained input-model copies, persistence annotations or runtime adapters.
Portable policy	`DOT-CLI/domain/policy`: in-memory invariants only. Pure domain packages cannot import feature adapters, filesystem/process APIs or network clients.
Input and resolution	Validate raw documents before generated-model decoding. Reuse the existing Contract parsers and verified-payload authority. Resolution may read verified inputs, but must not deploy or adopt runtime state.
Execution	Explicit preparation, runtime mutation, recorded ownership and cleanup. Keep adapter differences visible; preserve the order of preflight, state recording and mutations. A rendering success is not runtime acceptance.
Presentation	Format existing results and diagnostics. Do not resolve compatibility, read Secrets or mutate state inside renderers.
Test fixtures	`internal/testsupport` is test-only. Production dependency checks reject even an indirect import through another production package.

The CLI's composition files separate command dispatch/options, document decoding, DOT/Contract loading, resolution, deployment-input validation, materialization, state/locking, recovery and rendering. Runtime files separately handle selection, forwarding, Helm/Kubernetes configuration

and execution. This is source organization, not a new portable model or a user-facing change.

Review a change

- Use meaningful names: declaration, resolved composition, deployment selection, verified payload and recorded state are different things.
- Prefer guard clauses and one clear responsibility per function. Share a rule only when it has the same meaning, not merely similar syntax.
- Pass external dependencies explicitly where they improve isolation. Materialization accepts invocation-scoped environment lookups and temporary file factories; new tests need not replace a process-wide hook.
- Preserve absent-versus-empty values, native Secret references, error causes and secret-safe command diagnostics. Never print resolved credentials.
- Check write counts as well as errors before publishing a generated file. Failed materialization must preserve the input and clean its partial output.
- Keep characterization, negative and ownership/cleanup regressions. Coverage is a gate, not proof of correct behavior. Existing shared OS hooks still require serial tests; do not add `t.Parallel` around their replacement.
- Put whole-suite scratch storage outside Git and Go module trees: repository discovery tests need genuinely unrelated ancestors. For focused Git-negative controls, bound discovery at the fixture's parent with `GIT_CEILING_DIRECTORIES` and exercise an ignored ancestor repository too.
- Use existing parser/resolver benchmarks before optimizing. Do not introduce caching that bypasses digest re-verification or hides corrupt-cache failures.
- Record a durable architecture decision when a boundary, compatibility promise or ownership policy changes. A routine function move needs no new ADR.

The CLI dependency regressions run in ordinary `go test ./⋯` and `task ci`. Java architecture tests reuse the existing test-only JavaParser dependency to check production imports, qualified types and annotations; they do not add a production dependency or change model serialization. Generated-artifact drift, shared model corpora and reviewed locks remain separate checks. Dependency allowlists require review when adding a domain dependency; they are architecture guards, not a runtime sandbox or a replacement for behavior tests.

For current-source diagnostic parser/resolver baselines, from the **DOT-Design checkout root**:

```
task cli:bench
```

From a **DOT-CLI checkout root** with the matching Design fixtures and catalogue, use `task bench`. Both routes run the existing operation-resolution and monitoring benchmarks with allocation reporting and three samples; the Taskfile owns the benchmark options.

Use the same source fixtures, verified cache, toolchain and otherwise idle host for comparisons. These benchmarks have no machine-dependent pass threshold. Fuzz targets in `authoring_fuzz_test.go` exercise the real four-standard selector and composition parser; seed execution is part of ordinary tests. Longer fuzz campaigns supplement, rather than replace,

regression tests. The recorded clean-code baseline retains all three measured samples and their concurrent-load limitations; it does not claim a speedup from moving declarations between files.

Editor assistance from the domain schemas

After changing DOT-Domain, generate schemas with the Java build, then run these from the DOT-Design checkout:

```
task cli:domain:generate
task cli:domain:check
task cli:domain:generation:test
```

The read-only check covers all generated Go domain bindings, embedded validation schemas, the authoring output projection and Composition artifacts. It reports changed, missing or orphaned generated files without repairing the checkout. Fix drift by regenerating and reviewing it, not by editing generated Go files. The authoring projection derives field names, types and order from the domain schemas/generated bindings; frozen outputs protect readable JSON and existing omission behavior. Node and quicktype remain build-time tools, not dependencies of the installed CLI.

Core model and field descriptions are generated from DOT-Domain's Jackson annotations. Editors can show these descriptions alongside the same types, required fields and exact enum spellings used by validation. Do not edit generated schemas to change model documentation.

For a DOT-Design workspace, a VS Code JSON association can be configured in your local user/workspace settings (not in `dot.json`):

```
{
  "json.schemas": [
    {
      "fileMatch": ["**/dot.json"],
      "url": "./DOT-Repository/Schema/schema.json"
    }
  ],
  "yaml.schemas": {
    "./DOT-Repository/Schema/composition-v1.schema.json": "**/composition.yaml"
  }
}
```

The YAML association requires an editor YAML language extension. In a standalone DOT-Repository checkout, use `./Schema/` instead. Keep associations editor-side: adding `$schema` to a portable Composition is not supported. Editor hints do not replace `dotx validate`, semantic validation, reviewed locks or live acceptance.

Validate a DOT

`dotx validate` reads `dot.json` at the project root, falling back to `dot/dot.json`. The selected file is the unique DOT deployment definition; unreferenced Dockerfiles, Compose files, Kubernetes manifests,

Helm charts, and scripts are not inferred.

```
dotx validate DOT-Repository/DOTs/testing/contract-standards/provider/1.3.0 \
  --schema DOT-Repository/Schema/schema.json
dotx validate DOT-Repository/DOTs/external/Keycloak-DOT/26.8.1 \
  --schema DOT-Repository/Schema/schema.json --profile reusable
dotx inspect DOT-Repository/DOTs/testing/contract-standards/provider/1.3.0 --json
```

Base validation checks the JSON Schema and DOT-level Anchor, Contract, service-port, CDI, and sensitive-default relationships. Every declared Contract must be a complete URL/SHA-256 declaration in the owning `dot.json`. The validator never searches repository-level Contract directories or sibling DOTs as implicit fallbacks. A provider containing only owned declarations can be copied and validated with an explicit schema path and access to its verified payloads. A consumer's external `contract.source` references still need their exact originating DOTs: pass the matching catalogue with `--repository DOT-Repository`. An explicit schema alone cannot supply those definitions; a detached consumer must fail rather than guess an origin. A URL-backed definition declares one structured payload:

```
"payload": {
  "url": "https://raw.githubusercontent.com/vendor/project/v1.2.3/openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

Raw Contract text and the former flat `payloadUrl`/`payloadSha256` shape are rejected. Validation downloads the exact bytes, rejects non-HTTPS or mutable raw-GitHub branch URLs, verifies the digest, parses the declared standard/format, and resolves Anchor selectors. Missing IDs, incomplete declarations, digest mismatches, and unresolved selectors always fail validation. Use `--profile reusable` before publishing a stable reusable DOT. That profile additionally requires at least one pinned Contract, rejects unused Contracts, and requires every deployment to bind each Contract-backed provided Anchor exactly once through a service that declares the same Contract. Explicit contractless Anchors retain base-validation evidence only; they are never silently promoted to Contract-backed interoperability.

For an external product, prefer a complete root Contract declaration pointing to an official versioned vendor raw file. A full Git commit or release-version tag is accepted for `raw.githubusercontent.com`; `main`, `master`, and other mutable branch names are rejected. When no official usable raw machine-readable Contract exists, catalogue declarations may point to a clearly attributed derived/adapter document hosted in a secret GitHub Gist. Use the HTTPS host `gist.githubusercontent.com` with path `owner/gist-id/raw/40-character-revision/filename` and the SHA-256 of the exact raw bytes. Unversioned Gist URLs, HTML Gist pages and Gist query parameters are rejected. A secret Gist is unlisted, not private; anyone with its URL can read it without authentication. The CLI does not send GitHub tokens to Gist downloads. Keep the original source, license and authority in the Gist's `PROVENANCE.json`; hosting does not make a derived document official or prove live acceptance. Existing unresolved license notices remain unresolved.

All current catalogue declarations now include their URL-backed metadata. No local Contract directory is required or used. Compositions select the exact owning DOT with `source:`

`repository://name@release`, retaining the originating Contract ID. Run `dotx validate DOT-Repository` online once, then use `dotx validate DOT-Repository --offline`. The shared digest cache lives under `$XDG_CACHE_HOME/dotx/contracts/sha256/` (normally `~/.cache/` on Linux). Every cache read rechecks SHA-256; a cold offline cache fails without downloads or writes, and corruption is never silently repaired. Payload availability is not producer acceptance. Shared Contract IDs still require identical metadata and bytes; root ID-only declarations cannot fall back to local or sibling files. For private raw GitHub URLs, `dotx` reads `MARE_READ_ONLY_TOKEN`, falling back to `GITHUB_TOKEN`, and sends it only to `raw.githubusercontent.com`; credentials must never appear in `payload.url`. Inspection provides a normalized, machine-readable view without deploying anything.

Validate and inspect a composition

The canonical example connects Keycloak, APISIX, and an OAuth client:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose validate \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --json

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --dry-run
```

All five commands above are non-mutating. `lock --check` verifies the committed content-addressed input graph; `validate` answers whether references and compatibility can be resolved; `inspect` shows the exact instances, endpoints, security status, and materializers; `apply --dry-run` renders supported runtime actions without writing composition state.

Read Validation, resolution, and lifecycle before a live apply.

Verify the authoritative repository

Recursively discover and validate every definition. External/internal paths record ownership; Contract-backed external/internal bundles receive the strict reusable profile:

```
dotx validate DOT-Repository
dotx repository inspect \
  contract-standards-provider@1.3.0 \
  --repository DOT-Repository --json
```

The same command is the required DOT-Repository GitHub Actions gate for every change below

DOTs/ or .gitmodules. The filesystem is authoritative; no separate inventory or validation wrapper is maintained.

Run task testing:contracts:portability to copy providers into isolated temporary directories and validate them offline against the explicit catalogue schema. For consumers, it first requires detached validation to fail with CONTRACT_SOURCE_RESOLUTION, then supplies an isolated catalogue containing only their explicitly declared origins. This is also part of ci:repository-conformance; it detects undeclared file fallbacks without pretending a dependency can resolve in isolation. Each owned Contract carries its complete URL/hash declaration. Offline verification uses the digest cache outside the bundle, never local Contract assets. See external Contract ownership and source references.

Run the composition conformance gate

```
task testing:conformance
```

This gate combines Java domain tests, Go tests, repository-wide schema validation, Contract checks, composition validation/resolution, and dry-run scenario checks. It does not claim that external partner systems or production networks were exercised.

Run only the versioned composition production profile with:

```
task testing:compose:acceptance
```

This gate writes revision-pinned, secret-free JSON evidence below DOT-Testing/e2e/target/.

Build the documentation

Install the documentation toolchain once:

```
task docs:setup
```

Then generate the Antora HTML site and PDF export:

```
task docs:check
task docs:build
```

Generated artifacts are written under DOT-Docs/build/site and are intentionally ignored by Git. To build and serve the site locally:

```
task docs:run
```

Open http://localhost:8000. Do not edit files under DOT-Docs/build; update the AsciiDoc sources

instead.

Mermaid sources are stored as Antora attachments and rendered locally by `task docs:diagrams`. The renderer produces SVG for the HTML site and high-resolution PNG for PDF export; both outputs are committed so a clean checkout can resolve every Antora resource. Each source includes an accessible title and description, and each AsciiDoc image macro supplies alternative text.

`task docs:build` regenerates the diagrams before building either output. The toolchain is version-pinned and does not send architecture content to an external rendering service. Edit the `.mmd` source, never the generated SVG or PNG, then run the full documentation build to verify both renderers.

Where to continue

- Guided demo for a presentation-ready walkthrough from catalogue validation to cross-standard resolution and optional live probes.
- Architecture for system and ownership boundaries.
- DOT Anchors and Composition for the model overview and task-oriented composition guides.
- Verification and examples for evidence levels and MARE PoC scenarios.
- DOT-Domain for Java model details.
- Integration handoff for catalogue and composer implementers.

Architecture

DOT-Design separates reusable component description from deployment-instance topology. That separation allows a DOT to be authored and validated once, then instantiated in multiple compositions without embedding addresses, credentials, or orchestrator-specific connection objects in the DOT aggregate.

The diagrams use a consistent visual language: blue identifies authored or persistent artifacts, teal identifies validation and resolution activities, amber identifies decisions or incomplete evidence, green identifies successful outcomes, and red identifies rejection. Labels and line styles carry the same meaning when the documentation is printed without color; dashed lines indicate optional, indirect, or evidence-only relationships.

System context

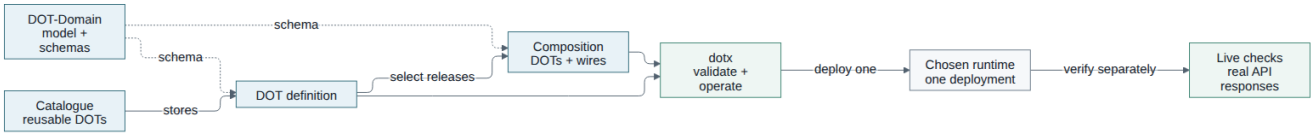


Figure 5. DOT-Design system context and runtime support

`DOT-Repository` is the authoritative catalogue of reusable definitions. `dotx` is the reference validation, resolution, deployment, and composition client. Platform-native systems remain responsible for workload execution and live state.

The runtime portion deliberately distinguishes **where** a workload runs from **how** it is deployed. The domain model supports four `PlatformEngine` values and six compatible manifest mappings:

- Docker accepts `DOCKER_IMAGE` and `DOCKER_COMPOSE`;
- Kubernetes accepts `KUBERNETES_MANIFEST` and `HELM_CHART`;
- Vagrant accepts `VAGRANTFILE`; and
- WebAssembly accepts `WASM_MODULE`.

The current `dotx` lifecycle implements Docker Image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and WebAssembly execution. The WebAssembly adapter uses Wasmtime with an explicit `WASI_CLI` or `WASI_HTTP` world, verifies the module's SHA-256 before execution, and grants only declared guest environment and directory capabilities.

Source-of-truth boundaries

All portable model definitions originate in `DOT-Domain`, including the separate `Composition` aggregate. The existing generation pipeline is Java/Jackson/Jakarta constraints → Victools JSON Schemas → quicktype Go input bindings. `DOT-Repository/Schema/` stores the generated artifacts; the CLI embeds the same Composition schema. There is no independent CLI input model or hand-authored production schema to keep in sync. Resolved compositions, locks and runtime state are executor representations, not alternative domain definitions.

Concern	Authoritative source	Explicitly excluded
DOT identity and supported capability	`dot.json`: `id`, `name`, `version`, Skills, and Contracts	Composition-local instance names and live state
Contract semantics	OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract payload	Repeated protocol, media, schema, or security fields on a contract-backed Anchor
Logical composition point	`Dot.anchors[]`	Hosts, URLs, credentials, placement, health, and policy decisions
Deployable endpoint declaration	`DeploymentService.ports[]` and `anchorBindings[]`	Allocated addresses, forwarded ports, and certificates
Runtime configuration surface	`DeploymentService.config`	Operator-supplied values already applied to one deployment
Multi-DOT topology and readiness	Composition document wires and `dependsOn`	Changes to either participating `dot.json`
Runtime state	Platform-native resources plus private `dotx` composition state	Contract or Anchor metadata

Repository architecture

```
DOT-Design
 ├──── DOT-Domain       Java model and validation
 ├──── DOT-CLI          independent Go CLI and reference executor
 ├──── DOT-Repository   DOTs, high-level Skills, and generated Schema (submodule)
 ├──── DOT-Testing      conformance, composition, interoperability, and E2E evidence
 └──── DOT-Docs         Antora sources, UI, playbooks, and generated diagrams
```

The Maven build contains the `DOT-Domain` Java module. `DOT-CLI` has its own Go module and lifecycle. `DOT-Testing` is deliberately outside both module hierarchies. `DOT-Repository` is the only admitted source for valid deployable DOT definitions used by shared tests. It also owns the generated JSON Schema and the optional high-level Skill classification catalogue, avoiding separate repositories for assets that evolve with the DOT catalogue.

DOT aggregate

`Dot` is the aggregate root. Its primary relationships are:

```
Dot
 ├──── Skills[]                human-readable capabilities
 ├──── Contracts[]             authoritative interaction definitions
 ├──── Anchors[]               stable logical composition points
 └──── Runtimes[]
     ├──── Platform            where execution occurs
     └──── Deployments[]
```

```
        ├──── DeploymentManifest   how deployment occurs
        └──── DeploymentService[]
            ├──── Contract references
            ├──── ServicePort[]
            ├──── AnchorBinding[]
            └──── DeploymentConfig
```

`Skill` and `Contract` are independently identified domain entities. Root `Dot.contracts` carries complete owned Contract declarations for catalogue/CLI publication, while deployment services reference those Contracts by ID only. ID-only root objects remain readable as model-level references; that does not make them complete, publishable declarations or enable a local-asset fallback. Anchors, Runtimes, Deployments, services, bindings, and configuration declarations are embedded values owned by the DOT aggregate.

The class-level `@ValidAnchor` constraint delegates aggregate traversal to `AnchorValidator`. It enforces Anchor identity, Contract membership, selector syntax, and service-port binding consistency without placing cross-reference logic in `Dot`.

Contract, Anchor, binding, and wire

These four concepts form a resolution chain but have different owners:

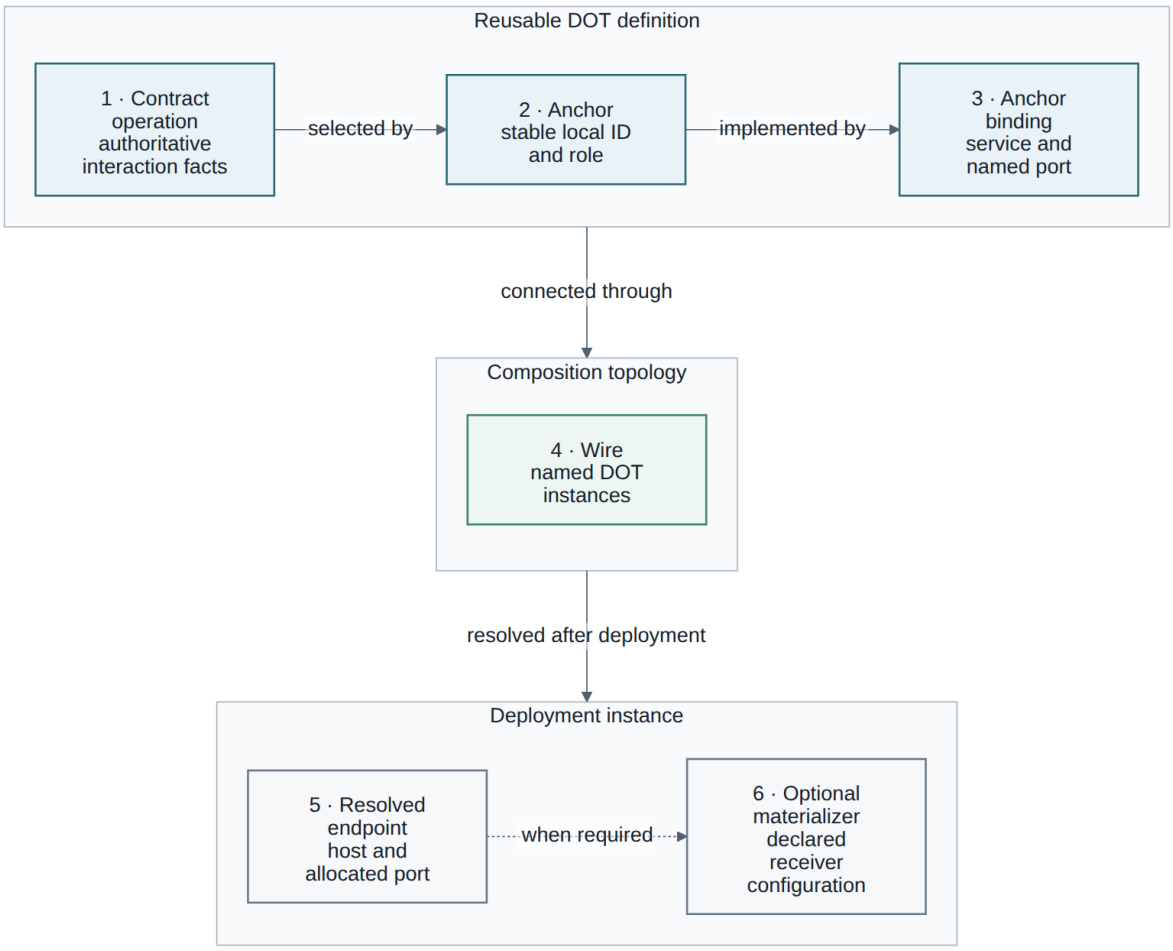


Figure 6. From an authoritative Contract operation to deployed configuration

- The **Contract** defines the interaction.

- The **Anchor** identifies one logical participation point.
- The **binding** says which declared service port implements that Anchor.
- The **wire** connects named deployment instances.
- The optional **materializer** writes a resolved value into configuration that the receiving service has already declared.

See DOT Anchors and Composition for the field model, compatibility rules, and examples.

Runtime and deployment separation

The model separates two orthogonal questions:

Concept	Field	Answers
`PlatformEngine`	`Runtime.platform.engine`	Where does the workload run? For example Docker, Kubernetes, Vagrant, or WebAssembly.
`DeploymentManifestType`	`Deployment.manifest.type`	How is it deployed? For example Docker image, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every manifest type declares its target platform. Runtime validation rejects a deployment whose manifest mechanism does not align with its platform.

`DeploymentService` is the unit that binds a deployed service to Contracts, named ports, Anchors, and supported configuration. Configuration is service-scoped; there is no deployment-level shared configuration surface.

One DOT, several services

Use one DOT when a tool's native Compose project, Kubernetes manifest or Helm chart deploys several cooperating services. List those services in the selected deployment's `services` collection. Each service has its own `config`, ports and Anchor bindings. You do not need a Composition just to describe the tool's internal containers; a Composition selects and connects independently reusable DOT instances.

For example, `api` and `worker` may both declare an environment variable named `MODE` and a file at `/etc/dot/service.json`. These declarations belong to different services and may have different defaults. Within one service, names and target paths must be unique. File defaults use the shared HTTPS URL and SHA-256 `payload`; sensitive files must not supply defaults.

Composition inputs identify the service explicitly. This fragment changes only the worker's environment and the API's file; it does not replace either service's other declarations:

```
deploymentInputs:
  environment:
    - service: worker
```

```
      name: MODE
      value: worker-kubernetes
  configFiles:
    - service: api
      source: ../api-service.json
      target: /etc/dot/service.json
```

The file path is relative to the Composition document. Review the complete service-to-workload example for the selected DOT, native workload mappings and Secret references.

The executor also needs an unambiguous native destination:

- **Docker Compose:** map each logical service to its Compose service key.
- **Kubernetes and Helm:** map each service to labels selecting exactly one composition-owned workload, plus the exact container when it has several. Two services may select different containers in the same Pod. Different selectors that resolve to the same workload/container are rejected before service configuration is patched.

Kubernetes configuration resources and Pod volumes distinguish the runtime instance, service, complete target path and resource kind. Two paths with the same filename must not share a configuration resource. Native Secret references remain operator-owned; destroying a DOT does not delete a referenced Secret.

After updating all selected services, dotx requests one configuration-triggered restart per distinct native workload. Each operation uses a fresh non-secret Pod-template token, so rapid applies do not depend on clock precision. Individual Pod-template patches may also trigger native rollouts; apply is not transactional. Paused Deployments requiring configuration-file restarts are rejected without automatic resume. Containers sharing a Pod also share that workload's restart lifecycle.

Manifest `envFiles` have a different purpose: Docker image defaults use `docker run --env-file`; Compose defaults use `docker compose --env-file` for project interpolation. They are not per-service configuration declarations. Use `services[].config` and service-targeted inputs for that purpose.

These guarantees apply to the supported Compose, Kubernetes and Helm execution profiles. They do not imply independent multi-service WASM execution or automatic wiring of native storage. Keep storage in the native manifest where the selected profile requires it. See Verification and examples for deterministic checks and separate live acceptance tasks.

Lifecycle scope remains narrower than configuration scope. For a custom Compose project, use its recorded runtime name for `dotx stop` / `start`; path-based Compose lifecycle uses the DOT name and may require the manifest's interpolation inputs again. Kubernetes path-based stop/start selects a single service-derived Deployment. Named Kubernetes/Helm stop/start operates on Deployments and restores one replica; it does not manage StatefulSets or DaemonSets. The multi-service acceptance below qualifies apply, status, resume and destroy, not general stateful recovery.

Contract resolution

The reference resolver maps an Anchor selector to an authoritative Contract element:

```
AnchorContract(id, selector)
        |
        v
load Contract by case-sensitive ID
        |
        v
standard-specific resolver
        |
        v
identifier + interaction + protocols + media types
          + input/output schema references + security scheme names
```

Resolution is read-only. Derived facts are returned to consumers and never written back into the Anchor. Contractless Anchors already carry their explicit direction, protocols, and media types and therefore have no selected Contract element.

Composition resolution

The stable Composition v1 executor uses a compile-like pipeline:

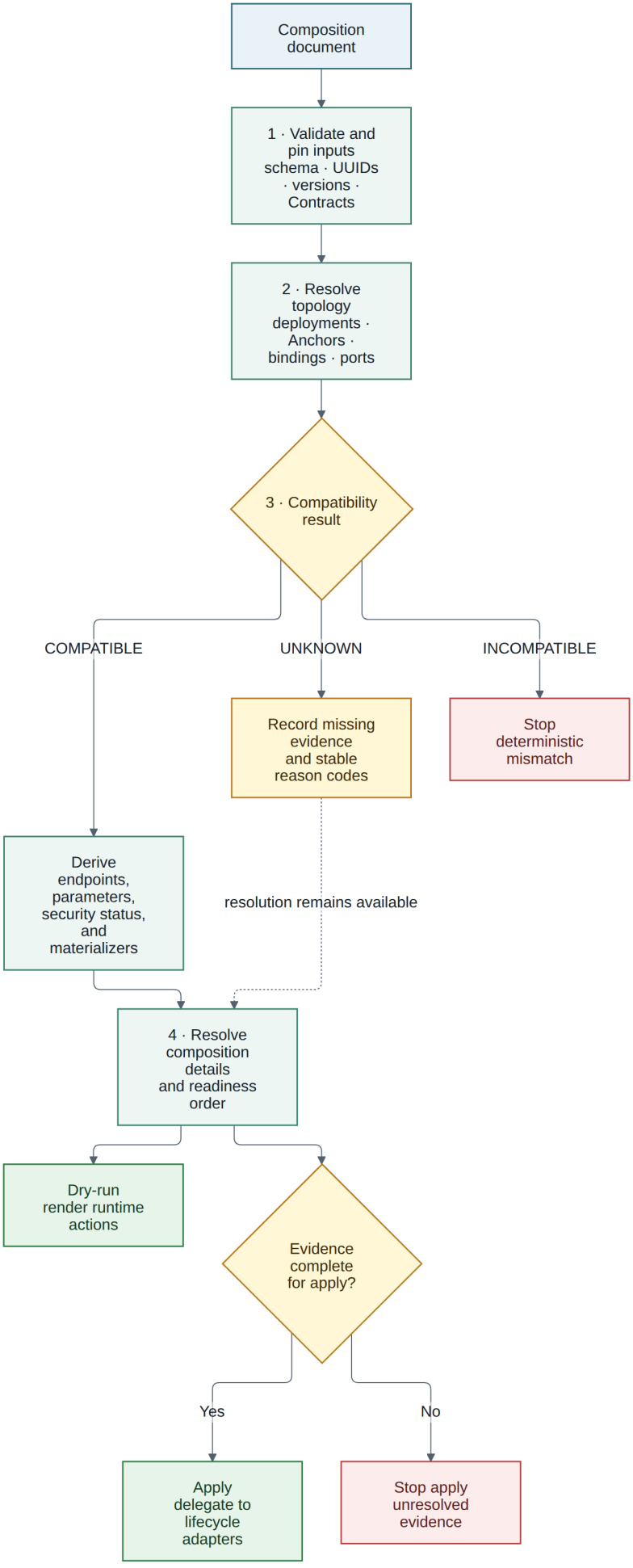


Figure 7. Fail-closed composition resolution pipeline

1. Validate the strict Draft 2020-12 composition shape.
2. Load each local DOT source and verify the pinned textual ID and version.
3. Load each Contract input and verify its case-sensitive textual ID.
4. Resolve exact external Anchor Contract sources without copying their declarations or searching other bundles; lock the originating definitions.
5. Select one deployment for every named DOT instance.
6. Resolve Anchor references and service-port bindings.
7. Evaluate three-state compatibility: `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`.
8. For provider-neutral OIDC requirements, verify the supported discovery profile against the actual selected provider Contract. Exact consumers retain strict same-Contract matching; this is not general OpenAPI substitutability.
9. Derive connection endpoints, explicit cross-platform network ownership, parameter values, security status, and materializers.
10. Build a deterministic resolved composition and readiness order.
11. On apply, prepare configuration and delegate workload operations to the existing single-DOT lifecycle adapters.

Resolution may retain `UNKNOWN` with reason codes so missing evidence is visible. Apply rejects `UNKNOWN` rather than treating absence of evidence as compatibility.

Security boundaries

DOT-Design models declarations and explicit grants; it does not infer trust.

- Contract security requirements remain in the Contract.
- Sensitive deployment values are supplied by reference, not embedded in a composition or persisted as resolved values.
- OCI capabilities, host namespaces, volumes, and CDI devices require exact DOT declarations and exact operator grants.
- A configured wire does not prove credential validity, authorization, remote attestation, policy compliance, or network reachability.
- An `EXTERNAL_GATEWAY` binding records operator ownership and `NONE` cleanup; it does not assert that `dotx` provisioned or verified the gateway.
- Cross-domain admission decisions belong to deployment or policy systems.

Active traffic and trust evidence must be collected by scenario-specific probes after deployment.

Catalogue and API boundaries

`DOT-Repository` versions reusable definitions and its catalogue metadata in Git. `dotx repository` validates and resolves those entries; `dotx compose` provides the executable compatibility reference. A future remote catalogue or API is free to use another language, framework, storage engine, and

transport provided it preserves the domain semantics and passes the consumer acceptance profile.

`dotx compose` stores private, local lifecycle state under the user's cache directory. Per-composition locking prevents concurrent local lifecycle commands from racing that state. The current model is therefore explicitly single-host; shared orchestration state remains future work.

Design principles

- Reuse established Contract standards instead of inventing parallel protocol models.
- Keep stable logical identity separate from runtime location.
- Make adapters, transformations, privileges, and secret access explicit.
- Fail closed on ambiguity and missing security evidence.
- Preserve independently versioned DOT definitions; keep topology in composition.
- Treat generated resolved compositions and test evidence according to what they actually prove.

DOT Anchors and Composition

This guide explains how independently authored Dynamic Open Trust (DOT) components participate in composition by referring to authoritative Contracts rather than duplicating contract-defined facts. It distinguishes the stable DOT Anchor model from the stable, versioned `dotx compose` format and follows the complete path from a logical capability to a deployed connection.

For a technology-neutral consumer boundary, see integration handoff.

Read this first

Keep these three boundaries intact:

1. A **Contract** describes what can be exchanged.
2. An **Anchor** is a named integration point that provides or requires an interface. It can select a Contract operation, express a supported provider-neutral requirement, or declare a contractless boundary.
3. A **Wire** connects compatible Anchors of selected DOT instances and remains outside both reusable DOT definitions. Declaring a Wire does not deploy it.



A structurally valid, deployable DOT is not automatically interoperability-ready. `dotx validate <bundle> --offline --profile interoperable` additionally requires a Contract for every Anchor and a Contract-backed provided Anchor for each exposed service port. The older `reusable` profile preserves mixed evidence during migration. Neither profile discovers undeclared implementation endpoints or replaces live producer acceptance. The catalogue Contract audit records official sources, source-derived internal descriptions, licensing boundaries, and remaining gaps. From DOT-Design, `task testing:contracts:readiness` produces a per-bundle strict-profile report; known gaps remain failures, not an interoperability certification.

If a value changes between installations—for example a host, URL, port, credential, realm, broker, namespace, or device—it does not belong in an Anchor.

If you want to…	Start with
Publish a composable DOT	Authoring two composable DOTs, then Contract-backed Anchors and deployment binding.
Connect existing DOTs	Composition documents and DOT instances, direct request/response wires, and materializers.
Model messaging through a broker	Brokered wires and path and channel parameters.
Handle authentication or secrets	Security and secrets.
Operate a composition	Validation, resolution, and lifecycle and troubleshooting.
Design a new 6G integration type	Anchor boundaries, extension points, and 6G security composition patterns.

Release status and supported profile

The DOT-owned Anchor model, contract selectors, service-port bindings, validation, generated schemas, repository resolution and conformance fixtures are part of the published 0.4.0 baseline. The separate composition document and its local lifecycle executor implement the `dot.mare6g.eu/v1` profile. Use a matching binary/catalogue set; a published format does not certify every catalogue interface or production deployment.

The current executable profile supports:

- local DOT definitions or exact catalogue releases selected by ID/version, with explicitly owned, digest-verified Contract documents;
- OpenAPI, AsyncAPI, Protobuf, GraphQL and supported SPARQL Contract selectors;
- contractless protocol/media declarations when no supported contract exists;
- named deployment selection and service-port bindings;
- explicit environment-variable and config-file materializers;
- explicit externally owned gateway bindings for cross-platform wires, with ownership and no-cleanup semantics recorded in resolved composition, lock, and state;
- Docker Image composition networking and per-instance identity;
- Kubernetes and Helm deployment/readiness through the existing single-DOT lifecycle;
- reference-only process-environment and Kubernetes Secret inputs;
- rollback, private local state, resumable partial apply, status, and reverse-order destroy;
- contract-authorized active probes for safe OpenAPI operations, GraphQL query/mutation/SSE lifecycles, finite side-effect-free unary/client/server/ bidirectional gRPC calls, and correlated AsyncAPI MQTT/NATS/Kafka/AMQP delivery within one Docker network or Kubernetes namespace; and
- fail-closed handling of ambiguity, unknown compatibility, unsupported privilege grants, and unreachable cross-platform materialization.

The authoritative machine-readable boundary is `DOT-Testing/acceptance/composition/v1/profile.json`. Run `task testing:compose:acceptance` to compare the installed CLI help and emitted schema with that profile, verify every committed lock, exercise all twenty-one cross-standard wires and six runtime dry-run adapters, and emit revision-pinned, secret-free JSON evidence.

It is not a general multi-host production orchestrator. Shared state, composition-owned cross-platform gateways or tunnels, external secret-store plugins, multi-scheme authentication, unsafe or unbounded gRPC, AsyncAPI event protocols other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, and cross-platform probe adapters remain outside this profile. Safe anonymous OpenAPI probes, single-scheme OAuth 2.0 client-credentials probes, GraphQL query/mutation/SSE probes, all four finite side-effect-free gRPC shapes, and AsyncAPI MQTT/NATS/Kafka/AMQP delivery probes within a shared Docker network or Kubernetes namespace are supported.

Single-DOT live lifecycle evidence for a runtime adapter is not automatically multi-DOT

composition acceptance. The profile records validation, resolution, dry-run, single-DOT live lifecycle, and composition live lifecycle separately for each adapter.

The five distinct objects

Object	Owner	Responsibility
Contract	DOT author	Authoritative operation, message, payload, protocol, and security description in OpenAPI, AsyncAPI, Protobuf, GraphQL, or current-source SPARQL.
Anchor	DOT author	Stable DOT-local ID and composition role selecting one contract element, or declaring explicit communication facts when no contract exists.
Anchor binding	DOT author	Maps one Anchor ID to a named port of a deployment service. It does not contain a live address.
Wire	Composer/operator	Connects one named DOT instance and Anchor ID to another. It may select a provider, consumer, or broker service-port endpoint.
Materializer	Composer/operator	Explicitly writes a resolved connection value into configuration declared by a participating service.

Where each fact lives

Fact	Source of truth	Referenced by
API operation, message, schemas, protocol, media type, security requirement	Contract payload	`Anchor.contract.selector`
Logical composition point and provider/consumer role	`Dot.anchors[]`	Wire `from` or `to`
Service and named port implementing an Anchor	`DeploymentService.anchorBindings[]`	Resolved service endpoint
Configuration the service permits a deployer to supply	`DeploymentService.config`	Deployment input or materializer target
Selected DOT version, topology, readiness, parameter values, and secret references	Composition document	`dotx compose` inspection and lifecycle

Fact	Source of truth	Referenced by
Concrete runtime address and port	Explicit composition network binding or resolved service endpoint	Materializer or exposed-Anchor forwarding
Live reachability, authentication result, and policy decision	Runtime probe or external policy system	Operational evidence; never persisted as an Anchor fact

The ownership chain is:

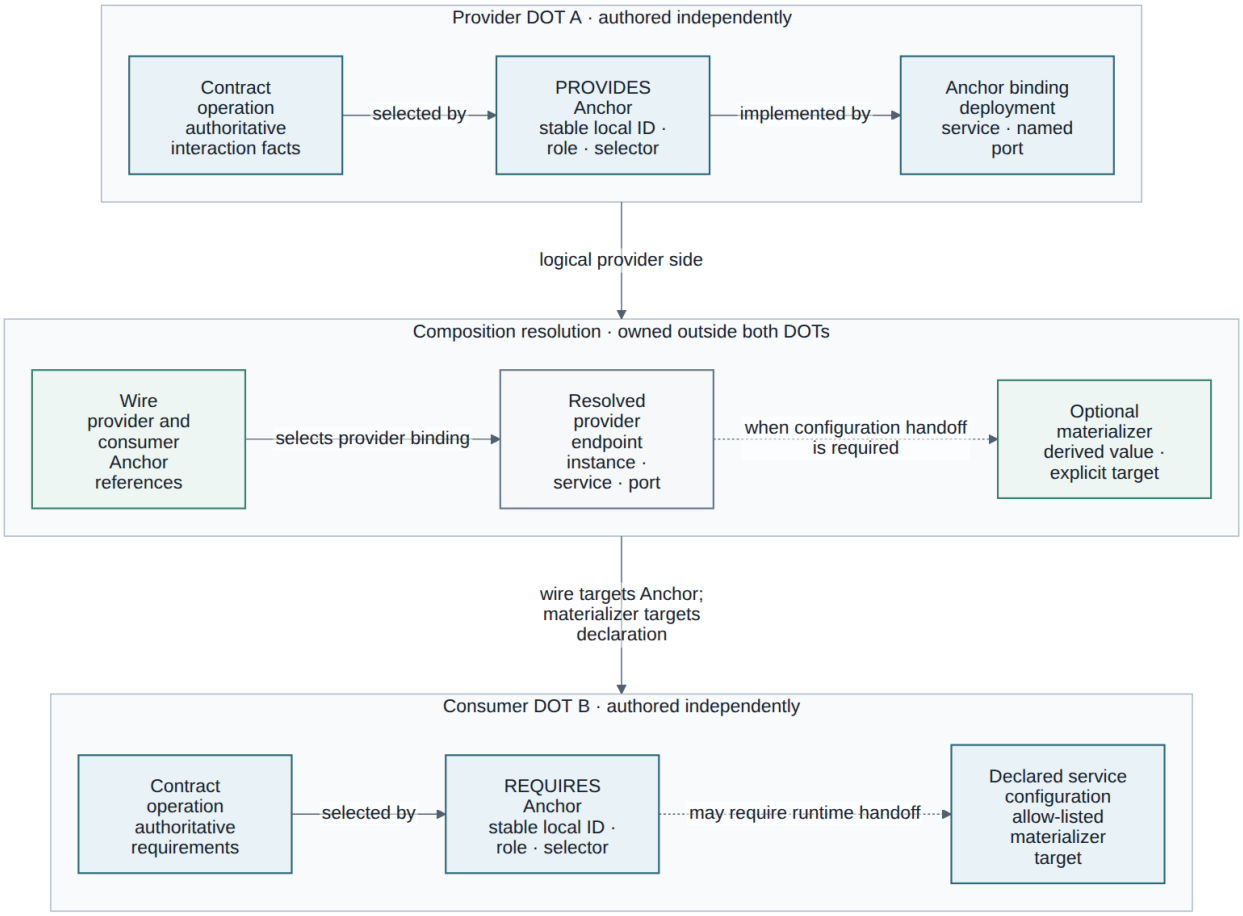


Figure 8. Provider-to-consumer ownership chain

These objects must not collapse into one schema. In particular, a wire is not part of either DOT, and a live URL is not part of an Anchor.

Resolution pipeline

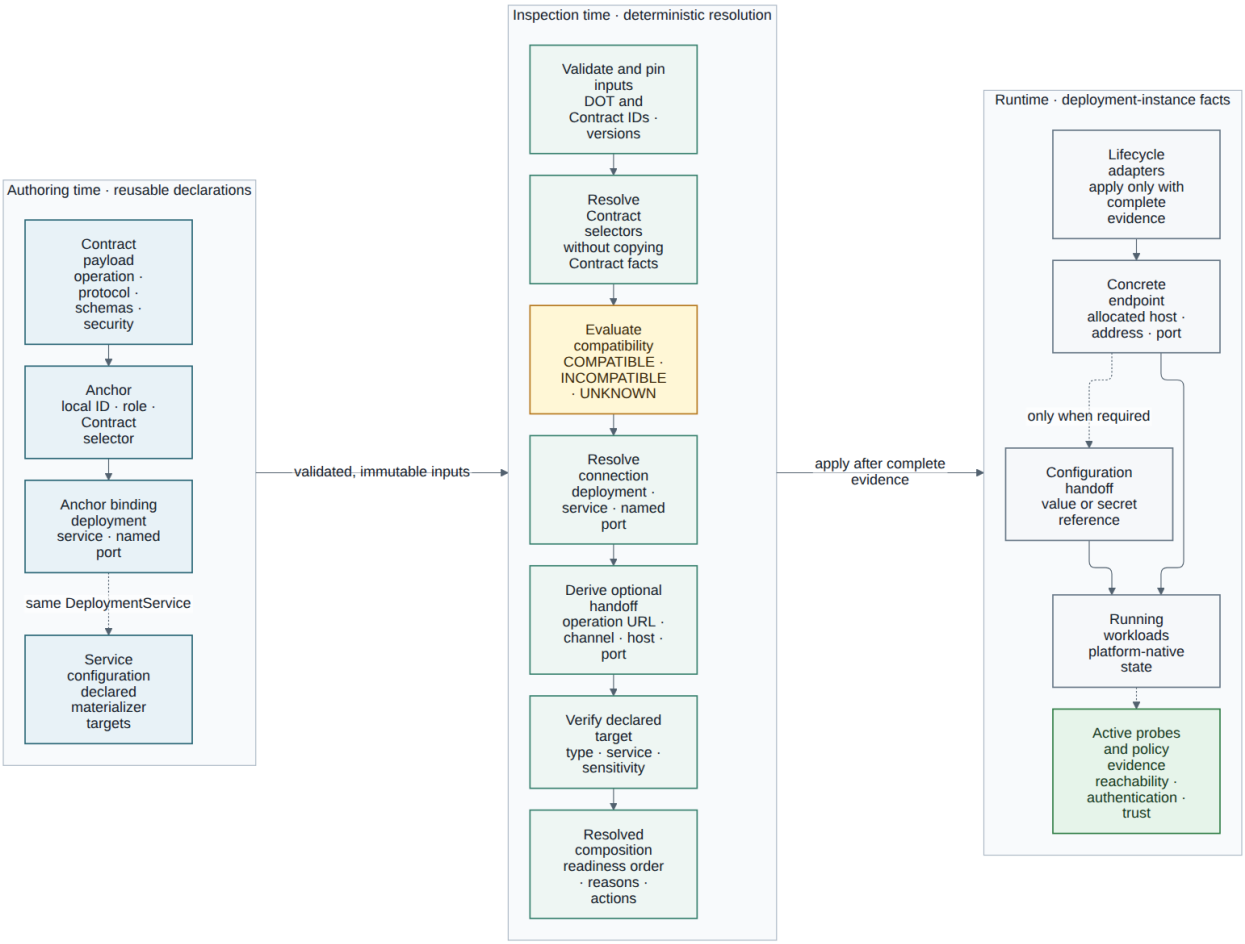


Figure 9. Authoring, resolution, and runtime resolution

This resembles compilation: validate the independently authored inputs, link stable references, resolve deployment details deterministically, and only then mutate runtime state. A failure at one phase is not evidence that a later phase was attempted.

Continue through the workflow

The detailed guide is organized around the work being performed:

Task	Guide
Publish stable composition points in a DOT	Authoring Anchors
Define pinned DOT instances and composition topology	Composition documents and instances
Connect endpoints, provide security inputs, and materialize configuration	Wiring, materialization, and security
Validate, resolved composition, apply, inspect, and evolve a composition	Composition lifecycle and design guidance

Readers implementing a separate composer should also consult the integration handoff.

Authoring Anchors

This page covers the DOT publisher's side of composition. Start with the model overview if the distinction between Contracts, Anchors, bindings, and wires is not yet familiar.

Authoring two composable DOTs

Use this sequence when introducing a new connection:

1. Put the operation or message boundary in an OpenAPI, AsyncAPI, Protobuf, GraphQL, or current-source SPARQL Contract within its supported profile. Use the contractless form only when no supported Contract describes the boundary.
2. Add a stable `PROVIDES` Anchor to the provider DOT and a stable `REQUIRES` Anchor to the consumer DOT. Select the authoritative Contract element from each Anchor; do not copy its protocol or payload fields.
3. On every deployable implementation, bind each locally provided, non-brokered Contract-backed Anchor to the named `ServicePort` implementing its listener. Required interfaces consume configuration, not an invented local listener. Brokered AsyncAPI participants use the composition's broker endpoint; a sender's local service port is not that broker endpoint.
4. Declare every consumer configuration input on its `DeploymentService`, including sensitivity. This declaration is the allow-list for composition.
5. Validate each DOT with `dotx validate` before writing a composition. Supply the matching catalogue with `--repository` for external Contract references; their exact originating DOTs must remain available.
6. Add named, ID/version-pinned DOT instances and ID-pinned Contract inputs to a separate composition document.
7. Connect the Anchor IDs with a wire. Add an explicit broker endpoint, parameter values, security binding, or materializer only when the connection requires it.
8. Run `dotx compose lock` and commit the generated lock beside the composition. Review digest changes as changes to the resolved DOT-to-DOT graph.
9. Run `dotx compose validate` to check the document and `dotx compose inspect` to inspect the resolved topology. Treat `UNKNOWN` compatibility as missing evidence, not as success.
10. Use `apply --dry-run` before execution. After apply, use `compose probe` for safe Contract-derived traffic and `status` for native runtime status, configuration evidence, and the latest redacted probe evidence. Dedicated acceptance tests remain necessary for authentication, authorization, mutable operations, and application-specific outcomes.

The reusable three-DOT worked example is under `DOT-Repository/Compositions/keycloak-apisix-oauth/`: `composition.yaml` owns the wiring and references canonical DOT definitions and their local Contracts; `runtime/` holds non-secret instance inputs. The executable acceptance harness remains in `DOT-Testing/e2e/api-gateway-oidc/`.



New Anchor IDs use explicit DOT-local kebab-case keys such as `build-information`. Composition ownership identifiers still use the canonical lowercase UUIDv4 text

> form defined by RFC 9562. Generate a fresh Composition UUID for a new independently owned graph; do not reuse demo ownership identities. Existing Anchor UUID strings remain readable only when explicitly declared; they are not aliases for the migrated catalogue keys. Catalogue DOT IDs use `<name>@<tool-release>` and Contract IDs use `<origin-DOT.id>::<standard>::<name>`, for example `Keycloak@26.8.1::OpenAPI::Admin`. The ID identifies the interface, not an operation: the Anchor's separate selector chooses the operation/path. Consumers keep the originating Contract ID. Each DOT references at least one meaningful Skill.

Contract-backed Anchors

A contract-backed Anchor carries stable identity and selects one authoritative contract element. It deliberately omits protocol, direction, media type, payload schema, and security fields that the Contract already defines. This excerpt selects Prometheus's real build-information operation; the Anchor key is DOT-local; this excerpt is not a second catalogue definition.

```
{
  "id": "build-information",
  "name": "build-information-http",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

The opposite DOT declares its own `REQUIRES` Anchor. An exact consumer selects the same Contract ID and operation; independently authored Contracts are not automatically interchangeable. AsyncAPI additionally requires complementary operations. Unsupported matches fail rather than being inferred from names.

The selector identifies an operation, not just a data type. Its request and response schemas belong to the verified Contract. OpenAPI parameters, request bodies, responses, media types and their schema references remain in that one authoritative document. A documentation-page URL does not add machine-readable semantics.

Ownership is not consumption

`Dot.contracts` contains this product's authoritative declarations. A consumer references an external interface on its Anchor without copying its Contract or payload. For example, a Prometheus API consumer uses:

```
{
  "id": "read-build-information",
  "name": "read-build-information",
  "role": "REQUIRES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
```

```
    "selector": "#/paths/~1status~1buildinfo/get",
    "source": "repository://Prometheus@3.15.1"
  }
}
```

The exact origin must declare that ID. Validation resolves its verified asset and selector; another bundle's copy cannot rescue a missing origin. Run from the matching catalogue context. A detached consumer with unavailable origins fails; `--offline` never skips these checks. A DOT implementing an externally defined interface may similarly reference its origin on a `PROVIDES` Anchor without claiming authorship. Service Contract bindings describe provided APIs, not every API the service calls.

Qualified root Contract IDs must start with the **exact** owning `Dot.id` plus `::`, including the tool release. For example, `Prometheus@3.15.1` owns `Prometheus@3.15.1::OpenAPI::HTTP`, not the older `Prometheus@3.15.0::OpenAPI::HTTP`. The catalogue uses the official specification from the selected release where available, retaining upstream bytes and SHA-256 pins. A Contract document's API version remains separate from the tool release; upstream `info.version` must not be edited merely to match an ID. Derived projections stay labelled as derived, and a minor-line specification is not exact patch-version acceptance.

Exact consumers must select the originating release actually used by the wire. A reusable consumer can declare distinct optional required Anchors for supported releases; the composition selects one explicitly. This does not create version aliases, infer cross-release compatibility, or upgrade deployed software. After a declaration changes, regenerate and review the affected locks and rerun live acceptance; previous receipts do not qualify the changed graph.

Current Contract inputs use `source: repository://<owner-name>@<release>`. The exact owner supplies the declaration, raw URL and digest. Locks fingerprint that `dot.json`, never a machine-specific cache path. `compose validate`, `inspect` and `lock` accept `--offline` after one online validation has warmed the shared verified cache; use a local pinned catalogue checkout in that mode. Historical wrappers remain readable, not a fallback for missing owner metadata.

Provider-neutral OIDC discovery

APISIX accepts an OIDC provider; its reusable root must not select Keycloak or embed a Keycloak discovery Contract. Its discovery Anchor instead declares:

```
{
  "id": "oidc-discovery",
  "name": "oidc-discovery",
  "role": "REQUIRES",
  "requirement": "OIDC_DISCOVERY_1_0"
}
```

Contract, requirement and explicit protocol/media modes are mutually exclusive. The existing composition wire selects Keycloak's provider Anchor, Contract and operation. The narrow profile verifies a GET configuration endpoint and a JSON HTTP-200 object schema requiring `issuer`, `authorization_endpoint`, `token_endpoint`, `jwks_uri`, `response_types_supported`,

`subject_types_supported` and `id_token_signing_alg_values_supported`, with URI strings and string-array types as appropriate. It conservatively requires token endpoint metadata; unsupported schema shapes fail closed. This is not generic OpenAPI substitutability, full OIDC certification, or live issuer/TLS validation.

`dotx compose inspect --json` reports `requirementProfile` alongside the actual provider Contract. Locks pin external origin definitions and selected payloads. Unwired optional interfaces are not reported as connected. The unused APISIX forwarded-discovery example Anchor was removed. Protected-metadata routes are still explicitly derived, composition-configured examples, not a native APISIX Admin API or unconditional route. Use the matching source CLI; the original released v0.2.5 predates these fields and checks.

Reading compatibility results

The resolver compares the provider and consumer across independently explainable dimensions:

Dimension	Meaning
Role and direction	The connection is from `PROVIDES` to `REQUIRES`, and the selected interaction directions can communicate.
Protocol and media	Resolved protocol and media-type sets have a usable intersection.
Interaction	Request/response, publish/subscribe, streaming, query, mutation, or subscription behavior agrees.
Schema	Ordinary Contract-backed wires require the same authoritative Contract and operation (complementary operations for AsyncAPI). This is interface identity, not a general provider-output/consumer-input schema compatibility algorithm. Different Contract IDs are rejected; matching shapes do not override that rule.
Security	Contract requirements are understood and, for execution, a complete supported binding has all required declared inputs.

`INCOMPATIBLE` means comparable evidence conflicts. `UNKNOWN` means evidence is missing or the current resolver cannot prove the dimension. `validate` and `inspect` may explain `UNKNOWN`; apply rejects it. An explicit mediator DOT is preferable to weakening compatibility when protocols or payloads genuinely differ.

Selector syntax

Standard	Selector	Example
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/operations/publishAlert`
Protobuf	Fully qualified service method	`mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate	`Mutation.publishAlert`

Standard	Selector	Example
SPARQL (0.4.0)	Exact absolute `sd:endpoint` IRI in the W3C Service Description	`http://localhost:7878/query`

For OpenAPI and AsyncAPI, `/` in a JSON Pointer token becomes `~1` and `~` becomes `~0`. Concrete path or channel template values belong to a composition wire's `parameters`, not to the Anchor.

Supported Contract representation matrix

Standard	Contract `format`	Authoritative interaction	Composition evidence
OpenAPI	`YAML` or `JSON`	Request/response operation, including path and security requirements	One shared selector; `REQUEST_RESPONSE` for both participants
AsyncAPI	`YAML` or `JSON`	Message operation and Channel Object	Distinct complementary selectors; `SEND` and `RECEIVE`, plus the resolved channel address
Protocol Buffers (Proto3; including gRPC service declarations)	`PROTOBUF`	Fully qualified RPC and unary or streaming shape	One shared method selector; `REQUEST_RESPONSE`, `CLIENT_STREAM`, `SERVER_STREAM`, or `BIDIRECTIONAL_STREAM`
GraphQL	`GRAPHQL`	SDL root field	One shared schema coordinate; `QUERY`, `MUTATION`, or `SUBSCRIPTION`
SPARQL 1.1 (0.4.0)	`N_TRIPLES`	W3C Service Description query endpoint advertising SPARQL 1.1 Query and SPARQL Results JSON	One exact endpoint IRI selector; `QUERY` for both participants

`dotx compose inspect --json` exposes the resolved `contractStandard`, `contractFormat`, `providerInteraction`, and `consumerInteraction` on each contract-backed wire. These fields are derived from the supplied Contract and therefore do not duplicate authoring metadata in either Anchor or composition. The deterministic `DOT-Testing/composition/contract-standards/` matrix connects provider and consumer DOTs through the established standard profiles; native query/metrics regressions qualify their own bounded source additions.

Native query and metrics profiles in 0.4.0

These additions are supported in **0.4.0**. Use matching Java-derived schemas, CLI and catalogue definitions and record exact revisions, modified state and binary digest. They do not change the published v0.3.0 binary or establish live production acceptance.

`SPARQL` is a canonical Contract standard, with `format: N_TRIPLES` and `version: "1.1"`. The pinned payload is a W3C Service Description; the selected service must advertise `sd:SPARQL11Query` and `http://www.w3.org/ns/formats/SPARQL_Results_JSON`. A selector is its exact absolute `sd:endpoint` IRI,

not the HTTPS payload URL, a blank-node label or a query string. Duplicate identical triples have RDF set semantics; conflicting or ambiguous endpoints fail. The parser accepts at most 1 MiB and 4096 unique triples, rejects named graphs and relative identifiers, and performs no remote RDF loading or inference.

The descriptor's escaped endpoint path is combined with the runtime service binding; its authority never selects a host to contact. Compatibility requires the same originating Contract ID and endpoint selector. This establishes only the declared query/result capability: dataset contents, arbitrary query obligations, updates, entailment, extensions and authentication remain outside the profile. A query-and-update service can expose its query capability; an update-only endpoint cannot satisfy it.

OpenMetrics is an HTTP response format under an ordinary `OpenAPI` Contract, not another interface-description language or enum value. Describe the actual successful operation response as either `application/openmetrics-text; version=1.0.0; charset=utf-8` or `text/plain; version=0.0.4; charset=utf-8` for Prometheus text. Contract.version continues to identify the OpenAPI document version. Error/default-only media declarations do not activate native metrics validation. Neither representation proves metric names, labels, units, application queries or physical host access.

The current Node Exporter 1.12.1 and cAdvisor 0.60.6 handlers serve Prometheus text 0.0.4. Their OpenAPI envelopes are source-derived projections of those handlers, not vendor-published specifications or evidence of OpenMetrics negotiation. Oxigraph 0.5.13 uses the unchanged Service Description generated by its official image; that runtime capture is not a static vendor URL. Preserve these provenance distinctions when reviewing payload pins or reusing the catalogue. GraphDB's configured repository/query boundary remains an explicit gap until qualified independently.

Operation identity and compatibility

Contract-backed Anchors select operations from a digest-verified document. Request and response schemas remain in that document; a schema pointer or an HTML documentation link cannot replace an operation selector. There is no separate ontology-URI matching field.

The current resolver checks selected interface identity, not arbitrary schema substitutability or automatic composition between independently authored APIs. Even identical JSON shapes can carry different meanings, such as Celsius and Fahrenheit. Structural matching and semantic-policy reasoning remain future work; runtime probes provide separate, bounded live evidence.

Contractless Anchors

Use a contractless Anchor only when no supported authoritative contract exists, for example a raw packet stream or a legacy metrics boundary. It must state the facts that cannot otherwise be derived:

```
{
  "id": "packet-stream",
  "name": "packet-stream",
  "role": "PROVIDES",
  "direction": "EGRESS",
```

```
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

Explicit contractless facts describe only the declared communication boundary; they provide no verified operation or payload schema. Keep these gaps visible in the catalogue readiness report.

Contract-backed and contractless forms are mutually exclusive. A contract-backed Anchor must not repeat `direction`, `protocols`, or `mediaTypes`. A contractless Anchor must declare all three.

Role is not traffic direction

`role` answers who offers the selected capability:

- `PROVIDES` means this DOT makes the capability available;
- `REQUIRES` means this DOT depends on that capability.

It does not mean that bytes move in only one direction. An OpenAPI request/response operation has a providing server and requiring client while both send data. An AsyncAPI provider may select a `send` operation and its consumer a complementary `receive` operation. For a contract-backed Anchor, the resolver derives that interaction from the selected operation. Only a contractless Anchor states `INGRESS`, `EGRESS`, or `BIDIRECTIONAL` explicitly.

When two DOTs implement the same published Contract, referencing the same Contract ID is required by ordinary Contract-backed Wire matching. Different Contract IDs are rejected even if names or document shapes look identical. Arbitrary structural substitutability across Contract identities is not an implemented matching mode. Supported provider-neutral requirements have their own explicit rules; they are not a fallback for a rejected identity.

Stable identity and version evolution

- The exact `id`, not `name`, identifies an Anchor in a wire. New catalogue IDs use DOT-local kebab-case keys such as `search-api`.
- IDs are unique within their owning DOT. Composition references pair `instance` with `anchorId`; different DOTs may use the same local key.
- A display-name rename or documentation correction preserves the ID. Do not derive an existing ID again from its current name.
- A compatible DOT version preserves the key for the same logical Anchor.
- An incompatible semantic replacement receives a new ID.
- Legacy UUID strings remain valid explicit keys, not aliases for migrated catalogue IDs. Regenerate and review affected locks when migrating references.
- Removal and any future `deprecated`/`replacedBy` metadata are publishing governance decisions; they are not inferred from contract changes.
- Names remain useful in logs and displays but are never foreign keys.

New keys follow `^[a-z](-[a-z0-9]+)$`: no spaces, uppercase, underscores, repeated hyphens or

path/fragment separators. The generated model also admits legacy UUID strings. Neither mode permits whitespace normalization. Contract identity, selector, protocol, role, tool release and deployment address remain separate; none is concatenated into the local Anchor key.

Binding an Anchor to a deployment

An Anchor is logical until a selected deployment maps it to a named service port:

```
{
  "id": "security-function",
  "name": "Security Function",
  "host": "security-function",
  "ports": [
    {"name": "http", "protocol": "HTTP", "port": 8080}
  ],
  "anchorBindings": [
    {
      "anchorId": "build-information",
      "port": "http"
    }
  ],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": []
  }
}
```

This says which logical service and named port implement the Anchor. The deployer still decides the concrete container DNS name, cluster Service, allocated address, forwarded port, certificate, or network path.

`@ValidAnchor` and `AnchorValidator` reject duplicate Anchor IDs, undeclared Contract references, invalid selector syntax, undeclared ports, undeclared Anchor bindings, and duplicate service-local bindings.

Runtime configuration portability

Every `DeploymentService` declares its supported configuration surface through `environment`, `configFiles`, and logical `volumes`. A composition may supply only values whose names or target paths the selected service declares. Required environment variables and files without defaults fail before runtime mutation; sensitive environment values require a `secretRef` instead of a literal. When an input is omitted, `dotx` reuses the non-sensitive `defaultValue` or `payload` declared by that service. Explicit composition values and wire materializers override defaults; conflicting explicit inputs fail closed. Declarations marked `sensitive` are forbidden from embedding either kind of default and must use a deployment-time secret reference or runtime-native secret facility. Dry runs print resolved environment values in stable name order for every adapter, but replace every value declared sensitive with `<redacted>`.

Each composition input carries the stable `service` id it targets. Authors may omit `service` only when exactly one service in the selected deployment declares that environment name, file path, or volume name; `dotx` resolves and records the service explicitly. If two services declare the same item, omission is an error. Duplicate detection is service-local, so two services can intentionally receive different values for the same environment-variable name.

Environment variables and configuration files are rendered by every currently executable `dotx` deployment adapter:

Manifest	Environment	Configuration files
`DOCKER_IMAGE`	Container environment	Read-only bind mounts
`DOCKER_COMPOSE`	Per-service override using process-environment staging	Generated per-service Compose override mounts
`KUBERNETES_MANIFEST`	Exact workload/container patch; existing Secret keys remain `secretKeyRef`	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`HELM_CHART`	Exact post-install workload/container patch; existing Secret keys remain native	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`VAGRANTFILE`	Vagrant process/provisioner environment	Files provisioned into the managed VM and exposed to the Vagrantfile
`WASM_MODULE`	Allow-listed WASI guest environment; sensitive values inherited by name	Staged configuration root plus explicit Wasmtime directory preopens

Configuration remains service-scoped. A Docker image or Vagrant deployment is one workload or VM, so several logical services may safely share its declared configuration surface. Docker Compose, Kubernetes, and Helm may materialize several independent workloads. Their composition therefore supplies `deploymentInputs.workloads`: a Compose binding names one native service; a Kubernetes/Helm binding supplies exact native labels and optionally a container. Dotx intersects those labels with its composition-owned instance label and requires exactly one workload. Missing, duplicate, cross-runtime, ambiguous workload, and ambiguous container bindings fail closed. This prevents environment values, secrets, or mounted files from reaching the wrong security function without adding platform-specific selectors to the reusable DOT model.

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Storage and privilege configuration is deliberately runtime-native. Docker image deployments support composition-scoped named volumes, explicit OCI bind sources, exact capability/host-namespace grants, and CNCF CDI devices. Compose payloads own their volume graph; Kubernetes and Helm own claims, storage classes, and volume topology; Vagrantfiles own disks and synced folders. A composer must not translate a host bind path into a cluster volume or silently grant privileged mode.

`WASM_MODULE` uses a reviewed Wasmtime/WASI profile. Each module is SHA-256 pinned and loaded from a relative/file or HTTPS URI. `WASI_CLI` executes command modules/components; `WASI_HTTP` serves a component on one declared TCP/HTTP service port bound to loopback. Dotx exposes only declared environment values, staged configuration files, and explicitly bound host directories. It does not translate OCI capabilities, host namespaces, or CDI devices into broader WASI permissions. Sensitive values are never printed or persisted. Because the Wasmtime CLI cannot enforce read-only permissions on a directory preopen, that request fails closed unless the host filesystem already supplies the boundary. The deterministic `DOT-Testing/composition/runtime-configuration/` matrix exercises every executable adapter twice: once using only declared non-sensitive defaults and once using explicit composition overrides. It checks the materialized default configuration document, the resolved service-scoped resolved composition, and the runtime-specific dry-run renderer. The five native cases reuse the external Prometheus DOT; WASI retains a separate conformance fixture. The real Prometheus acceptance runner verifies loaded configuration and environment through its official status APIs, PromQL results and invalid-query responses, stop/start, Git acquisition, and failure cleanup:

```
task testing:compose:runtime-configuration
task testing:compose:runtime-configuration:live RUNTIME=docker-image
```

Select `docker-compose`, `kubernetes-manifest`, `helm-chart`, or `vagrantfile` for the other native mechanisms. These are ephemeral single-server examples, not production monitoring configurations. The `DOT-Testing/composition/service-workloads/` matrix verifies isolation between two services across Compose, Kubernetes, and Helm dry-run adapters.

Discovering and resolving Anchors

The authoritative test and reference definitions are stored in `DOT-Repository` and addressed as `repository://<name>@<version>` for qualified IDs (legacy unqualified IDs use `repository://<id>@<version>`). Consumers can inspect one immutable definition or validate the complete repository:

```
dotx repository inspect Prometheus@3.15.1 --repository DOT-Repository --json
dotx validate DOT-Repository
```

`dotx compose validate` and `dotx compose inspect --json` resolve Anchor IDs, service-port bindings, and selected Contract elements conservatively. The resolved composition reports `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`; missing evidence is never treated as a match. Contract payloads remain authoritative and are not copied into Anchors or composition wires.

This command boundary is the reference behavior, not a requirement to embed dotx. Another

catalogue or composer may use any language, framework, storage, or API shape if it preserves the same identities, fail-closed resolution, and ownership boundaries. See integration handoff.

Next, define the concrete participants and topology in a composition document.

Composition Documents and DOT Instances

A composition document is owned by the composer or operator, not by any member DOT. It selects immutable DOT identities and creates deployment instances whose topology may vary between environments.

Reusable composition catalogue

`DOT-Repository/Compositions/` distributes reusable multi-DOT graphs, reviewed locks, READMEs, and non-secret deployment inputs. Start with `keycloak-apisix-oauth/` (Keycloak, APISIX, OAuth client) or `keycloak-prometheus-openapi-validation/` (official Keycloak and Prometheus OpenAPI operations). Each directory has `composition.yaml` and `composition.lock.json`.

From the standalone DOT-Repository checkout root:

```
dotx validate . --offline
composition=Compositions/keycloak-apisix-oauth/composition.yaml
dotx compose inspect "$composition" --locked --json
dotx compose apply "$composition" --locked --dry-run
```

Repository-wide composition validation requires the current source-built CLI; the released 0.2.4 binary predates this gate. It recursively checks declarations, semantic resolved compositions, adjacent locks, and READMEs without deploying or resolving Secrets. `task compositions:render` additionally renders every catalogue composition. There is no separate composition inventory or Scenario domain type.

A scenario is the broader use-case narrative and acceptance procedure described by the README. Synthetic matrices, negative controls, source-adapted PoC fixtures, executable harnesses, and evidence remain in DOT-Design's `DOT-Testing/`. See Getting started for the gateway's three Wires and Guided demo for live acceptance. Catalogue inclusion or a valid resolved composition never implies production or live producer acceptance.

Composition documents and DOT instances

A composition is a separate versioned document. It names concrete instances of DOT definitions and pins their identities through the authoritative `DOT-Repository` catalogue. The `repository://<dot-id>@<version>` form is stable across checkout locations and prevents a scenario-local copy from drifting away from the reviewed definition:

This structural excerpt omits the required instance inputs and wires. Use `DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml` for the complete runnable document; Contract paths below are relative to that directory.

```
apiVersion: dot.mare6g.eu/v1
kind: Composition
id: e3000000-0000-4000-8000-000000000001
name: keycloak-apisix-oauth
version: 1.0.0
dots:
  - name: identity
    source: repository://Keycloak@26.8.1
    dotId: Keycloak@26.8.1
    dotVersion: 26.8.1
    deployment: kubernetes
  - name: gateway
    source: repository://APISIX@3.19.0
    dotId: APISIX@3.19.0
    dotVersion: 3.19.0
    deployment: kubernetes
    dependsOn: [identity]
contracts:
  - id: Keycloak@26.8.1::OpenAPI::Discovery
    source: repository://Keycloak@26.8.1
```

`name` identifies an instance inside this composition. `source` selects the repository definition by ID and exact version. `dotx compose` locates the nearest `DOT-Repository` containing a `DOTs/` directory, discovers its definitions recursively, resolves the ID/version to its reviewed bundle, and then verifies that `dotId` and `dotVersion` match the loaded `dot.json`. Neither a repository definition nor a local path can silently substitute a different DOT. `deployment` selects a deployment by its stable deployment ID or unambiguous manifest alias.

Local discovery is the convenience default. `--repository <path-or-Git-URL>` selects another authoritative catalogue and `--git-ref <revision>` selects a Git revision. Locked and runtime-mutating commands require a full 40-character commit SHA for remote repositories; branch and tag names are allowed only for unlocked, read-only exploration. Exact commits are cached in a private user cache by normalized repository identity and commit, so the same resolved composition and lock can be reproduced offline. Resolved compositions, locks, and lifecycle state record only the credential-free identity and resolved commit. Credential-bearing URLs are rejected; authentication remains external through Git credential helpers, SSH agents, or environment-backed CI credentials.

Relative paths remain supported for authoring experiments and external Contract documents. Reusable stable, sample, and test-only DOT definitions belong in `DOT-Repository`, under their corresponding catalogue scope. Contracts remain authoritative protocol documents and are referenced independently; placing a DOT in the repository does not copy OpenAPI, AsyncAPI, Protobuf, or GraphQL facts into its Anchors.

`dependsOn` is a readiness graph, not a connection graph. A wire does not imply startup order. This distinction permits reciprocal peer wires and brokered topologies without creating false dependency cycles.

Composition field reference

The emitted Draft 2020-12 JSON Schema is normative for the serialized `v1` shape. This table is the study guide for its top-level ownership:

Field	Required	Meaning
`apiVersion`, `kind`	Yes	Fixed discriminators: `dot.mare6g.eu/v1` and `Composition`.
`id`	Yes	Stable UUID of this composition, also used to derive local state and runtime identities.
`name`, `version`	Yes	Human-readable identity and version of the composition document.
`dots[]`	Yes, at least one	Named deployment instances. Every entry pins `source`, `dotId`, and `dotVersion`; published scenarios use an exact `repository://<name>@<tool-release>` source. `deployment`, `dependsOn`, and `deploymentInputs` refine execution.
`contracts[]`	When a referenced Contract is needed	Maps an exact Contract ID to its originating DOT, normally selected by `source: repository://<name>@<tool-release>`. The resolver verifies the matching complete owned declaration and its URL/hash payload. Explicit local full Contract wrappers remain a historical compatibility mode, not a raw OpenAPI file or an implicit adjacent-file fallback.
`wires[]`	No	Connections between one `PROVIDES` and one `REQUIRES` Anchor. A composition with no wires can still expose a standalone capability.
`exposedAnchors[]`	No	Composition-local names for member Anchors that an operator may expose or forward.

Each wire requires `name`, `from`, and `to`. Both ends contain an instance name and Anchor ID. Optional `endpoint` selects a third-party or otherwise non-default service port; `parameters` supplies only Contract-declared template values; `materializer` or `materializers` writes resolved values; and `security` binds Contract security requirements to declared participant configuration. Unknown fields are rejected.

Deployment inputs and native workload bindings

`deploymentInputs.environment`, `configFiles`, and `volumes` identify the logical `DeploymentService` that owns each input. `ociRuntime` and `cdiDevices` authorize only requirements already declared by that selected DOT deployment. These fields supply instance values and grants; they do not expand the DOT's declared configuration surface.

When a Docker Compose, Kubernetes, or Helm deployment contains several logical services with configuration, `deploymentInputs.workloads` is required for every configured service. The binding is deliberately instance-owned because native service names, labels, and containers can differ between deployment variants or environments:

```
deploymentInputs:
  workloads:
    - service: api
      compose:
        service: backend
```

The Kubernetes/Helm form uses exact-match standard labels and may select one container:

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Labels are combined with dotx's instance ownership selector. A live apply must resolve exactly one Deployment, StatefulSet, or DaemonSet, and an omitted container is accepted only when that workload has one container. Two logical services cannot target the same Compose service or the same Kubernetes workload/container pair. Resolved compositions expose the binding and secret identifiers, and locks cryptographically pin them; neither contains resolved secret values.

DOT definition versus DOT instance

One pinned DOT definition can appear more than once in a composition. Each entry has a distinct composition-local `name`, runtime identity, deployment inputs, and wires while retaining the same `dotId` and `dotVersion`:

```
dots:
  - name: site-a
    source: ./federation-agent
    dotId: poc11-federation-agent@0.0.0-source.g303b353
    dotVersion: 0.0.0-source.g303b353
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-a}
  - name: site-b
    source: ./federation-agent
    dotId: poc11-federation-agent@0.0.0-source.g303b353
    dotVersion: 0.0.0-source.g303b353
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-b}
```

Wires reference `site-a` or `site-b`, never the source path. Docker Image deployments receive

deterministic per-instance container, DNS, network, and named-volume identities. Apply rejects repeated instances for deployment mechanisms whose manifest-owned resource names cannot yet be isolated safely. One native Kubernetes instance may coexist with isolated Docker Image/Compose instances of that same DOT because they occupy separate resource scopes. Two native Kubernetes instances of the same DOT remain rejected, including when a Docker instance separates them in document order. See the Prometheus portability demonstration.

Changing a DOT's version-specific ID does not isolate fixed resource names or ports. Apply and upgrade retain exact-ID collision checks and also check name-version product families, including fixed-port WASI HTTP instances. This conservative guard is not an identifier alias and does not change locks.

The executable example is `DOT-Testing/composition/federation-peers/scenario.yaml`.

After defining the participants, wire and configure their selected Anchors.

Wiring, Materialization, and Security

Wires express composition topology outside every participating DOT. They refer to stable Anchor IDs, resolve deployable endpoints through service bindings, and materialize only values that the target service has declared as configurable.

Direct request/response wire

A wire uses instance names and stable Anchor IDs:

```
wires:
  - name: oauth-token-to-client
    parameters:
      realm: {value: dotx}
    from:
      instance: identity
      anchorId: client-credentials-token
    to:
      instance: client
      anchorId: oauth-token
    materializer:
      type: ENVIRONMENT_VARIABLE
      valueFrom: OPERATION_URL
      target:
        instance: client
        service: oauth-client
        environmentVariable: TOKEN_URL
```

Composition resolution performs the following sequence:

1. Resolve the two pinned DOT instances.
2. Resolve both Anchor IDs and require complementary `PROVIDES`/`REQUIRES` roles.

3. Resolve the selected Contract elements.
4. Check selected interface identity and supported interaction, protocol, media and security requirements; do not infer arbitrary schema or domain-meaning compatibility.
5. Resolve the connection endpoint through the selected deployment's `anchorBindings`.
6. Verify that the target service declares `TOKEN_URL`.
7. Build `OPERATION_URL` from the runtime endpoint and the authoritative OpenAPI server/operation path.

The resulting URL is deployment data. It is never copied back into `dot.json` or the Anchor.

Cross-platform external gateway

A wire whose participants use different runtime platforms must declare a reviewed network adapter. The first supported adapter represents a gateway that the operator provisions outside `dotx`:

```
wires:
  - name: identity-discovery-to-proxy
    from: {instance: identity, anchorId: oidc-discovery}
    to: {instance: proxy, anchorId: oidc-discovery}
    network:
      adapter: EXTERNAL_GATEWAY
      host: identity-gateway.example.net
      port: 8080
```

The Contract and bound service port remain authoritative for protocol and interaction semantics. The binding replaces only the deployment-specific host and port used by materializers. The resolved composition, lock, and private lifecycle state record `ownership=EXTERNAL` and `cleanup=NONE`; apply, resume, status, and destroy never claim to create, inspect, or remove the gateway. Cross-platform wires without a binding, same-platform wires with a redundant binding, unknown adapters, URL-shaped hosts, and invalid ports fail before runtime mutation.

Composition-owned gateways, ingresses, tunnels, and brokers require dedicated lifecycle adapters and remain unsupported.

A wire and a materializer are intentionally separate. The wire says *these Anchors are connected*; the materializer says *write this derived value into this declared consumer setting*. A wire needs no materializer when the runtime already provides discovery, a service mesh performs routing, both participants use a fixed broker declared elsewhere, or the composition is documenting an Anchor relation without an executable handoff. Apply still requires every actually required service configuration value to have a default or explicit input.

For the example above, the human-readable resolved composition makes every resolution step inspectable before apply:

```
oauth-token-to-client:
  identity/client-credentials-token
```

```
    -> client/oauth-token [COMPATIBLE]
  endpoint: keycloak:8080
    (HTTP, instance=identity service=keycloak port=http)
  contract: Keycloak@26.8.1::OpenAPI::OIDC
    selector=#/paths/~1realms~1%7Brealm%7D~1protocol~1openid-connect~1token/post
  materializer: OPERATION_URL -> client/oauth-client env:TOKEN_URL
```

The same resolved composition with `--json` is the machine-readable automation output. It includes stable instance and Anchor references, compatibility reasons, the selected Contract, resolved endpoint, derived operation/channel value, materializer targets, security status, and exposed-Anchor endpoints. Consumers should parse JSON rather than scrape the human-readable form.

Brokered wire

For messaging, the semantic provider, semantic consumer, and connection endpoint may be three different instances. Select the broker explicitly:

```
wires:
  - name: openc2-command-via-mqtt
    from:
      instance: interpreter
      anchorId: send-openc2-command
    to:
      instance: actuator
      anchorId: receive-openc2-command
    endpoint:
      instance: broker
      service: mqtt
      port: mqtt
    materializers:
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_HOST
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_HOST
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_PORT
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_PORT
```

The AsyncAPI operations and message schema stay in the Contract. The broker's host and port stay in the resolved composition. If a channel has a templated address, the wire supplies only values for parameters declared by the Contract.

Materializers

Materializers are explicit adapters between a resolved endpoint and a configuration surface already declared by a service.

Type	Behavior
`ENVIRONMENT_VARIABLE`	Writes one resolved scalar to a declared environment variable.
`CONFIG_FILE`	Mounts a declared file and optionally replaces one YAML/JSON value selected by an RFC 6901 JSON Pointer.

Supported `valueFrom` values are:

Value	Result
`ENDPOINT_HOST`	Resolved service host only.
`ENDPOINT_PORT`	Resolved numeric service port only.
`ENDPOINT_HOST_PORT`	`host:port`.
`ENDPOINT_URL`	`scheme://host:port`, using the bound port protocol.
`OPERATION_URL`	Endpoint URL plus the authoritative OpenAPI server base and operation path.
`CHANNEL_ADDRESS`	Address of the authoritative AsyncAPI Channel Object.

An optional RFC 6570 `uriTemplate` can derive an application-specific URI from one resolved scalar. It is explicit composition behavior, not an Anchor or Contract fact.

Configuration-file materialization

A `CONFIG_FILE` materializer always names an operator-controlled source and a file path already declared by the target service. Without `valueFrom`, the source is supplied unchanged. With `valueFrom`, the source must be YAML or JSON and `jsonPointer` selects the scalar to replace:

```
materializer:
  type: CONFIG_FILE
  source: ./prometheus/prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

This produces a deployable Prometheus configuration without putting the provider's runtime address in either DOT. The reusable executable graph is `DOT-Repository/Compositions/node-exporter-prometheus-monitoring/composition.yaml`; the broader PoC4 validation-only fixture remains under `DOT-Testing/composition/`.

URI-template derivation

Use `uriTemplate` only when the application needs an explicit deterministic derivation that is not already the selected Contract operation URL:

```
materializer:
  type: ENVIRONMENT_VARIABLE
  valueFrom: ENDPOINT_URL
  uriTemplate: "{+value}/realms/drax"
  target:
    instance: proxy
    service: proxy-xapp
    environmentVariable: OIDC_ISSUER
```

The RFC 6570 variable is named `value`. The template belongs to the composition because it adapts one deployment to another; it does not amend the Contract or redefine the Anchor.

Materialization fails when the target configuration is undeclared, the Contract path/channel remains unresolved, alternative server paths are ambiguous, two wires produce conflicting values for one target, or the endpoint and target are on different runtime platforms without an explicit network binding.

Path and channel parameters

Only deployment-specific scalar values belong in the wire:

```
parameters:
  realm:
    value: drax
```

The Contract remains authoritative for the parameter name, location, requirement, schema, default, and serialization rules. Current composition bindings cover OpenAPI path parameters and AsyncAPI channel parameters. Query/header/cookie parameters, complex serialization, and runtime-derived parameter values require explicit future adapters.

Security and secrets

Contract security remains authoritative. A wire may demonstrate how one declared scheme is satisfied by deployment configuration:

```
security:
  bindings:
    - scheme: bearerAuth
      mechanism: https://www.rfc-editor.org/rfc/rfc6749#section-4.4
      participant: {instance: client, service: oauth-client}
      parameters:
        tokenUrl: {environmentVariable: TOKEN_URL}
```

```
        clientId: {environmentVariable: CLIENT_ID}
        clientSecret: {environmentVariable: CLIENT_SECRET}
```

Sensitive inputs are references, never secret values. They are scoped to the DOT instance whose selected deployment consumes them:

```
dots:
  - name: client
    source: repository://oauth-client@1.0.0
    dotId: oauth-client@1.0.0
    dotVersion: 1.0.0
    deployment: kubernetes
    deploymentInputs:
      environment:
        - name: CLIENT_SECRET
          secretRef:
            store: kubernetes
            name: oauth-client
            key: client-secret
```

Resolved compositions and composition state keep only the reference. Docker deployments can resolve `store: environment` at final handoff; Kubernetes and Helm preserve `store: kubernetes` as `secretKeyRef` without reading the Secret. A `CONFIGURED` resolved composition proves configuration completeness, not token validity, issuer trust, mTLS, workload identity, or remote attestation.

Exposed Anchors

A composition may re-export a member Anchor under a composition-local name:

```
exposedAnchors:
  - name: protected-metadata-through-gateway
    target:
      instance: gateway
      anchorId: protected-metadata
```

At apply time, an operator may request a local forward:

```
dotx compose apply scenario.yaml \
  --forward protected-metadata-through-gateway=19080
```

The exposure name belongs to the composition boundary. It does not rename or replace the member Anchor ID.

Before runtime mutation, follow the validation and lifecycle workflow.

Composition Lifecycle and Design Guidance

The lifecycle deliberately separates reproducible input locking, model validation, deterministic resolution, runtime mutation, recorded state, and active acceptance evidence. This keeps review and CI read-only while making execution recoverable and auditable.

Use `dotx compose inspect composition.yaml --locked` to explain the resolved DOTs, deployments, Contracts, wires, endpoints and configuration destinations before execution. It does not deploy workloads or probe services, although it may fetch and cache referenced definitions and Contracts. Use `topology` for a graph, `apply --dry-run` for deployment commands, and `status` for recorded runtime state. `inspect --json` returns the resolved composition as structured JSON.

The resolved composition is computed from your document and the referenced DOTs; it is not another file to author or another portable domain model. Locks and saved execution state use `resolutionDigest`. Regenerate and review locks after updating the CLI. Before changing binaries, clean up running compositions with their original CLI: saved state is not migrated or adopted.

Validation, resolution, and lifecycle

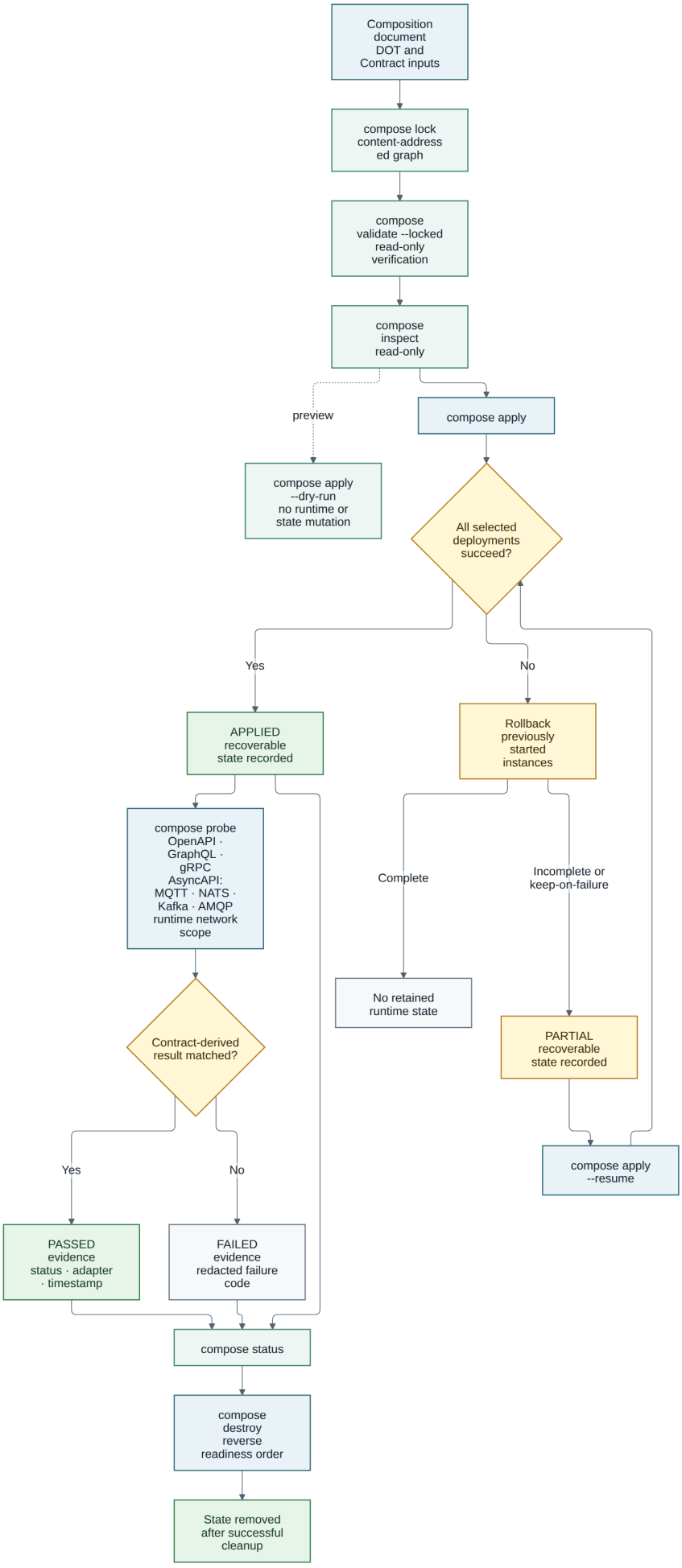


Figure 10. Composition lifecycle

What each command proves

Command	Proves	Does not prove	Mutates runtime/state
`compose lock`	The exact composition, selected DOT definitions and deployments, Contracts, configuration inputs, and portable resolved composition are content-addressed.	Runtime availability, credentials, or policy acceptance.	Writes only the requested lock file; `--check` is read-only
`compose validate`	The strict document shape, references, selected deployments, Anchors, Contracts, compatibility evidence, and materializer declarations resolve.	The exact runtime actions or live availability.	No
`compose inspect`	Everything validated above plus the deterministic instance order, runtime identities, endpoints, Contract standard/format and participant interaction, derived values, security status, and materializers.	That the resolved hosts are reachable or credentials work.	No
`compose apply --dry-run`	The composition reaches lifecycle preparation and can render supported deployment actions without persisting composition state.	Successful deployment or application-level traffic.	No
`compose apply`	The supported deployment adapters completed and recoverable state was recorded, subject to rollback behavior.	End-to-end Contract behavior unless separately probed.	Yes
`compose upgrade --dry-run`	The exact current and target locks resolve, the target SemVer increases, and a deterministic per-instance `ADD`/`UPDATE`/`REMOVE`/`UNCHANGED` diff passes runtime, storage, ownership, and recovery preflights.	That an application-specific persistent-data migration is safe.	No

Command	Proves	Does not prove	Mutates runtime/state
`compose upgrade`	The target revision was applied through a checkpointed transaction and the last-known-good control-plane revision remained available until promotion.	Reversal of application data writes; stateful mutations are rejected.	Yes; failures automatically restore the verified previous revision, while interrupted transactions require `--resume` or `--rollback`
`compose probe`	A selected safe OpenAPI operation, GraphQL lifecycle, finite side-effect-free gRPC method, or correlated AsyncAPI MQTT/NATS/Kafka/AMQP message is observable from the composition's shared Docker network or Kubernetes namespace. GraphQL mutations/subscriptions include mandatory verification or trigger and cleanup.	Complete application semantics, unsafe or unbounded gRPC, other AsyncAPI protocols, multi-scheme authentication, authorization-policy quality, or unprobed operations.	Creates and removes an ephemeral probe runner without changing member workloads; writes redacted probe evidence to composition state
`compose status`	Native status for recorded instances, wire/exposure configuration evidence, and the latest recorded active-probe outcomes.	Current reachability after the probe timestamp, authentication mechanisms not explicitly identified by the evidence, authorization semantics, response bodies, or policy compliance.	No runtime mutation; reads state while holding the lifecycle lock
`compose destroy`	Owned instances were destroyed in reverse order and successful cleanup removed composition state.	Cleanup of resources not owned or recorded by this composition.	Yes

Current-source upgrade admission inspects the selected deployment in both the previous and target revisions, not unrelated deployment alternatives. Compose YAML/JSON is parsed structurally; Kubernetes persistent volumes, claims, StatefulSets and non-ephemeral volume sources are refused. Missing, ambiguous or malformed inputs fail closed. Changed Helm and Vagrant instances are refused until chart defaults and dynamic Ruby storage can be established safely; unchanged instances and normal apply/status/destroy remain supported. This hardening is newer than the published CLI `v0.2.5`: use the reviewed fixed source for upgrades until a later release includes it. No control-plane rollback promises to undo application writes or preserve data in writable container layers.

`validate` answers *is this composition internally resolvable?* `inspect` answers *what exactly would be connected and materialized?* Keeping both commands read-only makes them suitable for review and CI.

`lock` makes those answers reproducible. Its JSON output preserves the original `repository://` and relative references while recording SHA-256 digests for the composition, every resolved `dot.json`, every supplied Contract, configuration inputs, and the portable resolved composition. For an explicit remote repository it also records the normalized repository identity and exact commit. It never records secret values, runtime addresses, or checkout-local absolute paths. Validate, inspect, and apply verify an adjacent lock automatically; `--locked` additionally requires it, and `--lock <path>` selects an explicit file. Any mismatch fails before runtime mutation.

`probe` deliberately reuses the interaction selected by the wire instead of introducing health-check fields in an Anchor or DOT. For OpenAPI, it derives the request from the authoritative operation, accepts only `GET`, `HEAD`, and `OPTIONS`, and requires one fully bound path. The OAuth 2.0 adapter reuses one configured RFC 6749 client-credentials binding that satisfies a complete, single-scheme OpenAPI Security Requirement alternative. It resolves the provider-derived token URL, client ID, scopes, and client-secret reference at execution time, then requires a Contract-declared `2xx` response from the protected operation. Kubernetes Secret values remain `secretKeyRef` references; Docker secrets are supplied only through the runner environment.

Development builds newer than the published CLI `v0.2.5` also accept an optional native JSON response assertion on safe OpenAPI `GET` or `OPTIONS` wires:

```
probe:
  openapi:
    expect: {pointer: /data/status, equals: success}
```

The existing Contract remains authoritative for the operation and declared status codes. With this fixture, the response must additionally be `2xx`, have `application/json` or `application/*+json` content type, and contain valid JSON whose RFC 6901 pointer equals the non-sensitive expected value. Array indices are supported; an empty pointer compares the entire document. Redirects are not followed, execution uses the probe timeout, and runner output is limited to 1 MiB. Only redacted outcomes and failure codes are recorded, never response bodies or actual values. This is explicit value checking, not complete OpenAPI response-schema validation. Without the fixture, ordinary operations retain status-only behavior; declared native metrics use the bounded profile below. The assertion is a lock-covered composition input, not a new DOT or Anchor property. `HEAD` cannot carry a JSON body assertion.

Native query and metrics probes in 0.4.0

The **0.4.0** release adds two native probes through the existing bounded HTTP runner and selected service binding. The published v0.3.0 probe behavior is unchanged.

A SPARQL query wire sends exactly one GET with the encoded fixed `query=ASK {}` and `Accept: application/sparql-results+json`. Success requires HTTP 200, that response media type, a JSON `head` object and `boolean: true`. It does not follow redirects, accept arbitrary query/update fixtures, manage datasets or establish authorization beyond that observed request. The probe uses the

runtime binding and the descriptor's escaped path, never its authority.

A safe OpenAPI GET declaring native metrics on a successful response requests OpenMetrics 1.0.0 or Prometheus text 0.0.4 and validates the entire bounded UTF-8 response with the official Prometheus parsers. The returned status chooses its own response media declaration in this order: exact status, `2XX` range, then `default`. Another response's media cannot qualify the body. Error-only or default-only native declarations do not activate this profile. Unsupported or ambiguous OpenMetrics versions fail instead of reverting to status-only success. A declared OpenMetrics response cannot pass using Prometheus text.

OpenMetrics requires the proper `# EOF` marker and rejects malformed or trailing content. Prometheus text requires a final newline and has no EOF marker; a syntactically complete truncated response cannot be detected by this format check. HTTP errors, wrong media/version, malformed or oversized responses fail. A JSON expectation remains JSON checking when a successful JSON representation is declared; it cannot validate a metrics-only response. Ordinary OpenAPI operations without a native metrics success declaration retain their existing status-only/explicit-JSON behavior.

These probes establish one negotiated response and syntax at their recorded time. Scrape/query behavior, metric semantics, dataset compatibility and production acceptance require separate producer/scenario evidence. Probe results retain redacted outcomes, not bodies or credentials.

Other Contract probe profiles

For GraphQL, the composition supplies the deployment-specific HTTP path and a non-sensitive operation document with optional operation name and variables. Before resolution succeeds, `dotx` parses the authoritative SDL, validates every document and variable set, requires exactly one direct top-level field, and requires that field to equal the Anchor-selected coordinate. It never guesses `/graphql`. Queries use HTTP (the default) and may declare an `expect` assertion; they cannot declare `verify`, `trigger` or `cleanup` operations. Mutations require HTTP, `expect`, an observable `verify` query and a `cleanup` mutation; they cannot declare `trigger`. Subscriptions require explicit `transport: SSE`, `expect`, a separate `trigger` mutation and a `cleanup` mutation; they cannot declare `verify`. The SSE connection is established before its trigger runs. Every phase must return HTTP `2xx` without GraphQL errors; the expectation is evaluated against the query response, mutation verification result, or delivered subscription event.

For gRPC, the composition supplies one protobuf-JSON `request` for unary and server-streaming methods, or an ordered `requests` sequence for client and bidirectional streams. The selected Proto3 RPC must declare the standard `NO_SIDE_EFFECTS` idempotency level and be discoverable through explicitly enabled server reflection. Every streaming fixture declares a positive exact `responseCount`; an optional expectation uses `ALL`, `ANY`, `FIRST`, or `LAST` matching. Requests are checked against the authoritative input descriptor before the pinned `grpcurl` runner starts, then every returned document is checked against the authoritative output descriptor. Unsafe, unbounded, wrong- count, or descriptor-incompatible calls fail closed.

For AsyncAPI, the active adapters require a provider `send` operation, an explicit MQTT/MQTT5, NATS, Kafka, or AMQP channel binding, one concrete destination, and a JSON-Schema message payload. The composition declares the message and RFC 6901 correlation pointer; MQTT additionally declares QoS. The Kafka adapter honors the official binding-level `topic` override and

restricts topics to the Kafka-supported ASCII alphanumeric, dot, underscore, and hyphen syntax. The digest-pinned official Mosquitto or NATS client subscribes before publishing a fresh UUID-correlated non-retained message. Kafka creates a unique consumer group, waits for its topic assignment before publishing the exact correlated record through the official Apache Kafka JVM image, and removes the group after the bounded check. Kafka records remain subject to broker/topic retention and are never represented as transient. The AMQP 0-9-1 adapter applies the official 0.3.0 channel and operation bindings, declares the exact direct/topic/fanout exchange, binds an exclusive auto-delete queue before publishing, acknowledges the correlated delivery, and closes the connection to remove the queue. It accepts credentials only as a password secret reference. Queue-bound channels, headers exchanges, AMQPS, AMQP 1.0, a binding/protocol mismatch, or MQTT-only QoS on NATS/Kafka/AMQP fail closed.

All adapters execute digest-pinned runners inside the common Docker network or Kubernetes namespace. Runner output is capped at 1 MiB before parsing. Secrets and bearer tokens never appear in the argument vector or state. Contractless, unresolved, unsupported-protocol, and cross-platform wires are reported as unsupported rather than inferred healthy. Diagnostics, documents, variables, requests, messages, and response bodies are not persisted; state contains only normalized identifiers, adapter, result metadata, failure code, and timestamp.

The live contract-probe fixture runs a real GraphQL.js producer with distinct GraphQL-over-SSE connections, a reflected `grpc-js` server implementing all four RPC shapes, Eclipse Mosquitto, NATS, Apache Kafka, and RabbitMQ. It demonstrates the full supported boundary; it does not turn those test services or interaction fixtures into reusable DOT metadata.

```
# Composition-level probe input; the DOT and Anchor still contain only the
# authoritative Contract reference and selector Query.health.
probe:
  graphql:
    path: /graphql
    operationName: Health
    document: |
      query Health {
        health
      }
    expect: {pointer: /data/health, equals: true}
```

```
# Finite bidirectional gRPC probe. The selected Proto method in the Contract
# declares stream input/output and NO_SIDE_EFFECTS; these fields supply only
# instance-specific test messages and acceptance criteria.
probe:
  grpc:
    plaintext: true
    serverReflection: true
    requests:
      - {component: contract-provider, count: 1}
      - {component: contract-consumer, count: 1}
    responseCount: 2
    responseMatch: LAST
```

```
    expect: {pointer: /sequence, equals: 2}
```

```
# NATS transport delivery. The selected AsyncAPI channel owns the subject,
# NATS binding, message schema, and send/receive semantics.
probe:
  asyncapi:
    protocol: NATS
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Kafka retained-stream delivery. The selected AsyncAPI Kafka 0.5.0 channel
# binding owns the physical topic override and the message schema.
probe:
  asyncapi:
    protocol: KAFKA
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Emit the canonical strict Draft 2020-12 schema
dotx compose schema > composition-v1.schema.json

# Generate once, commit, and verify without rewriting
dotx compose lock scenario.yaml
dotx compose lock scenario.yaml --check

# Structure, references, selectors, compatibility, and materializer declarations
dotx compose validate scenario.yaml --locked

# Deterministic resolved topology; no runtime mutation
dotx compose inspect scenario.yaml --locked
dotx compose inspect scenario.yaml --locked --json

# Resolve the catalogue from one immutable remote Git revision
dotx compose inspect scenario.yaml --locked --json \
  --repository https://github.com/MARE-EU-Project/DOT-Repository.git \
  --git-ref <40-character-commit>

# Render runtime actions without state or runtime mutation
dotx compose apply scenario.yaml --locked --dry-run

# Execute, inspect recorded evidence, and clean up owned resources
dotx compose apply scenario.yaml --locked
dotx compose probe scenario.yaml --locked --wire oidc-provider-to-gateway
dotx compose probe scenario.yaml --locked --wire protected-gateway-to-client
dotx compose status scenario.yaml
dotx compose destroy scenario.yaml
```

```
# Review and execute a locked upgrade
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked --dry-run --json
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked
```

Apply records each pending instance before runtime mutation, rolls back earlier instances on failure, and preserves recoverable state when rollback is incomplete or `--keep-on-failure` is selected. `apply --resume` requires an identical composition source, resolved composition, runtime options, inputs, ordering, and previously applied runtime state. Resume also reconciles an interrupted composition-network creation: it reuses the correctly owned network or creates the still-missing network before continuing. Destroy checkpoints every successfully removed instance, so a retry targets only cleanup that did not finish.

Upgrade is a separate lifecycle; `apply` never mutates an already recorded resolved composition. Upgrade requires the exact previous and target locks, the same composition UUID, and a strictly greater target SemVer. Existing instance names must retain their platform and deployment mechanism. Docker image, Docker Compose, Kubernetes manifest, Helm, Vagrantfile, and WASM module instances use their stable managed identities. The private transaction journal stores only paths, identifiers, phases, and digests—not resolved secrets or input payloads. Stateful and host-bound mutations fail before the first runtime action because restoring a deployment specification cannot undo workload data changes.

Recover an interrupted upgrade

Only use these commands for an upgrade with a recorded pending transaction. Keep the **original** binary, cache (`XDG_CACHE_HOME`), cluster context (`KUBECONFIG`), repository, runtime options, two declarations and two locks. The filenames below continue the preceding `scenario-v1`/`scenario-v2` example; substitute the original paths from your run. Do not regenerate either lock or edit inputs to make recovery pass. Retain any original upgrade `--repository` and `--git-ref` selections. Apply-only options such as `--namespace` and `--forward` are not accepted by upgrade; it uses the recorded execution context.

Choose **one** recovery action. To continue toward the reviewed target:

```
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml --from-lock scenario-v1.lock.json \
  --lock scenario-v2.lock.json --resume
```

To abandon the pending target and restore the recorded previous revision:

```
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml --from-lock scenario-v1.lock.json \
```

```
  --lock scenario-v2.lock.json --rollback
```

Both commands name the **target** first; `--from` and `--from-lock` still identify the previous revision. Rollback is not `apply` of an older document and does not restore arbitrary application data. An input/ownership mismatch is a stopping point: recover the original context rather than bypassing it.

After a completed resume, inspect `dotx compose status scenario-v2.yaml`; after a completed rollback, inspect `dotx compose status scenario-v1.yaml`. Perform the original API checks again. When finished with a disposable run, destroy that successfully recorded revision and verify only its owned resources are gone. Retain the recovery inputs if status, verification or cleanup fails.

The deterministic recovery profile injects an interruption immediately before and after network creation, instance deployment, instance destruction, and network removal. It also verifies lock contention, lock release after a panic, partial cleanup, idempotent resume, and fail-closed composition/runtime drift:

```
task testing:compose:lifecycle-recovery
task testing:compose:upgrade
```

The task writes revision-pinned, secret-free JSON evidence under `DOT-Testing/e2e/target/` and is enforced by the Composition lifecycle recovery workflow.

Apply, probe, status, and destroy acquire a private per-composition process lock. A concurrent lifecycle command fails immediately rather than racing local state or runtime mutations.

`compose status` reports:

- the recorded phase and native runtime status of each DOT instance;
- recorded wire compatibility, security configuration, selected endpoint, and materializer count; and
- recorded exposed-Anchor bindings; and
- the latest redacted active Contract-probe result for each probed wire.

Wire and exposure entries remain configuration evidence. Only entries under active Contract-probe evidence demonstrate a runtime request and declared OpenAPI response match, bounded by their timestamp and adapter scope.

Fail-closed rules

Apply rejects:

- `UNKNOWN` wire compatibility;
- missing, duplicate, or mismatched IDs and versions;
- ambiguous deployments, bindings, services, ports, or OpenAPI server paths;
- unresolved OpenAPI paths or AsyncAPI channel templates;

- undeclared materializer targets and conflicting target values;
- readiness dependency cycles;
- unsupported secret-store/runtime combinations;
- ungranted OCI capabilities, host namespaces, volumes, or CDI devices;
- repeated DOT instances on deployment mechanisms that cannot isolate their resource names; and
- materialized cross-platform wires without an explicit network binding.

Resolution may retain `UNKNOWN` to explain incomplete evidence. Execution never silently upgrades `UNKNOWN` to compatible.

What does not belong in an Anchor

Do not add these as optional Anchor fields:

- hosts, URLs, numeric ports, broker addresses, or Kubernetes names;
- credentials, tokens, certificates, or secret values;
- Camel/Kamelet route syntax;
- transformation logic;
- placement, slice, tenant, trust-zone, latency, or availability policy;
- current health or live authorization state;
- host paths or operator device names; or
- orchestration dependencies.

Use Contracts for interaction semantics, service bindings for deployable ports, composition inputs for operator-controlled configuration, wires for topology, materializers or mediator DOTs for explicit adaptation, and policy engines for deployment/security decisions.

Choosing an extension point

Requirement	Preferred representation
New API/message operation	Add or update the authoritative Contract and select it from an Anchor.
Consumer needs a provider address	Wire materializer targeting declared service configuration.
Payload/protocol transformation	Explicit mediator DOT with two Contracts and two Anchors.
Startup/readiness ordering	Composition `dependsOn`.
Secret or certificate	Reference-only deployment input and platform/external secret provider.

Requirement	Preferred representation
Privileged runtime resource	Typed deployment requirement plus exact composition grant.
Broker service	Explicit wire `endpoint` selecting the broker service port.
Network attachment, eBPF/XDP, WASM import/export, or lifecycle event	Separately reviewed polymorphic binding type when a real use case requires it; do not grow `Anchor` with nullable fields.

6G security composition patterns

The same model supports security-function composition without baking one orchestrator's topology language into every DOT:

- **Zero-trust API chain:** identity-provider and policy-decision DOTs expose token, discovery, and authorization Anchors; an API gateway consumes them and exposes a protected application Anchor. Contracts carry OAuth/OpenID Connect and application semantics, while the composition supplies client-specific secret references and issuer URLs.
- **Closed-loop detection and response:** telemetry, inference, CACAO interpretation, and OpenC2 actuation remain separate DOTs. HTTP and MQTT Contracts define the boundaries; the wire selects the broker endpoint and supplies only deployment-specific channel values.
- **Multi-domain federation:** peer DOT instances can have reciprocal wires while `dependsOn` contains only real readiness dependencies. Trust-domain, roaming, and federation policy belongs in policy inputs or a dedicated policy DOT, not in the Anchor.
- **Near-real-time RAN control:** E2 or management APIs are Contract-backed Anchors. Host networking, accelerators, eBPF/XDP attachments, and radio-device access are explicit deployment requirements and grants, never inferred from a wire.
- **Observability and digital twins:** metrics, logs, events, and integrity APIs are distinct Anchors. A storage or message broker is selected explicitly as a connection endpoint; a shared host path is not treated as a portable integration boundary.

For cross-domain 6G deployments, a successful resolved composition proves model consistency, not trust. Admission policy must still evaluate publisher identity, artifact provenance, attestation, slice/tenant placement, data-sovereignty constraints, and least-privilege grants before execution. Runtime acceptance should then probe the selected Contract operation over the actual network path.

Troubleshooting

Symptom	What to check
Anchor ID is not found	The wire references the pinned DOT version, the exact local ID is stable, and the Anchor is present in that version's `dot.json`.

Symptom	What to check
Contract or selector cannot be resolved	For a `repository://` source, select the exact originating DOT release and verify it owns the complete matching Contract declaration and URL/hash payload. The owner's DOT ID is different from the Contract ID. Check the operation selector separately. Only historical local Contract wrappers carry their own matching Contract `id`; do not add that field to a raw standard document.
Anchor has no deployable endpoint	The selected deployment contains exactly one service binding for the Anchor, and its named port exists on that same service.
Compatibility is `UNKNOWN`	Inspect `compatibilityReasons` in `compose inspect --json`; add authoritative Contract evidence or use an explicit mediator. Do not override the result.
Materializer target is rejected	The target service declares the environment variable or config-file path, the JSON Pointer is valid, and no other wire writes a conflicting value.
Security remains unresolved	Bind a Contract-required supported scheme, declare all participant variables, and supply sensitive values by `secretRef` rather than plaintext.
Apply reports the composition is busy	Another apply, status, or destroy holds the per-composition lock. Let that command finish; do not delete the lock while a lifecycle process is running.
Status shows a configured wire but no active evidence	Run `dotx compose probe scenario.yaml --wire <wire>`. If the wire is skipped, its operation requires a dedicated adapter or application-level acceptance test. Configuration evidence alone is not reachability evidence.
An OAuth probe reports `AUTHENTICATION_ERROR`	Check the bound token URL and client ID, the referenced Secret name/key or environment-store source, required scopes, issuer reachability, and client authorization. Dotx deliberately does not persist the token endpoint's body.

Executable examples

- `DOT-Testing/conformance/anchors/` covers all four Contract standards, contractless Anchors, validation failures, bindings, and compatibility.
- `DOT-Testing/e2e/api-gateway-oidc/` is the canonical authenticated request/response lifecycle example.
- `DOT-Testing/composition/poc6-closed-loop/` demonstrates an explicit MQTT broker endpoint and OpenC2/AsyncAPI operations.
- `DOT-Testing/composition/federation-peers/` demonstrates reciprocal wires without false readiness cycles.
- `DOT-Testing/e2e/poc1-ric-aaa/` demonstrates identity, OPA authorization, forwarding, metrics

feedback, and deny-list update across seven wires.

- `DOT-Testing/interoperability/pocs.yaml` is the evidence ledger for all eleven MARE PoCs and records every remaining acceptance gap.

Run the repository gate with:

```
task testing:conformance
```

The source-adapted E2E tests prove only their documented boundaries. They do not claim partner-runtime, live-RIC, production-network, trust-policy, or SFC acceptance unless that exact behavior is asserted by the scenario.

Verification and examples

DOT-Design keeps model correctness, composition resolution, deployment dry-runs, and live behavioral acceptance as separate evidence levels. A passing lower level never implies a higher one.

Evidence model

Level	Demonstrates	Does not demonstrate
Model and schema conformance	`dot.json` shape, Bean Validation constraints, stable references, selectors, service-port bindings, and generated-model compatibility	Contract payload resolution or multi-DOT topology
Contract resolution and compatibility	Selected operations exist and normalized semantic, interaction, protocol, media, schema, and security facts can be evaluated conservatively	Runtime reachability or valid credentials
Composition validation and resolved composition	Pinned DOTs, wires, endpoints, materializers, parameters, grants, and readiness ordering resolve deterministically	Successful deployment
Lifecycle dry-run	The reference executor accepts the supported profile and renders runtime actions without writing composition state	Workload startup or traffic
Source-adapted E2E	Pinned upstream logic or artifacts execute behind explicit test adapters and satisfy the assertions documented by that scenario	Unmodified partner runtime, production infrastructure, or full PoC acceptance
Live production acceptance	The intended workloads, network, identity, policy, and Contract operations work in the target environment	Nothing beyond the probes and acceptance criteria actually executed

Read catalogue readiness correctly

Catalogue validity, interface completeness and live acceptance are separate results. A green repository workflow validates declarations and renders locked compositions; it does not deploy every DOT or certify production readiness.

Result	Interpretation
Base/reusable validation	Checks definition structure, exact references, verified Contract bytes and applicable bindings. It permits explicitly lower-evidence runtime-only entries.

Result	Interpretation
`INTERFACE_PROFILE_PASSED`	Passes `--profile interoperable` for declared Anchors and exposed ports. This is not live producer acceptance or proof of complete API coverage.
`INTERFACE_GAP`	Lacks machine-readable coverage required by that strict profile. It is not automatically a broken deployment or proof that no upstream specification exists.
Live receipt	Records only the tested workloads, revisions, assertions and cleanup outcome. It does not qualify other deployments or the whole catalogue.

Also distinguish **official Contracts**, **source-derived Contracts**, **runtime-only components and brokers**, **planning definitions and stubs**, and **test fixtures, adapters and producer doubles**. Official provenance does not establish runtime behavior; source-derived documents need producer review; a broker's protocol does not define application message semantics. Do not fabricate a Contract for a client-only worker or planning API just to turn a profile result green. An ownership folder is not an evidence level.

Every current catalogue DOT README includes an expandable interface inventory: owned Contracts, provided and required Integration Points, exact operation selectors and service-port bindings. The complete interface inventory links all bundles, including entries with missing Contracts. It is generated from `dot.json`; it is not a second catalogue index or a qualification result. Required dependencies do not document a component's exposed APIs. An explicitly implemented external interface retains its original Contract ID and origin.

Grafana now includes its release-matched official HTTP OpenAPI declaration, with health and data-source-list Integration Points bound to port 3000. Its browser UI and consumed Prometheus/Elasticsearch APIs remain separate boundaries. The vendor's global security declarations are preserved; selecting a documented operation does not establish live authorization or full API coverage.

SPARQL 1.1 query interfaces use the canonical `SPARQL` Contract standard and `N_TRIPLES` Service Descriptions. The bounded profile resolves exact endpoint selectors and supports a read-only ASK probe. Updates, arbitrary dataset semantics and other RDF encodings remain outside this profile.

OpenMetrics is an HTTP response representation, not another Contract standard. Existing OpenAPI operations can describe and probe Prometheus text 0.0.4 and OpenMetrics 1.0.0 exposition. A metrics sample or an HTML documentation URL alone does not describe a tool's complete interface. HTTP/WebSocket AsyncAPI flows and imported Protobuf graphs remain separate implementation gaps.

From the **DOT-Design checkout**, warm the verified cache and regenerate the report:

```
task testing:contracts:warm
task testing:contracts:readiness
jq '.dots | group_by(.status) | map({status: .[0].status, count: length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

The report records `liveProducerAcceptance: false`, each definition's SHA-256 and its strict-profile diagnostic. Regenerate it after catalogue changes; counts describe that exact snapshot, not a permanent readiness percentage. The report also includes each diagnostic code and `review.category` with a concrete next action. These context categories do not change strict outcomes: planning entries, clients and runtime fixtures are not automatically exempt or interoperability-ready. A download/cache/execution failure aborts report publication instead of being counted as an interface gap. The previous complete report is retained when generation fails.

```
jq '.dots | group_by(.review.category) | map({review: .[0].review.category, count:
length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

The post-0.4.0 Loki update adds a source-derived HTTP subset and real log ingestion/query assertions for the same selected release on Helm and Compose. Its native gRPC port and OTLP/import coverage remain explicit gaps; HTTP acceptance does not turn those gaps into passes. These reporting Tasks do not belong to the standalone catalogue checkout. There, use `dotx validate .` and inspect the per-bundle README and Contract audit at the selected catalogue revision. For historical evidence, replace `main` in that link with the recorded commit; do not attribute today's audit to an old run.

Primary verification commands

Command	Scope	External requirements
`mvn -pl DOT-Domain test`	Domain value objects, manifests, Anchors, and aggregate validation	Java and Maven only
`go -C DOT-CLI test ./…`	CLI parsing, validation, resolution, deployment adapters, and lifecycle behavior	Go; individual tests use doubles where appropriate
`task testing:dots:repository`	Every `dot.json` discovered recursively below `DOT-Repository/DOTs/`, with the reusable profile for external/internal bundles declaring owned Contracts or explicitly providing an external Contract-backed interface, ownership-path isolation, mandatory per-bundle README files, schema validity, CLI cross-references, and identity uniqueness	Go and the checked-out `DOT-CLI` and `DOT-Repository` revisions; repository validation uses its co-versioned `Schema/`, validates the authoritative `Skills/` catalogue, and resolves every DOT Skill ID to one categorized capability
`task testing:dots:external:dry-run`	Reusable Keycloak/Prometheus DOTs plus the partial HTTP Loki and remote-desktop Wireshark definitions: profile-appropriate validation, inspection, deployment dry-run, exact-version Helm rendering, expected service/workload shape, and pinned OCI image metadata	`dotx`, Helm, Python/PyYAML, Docker, and network access to the pinned public artifacts

Command	Scope	External requirements
`task testing:quickstart:dry-run`	Execute the command blocks included by Getting started: validate and render Prometheus, then check the locked gateway resolved composition and dry-run	`dotx`, `jq`, initialized catalogue, and network access for official Contracts
`task testing:quickstart:live`	Run Prometheus at the default and overridden port, assert the build-information API/version, and verify container/network/volume cleanup on success and injected failure	Docker Compose, `dotx`, `curl`, `jq`, network access, and free host ports `18909`, `18919`, and `18929`; all example metrics are deleted
`task testing:dots:external:stress`	Repeated Linux deployment, readiness, HTTP/HTTPS access, destroy, and cleanup across two mechanisms per external product: three Helm/Compose pairs plus the Wireshark Docker-image/Compose pair; Keycloak and Prometheus call selected official Contract operations. Loki verifies its actual selected build, fresh correlated JSON log push/query and invalid-timestamp rejection against the explicitly source-derived HTTP subset	Docker, Minikube/Kubernetes, Helm, network access, and enough resources for Keycloak, Loki, Prometheus, and Wireshark; emits revision-pinned evidence
`task testing:compose:keycloak-prometheus-openapi-validation`	Reusable-profile validation and a locked three-DOT resolved composition proving the stable Keycloak and Prometheus Anchors resolve against exact official OpenAPI payloads, compose compatibly, bind the Keycloak realm parameter, and materialize the expected consumer operation URLs	`dotx`, `jq`, and the checked-out repository/schema revisions; no live runtime
`task testing:compose:contract-standards`	Two canonical test DOT definitions connected through OpenAPI 3.0/3.1/3.2 and AsyncAPI 3.0/3.1 YAML/JSON, Protocol Buffers (including gRPC service declarations), and GraphQL SDL; asserts selectors, participant interactions, endpoints, probe fixtures, and materializers across twenty-one wires	`dotx`, `jq`, and the checked-out repository/schema revisions

Command	Scope	External requirements
`task testing:compose:contract-probes:live`	A standards-native test producer and four real brokers prove GraphQL query, mutation verification/cleanup, GraphQL-over-SSE delivery, all four finite side-effect-free gRPC shapes through reflection, correlated non-retained AsyncAPI MQTT/NATS delivery, correlation-scoped Kafka delivery under broker retention, and acknowledged AMQP 0-9-1 delivery through an exclusive probe queue; rejects existing resources and emits redacted success evidence only after verified cleanup	Docker, network access for pinned images, and the checked-out `dotx`
`task testing:compose:acceptance`	Authoritative `v1` command/schema parity, every committed lock, twenty-one Contract wires, contractless compatibility, all six runtime dry-runs, service-scoped configuration, recovery/status/destroy, and focused fail-closed controls; emits revision-pinned JSON evidence	A globally installed `dotx` from the checked-out CLI revision, Go, Task, and fixture-specific dry-run CLIs
`task testing:contracts:standards`	Extracts the twelve distinct Contract payloads and validates them with dated official OAI schemas, the official AsyncAPI CLI, `protoc`, and GraphQL.js; each family also has an invalid control which must be rejected	Python with PyYAML/jsonschema, Node.js, network access for pinned validators, and `protoc`
`task testing:compose:runtime-configuration`	The same catalogued DOT receives declared environment and configuration-file inputs through Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and Wasmtime/WASI dry-run adapters	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads`	A two-service DOT receives isolated defaults, overrides, configuration files, and secret references through exact Compose, Kubernetes, and Helm workload bindings	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads:live`	Two concurrent copies of the same multi-service workload prove exact environment, secret, config-file, and volume isolation plus readiness, status, resume, selective destroy, surviving-peer health, and cleanup across Docker Compose, Kubernetes, and Helm	Docker Compose, Minikube/Kubernetes, Helm, and network access for runtime images; emits redacted revision-pinned evidence

Command	Scope	External requirements
`task testing:compose:runtime-isolation:live`	Two locked instances of one Vagrant DOT and one WASI CLI DOT prove distinct runtime identities, concurrent state, status, selective destroy, peer survival, and cleanup. Repeated fixed-port WASI HTTP fails before mutation.	Vagrant 2.4.9 with libvirt/KVM and Wasmtime 49.0.2; emits redacted, revision-pinned evidence
`task testing:cli:coverage`	Complete DOT-CLI test suite plus machine-readable total and high-value package statement-coverage profiles	Go; writes reports below `DOT-Testing/e2e/target/`
`cd DOT-CLI && task ci:security`	Scans the Go module dependency graph and reachable code against the current OSV vulnerability database with the exact pinned scanner used by CI	Go and network access; intentionally separate from deterministic `task ci`
`task testing:runtime:kubernetes-helm:live`	Real Kubernetes-manifest and Helm-chart deploy, rollout readiness, status, service-scoped environment materialization, and destroy lifecycle	Docker, Minikube, Kubernetes, Helm, and network access for runtime images
`bash DOT-Testing/runtime/wasm/live.sh`	Real Wasmtime lifecycle for WASI CLI; CI additionally builds the pinned Bytecode Alliance `wasi:http/proxy` sample and proves an HTTP response	`dotx`, Wasmtime, and the optional prebuilt HTTP component path
`task testing:conformance`	Cross-language Anchor and composition gate, including all deterministic PoC fixtures and lifecycle dry-runs	Java, Maven, Go, Task, and fixture-specific local tools
`mvn clean install`	Full Java domain build, schema generation, coverage gates, and SBOMs	Java and Maven only
`task testing:e2e:<scenario>`	Scenario-specific live behavior	Read that scenario's README first; commonly Docker and network access

Test asset layout

```
DOT-Testing/
 ├──── acceptance/        versioned, implementation-neutral consumer profiles
 ├──── conformance/       deterministic valid and invalid DOT fixtures
 ├──── composition/       read-only validate, inspect, and dry-run scenarios
 ├──── coverage/          executable code-coverage ratchets
 ├──── e2e/               opt-in deployable behavioral scenarios
 ├──── interoperability/  revision-pinned evidence and remaining-gap ledger
 └──── runtime/           isolated native-runtime lifecycle harnesses
```

`testing` is repository-level infrastructure, not a Maven module and not a deployable DOT.

Authoritative test-only definitions live under `DOT-Repository/DOTs/testing/` and are selected from compositions with `repository://<dot-id>@<version>`. Intentionally invalid conformance inputs stay under `DOT-Testing/conformance/`; they must never be admitted to the repository catalogue. Generated secrets, logs, reports, coverage profiles, and materialized configuration belong under scenario-local `target/` directories and must not be committed.

The focused `task testing:anchors:acceptance` gate verifies the versioned DOT Anchor consumer profile. It resolves pinned test definitions from `DOT-Repository`, verifies the composition lock, and asserts the complete cross-standard resolved composition without a deployed workload.

The separate `task testing:compose:acceptance` gate freezes the complete `dot.mare6g.eu/v1` production profile. Its machine-readable source is `DOT-Testing/acceptance/composition/v1/profile.json`; the gate binds that source to CLI help, the tracked schema, repository locks, positive evidence, and negative fail-closed behavior. Runtime rows distinguish underlying single-DOT live lifecycle proof from multi-DOT composition lifecycle proof.

The focused `task testing:compose:contract-standards` gate complements that API profile with an executable composition matrix. It proves all six supported standard/format representations between the same provider and consumer DOTs; it does not promote resolution evidence to live protocol or application acceptance.

The separate `task testing:contracts:standards` gate validates those payloads outside the DOT resolvers with standards-native tooling. The positive fixtures must pass and deliberately malformed OpenAPI, AsyncAPI, Protobuf, and GraphQL controls must fail. This catches both invalid examples and accidentally permissive validator invocations.

The runtime-configuration gate separately proves that selecting another executable deployment does not discard a service's declared environment or configuration-file surface. Each adapter is exercised first with its declared `defaultValue` and `payload`, then with explicit composition overrides; the gate validates the cached default document, resolved service id, override values, and runtime-specific dry-run. Runtime-native storage, secret, privilege, and device semantics retain dedicated tests rather than being flattened into a misleading universal volume abstraction.

The multi-service live profile is the higher evidence layer for the exact workload bindings. Compose projects and their named volumes are keyed by the composition runtime identity; Kubernetes repetitions use separate namespaces; Helm repetitions use identity-derived release names in one namespace. The profile verifies that destroying one copy leaves the other healthy and that dotx-created ConfigMaps/Secrets are cleaned without deleting referenced operator-owned Secrets. The tracked Helm chart is packaged from the pinned checkout, and its SHA-256 is included in the redacted evidence artifact.

The Kubernetes and Helm profiles also cover separate service containers in one Pod, same-filename configuration paths and service IDs that would collide after sanitization. A private test-only definition and verified payload cache keep these stress cases outside the catalogue. Service mappings must identify distinct actual workload/container pairs; selector spelling alone is not an isolation guarantee. See One DOT, several services for configuration scopes and supported execution boundaries.

Coverage claims and gates

Coverage is reported by evidence dimension; the project does not collapse different meanings of "covered" into one percentage:

Dimension	Gate	Interpretation
Repository inventory	`dotx validate DOT-Repository`, `dotx repository audit`, and the DOT repository conformance workflow	Must be 100%: every filesystem definition is discovered, parsed, schema-valid, cross-reference-valid, uniquely identified, and stored in a supported category; no deployable copy may remain outside `DOT-Repository`.
CLI statements	`task testing:cli:coverage`	The whole CLI and every package containing executable statements must retain 100% statement coverage. The constant-only `feature/sources` package has no testable statements. This is a code-execution metric, not proof that a workload or security control works.
Java instructions and branches	`mvn verify` through JaCoCo 0.8.15	The Java domain module must retain 100% instruction and branch coverage. JaCoCo HTML/XML/CSV reports are uploaded with the Java build evidence.
Native runtime lifecycle	WebAssembly and Kubernetes/Helm runtime E2E workflows	Proves real Wasmtime CLI and HTTP behavior plus Minikube manifest/Helm lifecycle. It does not imply that every external DOT workload is accepted.
Composition scenarios	`task testing:e2e:all:dry-run`	Every listed deterministic scenario must validate, inspect, and complete its supported lifecycle dry-run.
PoC interoperability	`DOT-Testing/interoperability/pocs.yaml` plus scenario-specific live workflows	Each of the eleven PoCs has traceable evidence, while remaining adapters, unavailable partner components, and unproved production behavior stay explicit.

The repository validation gate fails on duplicate identities, mis-scoped definitions, invalid DOTs, invalid Skill/category assets, or unresolved DOT Skill references. The workspace audit separately classifies uncatalogued, identical, divergent, and malformed definitions and fails on every deployable `dot.json` outside `DOT-Repository`. `DOT-Testing/conformance/` fixtures are reported but exempt because invalid definitions are their purpose. During the staged migration, run:

```
dotx repository audit --repository DOT-Repository --workspace . --json
```

The coverage gates enforce 100% Java instruction/branch coverage and exact 100% Go statement coverage overall and for every package containing executable statements. Those thresholds do not replace negative tests, lifecycle checks, live probes, or partner acceptance.

Canonical Anchor scenario

`DOT-Testing/e2e/api-gateway-oidc/` is the primary tutorial and lifecycle fixture. Its composition resolves three independently valid, test-scoped definitions from `DOT-Repository`:

- Keycloak provides OpenID Connect discovery and token operations;
- APISIX requires discovery information and provides a protected operation; and
- an OAuth client requires the token and protected operations.

The composition uses three OpenAPI Contracts, three wires, explicit security configuration, Kubernetes Secret references, configuration-file and environment materializers, readiness dependencies, and one exposed Anchor.

```
task testing:compose:keycloak-apisix-oauth
task testing:demo:keycloak-apisix-oauth:dry-run

# Opt-in live execution
task testing:demo:keycloak-apisix-oauth:live
```

Read DOT Anchors and Composition alongside `DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml`.

MARE PoC-derived coverage

The interoperability ledger pins the source revision used for every analysis, records the modeled interactions, identifies the executable fixtures, and keeps remaining acceptance gaps explicit. The current repository contains source-adapted evidence for all eleven PoCs; this is not a claim of unmodified partner-runtime acceptance.

PoC	Composition focus	Fixture
1	OIDC identity, OPA authorization, RIC forwarding, metrics feedback, and deny-list update	`DOT-Testing/composition/poc1-ric-aaa/`
2	Repeated Flower trainers over gRPC/Protobuf and exposed BentoML inference	`DOT-Testing/composition/poc2-federated-inference/`
3	Five-stage contractless JSON insider-threat analytics dataflow	`DOT-Testing/composition/poc3-insider-threat-pipeline/`

PoC	Composition focus	Fixture
4	Prometheus metrics, Elasticsearch logs, Grafana, and Kibana configuration	`DOT-Testing/composition/poc4-continuous-monitoring/`
5	Monitoring-to-analysis-to-assessment-to-mitigation security pipeline	`DOT-Testing/composition/poc5-security-pipeline/`
6	OpenAPI plus brokered AsyncAPI/MQTT, CACAO, OpenC2, and NWDAF remediation	`DOT-Testing/composition/poc6-closed-loop/`
7	Least-privilege packet capture, explicit CloudEvents mediation, and trust scoring	`DOT-Testing/composition/poc7-trust-scoring/`
8	cAdvisor/Prometheus observability and physical/twin integrity checks	`DOT-Testing/composition/poc8-ndt-observability/` and `poc8-twin-integrity/`
9	Amarisoft/Open5GS collection, InfluxDB storage, and data export	`DOT-Testing/composition/poc9-edge-ddos-collection/`
10	OIDC-protected capability exposure, SPARQL, OPA policy decisions, and observability	`DOT-Testing/composition/poc10-secure-capability-exposure/`
11	Reciprocal federation peers and local-core adapters with separate readiness edges	`DOT-Testing/composition/federation-peers/`

The authoritative coverage and gap list is `DOT-Testing/interoperability/pocs.yaml`. The official project descriptions are available from the MARE PoC catalogue.

Negative evidence is intentional

Rejected fixtures are part of the design evidence. They demonstrate that the model does not invent connectivity where the pinned source lacks a declared consumer input, does not treat stdout or a host path as a portable connection, and does not silently grant privileged runtime access.

Examples include:

- PoC7 rejects a collector-to-scorer wire when the collector declares no such Anchor or configuration input;
- PoC8 rejects an undeclared Prometheus-to-NDT handoff; and
- invalid conformance fixtures cover missing Contracts, selectors, ports, duplicate bindings, and incompatible Anchor pairs.

Failing closed is a successful test outcome when the available evidence cannot support the proposed composition.

Adding or promoting a scenario

1. Pin the exact external source revision used as evidence.

2. Document what the source actually produces and consumes.
3. Add deployable definitions according to ownership: third-party software under `DOT-Repository/DOTs/external/`, MARE-owned software under `DOT-Repository/DOTs/internal/`, and only synthetic fixtures under `DOT-Repository/DOTs/testing/`. Record their evidence and purpose in the applicable test evidence ledger and select them by exact `repository://<name>@<tool-release>` identity.
4. Define Contracts only for supported, observable interactions.
5. Use contractless Anchors only when no supported Contract standard applies.
6. Declare all service ports, configuration targets, privileges, and devices.
7. Add negative fixtures for tempting but unsupported connections.
8. Start under `DOT-Testing/composition/` with validate and resolved composition assertions.
9. Add a lifecycle dry-run before any live execution.
10. Promote to `DOT-Testing/e2e/` only with isolated resources, explicit prerequisites, reliable cleanup, and externally observable assertions.
11. Record what remains unproved in the interoperability ledger.

Interpreting results



A successful resolved composition proves model consistency. A successful apply proves that the supported deployment adapters completed. Only an active scenario probe can prove that traffic, authentication, authorization, policy, or security behavior worked as intended.

External DOT examples

The external-product examples demonstrate how an existing open-source workload can be described as a DOT without modifying that workload. The upstream project remains authoritative for the software, configuration, and interface Contract. The DOT descriptor selects a pinned artifact and declares the portable service surface that `dotx` can validate and deploy.

Upstream documentation is useful source evidence, but it is not acceptance evidence. DOT-Design therefore tests each example at separate levels:

1. direct `dotx validate`, using the reusable publication profile where applicable;
2. Contract-selector and DOT-to-DOT Anchor composition for reusable definitions;
3. offline inspection and deployment rendering;
4. networked rendering of the exact pinned Helm charts;
5. one real Linux lifecycle on Kubernetes/Helm or Docker; and
6. bounded repeated lifecycle stress with cleanup verification.

Canonical external catalogue

The external catalogue currently has one product folder for each of these 20 products, with exact release leaves: APISIX, cAdvisor, Elasticsearch, Filebeat, Grafana, GraphDB, InfluxDB, Keycloak, Kibana, Loki, Eclipse Mosquitto, Prometheus node-exporter, OPA, OpenTelemetry Collector, Oxigraph, Prometheus, Wireshark, NATS, Kafka, and RabbitMQ. PoC compositions reuse those identities and supply routes, realms, policies, scrape targets, data sources, credentials, files, and deployment selection as instance inputs. They do not create `pocN-*` copies of the product.

The following four products form the bounded release stress matrix. They are not the complete external catalogue.

Release stress matrix

This table describes the **current source checkout's** stress profile. The released quickstart retains its own manifest-pinned catalogue; use that exact artifact set rather than treating this table as an instruction to upgrade it.

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Keycloak@26.8.1`	Stable reusable external DOT	Codecentric `HELM_CHART 7.4.0`; official Keycloak `DOCKER_COMPOSE`, `DOCKER_IMAGE` and native Kubernetes deployments, all targeting 26.8.1 in one release definition	Exact Keycloak 26.8.1 Admin REST OpenAPI plus an attributed OIDC endpoint projection; Admin realm, JWKS, and client-credentials provider Anchors	Exact StatefulSet binding and Compose `start-dev`; readiness; obtain an admin token and assert `GET /admin/realms/master` returns the master realm through both mechanisms
`Loki@3.7.9`	Contract-backed HTTP subset; strict interface gaps remain	Grafana Community `HELM_CHART 20.2.2`; official Loki `DOCKER_COMPOSE` image pinned by digest	Attributed source-derived OpenAPI subset for readiness, build information, JSON log ingestion and querying; not a vendor-published complete HTTP/gRPC/OTLP specification	Exact single-binary StatefulSet binding and readiness; ingest and query a uniquely labelled real log through both mechanisms

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Prometheus@3.15.1`	Stable reusable external DOT	Prometheus Community `HELM_CHART 29.38.2`; official Prometheus `DOCKER_COMPOSE` image pinned by digest; `DOCKER_IMAGE` for composition-owned scrape configuration	Exact Prometheus 3.15.1 OpenAPI 3.1 artifact for build information and PromQL; a required Node Exporter metrics interface materialized through `prometheus.yml`	Exact server Deployment binding; readiness; assert `/api/v1/status/buildinfo` reports version `3.15.1` through both mechanisms
`Wireshark@4.6.1`	Runtime sample, not reusable	LinuxServer versioned `DOCKER_IMAGE`; equivalent `DOCKER_COMPOSE` image pinned by digest	No fabricated Contract: the image exposes a human web desktop, not a supported machine API	Container state/environment/volume/ports plus HTTPS for both mechanisms

Each product is kept under `DOTs/external/` because it is owned outside MARE, with every supported mechanism for its selected actual tool release in the same release `dot.json`. Additional supported releases occupy separate exact release folders. See Catalogue release selection. Ownership does not imply interoperability maturity. Keycloak and Prometheus declare official specification payloads and pass the reusable profile for the documented interfaces. Loki has real HTTP-subset acceptance but retains native gRPC/OTLP coverage gaps; it is not full interface certification. Wireshark supplies runtime and web-desktop reachability evidence, not a machine API integration. Chart versions, Compose image indexes, and the direct Wireshark image tag are deliberately pinned. `latest` is convenient for exploration but unsuitable for release evidence because a repeated run could execute different bytes. The Wireshark OCI index currently contains Linux amd64 and arm64 manifests. Stable `0.2.0` retained a historical cross-built arm64 preview, but subsequent DOT-Design release packaging and automated acceptance intentionally target Linux amd64 only.

Authoritative upstream references:

- Keycloak 26.8.1 release assets, Keycloak Admin REST API, Codecentric Keycloak chart, and Keycloak containers
- Grafana Loki Helm documentation
- Prometheus 3.15.1 official OpenAPI 3.1 artifact, Prometheus Community chart, and Prometheus installation
- LinuxServer Wireshark image

Run the examples

Run commands from the `DOT-Design` root with a `dotx` built from the checked-out `DOT-CLI` revision.

```
# Fully offline and non-mutating
task testing:dots:external:offline

# Downloads and renders the pinned charts, but does not deploy them
task testing:dots:external:dry-run

# Deploys each example once and cleans it up
task testing:dots:external:live

# Repeats the complete lifecycle three times by default
task testing:dots:external:stress
```

The live profiles require Linux, Docker, Kubernetes, Helm, `kubectl`, `curl`, `jq`, and network access to the pinned public artifacts. Prepare a Ready selected cluster; `dotx tool check kubernetes --start` explicitly permits starting Minikube when needed. Plain `dotx tool check kubernetes` checks readiness without starting a cluster. Set `DOTX_E2E_SKIP_HELM=1` or `DOTX_E2E_SKIP_DOCKER=1` only for focused local diagnosis; a release acceptance run must not skip either runtime.

What the stress profile adds

`DOTX_E2E_STRESS_REPEATS` accepts a bounded value from 1 through 5. Every cycle uses a new namespace, Docker runtime name, Compose project names, marker, and host-port range. It executes validation again before runtime mutation, selects each mechanism explicitly, verifies workload readiness and HTTP/HTTPS reachability, and destroys all resources. The runner then fails if the cycle namespace, Wireshark container, or any of the four Compose projects still exists. If a Kubernetes deployment or readiness check fails, the runner records pod, workload, event, and bounded container-log diagnostics before cleanup and skips only the assertions that are causally blocked by that failed workload. The root failure still fails the cycle; dependent timeout noise is not counted as independent evidence.

The resulting matrix contains eight cells:

- Keycloak: `HELM_CHART` and `DOCKER_COMPOSE`;
- Loki: `HELM_CHART` and `DOCKER_COMPOSE`;
- Prometheus: `HELM_CHART` and `DOCKER_COMPOSE`; and
- Wireshark: `DOCKER_IMAGE` and `DOCKER_COMPOSE`.

This is a runtime-mechanism matrix, not a reusable-interface count. Only the Keycloak and Prometheus cells carry official machine-readable Contracts; Loki's HTTP cells use an explicitly source-derived subset. A separate locked three-DOT composition wires those two providers to a headless consumer, resolves the exact official selectors, binds the Keycloak `{realm}` parameter, and asserts the resulting operation paths and environment materializers:

```
task testing:compose:keycloak-prometheus-openapi-validation
```

```
DOTX_E2E_STRESS_REPEATS=5 task testing:dots:external:stress
```

Evidence is written to `DOT-Testing/e2e/target/external-dots/`:

- one log per cycle;
- `summary.json`, including the DOT-CLI and DOT-Repository revisions; and
- `SHA256SUMS` covering the summary and logs.

The dedicated `External DOT examples` GitHub Actions workflow runs the non-mutating profile for relevant changes. Its weekly and manually dispatched Linux job runs the stress profile and uploads the evidence bundle. Push, schedule, and manual runs use distinct concurrency lanes, so requesting a live stress run cannot cancel the required push conformance result for the same revision.

Recorded `0.2.0-rc.3` evidence

External DOT examples run 38708940363 completed two independent Linux lifecycle cycles against DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`. Each cycle passed all 47 checks and cleaned up its resources. The complete eight-cell mechanism matrix was:

- Keycloak: Helm chart and Docker Compose;
- Loki: Helm chart and Docker Compose;
- Prometheus: Helm chart and Docker Compose; and
- Wireshark: Docker image and Docker Compose.

For both Keycloak mechanisms, the test invoked the exact Admin REST operation selected from the pinned official Contract. For both Prometheus mechanisms, it invoked the exact build-information operation selected from the pinned official Contract. The uploaded evidence contains `cycle-1.log`, `cycle-2.log`, `summary.json`, and `SHA256SUMS`; local checksum verification of the downloaded artifact passed for both cycle logs and `summary.json`. This evidence supersedes the earlier `rc.2` run because it exercises Keycloak-DOT `0.1.1` with the corrected Helm resource profile and pins the exact candidate CLI and repository revisions.

Interpreting a pass

A pass means the pinned public artifacts satisfied the assertions executed in the disposable test environment. For Keycloak and Prometheus this includes a successful request to the exact operation selected by the DOT Anchor, in addition to readiness. It does not establish production sizing, availability, security hardening, backup/restore, or every upstream feature. In particular, the Wireshark example checks its web application surface; it does not grant packet-capture capabilities, host networking, or a CDI device. Those requirements must be declared and explicitly authorized in a separate acceptance profile.

When adding another external DOT, place it under `DOTs/external/` because of its ownership and provide an immutable runtime artifact. Use a vendor/project- official Contract whenever available. Otherwise either pin a runtime-extracted Contract, maintain a clearly identified derived Contract

with live conformance, or expose an adapter-owned Contract. Every Contract asset declares its authority, source revision, license, and SHA-256. A runtime-only external DOT remains valid without Contract declarations but makes no interoperability claim. Owned declarations belong in `dot.json` as raw HTTPS URLs plus SHA-256; no local Contract directory is required. Add production claims only after a test explicitly proves them.

Production integration checklist and limits

Qualify the integration you will operate. A successful release qualifies its recorded tests and artifact set, not every DOT, API, environment or data store. The supported execution boundary remains Linux amd64 and existing single-host profiles. No new portable-model fields or managed multi-host runtime are needed to use those profiles.

Choose the evidence you need

Evidence	Meaning
Definition validation	Structure, exact references, verified payloads and applicable bindings pass.
Strict interface profile	Declared Anchors and exposed ports have the required machine-readable coverage. This still does not establish live behavior or authorization.
Actual producer acceptance	The selected real tools, versions, configuration and interactions passed. Protocol fixtures and producer doubles remain labelled separately.
Operational acceptance	The operator has verified access controls, storage/retention, upgrade/recovery, monitoring and ownership-safe cleanup in the intended environment.

Use the release record for exact 0.4.0 identities. The manifest-pinned released catalogue retains its original tool versions. Current Next may update a product without changing the models: the post-release Loki definition selects 3.7.9 and chart 20.2.2. Do not attribute a new catalogue deployment to an older receipt or silently upgrade an existing data store.

Triage a catalogue gap

From the **DOT-Design checkout root**, run the existing report procedure in Verification and examples. Each strict result now carries its diagnostic code, definition hash and review category. Categories describe declarations and paths; they are not a new validator or an exemption from the strict profile.

- **Partial interface:** inspect the reported Anchor or port. Complete a real supported operation and its service binding. Keep UI, native-protocol and provider-neutral boundaries explicit when they cannot be represented completely.
- **Missing exposed Contract:** inspect the actual implementation and prefer its release-matched official raw specification. A source-derived subset must say which operations were verified and which remain outside it.
- **Client or broker:** a consumed API or application message does not become the client's or

broker's owned API. Verify real configuration and transport behavior; do not invent an HTTP endpoint or arbitrary application message schema.

- **Planning or fixture:** require the actual producer and deployment before claiming producer readiness. Automated doubles remain useful test evidence.

For example, Loki's HTTP subset can verify real JSON log ingestion and querying while the native gRPC port remains a strict-profile gap. Similarly, a successfully composed OIDC requirement does not create an APISIX-owned Keycloak Contract.

Record one production acceptance

Record the exact binary, catalogue, DOT/deployment IDs, payload/image/chart pins, reviewed lock, environment and operator. Use private state and inputs, reserve names/ports/namespaces, and reject collisions. Keep real credentials out of commands, documents and evidence; use the supported Secret/input mechanisms.

Verify the actual consumer reaches the intended producer. Include the normal application result, unauthorized/invalid requests, version identity, a bounded failure/recovery case and absence of test-owned leftovers. Preserve unrelated resources and externally owned gateways. A running Pod or matching protocol name is insufficient evidence.

Retain the original binary, source hashes, frozen kubeconfig and private resource receipts until cleanup is verified. The external harness persists the actual namespace creation response with owner-only permissions. Recovery checks its recorded name/UID against the current API object; replacement objects are rejected. A missing or partial response remains unconfirmed and requires operator review. Do not reconstruct ownership from a later matching name. Namespace removal alone also does not prove persistent-volume reclamation. The shared harness waits for PVs referencing that namespace to disappear and reports cleanup failure if storage remains or discovery is unavailable. It performs no forced PV deletion or provisioner-identity adoption. In particular, a restarted Minikube host-path provisioner can refuse its earlier volumes; retire confirmed test data through a separately reviewed operator action.

A post-release Helm fix removes the automatically injected `rolloutToken` chart value. Strict chart schemas must remain enabled; a CLI must not invent a chart option to request a restart. This source fix is separate from the immutable released `v0.4.0` binary. A second fix makes path-based Docker `start`/`stop` honour `--name`; the restart acceptance checks the selected container actually stops and retains its identity and original log after restart. Explicit Docker container names retain their exact case, matching deployment and destruction; Compose project names retain Docker Compose's normalization. Keep the original binary and recorded context when operating existing resources. Kubernetes/Helm path start/stop reject `--name` rather than silently targeting a product peer; use reviewed native workload lifecycle commands with an explicit namespace.

The Loki external harness now asserts its selected build version, an initially empty uniquely labelled stream, accepted JSON ingestion, the exact correlated log result and rejection of numeric timestamps on Helm and Compose. Its source-derived OpenAPI subset does not claim every LogQL query, metric query, gRPC method, OTLP signal or authorization policy. The shared external lifecycle harness owns deployment and cleanup; the HTTP assertion helper does neither. The Compose configuration points filesystem data at its named `/loki` volume. A separate owned-container recreation check requires a different container ID, the original volume and the original

log without a replacement push. Unrelated network/volume changes are rejected. This is local retained-storage evidence, not a consistent backup, restore, migration or HA claim.

Remaining implementation boundaries

Work	Current boundary	Acceptance before extending it
External Protobuf import graphs	Embedded standard imports are supported. Arbitrary producer imports are not resolved by fetching mutable adjacent files.	Preserve a cryptographically closed root and dependency graph, official package and operation names, deterministic selection and byte limits. Compare a complete descriptor-set artifact with an explicitly pinned source bundle. Reject missing, conflicting, cyclic, ambiguous or path-escaping dependencies. A new format/profile requires a separately reviewed design; no new model fields are added here.
Complete Loki and OTLP interfaces	The Loki HTTP log subset is separate from imported native gRPC/OTLP schemas.	Use the selected release's official graph and actual receiving implementation. Test real ingestion and retrieval/export, authentication and malformed input for each claimed signal. A successful HTTP push or broker fixture cannot qualify it.
Stateful recovery	Existing lifecycle controls and stateless rollback do not restore application data.	In an isolated owned deployment, write identifiable data, verify persistence across stop/start, create a native consistent backup, restore into a separate owned target and compare application results. Inject interruption and preserve the original data and unrelated peers. Record RPO/RTO and storage/version limits; do not treat restarting a container as backup/restore or infer schema rollback.
Distributed orchestration	Operator-owned external gateways support explicit connections; they are not managed multi-host networking or automatic infrastructure provisioning.	First demonstrate the actual consumer route and selected-producer attribution, denial/failure/recovery, credentials and independent resource ownership. Record the required infrastructure and failure model before proposing a new execution profile. Keep the present single-host boundary unchanged.

Until these criteria are met, retain the specific limitation. Do not make an unsupported path look successful through a placeholder Contract, automatic state adoption, a weaker timeout or a test double described as the producer.

Human trials and owner handoff

Three actual unfamiliar developers must still follow the complete onboarding protocol. Start from one verified binary/catalogue kit; retain first failures, assistance, timing, Contract selection, safe

edit/cancel, composition/lock review, live results and cleanup. Automated terminal tests and maintainer rehearsals do not count as participants. Use the existing evaluation kit; no Java or Go is required for the first participant monitoring path.

The kit's `asset-inventory.json` supplies source declarations and pending owner questions. The owner must record terms, permitted distribution, custodian, hosting/recovery responsibility and access policy for each relevant asset. Unlisted Gist URLs are anonymously readable and currently use a personal account; hashes verify bytes but do not grant rights or guarantee future availability. Do not change custody, publish participant records or infer approvals on the owner's behalf. These decisions and consent/retention remain pending.

Integration handoff

This handoff defines the interoperability boundary for a team implementing a catalogue, composer, Security Function Composer adapter, or other DOT consumer. It intentionally does not prescribe a service, database, API style, language, or framework.

Authoritative inputs

The reference source of reusable definitions is `DOT-Repository`. Resolve a DOT by its exact textual ID and pinned version, then verify the returned `dot.json` matches both. Resolve referenced Contract payloads without copying their protocol, schema, media, or security facts into an Anchor.

Use these commands as executable reference behavior:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx validate DOT-Repository
dotx compose validate composition.yaml
dotx compose inspect composition.yaml --json
```

An implementation may expose equivalent behavior through REST, gRPC, events, an SDK, or an embedded library. Its external contract should be versioned and tested independently from the DOT model.

Disposition of legacy SFC requests

The original SFC issues 6 through 9 predate the stable repository-backed model. Their consumer needs are handled at the integration boundary without creating parallel sources of truth:

- **Parameter forms:** derive an SFC-facing form from exact service-scoped environment and configuration declarations plus authoritative Contract schemas. Composition validation rejects missing or ambiguous required inputs before runtime mutation. A duplicate root `params` model is not defined.
- **Skill search:** resolve stable, case-sensitive textual `Skill` IDs through the authoritative `DOT-Repository/Skills/` catalogue. Every referenced capability has one category there; `dotx repository list --skill <id-or-name>` and `--category <id-or-name>` expose the tested consumer projection without copying names or categories into every DOT as a second controlled vocabulary.
- **Catalogue API:** `DOT-Repository` and `dotx repository` define reference behavior. A catalogue team may expose equivalent queries through any versioned API; DOT-Design does not own or prescribe that service.
- **Low-barrier authoring:** a DOT author may hand-write one `dot.json`, place its folder under the appropriate recursive `DOT-Repository/DOTs/` category, and validate the whole filesystem catalogue with `dotx validate DOT-Repository`. There is no duplicate inventory file or wrapper script. This path requires no Maven, Jsonnet, MongoDB, or running service. `dot.json` remains the sole deployment definition; a parallel YAML format and inline copies of referenced entities

are intentionally not introduced.

These dispositions settle DOT-side model ownership. They do not replace joint SFC acceptance or prove a partner catalogue/search implementation.

Required semantics

An interoperable consumer must:

- address DOTs by exact textual ID and pinned version; preserve exact DOT-local Anchor IDs and Composition ownership UUIDs;
- resolve Skills and Contracts by their exact, case-sensitive textual IDs; legacy UUID-shaped IDs remain valid, but names and case-folded aliases do not replace identity;
- require at least one meaningful Skill reference per DOT and consume the generated nonempty-list constraint rather than a separate handwritten rule;
- resolve contract-backed selectors against the authoritative OpenAPI, AsyncAPI, Protobuf, GraphQL or supported SPARQL document;
- use `anchorBindings` to map a logical Anchor to a named service port;
- keep instance topology, endpoints, parameter values, credentials, and policy outside reusable DOT definitions;
- evaluate compatibility conservatively and preserve `UNKNOWN` as distinct from `COMPATIBLE`; and
- keep secret values out of DOTs, resolved compositions, persisted state, and logs.

Contractless Anchors are valid only when a supported Contract standard cannot describe the interaction. Their explicit direction, protocols and media types must be compatible before a wire is accepted. These communication checks do not establish application meaning or arbitrary schema substitutability. Contract-backed compatibility uses verified operation selectors and the supported interface rules; documentation URLs and capability names do not establish a match.

Acceptance evidence

The versioned consumer profile is under `DOT-Testing/acceptance/dot-anchors/v1/`. It resolves two catalogued test DOTs, verifies the committed composition lock, and resolves 21 compatible wires across OpenAPI, AsyncAPI, Protobuf and GraphQL. The supported SPARQL query profile is outside this focused matrix; passing it does not establish SPARQL runtime acceptance. See the individual probe profiles for their separate supported operations and evidence boundaries.

```
task testing:anchors:acceptance
task testing:conformance
```

Passing the deterministic profile proves discovery and resolution semantics. A joint live test must separately prove endpoint reachability, credential and token handling, authorization, policy enforcement, and application behavior.

Future implementation freedom

The removed catalogue-service prototype is not part of the production architecture. A future team may replace it or implement discovery elsewhere in a different language or framework. Compatibility is judged by the stable DOT model and executable acceptance profile, not by reproducing the prototype's internal classes or former HTTP response shapes.

Release notes

Next

The catalogue now groups definitions as `<product>/<tool-release>/dot.json`. Each release retains all its deployment mechanisms; product folders contain navigation only. Existing definition bytes, exact IDs, payload pins and reviewed composition locks are preserved. Discovery and guided authoring list product releases together, newest first, without choosing an implicit upgrade.

This is a current-source filesystem migration, not a new released binary or qualification of historical tool versions. Stable snapshots remain unchanged. Use exact release selection and the matching source checkout. No portable model or deployment-state migration is introduced.

0.4.0

0.4.0 adds guided Contract authoring across the supported standards, safe editing into a separate candidate, and composition creation from exact catalogue DOTs. Shared native forms provide searchable selections and explicit draft approval. Runtime diagnostics and architecture guards retain the existing model authority and execution safety checks.

The approved file-document migration replaces inline/Base64 contents with verified HTTPS URL/hash `Payload` values and renames configuration-file `defaultContent` to `payload`. This intentionally breaks those older input shapes; it is not a transparent 0.3.0 patch. Contract payload JSON is unchanged. Use the 0.4.0 CLI, generated schemas and pinned catalogue together, and follow the migration procedure. Existing released artifacts and stable-tag documentation are preserved.

The owner authorized direct 0.4.0 publication. Three unfamiliar-developer trials and owner reviews remain pending follow-up; automated qualification does not claim independent adoption or legal clearance. See release qualification and known limits. Native SPARQL query Contracts and metrics response validation are included; OpenMetrics is a representation within OpenAPI, not a separate standard enum.

0.3.0

The aligned Linux-amd64 release promotes `0.3.0-rc.1` without changing portable Java models, generated schemas or Go bindings, catalogue definitions or locks. The model archive retains all 77 schemas and the reviewed compatibility corpora; its release label advances without rewriting historical corpus identities. See Compatibility and migration and Catalogue readiness.

Use the selected CLI release and the exact catalogue revision in its manifest. Offline validation requires its warm verified Contract cache; `compose new` creates a template-based draft, not an automatic graph or state migration. Source, rendered-command and evidence-boundary regressions protect the guidance. Final qualification still requires the local release gates, a fresh exact-source demo and successful mandatory remote checks; downloaded-binary acceptance is separate from publication. The supported single-host profile is not certification of every MARE producer or a final 1.0 API guarantee.

Immutable stable documentation is cut at `0.3.0`. The development branch resumes the Next descriptor afterwards; historical tags and documentation are preserved.

0.3.0-rc.1 candidate

The candidate model surface is reviewed in `DOT-Domain/MODEL-SPEC.md`. A reproducible, checksummed model-baseline archive includes all 77 public schemas, shared compatibility corpora and hash inventory. Java, generated schemas and Go remain one authority chain. This is not a final stable release or certification of all producers; publication requires the separate release/runtime gates.

`dotx compose topology <composition-or-directory> --write` now maintains the README Mermaid block, wire legend and adjacent `topology.mmd` natively in Go. `--check` rejects drift without writing. Both require verified locks and preflight all graphs/targets; Task wrappers delegate to the command instead of the removed `Compositions/topology.sh` generator. These remain resolved composition artifacts, not live acceptance evidence.

Catalogue DOT IDs now follow `<name>@<actual-tool-release>` and all 50 distinct Contract IDs follow `<origin-DOT.id>::<standard>::<name>`. For example, `Prometheus@3.15.1` declares `Prometheus@3.15.1::OpenAPI::HTTP`. DOT and composition DOT-reference types are now textual, generated from DOT-Domain. Legacy UUID strings remain valid inputs, but migrated entries do not provide aliases. Catalogue Anchors now use DOT-local kebab-case keys; Composition ownership UUIDs are unchanged. Every DOT now requires at least one Skill reference; native authoring accepts repeated `--skill <id>` flags and prompts interactively. See the catalogue's `IDENTIFIER-MIGRATION.md` for exact mappings and state safety. Destroy old instances using their original checkout and CLI before migrating; regenerate and review locks, then rehearse fresh deployments. Do not reuse old runtime state or treat previous rehearsal receipts as evidence for the new graph. These development-format migrations are included in the 0.3 line, not silently backported to historical 0.2 releases.

The catalogue has 67 canonical product/component definitions, each with one root `dot.json` containing all supported deployment choices. Nested `versions/` bundles and duplicate product definitions are rejected by catalogue CI. Tool upgrades update all mechanisms, release-matched Contracts, explicit references and reviewed locks together; Git retains previous releases. APISIX 3.19.0, Keycloak 26.8.1, Prometheus 3.15.1, OPA 1.21.4 and Elasticsearch 9.5.5 use stable upstream artifacts verified on 2026-10-26. Unreleased internal/validation-only/demo components remain labelled as such. Official Contract documents remain byte-for-byte upstream artifacts; derived configured-route/policy projections remain explicitly derived. Static validation does not establish full producer certification.

The catalogue now uses explicit readable IDs for its five Skill categories and eight capabilities. DOT references contain only `id`; labels and descriptions resolve from `Skills/`. Existing references and affected composition locks have been migrated together. UUID-shaped Skill IDs remain supported in independently authored or historical catalogues, but old UUIDs are not implicit aliases in the current catalogue. That earlier Skill-only migration did not change the other identities; the subsequent DOT/Contract migration is described above.

Automated native Prometheus and quickstart acceptance now coordinate unused TCP ports with

per-user leases, check IPv4/IPv6 listeners, and reject occupied explicit overrides without stopping existing services. Leases are released after cleanup, including controlled failures. The manually followed Prometheus walkthrough retains its documented port defaults.

Contract `standard` now accepts only `OpenAPI`, `AsyncAPI`, `Protobuf`, and `GraphQL`, generated from the Java domain enum's Jackson values. This is an intentional input-format change: uppercase values and case/whitespace aliases are rejected. Migrate owned Contract declarations and explicit origin references, then explicitly regenerate and review affected composition locks with `dotx compose lock <composition.yaml>`. Existing locks fail closed; do not reuse saved runtime state against a changed graph. The `format` values remain `YAML`, `JSON`, `PROTOBUF`, and `GRAPHQL`. Use the selected 0.3 release or matching source; historical 0.2 binaries retain the earlier vocabulary. Stable-tag docs remain unchanged.

`dotx compose inspect` is the read-only command for resolving and explaining a composition. Derived execution details are called the **resolved composition**, not a separate authored model. Deployment commands are previewed with `dotx compose apply --dry-run`. Locks and saved state now use `resolutionDigest`; locks must be regenerated and reviewed. Clean up running compositions with the original binary before upgrading: saved state is not migrated, and retired commands have no aliases. Use the selected release or its matching source checkout.

The official ecosystem presenter route now connects reuse across Docker and Kubernetes, secure multi-DOT policy enforcement and native stateless recovery. Shared preparation/preflight cover five lanes; the full clean-source rehearsal runs seven live paths twice and links cleanup-verified child evidence. The existing runtime workflow offers an explicit long-running full-demo dispatch.

Development CLI builds add `compose topology --status`: a locked composition graph with scoped, bounded runtime observations and a distinct historical probe table. Matching saved execution inputs are required before runtime queries. Presenter pauses use this view without treating running workloads or a recent probe receipt as current API acceptance. Mermaid export remains declaration-only; there is no portable-model or schema change, and no new release is implied.

Contracts now resolve only from their declaring DOT bundle or a complete digest-pinned URL declaration. The repository-level Contract catalogue and validator fallback have been removed; all remaining assets live in their owning bundles. Repository validation still rejects conflicting definitions of the same Contract ID. The shared local/GitHub repository gate additionally validates every bundle in isolation and offline.

Getting started now begins with a checksummed Linux-amd64 CLI installation and a complete Prometheus Docker Compose lifecycle, with API/version verification, cleanup, and troubleshooting. Contributor toolchains and shared CI gates have their own guide. Antora includes the same command blocks executed by `testing:quickstart:dry-run` and `testing:quickstart:live`; the external-DOT workflow rehearses both, including port overrides and cleanup after failure. Development pages now export their own PDF alongside the latest stable PDF, matching the existing download link in the site header.

The Antora site now includes a guided demo for presentations and onboarding, with a tested five-minute read-only path, expected evidence, an optional live Contract-probe path, and explicit support boundaries.

`dotx compose apply --dry-run` no longer creates or changes the composition state/lock directory. Lifecycle rendering therefore works with a read-only cache while retaining the same fail-closed resolution and runtime checks.

0.2.4

`0.2.4` is a backward-compatible Linux-amd64 hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, deployment, and CLI interfaces published in `0.2.3`. The exact pinned revisions are DOT-CLI `e88daa83221ae5d49a1317b7412045ab2b78315f` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

DOT-CLI now exposes a local/GitHub `task ci:security` gate using the pinned stable OSV-Scanner release. It checks the module dependency graph and performs Go call analysis against the current OSV database. The network-backed scan is kept separate from deterministic `task ci` because vulnerability data changes independently of a source revision.

Standalone DOT-CLI packaging now emits a deterministic CycloneDX 1.6 SBOM for the exact Linux-amd64, CGO-disabled build profile. Release tests validate the SBOM identity, platform properties, dependency content, omitted timestamp and serial number, checksum coverage, and byte-for-byte equality across two package builds. Downloaded-binary acceptance retains the published SBOM as evidence.

Readable workflow badge publication now refreshes the complete status set on a coalesced 15-minute schedule, with manual dispatch available for immediate refresh. It no longer consumes one self-hosted-runner job for every completed workflow in the already large validation matrix.

Wasmtime lifecycle status and inventory checks now tolerate the short, bounded interval in which Linux procfs may expose an empty process command line during `exec`. A non-empty command line that does not match the recorded Wasmtime module identity is still rejected immediately, preventing lifecycle operations from acting on a reused PID.

Antora publication now deduplicates an exact stable tag from its identical release-commit worktree, closing the tag-push race while preserving immutable stable documentation. Current-profile documentation is guarded against the machine-readable acceptance profile: twenty-one wires, all four finite side-effect-free gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery.

The complete pushed workflow matrix passed on `77d8f099e5f42430f1d30e8d01c27b2c534b8d49`. It includes 100% DOT-Domain instruction and branch coverage, 100% DOT-CLI aggregate and per-executable- package statement coverage, repository conformance, all twenty-one cross-standard wires, real-producer and real-broker active probes, Docker, Kubernetes, Helm, Vagrant, and WebAssembly lifecycle evidence, and versioned Antora publication. DOT-CLI `v0.2.4` additionally passed two-pass reproducible packaging and downloaded-binary acceptance.

The release publishes the versioned DOT-Domain JAR, DOT and Composition v1 JSON Schemas, JSON and XML CycloneDX SBOMs, the Linux-amd64 `dotx` binary, an exact source manifest, and checksums. Linux amd64 remains the only supported release target. DOT-Registry REST API work, native arm64, non-Linux targets, multi-host orchestration, and joint Security Function Composer or partner acceptance remain outside this release boundary.

0.2.3

`0.2.3` is a backward-compatible production-hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces published in `0.2.2` while broadening standards-authorized runtime evidence and strengthening deployment-configuration acceptance. The exact pinned revisions are DOT-CLI `b07ec3b568b59cb25631a9b51700a14ee124bb05` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

The cross-runtime configuration matrix now exercises every executable adapter through separate default-only and explicit-override paths. It verifies non-sensitive `defaultValue` and `defaultContent` materialization, explicit environment/file precedence, resolved service targeting, and redacted dry-run diagnostics.

Contract-authorized gRPC probes now cover unary, server-streaming, client-streaming, and bidirectional methods. All methods still require the Proto3 `NO_SIDE_EFFECTS` option and server reflection. Streaming fixtures must declare an exact finite response count; input and output messages are validated against the authoritative descriptors; and expectations explicitly select `ALL`, `ANY`, `FIRST`, or `LAST` response matching.

AsyncAPI delivery now supports NATS core messaging beside MQTT. NATS fixtures require the official channel binding and a concrete non-wildcard subject, then prove UUID-correlated subscribe-before-publish delivery through digest-pinned official NATS client and server images. The live standards fixture exercises all four gRPC shapes plus MQTT and NATS while retaining redacted state evidence.

AsyncAPI delivery also supports Kafka through the official 0.5.0 channel binding. `dotx` honors a binding-level `topic` override, rejects invalid Kafka topic names and MQTT-only QoS, waits for a correlation-scoped consumer-group assignment before publishing, and verifies the exact correlated record through the digest-pinned official Apache Kafka 4.3.1 JVM image. The ephemeral consumer group is removed; the record itself remains subject to the topic's broker retention policy. The canonical runtime-only Kafka DOT expands the locked standards composition to twenty wires and three real message brokers.

AsyncAPI delivery now also supports AMQP 0-9-1 through the official 0.3.0 binding. `dotx` validates exact exchange, routing-key, virtual-host, durability, auto-delete, and delivery-mode semantics; binds an exclusive auto-delete probe queue before publish; acknowledges and correlation-matches the delivered JSON; and obtains RabbitMQ credentials only through a secret reference. Queue-bound channels, headers exchanges, AMQPS, and AMQP 1.0 fail closed. The canonical digest-pinned RabbitMQ 4.3.6 runtime DOT expands the locked standards composition to twenty-one wires and four real message brokers.

The complete pushed workflow matrix passed on `79807aacebe7aab1112fe82104898ab4a0929db0`, including DOT-CLI CI, Contract active probes, compatibility matrix, coverage, Kubernetes and Helm runtime E2E, DOT repository conformance, and versioned Antora publication.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, Linux-amd64 `dotx` binary, immutable release manifest, and checksums. The tag workflow performs deterministic two-build packaging before publication, and post-publication

acceptance downloads and exercises the released binary.

Existing `0.2.0` through `0.2.2` definitions, compositions, and locks remain valid. Unsafe or unbounded gRPC, AsyncAPI transports other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, multi-scheme authentication, multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, native arm64, non-Linux executables, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.2

`0.2.2` is a backward-compatible extension of the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces from `0.2.1` while completing the standards-authorized active-probe profile. The exact pinned revisions are DOT-CLI `2dafb492bb657da4d6e6dfe3eab4e9ee4ee229c9` and DOT-Repository `31357e57ab1332e93de6afc25a2b0d67aafed857`.

GraphQL query probes validate documents, variables, and optional exact response expectations against the authoritative SDL. Mutations require verification and cleanup operations; GraphQL-over-SSE subscriptions require a separately triggered event plus cleanup. Unary gRPC probes require server reflection, protobuf-JSON request validation, and the Proto3 `NO_SIDE_EFFECTS` option. AsyncAPI MQTT probes require an explicit binding, concrete topic, JSON-Schema-valid correlated message, QoS, subscribe-before-publish ordering, bounded timeout, and exact non-retained delivery. All fixtures remain composition-owned and persisted evidence remains redacted.

The fifteen-wire deterministic matrix covers OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Proto3, and GraphQL. A real GraphQL.js/gRPC producer and Eclipse Mosquitto broker provide live Docker acceptance. The complete pushed workflow matrix passed on `70b0e94760d60bc0fc37f887db0466796627829a`, including Contract active probes, coverage, compatibility, and Antora publication.

Existing `0.2.0` and `0.2.1` definitions and compositions remain valid; probe fields are optional and additive. Streaming or safety-unclassified gRPC, non-MQTT AsyncAPI transports, multi-scheme probe authentication, native arm64, non-Linux executables, multi-host orchestration, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.1

`0.2.1` is a backward-compatible hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the `0.2.0` DOT, Contract, Anchor, composition-schema, repository, and deployment interfaces. The exact pinned revisions are DOT-CLI `82d7de45800f4e4a435ef08202d4f1e42c16eca8` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

The patch adds overflow-safe SemVer precedence, atomic WASM module-cache replacement and restart-profile verification, fail-closed empty Docker port forwarding, correct Minikube home-directory expansion, and child-process checks for detached Kubernetes forwarding. Repository commands now share one verified private-cache acquisition path. DOT-Domain enforces 100%

instruction and branch coverage, while DOT-CLI enforces 100% aggregate and per-executable-package statement coverage. The installer missing-`curl` negative test is hermetic and cannot fall through to host networking or `sudo`.

The complete 13-workflow matrix passed on the pushed hardening baseline `7c71e60393b1758d45e3558a15a2073450cb5cfc`, including coverage, Anchor conformance, and repository conformance. The tag workflow must also pass deterministic two-build packaging and checksum verification. Post-publication acceptance downloads and exercises the released binary rather than trusting only source-tree tests.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, release manifest, checksums, and `dotx` for Linux amd64. Linux amd64 is the only supported executable target. DOT-Registry REST API work, native arm64, non-Linux targets, and joint Security Function Composer/partner acceptance remain outside this release boundary.

0.2.0

`0.2.0` promotes the fully accepted `0.2.0-rc.8` candidate as the first stable release of the `dot.mare6g.eu/v1` composition profile. There are no behavioral changes from that candidate: the stable tag regenerates the versioned domain artifact, schemas, SBOMs, documentation, and Linux CLI assets from the same pinned DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3` revisions.

The release pipeline classifies `0.2.0` as stable and publishes it as the latest GitHub release. Before promotion, `0.2.0-rc.8` passed the full pushed commit matrix, reproducible release packaging, and post-publication Linux amd64 acceptance using the downloaded checksummed binary, including two repeated external-DOT lifecycle and cleanup cycles. Arm64 remains a cross-built preview until equivalent native evidence exists, and joint SFC acceptance remains a separate integration activity.

0.2.0-rc.8

`0.2.0-rc.8` supersedes `0.2.0-rc.7` with a current stable build and CI toolchain while retaining the accepted `dot.mare6g.eu/v1` composition, runtime, lifecycle, Contract, Skill-taxonomy, reusable-catalogue, and WASI profiles. Node.js 26 is now the explicit documentation and Contract-validation baseline; Go 1.27.2, Rust 1.98.1, Ruby 4.0, Java 27, and Maven 3.10.0 provide the remaining release toolchains. Direct Maven and Go dependencies and all third-party GitHub Actions were updated to compatible stable releases and immutable Action SHAs, with weekly Dependabot checks for Actions, Maven, Go, and the independently cloneable repositories. Documentation CI now rejects prerelease Ruby runtimes before Antora/PDF assembly.

Release publication now uses a tested, fail-closed SemVer channel classifier: prerelease versions such as `0.2.0-rc.8` are published as GitHub prereleases, while `0.2.0` is published as the latest stable release. The exact pinned revisions are DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

0.2.0-rc.7

`0.2.0-rc.7` supersedes `0.2.0-rc.6` with two release-evidence hardening changes while retaining the same stable `dot.mare6g.eu/v1` profile and pinned DOT-CLI and DOT-Repository revisions. Source-adapted PoC harnesses now acquire private producer source through one guarded helper: it checks out the full pinned commit in a fresh temporary directory, verifies `HEAD` and the expected GitHub origin, retries bounded transient failures, and rejects partial, untrusted, missing-revision, or unsafe-destination results. Deterministic tests exercise those positive and negative boundaries without network access.

Contract conformance now installs the official Protocol Buffers 36.2 compiler for Linux amd64 or arm64 through the shared local bootstrap, verifies the download with an architecture-specific SHA-256, and uses that exact binary for both positive and deliberately invalid Protobuf fixtures. Local Taskfile runs and GitHub Actions no longer depend on different compiler setup paths. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

0.2.0-rc.6

`0.2.0-rc.6` supersedes `0.2.0-rc.5` with one shared, fail-closed bootstrap for the official WASI HTTP fixture. Local `task ci:live` and GitHub Actions now fetch and build the same exact Bytecode Alliance `sample-wasi-http-rust` revision before executing the same Wasmtime lifecycle assertions. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Contract model, Skill taxonomy, and reusable catalogue structure are otherwise unchanged. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

The bootstrap verifies the exact upstream URL and commit, rejects non-Git, wrong-origin, and modified tracked source caches, and rebuilds the `.wasm` component rather than trusting an ignored build artifact. The GitHub workflow no longer contains a separate checkout/build implementation; it invokes the same `ci:wasm-runtime` Taskfile gate available locally. Documentation records the Rust 1.98.1, `wasm32-wasip2`, and first-build network prerequisites and the optional `WASI_HTTP_COMPONENT` override for a deliberately supplied component.

0.2.0-rc.5

`0.2.0-rc.5` supersedes `0.2.0-rc.4` with one fail-closed representation for complete Contract payloads. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Skill taxonomy, and reusable catalogue structure are unchanged. The exact pinned revisions are DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

Complete Contract declarations now carry a structured `payload` object:

```
"payload": {
  "url": "https://vendor.example/contracts/service-openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
```

```
}
```

The URL must be absolute HTTPS without user information or a fragment, and the digest must be exactly 64 lowercase hexadecimal characters. Raw embedded Contract documents and the former flat `payloadUrl`/`payloadSha256` fields now fail schema and `dotx validate` checks. UUID-only Contract references remain a migration-compatible form for verified repository-local wrapper definitions.

`dotx validate` downloads the authoritative bytes, verifies their digest, and uses `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN` only for `raw.githubusercontent.com`. Authorization is removed before following a cross-host redirect. Unit and repository tests cover malformed URLs and digests, unavailable assets, digest mismatch, authentication scoping, redirect credential leakage, inline payload rejection, flat-field rejection, and the official external Contract composition. The standalone DOT-Repository gate also runs the pinned validator directly for every catalogue revision.

GitHub Actions and local pre-push verification now share root `ci:*` Taskfile gates. `task ci:deterministic` covers the deterministic non-live release boundary; `task ci:live` covers the mutable Linux runtime workflows and requires the same official WASI HTTP component exercised by Actions, preventing a reduced local test path from being mistaken for workflow parity.

0.2.0-rc.4

`0.2.0-rc.4` retains the runtime and lifecycle profile accepted in `0.2.0-rc.3` and completes the closed SFC-requirement audit with repository-backed Skill discovery. `DOT-Repository/Skills/` now contains an authoritative asset for every capability UUID referenced by the catalogue and links each capability to one of the five controlled MARE categories.

`dotx validate DOT-Repository` resolves all 67 DOT definitions against that catalogue and rejects missing capabilities, category assets used as capabilities, duplicate references, and invalid category links. `dotx repository list --skill <uuid-or-name>` and `--category <uuid-or-name>` provide resolved, machine-readable discovery without copying category fields into every `dot.json`. The exact published revisions were DOT-CLI `3069f907bfd2056b16b92800bb5b98552c28832d` and DOT-Repository `3117a18da3004c57f4edf3714494fd3954225ae7`.

The audit also reconfirmed the existing DOT-side dispositions for stable Anchor identity and semantics, authoritative payload Contracts, service-scoped pre-deployment configuration validation, and single-file `dot.json` authoring. The separate DOT-Registry REST API remains outside this candidate, and joint SFC/partner acceptance remains a manual gate.

External products now have one canonical reusable DOT root rather than PoC-specific identities. Seventeen product roots plus two shared external test fixtures replace the former duplicated Keycloak, OPA, Prometheus, Elasticsearch, and APISIX definitions. PoC realms, policies, routes, scrape targets, data sources, secrets, and files remain composition-owned inputs; unavoidable source adaptations are named deployment variants under the same product UUID. Repository validation rejects any new deployable definition below `DOTs/external/pocs/`.

Composition lifecycle commands preserve the exact deployment ID rather than reducing it to a manifest type. The `--deployment` selector prefers an exact ID and accepts a manifest type or alias only when it identifies one deployment, so source-adapted and upstream deployments of the same

type cannot be selected silently.

Catalogue placement now records ownership rather than Contract maturity: third-party software is under `DOTs/external/`, MARE-owned/developed software is under `DOTs/internal/`, and only synthetic fixtures remain under `DOTs/testing/`. Every DOT bundle includes a local quickstart `README.md`. External/internal bundles opt into strict reusable validation by publishing a `contracts/` evidence directory whose assets identify their authority as upstream-official, runtime-extracted, DOT-ecosystem-derived, or adapter-owned, along with source revision, license, and SHA-256. Loki and Wireshark are now correctly classified as external runtime-only definitions without fabricated vendor Contracts.

0.2.0-rc.3

`0.2.0-rc.3` supersedes `0.2.0-rc.2` after post-publication, exact-pin lifecycle acceptance found that the Keycloak Helm workload could not reliably complete Quarkus augmentation within its declared 1536-MiB container limit. Three independent real-cluster runs retained pod and event diagnostics showing `OOMKilled` exit 137; the runner had 62 GiB of host memory and Minikube exposed 16.8 GB and 20 CPUs, isolating the defect to the DOT resource declaration.

Keycloak-DOT `0.1.1` declares a 2-GiB memory request and 4-GiB limit while retaining the same official Keycloak 26.3.3 Contract, Anchor selectors, chart, image, service, and Docker Compose mechanism. The external-DOT render suite now inspects the real upstream chart output and rejects any mismatch in the Keycloak container's CPU or memory requests and limits before live deployment. The official external-Contract composition and content-addressed lock resolve the versioned `0.1.1` definition.

External DOT examples run 38708940363 passed both the non-mutating chart conformance job and two complete Linux lifecycle cycles. Each cycle passed all 47 checks with zero failures and cleanup verification across Keycloak, Loki, Prometheus, and Wireshark's eight deployment-mechanism cells. The downloaded, checksum-verified evidence pins DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`.

This candidate also adds a post-publication workflow that downloads the actual checksummed Linux amd64 `dotx` release asset, verifies its release manifest and source revisions, then runs repository validation, Anchor acceptance, Composition v1 acceptance, and two complete external-DOT lifecycle cycles. Maven dependency submission is now an explicit self-hosted workflow; the billing-blocked GitHub-managed automatic submission feature is disabled. Linux amd64 is the release-accepted executable target. The cross-built arm64 asset remains preview-only until equivalent native arm64 evidence exists. Joint SFC/partner acceptance remains a separate manual gate and is not claimed by this candidate.

0.2.0-rc.2

`0.2.0-rc.2` supersedes the first candidate with the fixes and acceptance evidence completed after `0.2.0-rc.1` was tagged. The stable serialized composition API remains `dot.mare6g.eu/v1`.

Changes since rc.1

- Exact service-to-workload selection and configuration isolation for multi-service Docker Compose, Kubernetes-manifest, and Helm deployments.
- Exact official, digest-pinned OpenAPI Contracts and Contract-backed Anchors for reusable Keycloak and Prometheus external DOTs, with every Helm and Docker Compose deployment binding the same interface.
- `dotx validate --profile reusable` now enforces declared HTTPS Contract provenance, payload integrity, selector resolution, Contract use, and per- deployment Anchor binding. Review establishes that the pinned source is the official upstream artifact. `dotx validate <repository-root>` recursively discovers every `dot.json`; the later rc.4 ownership migration derives strict reusable validation from adjacent Contract evidence rather than category path, and is the single DOT-Repository GitHub Actions gate; no duplicate inventory or shell wrapper is required.
- Same-DOT alternative deployment mechanisms for the Keycloak, Loki, Prometheus, and Wireshark external-product examples. Loki and Wireshark were initially retained as runtime-tested samples; rc.4 reclassifies them as external runtime-only DOTs without claiming Contract-backed interoperability.
- A locked Keycloak/Prometheus/consumer composition proves that both stable external Anchors resolve and materialize the exact official operations.
- Failed Kubernetes/Helm workloads now retain pod, event, workload, and bounded log diagnostics before cleanup; causally blocked endpoint assertions are skipped rather than counted as independent failures.
- Repeated hosted-Linux acceptance run 38600327954 covering all eight external-DOT mechanism cells: two complete lifecycle cycles, 46 checks per cycle, zero failures, verified cleanup, and four successful official- Contract operation checks per cycle. The checksummed evidence pins DOT-CLI `848b2d56e9e416ae87fa46d221c9649041b64541` and DOT-Repository `b681c937422fa3db13ad16587cf271700eff5d79`.
- Portable evidence checksums and retained GitHub Actions artifacts.
- Separate workflow concurrency lanes for push conformance and manually requested live stress, preventing one evidence class from canceling the other.

Linux amd64 is the release-accepted executable target. The published arm64 executable is a cross-built preview and is not a supported final-release target until equivalent native arm64 evidence exists. Producer/partner acceptance, non-Linux executables, multi-host orchestration, general stateful migration, and automatic provisioning of cross-platform gateways remain outside this candidate. GraphQL, gRPC, and event-driven runtime probes beyond the documented safe profile remain standards-led roadmap work rather than an `rc.2` blocker.

The candidate assets and verification procedure are the same classes described for `rc.1` below, with filenames versioned as `0.2.0-rc.2`. The tag workflow rebuilds the Java artifact, both schemas, SBOMs, and Linux `dotx` binaries from the immutable revisions recorded in `release-manifest.json`, then verifies reproducibility and `SHA256SUMS` before publication.

0.2.0-rc.1

`0.2.0-rc.1` is the first integrated release candidate for the stable `dot.mare6g.eu/v1` composition profile. It replaces the narrower 0.1.0 artifact set with a traceable domain model, both machine-readable schemas, and version-stamped `dotx` binaries built from the exact DOT-CLI revision pinned by this repository.

Highlights

- Stable, repository-backed Composition v1 lifecycle, probe, and upgrade commands.
- Contract-driven Anchors for OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Protobuf/gRPC, GraphQL, and declared contractless interactions.
- Immutable repository resolution, content-addressed locks, resumable lifecycle operations, and last-known-good upgrade rollback.
- Managed Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrant/libvirt, and Wasmtime/WASI profiles.
- Explicit cross-platform networking through externally owned gateway bindings with pinned host, port, ownership, and no-cleanup semantics.
- Service-scoped configuration, secrets, OCI requirements, CDI devices, and exact native-workload bindings.

Assets and verification

The candidate includes the DOT-Domain JAR, DOT and Composition v1 schemas, CycloneDX SBOMs, and `dotx` binaries for Linux amd64 and arm64. `release-manifest.json` records the immutable DOT-Design, DOT-CLI, and DOT-Repository revisions. The repository revision pins its `DOTs/`, `Skills/`, and `Schema/` assets together. Verify the download set with:

```
sha256sum --check SHA256SUMS
```

The tag workflow publishes only after the Java build, generated-schema drift check, DOT-CLI tests, repository test, Composition v1 acceptance suite, deterministic packaging comparison, manifest validation, and checksum verification pass.

Compatibility and boundaries

The serialized composition API is `dot.mare6g.eu/v1`; breaking serialized changes require a new API version. The experimental local catalogue/registry prototype from the 0.1.0 era is not included: DOT-Repository is authoritative.

This candidate does not claim multi-host orchestration, automatic provisioning of cross-platform gateways/tunnels/brokers, general stateful data migration, arbitrary privileged workloads, non-Linux `dotx` executables, or producer/partner acceptance. The current CLI includes Linux process controls for its managed WASI runtime, so this candidate publishes Linux amd64 and arm64 binaries only. Closed legacy SFC issues record DOT-side disposition; joint consumer acceptance

remains a separate manual gate.

See the repository-root `RELEASE_NOTES.md` included with the source for the complete candidate notes and upgrade guidance.

0.4.0 readiness and remaining gates

The **0.4.0 release baseline** targets **Linux amd64** and the existing single-host execution profiles. The owner authorized direct stable publication on 2026-11-03, without requiring a public candidate first. Three unfamiliar-developer trials and licensing/redistribution/hosting reviews remain **pending follow-up**; they are not reported as complete. Publication in the existing private repositories does not establish independent adoption or blanket production certification.

Use the selected release's actual assets and `release-manifest.json` for final source identities. The CLI acceptance manifest names immutable test fixtures; the Design manifest separately names its CLI/catalogue gitlinks. These are separate coordinates, not a circular chain of floating latest revisions.

Published stable release — 2026-11-03

DOT-Design 0.4.0 and DOT-CLI v0.4.0 are published stable releases. Their verified manifests identify:

- DOT-Design: `8746d199fce9f19306d94f570d2ddc53ad0e3de3`
- DOT-CLI: `87c87e20e04fabe4d4fae47ca9b330d5052cff16`
- DOT-Repository: `e9a9342602e811a1b236a551ec05697d3fbbab4e`

Actual GitHub downloads passed complete inventory, checksum and source-identity checks. Both releases contain the same CLI bytes, SHA-256 `ba1fd82250217355ce9176aff5cf9bf433637c9e619e74a97ae445392cefca63`. All 151 published Domain class files match the qualified frozen models.

The fresh final demo passed in 28 minutes: seven live paths twice, failure and real SIGINT/SIGTERM cleanup, four presenter chapters and its matching fallback. Its Design source was `653e9e78bc177fe5d553504e964344828710438d`; the stable cut changes only the Antora version descriptor. The post-release Next tree at `8ed1a1d6d72ef5903e5921edd9a8812ce5b93e00` is identical to that qualified demo tree.

Release packaging and released-binary acceptance succeeded. The latter downloaded the published assets and passed two complete 48/48 external-tool lifecycle cycles. Its checksummed artifact and exact binary identity were retrieved into the restricted durable evidence archive. The CLI's initial acceptance-workflow checkout defect was fixed in a CI-only follow-up; acceptance of the same immutable CLI binary then succeeded. The original failed run remains historical evidence.

All eleven final or superseding workflows succeeded; two optional source `linux-stress` jobs were deliberately skipped. Those skips are not passes; the separate required released-binary stress job above passed. Next and stable documentation, exact PDF hashes, the default Next route and preservation of the 0.3.0 PDF were verified online. Human trials and owner reviews remain

pending.

Maintainer receipts: `demo/run-2th_ez3x/summary.json`, `rehearsal/run-wxlkwv59/summary.json`, and `presenter/run-23m3u_0a/summary.json`. The archive retains both passing and failed attempts outside disposable build trees. Publication qualifies the tested scope; it does not certify the whole catalogue.

Historical candidate foundation — 2026-11-03

The clean candidate foundation used source version `0.4.0-rc.1`:

- DOT-Design: `a7e55064cc33d9933bd51d71ec3e7a04c44f79f2`
- DOT-CLI: `e9f2a86573001e8f49235ffde1daff5b211636d2`
- DOT-Repository: `c7cc840ba0f137aea0f7cdb9382605d05756f7e2`

Twelve exact-source workflows and all 31 jobs passed. The fresh complete demo passed all five phases in 27 minutes: seven live paths twice, 19 qualification steps, injected-failure and real SIGINT/SIGTERM cleanup, four terminal-driven presenter chapters and the matching sanitized historical fallback. Source and binary identities remained unchanged. Native source qualification, synthetic standards fixtures and real producer acceptance retain their separate labels.

Design and CLI packages reproduced exactly in two builds, with identical CLI bytes in both packages and the private evaluation kit. Downloaded copies over a loopback-only server passed inventory and packaged-command acceptance. The exact packaged binary additionally passed live monitoring, cAdvisor and secure-API checks, each with verified cleanup. This was not a GitHub release download. Fresh Go-source and Java-runtime-SBOM scans found no issues. Java/schema drift, all 77 exported model references and the published Next model PDFs were checked.

Maintainer receipts are recorded as `demo/run-q5cve_jf/summary.json`, `rehearsal/run-kpdvxb71/summary.json` and `presenter/run-9z_n6767/summary.json`. The restricted evidence archive retains their hashes and exact artifact sets outside disposable build trees. They qualify that foundation, not an unbuilt future stable artifact; final version metadata, packages, remote jobs and real release downloads are checked again during promotion.

The declaration-only report had 24 `INTERFACE_PROFILE_PASSED` definitions and 43 `INTERFACE_GAP` definitions among 67 entries, with `liveProducerAcceptance: false`. Refresh it after catalogue changes. A green release does not certify every catalogue producer, deployment or exposed API.

Historical qualified source snapshot — 2026-10-31

Audit date: **2026-10-31**. The tested artifact set is:

- DOT-Design: `caa08754c2e0813686213678d402c8b0c2cfb48a`
- DOT-CLI: `a140194f8185eed3b817c52b617d766b3e54d819`
- DOT-Repository: `9c074de005810ae8d1dd3dee2e29ffd30cd592cd`

These are source revisions, not 0.4.0 release artifacts. Later documentation-only commits do not turn this snapshot into qualification of a future release tag.

Evidence	Result and boundary
Local deterministic checks	Full CLI CI and Design deterministic CI passed: 163 Java tests, generated schemas/bindings, race tests, exact Go coverage, catalogue/locks, all 12 E2E dry-runs and Antora HTML/PDF/publication guards. No production model, CLI, catalogue definition or lock changed in this qualification increment.
Fresh clean-source demo and presenter qualification	All five `task testing:demo` phases passed in 32.9 minutes. All 19 qualification steps passed: seven live paths twice, failure and SIGINT/SIGTERM cleanup, and four terminal-driven presenter chapters. A separate short-guide rehearsal passed full flow, early finish, EOF, timeout and both signals. Sources and installed binary were unchanged before/after; owned cleanup was verified. Source-adapted and standards-native fixtures retain their labels; this is not full producer certification.
GitHub checks	The preceding refactor's 17 validation/publication workflows and separate badge workflow succeeded, as did CLI CI. Its optional external `linux-stress` job was skipped, not passed. Those results belong to the preceding revisions, not the new source set above; fresh remote checks remain required. See Design runs and CLI CI.
Published documentation	The preceding Next publication and contributor guide were verified online. This updated audit still requires publication. Publication proves document availability, not live API acceptance.
Catalogue declarations, refreshed on the audit date	67 DOTs and five reusable compositions validate. 27 DOTs pass the strict interface-declaration profile; 40 report `INTERFACE_GAP`. The report explicitly sets `liveProducerAcceptance: false`. These counts apply to this catalogue SHA, not a permanent readiness percentage.

The local receipts are private, ignored artifacts under `DOT-Testing/e2e/target/`:

- Five-phase run: `demo/run-4eva88fs/summary.json`.
- Exact qualification: `rehearsal/run-qbv4zxn8/summary.json`; its adjacent `fallback/` contains the matching sanitized historical summary.
- Four official chapters: `presenter/run-ky9b5lmw/summary.json`.
- Short-guide controls: `guided-rehearsal/run-wle9fkdp/summary.json`.

An earlier clean run, `demo/run-s5qzi8m7/summary.json`, remains **failed**: its SIGTERM case stopped at Kubernetes API preflight before workload adoption. The API was subsequently Ready and the entire fresh run above passed, but the transient failure's cause is not proven. No timeout was weakened or cluster restarted; the failed receipt is not replaced by the later success.

Source-packaging checks produced byte-identical model archives and CLI packages; the packaged binary passed catalogue/interface acceptance. Full package verification nevertheless **failed the SBOM/version identity gate**: the untagged source still declares 0.3.0 while SBOM generation derives a pseudo-version. These are diagnostic source builds, not qualified 0.4.0 assets. Matching release metadata, exact tags and fresh artifact verification remain necessary; do not rewrite SBOM identities to turn that failure into a pass.

A subsequent **private local tagged rehearsal** aligned Design and CLI at 0.4.0 and passed version/SBOM verification without changing those identities by hand. The complete Design package was also reproduced byte-for-byte after a second clean Java verification build. This demonstrates the packaging procedure, not publication or acceptance of future release tags: the real repositories' versions and published tags are unchanged. A discovered incomplete-checksum inventory gap is covered by shared package checks and pre-execution regression tests; see the contributor guide. Versioned documentation also checks that its installation instructions select the aligned CLI release; private packaging alone does not qualify documentation.

Guided four-standard authoring, separate-candidate editing and from-DOT composition creation are implemented in source. Start with authoring or composition creation. The URL/hash payload migration and the explicitly approved removal of `Anchor.semanticTypes` are exceptions to 0.3.0 model compatibility. Contract-backed Anchors use verified operation selectors; undeclared Anchor keys are rejected without an alias. Affected locks require review. Clean existing workloads with their original CLI and inputs before switching; unrelated model changes remain out of scope.

Release checks and pending follow-up

Item	Required result or recorded status
Final clean-source qualification	**Passed for the published 0.4.0 artifact set above.** Recheck after changes. Retain exact source/binary identity, cleanup, presenter controls and matching fallback. Earlier source receipts are not silently relabelled as stable-tag acceptance.
Coordinated release artifacts	**Passed:** aligned Design `0.4.0` and CLI `v0.4.0`, reproducible packages, complete checksums, all model archives/SBOMs and actual downloaded-binary acceptance. The `v` prefix belongs to Go module tagging, not a different product version.
Remote checks and publication	**Passed for the recorded final sources and artifacts.** Require every applicable job to succeed and verify released assets and documentation. Queued, running, failed, cancelled and skipped jobs are not passes; optional skips are reported separately. Required Vagrant and other adapters are not waived because a different adapter passed.

Item	Required result or recorded status
Three unfamiliar-developer trials	**Pending follow-up under the owner's direct-publication decision.** Earlier assisted feedback and automated PTY/maintainer tests are not independent success. Use the trial record, retaining interventions, failures, exact artifacts and cleanup outcomes.
Owner reviews	**Pending follow-up under the owner's direct-publication decision.** Licensing, redistribution and hosting reviews have not been declared complete. Approval to create unlisted Gists is not blanket redistribution clearance; technical checks do not supply legal conclusions.

Keep these statuses visible after publication. The owner changed the publication sequence; missing trial results and owner reviews have not become passing checks. Use the migration guide before switching existing installations and keep their original binary until cleanup.

Recheck from one checkout

From a clean **DOT-Design checkout root**, with its pinned submodules initialized and the contributor prerequisites installed:

```
task ci:deterministic
task ci:live
task testing:demo
```

These are full qualification commands, not presentation commands. Live gates start workloads, take substantially longer than coding tests and must verify test-owned cleanup. Do not run other rehearsals against the same owned profile concurrently. `task demo` is the short interactive presentation; it does not replace this qualification. See the full demo procedure for receipts, recovery and fallback selection.

To refresh only the catalogue evidence, follow the readiness report procedure. Keep official specifications, source-derived interfaces, runtime-only brokers, planning entries and fixtures distinguishable. Imported Protobuf graphs, complete Loki/OTLP coverage, stateful recovery, distributed orchestration and independent production-producer acceptance remain separate follow-up work.

See From a working demo to a production integration for per-gap review actions, selected-producer qualification, stateful/import acceptance criteria and the pending human/owner handoff. These are explicit follow-up boundaries, not additional capabilities claimed by this release.

DOT-Domain

Domain model

`DOT-Domain` is the Java library that defines reusable DOTs, compositions and their validation rules. These models generate the JSON Schemas and CLI input types. The library does not deploy services or manage databases.

Aggregate overview

A DOT describes one reusable tool. A composition selects exact DOT releases, chooses one deployment per instance and connects their Anchors. Read the core relationships below first; field-level and validation details follow separately.

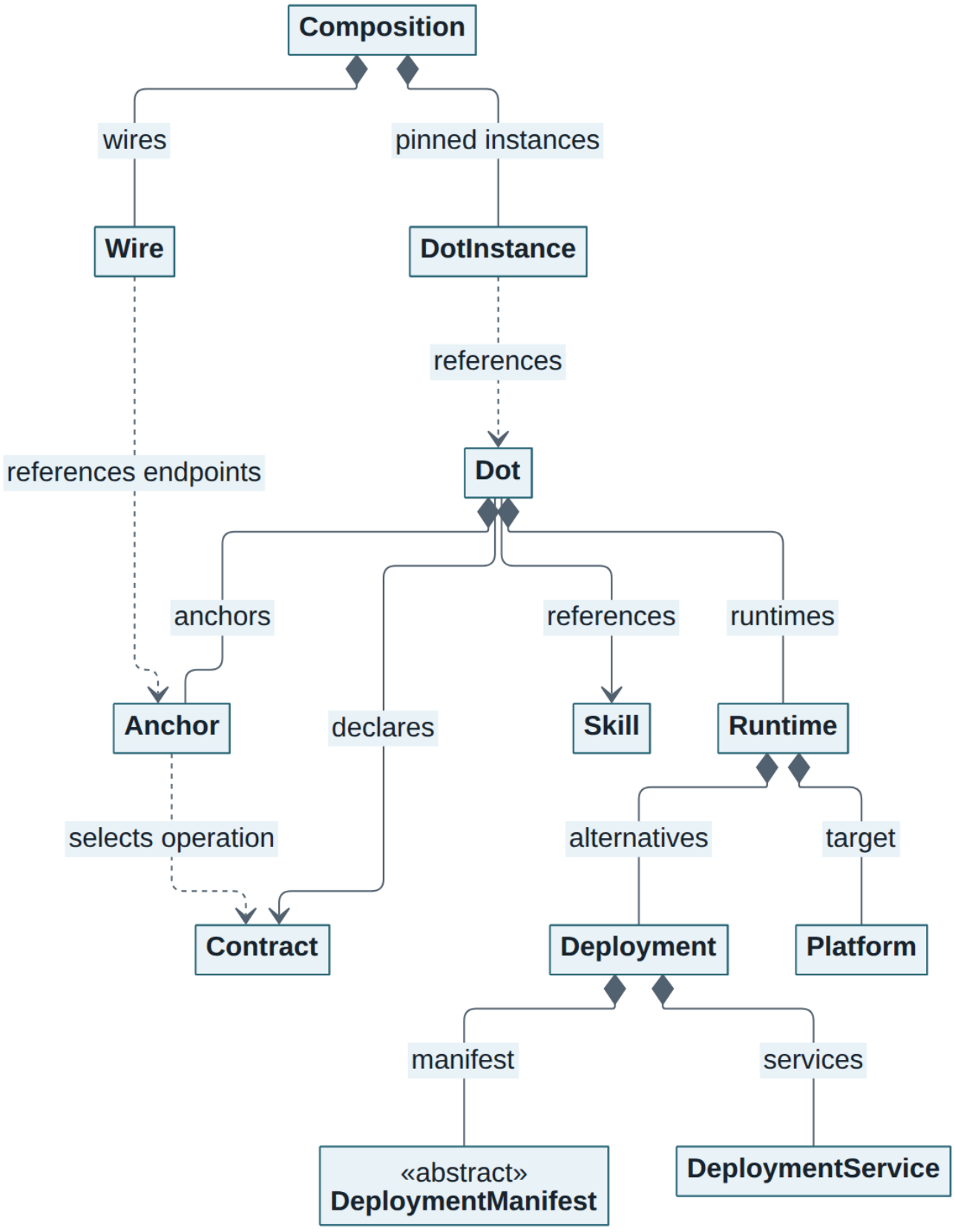


Figure 11. Simplified domain model: reusable DOT definitions and a separate Composition

Diamonds indicate embedded values; arrows indicate declarations or references. The dashed arrow applies to Contract-backed Anchors. Field lists, collection sizes, configuration, ports, binding helpers and audit metadata are omitted for readability; see the full generated class diagram for implementation detail. `Composition` is a separate DOT-Domain aggregate: its instances reference DOTs and its Wires reference their Anchors. Neither is embedded in the reusable Java `Dot`

aggregate.

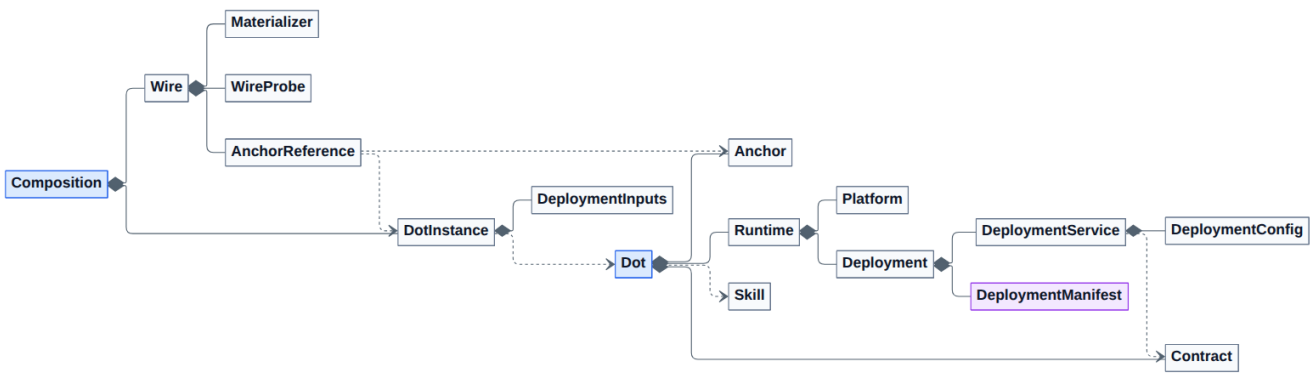
Type	Responsibility
`Dot`	Aggregate root: identity, version, description, classification groups, metadata, Skills, Contracts, Anchors, and Runtimes.
`Skill`	Standalone human-readable capability description.
`Composition`	Separate aggregate: pinned DOT instances, Anchor Wires, exposed Anchors, deployment inputs, Secret references and active probe expectations.
`Contract`	Standalone authoritative OpenAPI, AsyncAPI, Protobuf, GraphQL or supported SPARQL Service Description.
`Anchor`	Embedded named integration point that provides or requires an interface; Wires connect compatible Anchors.
`Runtime`	A target `Platform` plus one or more compatible deployment definitions.
`Deployment`	Stable deployment ID, one polymorphic manifest, and the logical services it materializes.
`DeploymentService`	Service identity, host, named ports, Contract references, Anchor bindings, and its supported configuration surface.

`Dot`, `Skill`, `Contract`, and `Composition` are independently identified domain entities. Other types are embedded values owned by these aggregates. The module carries no database or ORM annotations; persistence is an adapter concern.

Model overview and complete reference

Use the complete model reference for every public type's purpose, fields, relationships, validation boundaries and JSON example. It is organized into five chapters and available as a comprehensive A4 PDF. The README stays a short entry point; these chapters provide the detailed reference.

The A4 landscape reference sheet combines a readable core class diagram with an inventory of every exported public model type (77 in this source revision). Scalars, generated accessors and validation implementation helpers are omitted. Supporting types are listed by their nearest core-model family, not presented as exclusively owned values, extra aggregates or guaranteed runtime features. Shared types are listed once.



The complete vector diagram shows all 77 public classes/enums and their relationships individually. Both the compact Mermaid source and complete Mermaid source come from the existing Java-generated diagram and exported schema inventory. The DOT-Domain README renders them natively and includes a schema/semantically validated, compact Prometheus `dot.json` projection using the real catalogue Contract and deployment, rather than another product descriptor.

After an intentional model/catalogue documentation change, run `task docs:domain:generate` from DOT-Design with the source-built CLI installed. `task docs:domain:check` verifies source/output drift and the exact JSON example offline; it is part of `task testing:models`, not a live deployment test.

One model authority

DOT-Domain defines **all** portable models, including Composition, not just `Dot`. The existing Victools build generates `Schema/schema.json` for DOTs, `Schema/composition-v1.schema.json` for Composition v1, and standalone `Schema/models/*.schema.json` files for every public domain data class and enum. Jackson reference views remain intentional: a Skill reference containing only `id` is not the full standalone Skill entity.

The CLI consumes generated Go input types and embeds the generated Composition schema. It does not require Java at runtime. From DOT-Design, regenerate with:

```
mvn -pl DOT-Domain process-test-classes
task cli:domain:generate
```

Never edit generated files manually. `task testing:composition:model` checks generation drift and v1 compatibility after the Contract validator bootstrap.

`Composition` references reusable DOTs; it neither inherits from `Dot` nor copies its runtimes or Contracts. Resolved execution details, lock digests, private lifecycle state, endpoint allocation and runtime policy enforcement remain CLI/executor concerns. Standard Jakarta constraints and Jackson shape annotations describe fields; model-owned JSON Schema annotations express conditional input rules such as exactly one literal value or Secret reference. Jakarta validation also checks the exclusive environment, workload, probe and gRPC request choices, and the fields allowed for each materializer destination. Cross-DOT resolution, secret authorization and ownership checks still run in the CLI after schema validation.

This migration preserves `dot.mare6g.eu/v1`, existing documents and reviewed locks. Composition `version` and `dotVersion` retain the existing v1 string semantics; they have not silently become stricter Java `Version` fields.

Domain stabilization

The immediate priority is a model compatibility freeze, before further CLI authoring features. Preserve valid Composition v1 JSON and reviewed locks while allowing targeted Java API improvements. Probe and materializer selections use typed Java enums: `AsyncAPIProtocol`, `GraphQLTransport`, `GRPCResponseMatch`, `MaterializerType`, and `MaterializerValueFrom`. Existing wire values remain unchanged and exact, for example `MQTT`, `SSE`, and `CONFIG_FILE`.

```
AsyncAPIProbe probe = AsyncAPIProbe.builder()
    .protocol(AsyncAPIProtocol.MQTT)
    .correlationPointer("/id")
    .build();
```

This constructs a declaration, not a broker connection or successful delivery probe. Validate Java objects with Jakarta Bean Validation and raw input with the generated schema. Raw-input validation must precede deserialization when required-property presence matters: defaulted Java fields cannot distinguish an omitted required property from an explicitly supplied default. Optional empty Java Wire collections serialize as absent; explicit empty JSON collections remain invalid where the schema requires a nonempty value. Materializer target pointers such as `/routes/0/uri` remain accepted without rewriting.

The Java build enforces 100% instruction and branch coverage. Compatibility tests compare generated schemas to the frozen v1 oracle, including existing composition inputs and all 128 materializer field combinations. These checks do not establish independent adoption or live producer acceptance.

The `0.3.0` baseline preserves the reviewed portable model surface, not a permanent 1.0 API guarantee. `DOT-Domain/MODEL-SPEC.md` records the cross-model invariant review and validation boundaries. The reproducible, checksummed `dot-domain-models-0.3.0.tar.gz` release asset contains all 77 exported schemas and shared compatibility corpora. Publication and full runtime qualification remain separate release gates.

The service-configuration audit now rejects duplicate environment/CDI names, configuration-file paths, and volume names/target paths within one service. Distinct services keep separate scopes, so the same key may occur in both. Declarations retain their order and are never silently collapsed. Java and installed-CLI tests share the same positive/negative corpus. These are semantic checks: standard JSON Schema cannot express uniqueness by one property of an array item. The generated JSON shapes and reviewed composition locks remain unchanged; use full DOT validation rather than schema-only acceptance. Resolve these in DOT-Domain and its existing generation pipeline, not in independent CLI models. No final stable-model claim or new release is implied.

Deployment-free model gate

```
task testing:models
```

This sequential, fail-fast local gate runs Java tests and coverage, shared positive/negative examples and serialized Java/schema agreement, generated Go drift checks, CLI race tests and coverage, offline catalogue validation and reviewed locks. Every exported model needs reviewed examples; all enum values and registered manifest subtypes are covered. This is a baseline, not an exhaustive proof of every field combination.

No deployment, runtime startup or GitHub workflow dispatch occurs. Contributor toolchains are required; initial bootstrap may download pinned validation tools. Use `task ci:live` separately for actual adapter/runtime acceptance and `task testing:demo` separately for full presenter qualification. A coding-only push is not a live-runtime qualification or documentation publication.

Optional null manifest/configuration fields serialize as absent, not schema-invalid JSON nulls; JSON assertions intentionally retain `equals: null`. OCI capabilities/host namespaces reject duplicates before Set conversion. Java callers of their previous Set-specific setters must recompile. Generated schemas now retain existing Java patterns for Helm repositories and WASM module references/digests, rejecting malformed inputs earlier without changing valid document shapes.

Reference-aware validation and collection safety

Anchors use exact DOT-local kebab-case keys such as `search-api`, not generated UUIDs for new catalogue entries. `Anchor.ID_PATTERN` is shared by the Java definition, service bindings and composition references and exported to the generated schemas/Go inputs. A wire's `instance` identifies the owning DOT, so two different DOTs may reuse the same local key. Display labels and operation selectors remain separate and never act as lookup aliases. Legacy UUID strings remain readable when explicitly declared. These three Java properties now use `String`; recompile callers and convert old UUID arguments explicitly. Composition ownership UUIDs are unchanged. Catalogue migration updates references and reviewed locks together, not private live state.

Use ordinary `validator.validate(dot)` to cascade validation. ID-only Skill and service Contract views use the portable Jakarta `IdentityReference` group; ignored entity metadata is not validated as a complete declaration. Validating a standalone `Skill` or `Contract` still requires a full entity. Root Contracts use the `ContractDeclaration` group, admitting either a complete declaration with validated payload/version or the existing ID-only migration reference. Partial declarations fail. Serializing a root ID-only reference does not add the Java default version to its JSON.

Service Contract references must identify a root declaration or an explicitly sourced external Anchor Contract. A different release or similar name is not an alias. The model does no catalogue or network I/O: Skill existence, verified Contract assets and operation resolution remain CLI/resolver checks.

Scope	Unique keys
Runtime	Deployment IDs
Deployment	Service IDs
Service	Port names
DOT	Root Contract IDs and Anchor IDs
Composition	Instance names, Contract-input IDs, wire names, exposed-Anchor names

Jackson's `Deployment.setServices(Collection<DeploymentService>)` rejects duplicate service IDs before constructing its ordered Set; it never silently chooses one conflicting service. Recompile Java callers of the former Set-specific setter. Validation also rechecks keys after in-place ID mutation. The CLI applies the corresponding semantic checks to raw definitions and emits field locations without copying the portable models into handwritten Go types.

Dependencies, security scopes, CDI bindings and OCI grants retain their existing v1 `uniqueItems` constraints, now checked in Java lists too. Generated schemas also retain their annotated item-level nonempty-string/capability constraints. The portable `@UniqueElements` constraint drives both Jakarta validation and the shared Victools schema module; there is no separate hand-maintained list of

unique fields. No blanket deduplication or reordering applies to arbitrary lists. Missing required values and null nested entries produce validation violations rather than aggregate-traversal exceptions. Validate raw required-property presence before deserialization, as described above. Optional null Dot/Skill fields serialize as absent, not schema-invalid nulls. Shared golden examples run through both Java round trips and generated-schema validation. The broader optional-value serialization audit remains open.

Identity, versioning, and references

`Dot`, `Skill`, and `Contract` use case-sensitive textual IDs. The catalogue uses `Prometheus@3.15.1`, `Metrics-Scraping`, and `Prometheus@3.15.1::OpenAPI::HTTP`, respectively. DOT IDs follow `<name>@<version>` using the actual tool release; Contract IDs follow `<origin-DOT.id>::<standard>::<name>`. The local Contract name is `HTTP` in this example. Consumers retain the originating identity, even when their own DOT has a different name or version. Contract standard versions and chart packaging versions are independent. IDs are explicit keys, not download URLs or filesystem paths. Existing UUID-shaped strings remain accepted, without implicit aliases to migrated entries. Anchor IDs are DOT-local kebab-case; Composition identities remain UUIDs. IDs are not URLs. The catalogue naming convention requires an explicit identity/reference update when changing a DOT name or tool release; mutating a Java object's fields does not silently regenerate its ID. Equality for `Dot`, `Skill`, `Contract`, and `Composition` is ID-based so changes to mutable metadata or collections do not destabilize sets or persistence identity.

Every DOT must reference at least one meaningful Skill. The catalogue's capability assets resolve through `Skills/`. Prometheus, for example, contains:

```
"skills": [{"id": "Metrics-Scraping"}, {"id": "Observability"}]
```

`dotx repository inspect <id>@<version> --repository DOT-Repository --json` resolves those IDs to the authoritative labels, descriptions and `Monitoring-and-Analysis` category. References require the exact ID; search filters may additionally match a name case-insensitively. A display-label change never implicitly changes identity. See the catalogue's `IDENTIFIER-MIGRATION.md` for the completed DOT/Contract migration and safe lock/state handling.

`Version` accepts Semantic Versioning 2.0.0 values and the special `latest` selector. It serializes as a string and implements SemVer precedence; `latest` sorts after concrete versions. A new `Dot` defaults to `0.0.0`, while `Platform.version` defaults to `latest` so no platform-specific release is implied.

`Contract.version` is a standard-specific string, not a software/API release: OpenAPI and AsyncAPI copy their exact top-level `openapi`/`asyncapi` version; Protobuf uses the native `syntax`/`edition` value (implicit `proto2` when absent). GraphQL SDL declares no version, so that field is omitted. There is no default or placeholder Contract version. `info.version` remains unchanged inside the upstream payload. `dotx validate`, composition loading and URL authoring reject mismatches even when the declared number is otherwise well formed.

JSON serializes `Dot.skills` and `DeploymentService.contracts` as ID-only references. Root `Dot.contracts` carries authoritative Contract declarations: identity, standard, format, and a structured `payload` with an HTTPS `url` and `sha256` pin. Raw API documents are not embedded in `dot.json`. Current catalogue roots must carry complete declarations; CLI validation rejects ID-only

roots instead of searching local or sibling files. Verified bytes are cached outside the DOT; cold offline validation fails until warmed online. An Anchor references the root declaration by Contract ID and selector.

Current source uses one shared `Payload` value for Contracts and file documents: `DockerCompose.payload`, `KubernetesManifest.payload`, `Vagrantfile.payload`, `HelmValues.payload`, `EnvFile.payload`, and optional `ConfigFile.payload`. Each reference selects exact raw document bytes, not an application endpoint. There is no inline-text, Base64 or adjacent-file fallback. This deliberately breaks former string-based file declarations and renames the Java `ContractPayload` type to `Payload`; valid Contract JSON keeps its URL/hash shape. Published stable-tag documentation retains its historical model baseline. See the current-source reference.

`dotx validate` enforces catalogue identity uniqueness, including DOT `(id, version)` and `(name, version)`, and rejects conflicting documents for one Contract ID across bundles. These catalogue rules are not database indexes or persistence requirements enforced by the Java entities.

Contracts

Contract `standard` accepts only the canonical JSON spellings `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`, and `SPARQL`. Use those values exactly; alternative casing and surrounding whitespace are rejected. The Java enum constants remain uppercase; Jackson serialization and the generated JSON Schemas use their canonical values. `DataFormat` is a separate enum and is unchanged.

Standard	Payload format	Anchor selector
`OpenAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`AsyncAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`Protobuf`	`PROTOBUF`	Fully qualified service method
`GraphQL`	`GRAPHQL`	Root-field schema coordinate
`SPARQL`	`N_TRIPLES`	Exact query endpoint IRI from a SPARQL Service Description

`Contract.isFormatCompatibleWithStandard()` rejects mismatched standard/format pairs. Payload parsing and selected-element existence are resolver concerns; the domain model validates selector syntax and references within the aggregate.

Anchors and bindings

A contract-backed Anchor contains:

- a stable DOT-local ID and a separate display name;
- a `PROVIDES` or `REQUIRES` role; and
- a Contract ID plus standard-specific selector.

It omits direction, protocol, and media type because those facts come from the selected Contract

operation. A contractless Anchor must declare `direction`, at least one protocol, and at least one media type explicitly.

`DeploymentService.anchorBindings` maps an exact Anchor ID to a named `ServicePort` on that same service. An Anchor contains no host, numeric port, credential, placement, or live policy state.

The class-level `@ValidAnchor` constraint delegates cross-reference validation to `AnchorValidator`. It rejects:

- duplicate Anchor IDs within one DOT;
- local Contract references not declared by the same DOT (external references instead require an explicit originating DOT; its actual declaration is resolved separately by the CLI);
- selectors invalid for the referenced Contract standard;
- bindings to undeclared Anchors;
- bindings to port names absent from that service; and
- duplicate bindings of one Anchor within the same service.

See DOT Anchors and Composition for authoring and wiring examples.

Runtime two-layer model

`Runtime` separates infrastructure from deployment mechanism:

Enum	Used by	Answers
`PlatformEngine`	`Platform.engine`	Where the workload runs: `DOCKER`, `KUBERNETES`, `VAGRANT`, or `WEBASSEMBLY`.
`DeploymentManifestType`	`DeploymentManifest.type`	How it is deployed: Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every `DeploymentManifestType` declares its required `PlatformEngine`. `Runtime.isDeploymentPlatformAligned()` validates every deployment against the runtime platform. `Runtime.isDeploymentIdsUnique()` enforces stable deployment selection within that runtime.

`Deployment.services` is a set. `DeploymentService` uses ID-based equality, so logical service IDs remain unique within the deployment.

Deployment manifests

`DeploymentManifest` is polymorphic JSON dispatched by its existing `type` property.

Type	Java subtype	Important fields
`DOCKER_IMAGE`	`DockerImage`	Image reference and ordered dotenv payloads
`DOCKER_COMPOSE`	`DockerCompose`	Complete YAML/JSON payload and ordered dotenv payloads

Type	Java subtype	Important fields
KUBERNETES_MANIFEST	KubernetesManifest	Complete YAML/JSON manifest payload
HELM_CHART	HelmChart	Repository, chart or OCI reference, version, and ordered values payloads
VAGRANTFILE	Vagrantfile	Complete Ruby DSL payload
WASM_MODULE	WebAssemblyModule	Relative/file or HTTPS module URI, WASI execution world, command arguments, and required SHA-256 integrity pin

Docker Compose and Kubernetes manifest payloads accept YAML or JSON only. Helm values also preserve YAML/JSON order: later entries override earlier entries as with repeated Helm `-f` arguments. These formats describe the downloaded bytes, not embedded strings in `dot.json`. Image/chart references and the WASM module URI/hash remain separate from the shared file-document `Payload` model. Digest verification alone does not establish publisher identity, execution safety or successful deployment.

Runtime instance concerns do not belong in a manifest. For example, a Helm release name and namespace are selected by deployment tooling, not embedded in `HelmChart`.

Service configuration declarations

Every `DeploymentService` owns one `DeploymentConfig` containing:

- environment variables, including default, required, and sensitive metadata;
- configuration-file paths, content type, defaults, and sensitivity;
- logical volume mounts;
- logical CNCF CDI device requirements; and
- optional OCI capability and host-namespace requirements.

These are supported inputs and requirements, not applied runtime state. An omitted `ConfigFile.payload` means no declared default. A supplied default is a pinned `Payload`; a sensitive file must not declare any non-null default, even a public or empty document. Scalar environment defaults remain strings. Java null omission does not imply that explicit JSON null is accepted.

`ConfigFile.payload` replaces the former `defaultContent` name. Only `payload` is accepted by current source; operator or composition inputs still override this optional default. A sensitive file must receive its content at deployment time, never through a reusable payload URL. Operator values, host paths, concrete CDI device names, and permission grants belong to deployment or composition input.

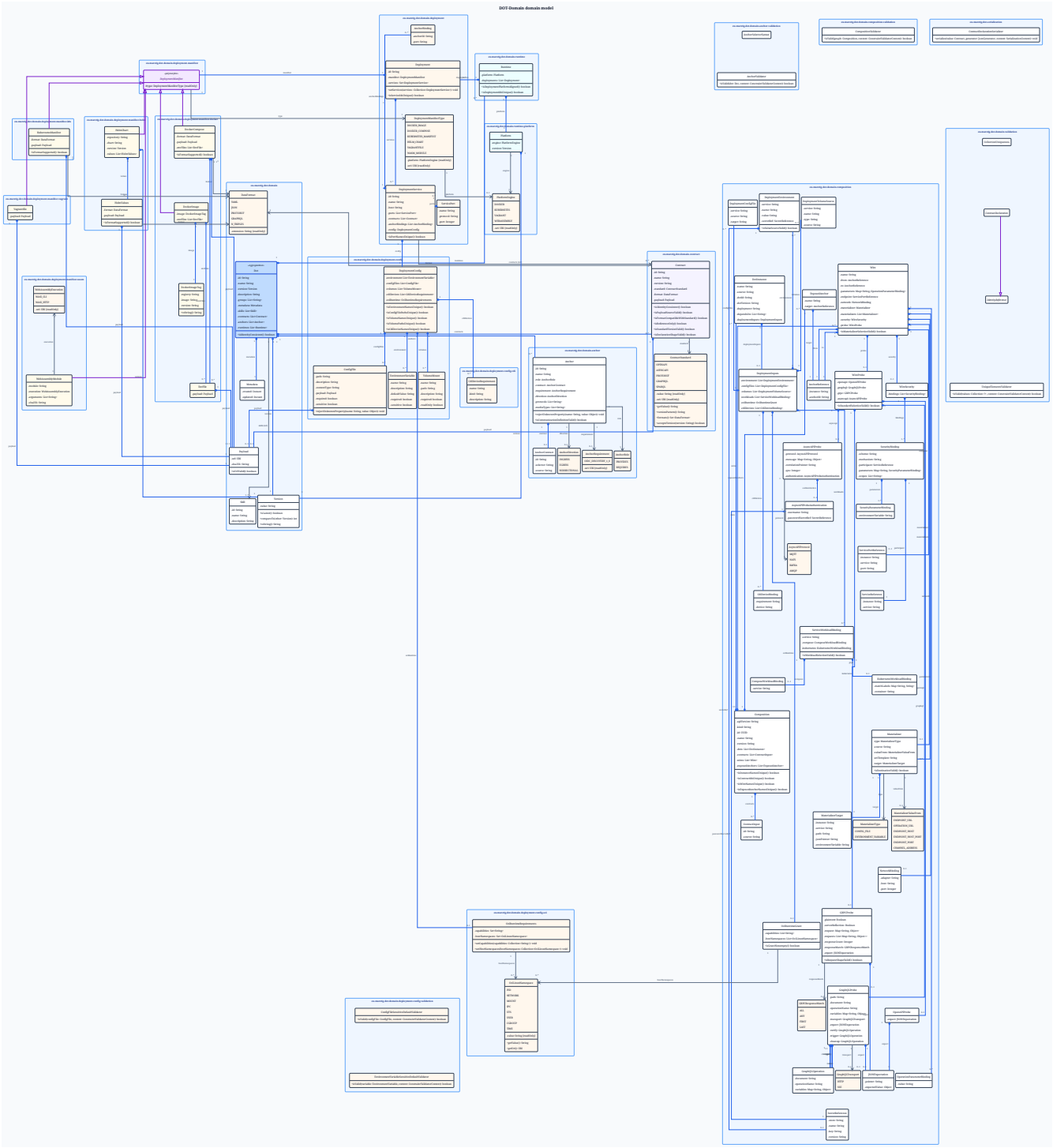
OCI requirements use standard `CAP_*` capability names and typed Linux namespaces. The model deliberately has no unrestricted `privileged` flag.

A CDI requirement declares a stable logical name and a `vendor.example/class` kind. A composition binds it to one exact `vendor.example/class=device` identifier. CDI installation, allocation, and authorization remain operator responsibilities.

Serialization and persistence adapters

- Collections have initialized builder defaults and do not become `null` when constructed through Lombok builders.
- `Version` is JSON-serialized as a normalized string.
- Persistence adapters must preserve UUID, textual identifier and version representations.
- Skill and service-level Contract references expose only `id`; published root `Dot.contracts` contains complete owned URL/hash declarations. Model-level ID-only values are not a local-asset fallback or publishable root declaration.
- Domain values carry no database or ORM annotations.

Class diagram



The published diagram is a reviewed export of the Java sources. Ordinary model builds keep generated PlantUML under `target` and leave these committed images unchanged. Maintainers refresh the exports with `task docs:domain:generate`, then review the differences. Open the diagram separately for the full-resolution view.

Build and test

```
# Unit tests only; no service or database required
mvn -pl DOT-Domain test

# Regenerate schema and diagram through the reactor lifecycle
```

```
mvn -pl DOT-Domain clean install
```

Representative tests cover version precedence, reference serialization, manifest polymorphism, runtime/manifest alignment, deployment configuration, Anchor rules, and entity equality.

Related pages

- Architecture
- DOT Anchors and Composition
- Integration handoff

Complete model reference

Start with the domain overview, then use these chapters to understand each building block, its fields and its relationships. The inventory is discovered from the Java-generated schemas, not maintained as a second model.

- Identity and capabilities: what a DOT represents, reusable Skills, audit timestamps and software versions.
- Contracts and Anchors: owned specifications, selected interactions, roles and provider-neutral requirements.
- Runtimes, deployment and configuration: platform versus mechanism, manifests, services, ports, configuration and requirements.
- Compositions and connections: exact instances, Wires, deployment inputs, materializers, Secret references and security.
- Probes and assertions: bounded checks for supported HTTP, GraphQL, gRPC and messaging profiles.

Download the comprehensive A4 reference for the same model explanations, generated field tables, relationships and examples. Its diagram page is landscape; the detailed chapters are portrait. The single-page A4 overview remains a compact visual companion, not a substitute for the detailed reference.

How to read a model entry

Each entry explains its purpose and use, followed by its generated fields, relationships and one JSON example. Field names, JSON types, enum values, base required properties and structural constraints come from the existing generated schema. Reviewed notes explain intent, invariants and common mistakes without redefining those shapes. Java and schema links lead to the authorities.

- **Required** means required by the base schema for a full standalone value.
- **Conditional / optional** means it is not unconditionally required by that base schema. A selected mode, parent context or cross-field rule can still require it. Follow the conditional schema and model validation.

- A Java constructor default does not waive a required raw JSON property. This reference does not infer undeclared defaults from example values.
- **Schema enum** lists values declared by that generated schema. It is not a promise that every value is valid in every owner: the field explanation and Java cross-field validation can narrow the set. For example, Compose, Kubernetes documents and Helm values accept only YAML or JSON.
- A `Skill` reference inside `Dot.skills`, or a service-level Contract reference, intentionally carries only `id`; it is not the full entity shown by its standalone field table.
- Relationships come from the existing Java-derived Mermaid graph. Multiplicities describe those associations; the schema and Java validation determine document validity. A supporting type can be used by several owners.

Follow the integration chain

For executable probes, pair the field reference with the supported probe profiles. Required transport, expectation and cleanup combinations are execution rules; a structurally valid fixture alone does not satisfy them.

1. A `Dot` identifies a reusable tool release and references meaningful `Skill`s.
2. Its owned `Contract` declarations locate exact specification bytes by URL/hash.
3. An `Anchor` selects one Contract interaction, expresses a supported requirement, or explicitly declares a contractless boundary.
4. A `Runtime` offers compatible `Deployment` alternatives for one `Platform`.
5. A deployment's `DeploymentService` uses `AnchorBinding` to connect logical interfaces to named `ServicePort`s and declares its supported configuration.
6. A `Composition` selects `DotInstance`s and connects their Anchors with `Wire`s.
7. `DeploymentInputs` and materializers supply reviewed instance-specific inputs; Secret references name secret sources without embedding their values.
8. A `WireProbe` declares a bounded acceptance check where the CLI supports it.

These relationships are not a packet-flow graph. In particular, `PROVIDES` to `REQUIRES` describes interface ownership and dependency, not who initiates a network request. See Anchor authoring for the distinction between operation identity, data schemas and semantic meaning.

Examples and acceptance boundaries

The small DOT, interface, file-manifest, Helm values and configuration-default examples reuse the canonical Prometheus declaration and its actual URL/hash references. There is no catalogue `EnvFile` declaration: that example remains explicitly shape-only, from the Java compatibility corpus, not a downloadable dotenv recommendation. Other examples come from the same reviewed corpus and illustrate individual value shapes. They may use placeholder URLs, images or references requiring an enclosing graph; they are **not runnable deployment recipes, producer claims or recommended product versions**. The ContractInput example uses the current explicit-owner source rather than a historical local wrapper. Use Getting started for a real service, readiness

checks and cleanup.

Java and generated-schema validation establish declaration invariants, not cross-DOT compatibility, downloads, network reachability, authentication or live producer conformance. The CLI performs the supported resolution and execution checks separately. Resolved compositions, lock digests, concrete endpoints and runtime state are CLI artifacts, not additional portable domain models.

Current source deliberately replaces former inline file documents with the shared `Payload` URL/hash value. Five file-document owners require it; `ConfigFile.payload` is optional and forbidden for sensitive files. There is no inline-text or Base64 fallback. Valid Contract JSON is unchanged, although Java callers replace `ContractPayload` with `Payload`. This approved change supersedes the former current-source payload freeze; published stable tags retain their historical model. Offline CLI validation requires every needed digest already cached, rehashes each read, and fails on missing or corrupt bytes rather than searching adjacent files or silently repairing them.

Maintaining the reference

From the DOT-Design source checkout, with its pinned catalogue and documentation toolchain available:

```
task docs:domain:generate
task docs:domain:check
task ci:docs
```

Generation discovers all exported types, reuses their Java/schema authority and the existing example corpus, and rejects missing prose, unexplained fields and orphan notes. Public documentation checks verify committed outputs and source hashes without requiring private catalogue access; they are not a replacement for the full source/schema drift gate. No deployment is performed.

Identity and capabilities

How to read this reference explains full objects versus ID-only views, conditional fields and validation boundaries.

Dot

Describe one reusable software product or component: its identity, capabilities, interfaces and supported deployment alternatives. A DOT definition is not a running instance or a complete multi-tool use case.

Keep one canonical product definition for the selected tool release. A composition selects instances and deployments, connects Anchors and supplies use-case inputs; do not copy product definitions for each scenario. The CLI deploys from the selected dot.json, not from nearby Dockerfiles or manifests. Java validation checks the aggregate and its declared relationships; catalogue resolution, verified Contract downloads and live acceptance are separate checks.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Stable, case-sensitive component identity. A release-qualified identity must agree exactly with name and version. Legacy textual and UUID-shaped keys remain readable, but are not automatic aliases for a different catalogue entry. minimum length: 1; pattern restrictions (see schema).
`name`	string	Required	Readable product or component name, kept separate from the bundle directory and runtime-instance name. For a qualified identity, changing this name also changes the identity relationship that must be reviewed. minimum length: 1.
`version`	Version	Required	Release of the represented software, not a revision counter for catalogue edits. Image packaging tags, Helm chart versions and Contract-language versions are separate. Label unreleased planning, source-adapted and test components honestly. pattern restrictions (see schema).
`description`	string	Conditional / optional	Optional human explanation of purpose, intended use and important limitations. It does not replace machine-readable Contracts, deployment inputs or acceptance evidence.
`groups`	array of string	Conditional / optional	Open classification labels rather than a closed domain enum. Ownership labels such as internal and external are interpreted by workflows; they do not grant permissions or establish production maturity.
`metadata`	Metadata	Required	Audit timestamps for this definition. Java construction provides initial values, but raw JSON still needs the required timestamp properties; metadata is not live deployment state.
`skills`	array of Skill (ID reference)	Required	Capability references serialized as ID-only objects. Choose at least one meaningful catalogue capability; its label and description belong to the Skill catalogue rather than being repeated in every DOT. minimum items: 1.

JSON field	Value	Base presence	Meaning and constraints
`contracts`	array of (Contract (ID reference) or Contract)	Required	Authoritative interface declarations owned by this exact DOT release. Published catalogue roots need complete URL-backed declarations. References to dependency interfaces belong on Anchors with an explicit origin, not as copied declarations here.
`anchors`	array of Anchor	Conditional / optional	Embedded logical integration points provided or required by the component. Their IDs are local to this DOT and are the keys used by service bindings and composition wires; addresses and credentials do not belong here.
`runtimes`	array of Runtime	Required	Alternative ways the component can run on its supported platforms. A deployment instance chooses one named deployment; declaring several alternatives does not request that all of them be launched.

Relationships

- To Metadata: containment via metadata; diagram multiplicity 1.
- To Skill: ID/name reference via skills (ref); diagram multiplicity 1..*.
- To Version: containment via version; diagram multiplicity 1.
- To Anchor: containment via anchors; diagram multiplicity 0..*.
- To Contract: containment via contracts; diagram multiplicity 0..*.
- To Runtime: containment via runtimes; diagram multiplicity 0..*.
- From DotInstance: ID/name reference via dotId / source.

Example derived from the canonical Prometheus definition (not a second product)


```
{
  "id": "Prometheus@3.15.1",
  "name": "Prometheus",
  "version": "3.15.1",
  "groups": [
    "external"
  ],
  "metadata": {
    "created": "2026-07-10T16:30:00Z",
    "updated": "2026-11-02T00:00:00Z"
  },
  "skills": [
    {
      "id": "Metrics-Scraping"
    }
  ],
```

```
  "contracts": [
    {
      "id": "Prometheus@3.15.1::OpenAPI::HTTP",
      "name": "HTTP",
      "version": "3.1.0",
      "standard": "OpenAPI",
      "format": "YAML",
      "payload": {
        "url":
"https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1/web/api/v1/testdata/o
penapi_3.1_golden.yaml",
        "sha256": "406b517ed3800c107b3afbc2d9712f2e487269fe8c8b55a075dce46823d8d7c0"
      }
    }
  ],
  "anchors": [
    {
      "id": "build-information",
      "name": "build-information",
      "role": "PROVIDES",
      "contract": {
        "id": "Prometheus@3.15.1::OpenAPI::HTTP",
        "selector": "#/paths/~1status~1buildinfo/get"
      }
    }
  ],
  "runtimes": [
    {
      "platform": {
        "engine": "DOCKER",
        "version": "latest"
      },
      "deployments": [
        {
          "id": "docker-compose",
          "manifest": {
            "type": "DOCKER_COMPOSE",
            "format": "YAML",
            "payload": {
              "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/docker-compose.yaml",
              "sha256":
"648687feae439ddc1a285c9c61e6bf855c9426a3bb4f16b36e56b68ae9529439"
            }
          },
          "services": [
            {
              "id": "prometheus",
              "name": "Prometheus Server",
              "host": "prometheus",
```

```
              "ports": [
                {
                  "name": "http",
                  "protocol": "HTTP",
                  "port": 9090
                }
              ],
              "contracts": [
                {
                  "id": "Prometheus@3.15.1::OpenAPI::HTTP"
                }
              ],
              "anchorBindings": [
                {
                  "anchorId": "build-information",
                  "port": "http"
                }
              ],
              "config": {
                "environment": [
                  {
                    "name": "DOTX_HTTP_PORT",
                    "defaultValue": "18909",
                    "description": "Host port used by the Docker Compose example."
                  }
                ],
                "configFiles": [],
                "volumes": [
                  {
                    "name": "prometheus-data",
                    "path": "/prometheus",
                    "description": "Prometheus TSDB storage for the Compose
mechanism."
                  }
                ]
              }
            }
          ]
        }
      ]
    }
  ]
}
```

Metadata

Record when a domain definition was created and last updated. These are audit values for the authored record, not workload start times or health observations.

Java constructors and builders initialize timestamps, but authors of raw JSON must supply the

properties required by the generated schema. Callers manage subsequent updates; changing another field does not automatically refresh updated. The model does not enforce chronological ordering between these two timestamps or turn them into a version history.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`created`	string	Required	Instant at which this definition record was created, serialized as an offset-bearing date-time. Keep it distinct from the release date of the represented software and from a deployment creation time. format: date-time; pattern restrictions (see schema).
`updated`	string	Required	Instant recorded for the latest authored change. Update it deliberately when appropriate; it is not an automatically maintained clock, a lock digest or proof that a live instance matches the definition. format: date-time; pattern restrictions (see schema).

Relationships

- From Dot: containment via metadata; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "created": "2026-10-26T00:00:00Z",
  "updated": "2026-10-26T00:00:00Z"
}
```

Skill

Name a reusable, high-level capability so that components can be classified and discovered by what they do. A Skill is not executable code, an API operation or a deployment recipe.

A standalone Skill has its own descriptive metadata, while Dot.skills serializes only its identity. The catalogue resolves that identity to the authoritative label, description and capability category. Domain reference validation checks the key; repository validation checks that the referenced capability exists and is not merely a category asset.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`description`	string	Conditional / optional	Optional explanation of the capability and its scope. Describe the ability without duplicating the interface Contract or claiming that a runtime has been verified.

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Stable, case-sensitive capability key. Moving a catalogue file or changing a display label must not silently create a different identity; unknown keys are not interpreted as names. minimum length: 1; pattern restrictions (see schema).
`name`	string	Required	Human-readable capability label for a complete Skill entity. It is omitted from the ID-only reference view inside a DOT. minimum length: 1.

Relationships

- From Dot: ID/name reference via skills (ref); diagram multiplicity 1..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "id": "Metrics-Scraping",
  "name": "Metrics scraping"
}
```

Version

Carry a semantic software version or the special latest selector as a single JSON string. Use it where the owning model needs a software or platform version rather than a free-form label.

The Java default is 0.0.0; it is not evidence of a released component. Concrete values follow SemVer precedence, including prereleases; build metadata is ignored for precedence comparisons. The latest selector sorts after concrete versions and delegates selection to its context, so it is not an immutable artifact pin. Contract.version is a separate document-language string, not this value object. There is no nested JSON value property.

Java authority | Generated JSON Schema

JSON representation: string; pattern restrictions (see schema).

Relationships

- From Dot: containment via version; diagram multiplicity 1.
- From HelmChart: containment via version; diagram multiplicity 1.
- From Platform: containment via version; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"0.0.0"
```

Contracts and Anchors

How to read this reference explains full objects versus ID-only views, conditional fields and validation boundaries.

Anchor

Give one provided or required interaction a stable, DOT-local integration key. Compositions connect these logical points; deployment service bindings connect locally provided, non-brokered interfaces to named ports.

Choose exactly one communication mode: a Contract operation, a supported provider-neutral requirement, or explicit contractless communication facts. Do not duplicate direction, protocols or media types for a Contract-backed Anchor. Aggregate validation checks references, selector syntax and deployment bindings; the CLI resolves actual Contract operations. Declarations and a compatible wire are not live reachability, authentication or production acceptance.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Stable key used by composition endpoints and deployment Anchor bindings. Prefer meaningful DOT-local kebab-case names. Legacy UUID strings remain readable, but display names and selectors are not aliases for this key. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema).
`name`	string	Required	Readable label for the interaction. It can help an author choose an Anchor, but wires and service bindings reference id rather than this display label. minimum length: 1.
`role`	AnchorRole	Required	Whether this component offers the interaction or depends on it. This logical provider/consumer relationship is independent of which participant initiates a network request or the direction of payload flow. schema enum: "PROVIDES", "REQUIRES".
`contract`	AnchorContract	Conditional / optional	Reference to an authoritative Contract plus one standard-specific operation selector. Local ownership resolves through this DOT; external ownership requires the exact originating DOT source. Communication facts come from the selected document.

JSON field	Value	Base presence	Meaning and constraints
`requirement`	AnchorRequirement	Conditional / optional	Provider-neutral, versioned requirement on a REQUIRES Anchor. A composition chooses and verifies the concrete provider. It cannot be combined with a Contract reference or explicit contractless communication facts. exact value "OIDC_DISCOVERY_1_0".
`direction`	AnchorDirection	Conditional / optional	Payload-flow direction relative to the declaring DOT, only for a contractless Anchor. Do not infer it solely from PROVIDES or REQUIRES; request/response interactions and transport initiation are different concepts. schema enum: "INGRESS", "EGRESS", "BIDIRECTIONAL".
`protocols`	array of string	Conditional / optional	Explicit communication protocols for a contractless interaction. Declare the actual supported boundary rather than treating a display label, file format or service name as a protocol.
`mediaTypes`	array of string	Conditional / optional	Explicit payload media types for a contractless interaction. These describe the exchanged representation, not a full payload schema or a guarantee that another component understands its domain meaning.

Relationships

- From Dot: containment via anchors; diagram multiplicity 0..*.
- To AnchorContract: containment via contract; diagram multiplicity 0..1.
- To AnchorDirection: typed association via direction; diagram multiplicity 0..1.
- To AnchorRequirement: typed association via requirement; diagram multiplicity 0..1.
- To AnchorRole: typed association via role; diagram multiplicity 1.
- From AnchorReference: ID/name reference via anchorId.
- From AnchorBinding: ID/name reference via anchorId.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "id": "build-information",
  "name": "build-information",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
```

```
}
```

AnchorContract

Select one composable interaction from an existing authoritative Contract. Use it on an Anchor rather than copying an interface document or declaring another product-owned Contract.

The referenced identity, origin and selector must agree. OpenAPI and AsyncAPI use operation references; Protobuf uses a service method, GraphQL uses a root-field coordinate, and SPARQL uses the exact sd:endpoint IRI of a supported query service. A data-schema pointer is not an operation. Java aggregate validation checks local relationships and selector syntax; the CLI resolves the exact originating declaration, verifies its bytes and checks the selected operation. Matching shapes in independently authored Contracts do not make their identities interchangeable.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Exact, case-sensitive Contract identity. An external dependency retains the producer identity; merely declaring a syntactically valid key does not make a Contract available. minimum length: 1; pattern restrictions (see schema).
`selector`	string	Required	Standard-specific operation reference within the authoritative document. Select the interaction rather than a request/response data type; required operation parameters and security still need composition and deployment inputs. minimum length: 1.
`source`	string	Conditional / optional	Exact originating DOT coordinate for an external Contract reference in the selected catalogue. Omit it for a locally owned declaration. An unavailable origin is an error, not permission to search sibling folders, infer another release or copy a dependency declaration. pattern restrictions (see schema).

Relationships

- From Anchor: containment via contract; diagram multiplicity 0..1.
- To Contract: ID/name reference via id.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "id": "Example@1.0.0::OpenAPI::HTTP",
  "selector": "#/paths/~1ready/get"
```

```
}
```

AnchorDirection

Describe the direction of data flow at a contractless interaction relative to the declaring DOT. It fills in a communication fact that cannot be derived from an authoritative Contract.

Ingress means flow into the DOT, egress means flow out, and bidirectional means both. Keep this separate from the Anchor provider/consumer role and from the direction of a TCP connection. The current contractless matcher requires complementary directions alongside compatible protocols and media types; the enum alone proves none of those other conditions.

Java authority | Generated JSON Schema

Exact JSON values: `INGRESS`, `EGRESS`, `BIDIRECTIONAL`.

JSON representation: string; schema enum: "INGRESS", "EGRESS", "BIDIRECTIONAL".

Relationships

- From Anchor: typed association via direction; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"INGRESS"
```

AnchorRequirement

State a narrow, versioned provider-neutral interface requirement instead of hard-coding a particular producer Contract. The current supported profile concerns OIDC discovery.

Use it only on a REQUIRES Anchor and do not combine it with Contract or explicit contractless fields. A composition wire selects the real provider and the CLI checks the supported requirement profile against its verified Contract. This is not an arbitrary semantic label, automatic provider discovery or full OIDC, TLS and production IAM acceptance.

Java authority | Generated JSON Schema

Exact JSON values: `OIDC_DISCOVERY_1_0`.

JSON representation: string; exact value "OIDC_DISCOVERY_1_0".

Relationships

- From Anchor: typed association via requirement; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"OIDC_DISCOVERY_1_0"
```

AnchorRole

Distinguish a component that provides an interaction from one that requires it. This is the logical relationship used to connect a provider Anchor to a consumer Anchor.

Role is not payload-flow direction, server/client process type or permission to call an API. A component can provide an HTTP operation even when another component initiates the request. Complementary roles do not replace Contract identity, operation, protocol, security or live acceptance checks.

Java authority | Generated JSON Schema

Exact JSON values: `PROVIDES`, `REQUIRES`.

JSON representation: string; schema enum: "PROVIDES", "REQUIRES".

Relationships

- From Anchor: typed association via role; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"PROVIDES"
```

Contract

Identify one authoritative interface specification and the exact downloadable document that describes it. Anchors select interactions from that document instead of copying its protocol and payload details.

Keep the declaration at its owning DOT and retain that identity when another component consumes or implements the interface. Qualified identities must agree with the exact owning DOT release, standard and local Contract name. Complete definitions carry a verified URL/hash payload; an ID-only reference view is not a publishable owned declaration. Domain validation checks declaration relationships and metadata, while the CLI fetches, hashes and parses the document and resolves operations. Neither establishes that a producer implements the interface at runtime.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`format`	DataFormat	Required	Encoding of the downloaded document, not its transport protocol. It must be supported by the chosen standard; an extension or a matching file name does not establish compatibility. schema enum: "YAML", "JSON", "PROTOBUF", "GRAPHQL", "N_TRIPLES".

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Stable interface identity, independent of its download URL. Catalogue identities include the originating DOT release and the exact standard/name suffix. Consumers retain this key rather than minting a consumer-owned copy. minimum length: 1; pattern restrictions (see schema).
`name`	string	Required	Local, case-sensitive interface name within the owning DOT. It is part of the qualified identity, not a hostname, an operation selector or the upstream API release number. minimum length: 1.
`payload`	Payload	Required	Structured raw-document location and integrity pin. The specification bytes are downloaded separately; this property does not embed the document or provide a local Contract-folder fallback.
`standard`	ContractStandard	Required	Interface language used to interpret the document and Anchor selectors. Use the exact generated wire spelling; the Java enum constant name is not necessarily the serialized value. schema enum: "OpenAPI", "AsyncAPI", "Protobuf", "GraphQL", "SPARQL".
`version`	string	Conditional / optional	Version of the document language declared by the verified payload: the OpenAPI or AsyncAPI marker, the native Protobuf syntax/edition, or 1.1 from the SPARQL Service Description query capability. Do not copy a tool release or info.version. GraphQL SDL has no such marker and omits this property.

Relationships

- From Dot: containment via contracts; diagram multiplicity 0..*.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To Payload: containment via payload; diagram multiplicity 0..1.
- To ContractStandard: typed association via standard; diagram multiplicity 1.
- From DeploymentService: ID/name reference via contracts (ref); diagram multiplicity 0..*.
- From AnchorContract: ID/name reference via id.
- From ContractInput: ID/name reference via id.

Example derived from the canonical Prometheus definition (not a second product)


```
{
  "id": "Prometheus@3.15.1::OpenAPI::HTTP",
  "name": "HTTP",
  "version": "3.1.0",
  "standard": "OpenAPI",
  "format": "YAML",
  "payload": {
    "url":
"https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1/web/api/v1/testdata/o
penapi_3.1_golden.yaml",
    "sha256": "406b517ed3800c107b3afbc2d9712f2e487269fe8c8b55a075dce46823d8d7c0"
  }
}
```


ContractStandard

Choose the interface-description language used by a Contract. This choice determines how the document is parsed, which encoding is valid and how an Anchor operation is selected.

Use the exact serialized values shown by the generated enum reference; alternative capitalization, whitespace and Java constant names are not aliases. Standard version and document format remain separate properties. Selecting a supported language is not a claim that every optional feature or live protocol behavior is implemented by the current CLI.

Java authority | Generated JSON Schema

Exact JSON values: `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`, `SPARQL`.

JSON representation: string; schema enum: "OpenAPI", "AsyncAPI", "Protobuf", "GraphQL", "SPARQL".

Relationships

- From Contract: typed association via standard; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)


```
"OpenAPI"
```


DataFormat

Identify the serialization format of a document or manifest payload. It describes how bytes are represented, not the purpose of the interface or the infrastructure that runs it.

The owning field determines which formats are valid. Contract formats must match their Contract standard, and Docker Compose, Kubernetes manifests and Helm values accept only their supported structured text encodings. A format enum does not add a new deployment mechanism, turn a Vagrantfile into YAML or describe an HTTP media type.

Java authority | Generated JSON Schema

Exact JSON values: `YAML`, `JSON`, `PROTOBUF`, `GRAPHQL`, `N_TRIPLES`.

JSON representation: string; schema enum: "YAML", "JSON", "PROTOBUF", "GRAPHQL", "N_TRIPLES".

Relationships

- From Contract: typed association via format; diagram multiplicity 1.
- From DockerCompose: typed association via format; diagram multiplicity 1.
- From HelmValues: typed association via format; diagram multiplicity 1.
- From KubernetesManifest: typed association via format; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"YAML"
```

Payload

Locate an exact raw document and pin its bytes. This shared value describes a downloadable file, never the address of a deployed API or an instruction to execute it.

Contract.payload, DockerCompose.payload, KubernetesManifest.payload, Vagrantfile.payload, HelmValues.payload and EnvFile.payload use this same value; ConfigFile.payload optionally uses it too. There is no inline-text, Base64 or adjacent-file fallback. URLs must be absolute HTTPS without embedded credentials or fragments. The current-source CLI reuses bounded verification and a digest-addressed cache outside DOT bundles, rehashes cached bytes and fails on corruption. Offline use needs every required document already cached; it does not prove that an origin is currently available. Java/schema validity alone does not download bytes. A matching hash establishes byte identity, not publisher authority, license permission, safe execution, producer conformance or live availability.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`sha256`	string	Required	Lower-case SHA-256 of the exact downloaded bytes. Formatting or newline changes alter this pin even if a reader considers the document equivalent; review changed bytes rather than silently replacing the expected digest. minimum length: 64; maximum length: 64; pattern restrictions (see schema).

JSON field	Value	Base presence	Meaning and constraints
`url`	string	Required	Download location for the raw document, not an HTML viewer, deployed endpoint, local path or operation fragment. The owner determines whether the verified bytes are a Contract, manifest, values, dotenv or default configuration document. format: uri; pattern restrictions (see schema).

Relationships

- From Contract: containment via payload; diagram multiplicity 0..1.
- From ConfigFile: containment via payload; diagram multiplicity 0..1.
- From DockerCompose: containment via payload; diagram multiplicity 1.
- From EnvFile: containment via payload; diagram multiplicity 1.
- From HelmValues: containment via payload; diagram multiplicity 1.
- From KubernetesManifest: containment via payload; diagram multiplicity 1.
- From Vagrantfile: containment via payload; diagram multiplicity 1.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/docker-compose.yaml",
  "sha256": "648687feae439ddc1a285c9c61e6bf855c9426a3bb4f16b36e56b68ae9529439"
}
```

Runtimes and deployment

How to read this reference explains full objects versus ID-only views, conditional fields and validation boundaries.

AnchorBinding

Maps a logical DOT Anchor to a named port of one deployment service.

Use an exact Anchor ID from the same DOT and an existing port name on the containing service. The binding connects logical interface selection to deployment structure; it does not contain a URL, credentials or a host-port publication rule.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`anchorId`	string	Required	The exact DOT-local Anchor identifier being materialized. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema).
`port`	string	Required	The name of the containing service port, not its numeric port value. minimum length: 1.

Relationships

- From DeploymentService: containment via anchorBindings; diagram multiplicity 0..*.
- To Anchor: ID/name reference via anchorId.
- To ServicePort: ID/name reference via port.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "anchorId": "build-information",
  "port": "http"
}
```

CdiDeviceRequirement

Declares a service's need for a device exposed through the Container Device Interface.

The DOT names a logical requirement and device kind, not a host device path or a concrete allocated device. Deployment inputs bind a fully qualified device of the matching kind. Device allocation, CDI specification installation and authorization remain operator or orchestrator responsibilities.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`description`	string	Conditional / optional	An explanation of why the service needs this device.
`kind`	string	Required	The CDI vendor/class identity that a supplied device must match. minimum length: 1; maximum length: 317; pattern restrictions (see schema).
`name`	string	Required	The service-local logical identifier used when binding a concrete device. minimum length: 1; maximum length: 63; pattern restrictions (see schema).

Relationships

- From DeploymentConfig: containment via cdiDevices; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "gpu",
  "kind": "vendor.example/gpu"
}
```

ConfigFile

Documents a configuration file expected by a service.

The path is the destination inside the workload, not a source file on the operator's machine. An optional payload is a shared Payload reference, never inline text or Base64; omit it when no default exists. Java null serializes as omission and does not expand raw-schema null acceptance. Instance inputs can supply replacement content. A sensitive file forbids every non-null default Payload, including an empty value or a publicly hosted document; a URL is not a secret-provisioning mechanism. This declaration alone creates no mount and does not verify remote bytes.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`contentType`	string	Conditional / optional	The media type describing how the file content should be interpreted.
`description`	string	Conditional / optional	An explanation of the file's role.
`path`	string	Required	The destination path where the workload expects the configuration file. minimum length: 1.
`payload`	Payload	Conditional / optional	Optional HTTPS URL and SHA-256 pin of the default file used when no instance replacement is supplied. It must be absent for a sensitive file.
`required`	boolean	Conditional / optional	Whether content must be supplied when the declaration provides none.
`sensitive`	boolean	Conditional / optional	Whether tooling must protect the file content as sensitive information.

Relationships

- To Payload: containment via payload; diagram multiplicity 0..1.
- From DeploymentConfig: containment via configFiles; diagram multiplicity 0..*.

Example derived from the canonical Prometheus definition (not a second product)

```
{
```

```
  "path": "/etc/prometheus/prometheus.yml",
  "contentType": "application/yaml",
  "payload": {
    "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/configuration-kubernetes-manifest-
prometheus-config-0.yaml",
    "sha256": "36d801295435ff6ba6f00dbc47fc1c16783ae84a493e405c392bbc20659364fc"
  },
  "description": "Prometheus scrape configuration. Explicit operator inputs replace
this self-scrape default."
}
```


Deployment

Describes one selectable deployment mechanism and the logical services it materializes.

The selected manifest, including its explicit pinned Payload references, is the complete deployment input; files beside dot.json are not an implicit fallback. Keep service IDs distinct within the deployment and select the deployment explicitly. Configuration belongs to each service, while instance names, live state and lifecycle ownership belong to the executor.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	The stable name used to select this deployment alternative. minimum length: 1.
`manifest`	DockerImage or DockerCompose or KubernetesManifest or HelmChart or Vagrantfile or WebAssembly Module	Required	The platform-specific specification used to create the workload. conditional constraints (see schema).
`services`	array of DeploymentService	Required	The logical services supplied by this particular deployment.

Relationships

- To DeploymentService: containment via services; diagram multiplicity 0..*.
- To DeploymentManifest: containment via manifest; diagram multiplicity 1.
- From Runtime: containment via deployments; diagram multiplicity 1..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "id": "image",
  "manifest": {
    "type": "DOCKER_IMAGE",
    "image": "docker.io/example/api:1.0.0"
  },
  "services": []
}
```

DeploymentConfig

Declares the configuration options and execution requirements supported by one service.

This is not a bag of applied operator values. Keep environment names, configuration paths, volume names and paths, and CDI requirement names unambiguous within the service; separate services can reuse those keys. Instance inputs supply overrides, storage bindings and secrets. Requirements never grant privileges by themselves.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`cdiDevices`	array of CdiDeviceRequirement	Conditional / optional	Logical requirements for devices supplied through CDI.
`configFiles`	array of ConfigFile	Required	Configuration files the service can consume, with optional pinned non-sensitive default documents.
`environment`	array of EnvironmentVariable	Required	Environment options understood by the service.
`ociRuntime`	OciRuntimeRequirements	Conditional / optional	Additional container capabilities or host-namespace requirements needing explicit authorization.
`volumes`	array of VolumeMount	Required	Logical mount points expected by the service.

Relationships

- From DeploymentService: containment via config; diagram multiplicity 1.
- To ConfigFile: containment via configFiles; diagram multiplicity 0..*.
- To EnvironmentVariable: containment via environment; diagram multiplicity 0..*.
- To VolumeMount: containment via volumes; diagram multiplicity 0..*.
- To CdiDeviceRequirement: containment via cdiDevices; diagram multiplicity 0..*.

- To OciRuntimeRequirements: containment via ociRuntime; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "environment": [],
  "configFiles": [],
  "volumes": [],
  "cdiDevices": []
}
```

DeploymentManifest

Provides the common base for concrete deployment specifications.

The existing type property selects the concrete manifest shape; it is not a second infrastructure selector. Supply the fields of that concrete manifest. Domain validation does not execute a payload, fetch a chart or establish deployment success.

Java authority | Generated JSON Schema

Manifest alternatives: DockerImage, DockerCompose, KubernetesManifest, HelmChart, Vagrantfile, WebAssemblyModule.

JSON field	Value	Base presence	Meaning and constraints
`type`	DeploymentManifestType	Required	The discriminator identifying how this manifest is deployed. conditional constraints (see schema).

Relationships

- To DockerCompose: inheritance.
- To DockerImage: inheritance.
- To HelmChart: inheritance.
- To KubernetesManifest: inheritance.
- To Vagrantfile: inheritance.
- To WebAssemblyModule: inheritance.
- From Deployment: containment via manifest; diagram multiplicity 1.
- To DeploymentManifestType: typed association via type; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "type": "DOCKER_IMAGE",
  "image": "docker.io/example/api:1.0.0"
}
```

DeploymentManifestType

Names how a workload is deployed and links that mechanism to its target platform.

Do not substitute this value for Platform.engine. The manifest subtype and its type must agree, and its target platform must match the enclosing runtime. Executor capabilities and installed tools remain separate from this model vocabulary.

Java authority | Generated JSON Schema

Exact JSON values: `DOCKER_IMAGE`, `DOCKER_COMPOSE`, `KUBERNETES_MANIFEST`, `HELM_CHART`, `VAGRANTFILE`, `WASM_MODULE`.

JSON representation: string; schema enum: "DOCKER_IMAGE", "DOCKER_COMPOSE", "KUBERNETES_MANIFEST", "HELM_CHART", "VAGRANTFILE", "WASM_MODULE".

Relationships

- To PlatformEngine: typed association via platform; diagram multiplicity 0..1.
- From DeploymentManifest: typed association via type; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"DOCKER_IMAGE"
```

DeploymentService

Describes a logical service provided by one deployment, including its interfaces and configuration surface.

Use stable service IDs for composition inputs and named ports for Anchor bindings. Contract entries are identity references, not copied declarations; their origin must resolve through the DOT interfaces. A logical service is not a native workload selector, and its host is not a promised public endpoint.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	The stable service identifier within its deployment. minimum length: 1.
`name`	string	Required	The readable name of the service. minimum length: 1.
`host`	string	Required	The host by which the service is reached inside the deployment. minimum length: 1.
`ports`	array of ServicePort	Required	Named service ports that can be selected by Anchor bindings.

JSON field	Value	Base presence	Meaning and constraints
`contracts`	array of Contract (ID reference)	Required	ID-only references to the interfaces exposed or consumed by this service.
`anchorBindings`	array of AnchorBinding	Conditional / optional	Mappings from this DOT's logical Anchors to the service's named ports.
`config`	DeploymentConfig	Required	The environment, files, storage and execution requirements supported by this service.

Relationships

- To Contract: ID/name reference via contracts (ref); diagram multiplicity 0..*.
- To AnchorBinding: containment via anchorBindings; diagram multiplicity 0..*.
- To ServicePort: containment via ports; diagram multiplicity 0..*.
- To DeploymentConfig: containment via config; diagram multiplicity 1.
- From Deployment: containment via services; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "id": "api",
  "name": "API",
  "host": "api",
  "ports": [],
  "contracts": [],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": [],
    "cdiDevices": []
  }
}
```

DockerCompose

References the exact Docker Compose document for a multi-container deployment.

The shared Payload identifies complete document bytes by HTTPS URL and SHA-256, not inline text, Base64 or a compose file beside dot.json. The format describes the downloaded document. Ordered environment-file references follow Compose precedence; instance overrides remain separate. Logical services and any native workload selection must agree with the selected deployment. Verification and cache readiness do not establish live deployment success.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`type`	DeploymentManifestType	Required	The discriminator selecting the Docker Compose manifest. exact value "DOCKER_COMPOSE".
`format`	DataFormat	Required	The downloaded Compose document must use YAML or JSON; other DataFormat values fail owner validation. schema enum: "YAML", "JSON", "PROTOBUF", "GRAPHQL", "N_TRIPLES".
`payload`	Payload	Required	The shared Payload reference for the complete Compose document.
`envFiles`	array of EnvFile	Conditional / optional	Ordered environment-file payloads applied to the Compose invocation.

Relationships

- From DeploymentManifest: inheritance.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To Payload: containment via payload; diagram multiplicity 1.
- To EnvFile: containment via envFiles; diagram multiplicity 0..*.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "type": "DOCKER_COMPOSE",
  "format": "YAML",
  "payload": {
    "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/docker-compose.yaml",
    "sha256": "648687feae439ddc1a285c9c61e6bf855c9426a3bb4f16b36e56b68ae9529439"
  }
}
```

DockerImage

Runs a pre-built container image as a Docker deployment alternative.

Reference a published image rather than expecting the CLI to build a nearby Dockerfile. Ordered environment-file payloads provide reusable defaults; later files and explicit instance environment values take precedence. Expose user-facing configuration through each logical service rather than adding deployment-level overrides to the model.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`type`	DeploymentManifestType	Required	The discriminator selecting the Docker image manifest. exact value "DOCKER_IMAGE".

JSON field	Value	Base presence	Meaning and constraints
`image`	DockerImageTag	Required	The compact image reference to pull and run.
`envFiles`	array of EnvFile	Conditional / optional	Ordered Docker-compatible environment-file payloads for reusable manifest defaults.

Relationships

- From DeploymentManifest: inheritance.
- To DockerImageTag: containment via image; diagram multiplicity 1.
- To EnvFile: containment via envFiles; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "type": "DOCKER_IMAGE",
  "image": "docker.io/example/api:1.0.0"
}
```

DockerImageTag

Represents a container image tag as a compact JSON string.

Although Java exposes registry, image and version components, JSON uses one image-reference string, not an object containing those fields. The Java parser is tag-based and rejects digest references. Image-reference parsing alone does not verify registry availability, artifact integrity or executor support.

Java authority | Generated JSON Schema

JSON representation: string.

Relationships

- From DockerImage: containment via image; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"docker.io/example/api:1.0.0"
```

EnvFile

References a reusable Docker-compatible environment document through a pinned Payload.

The URL must identify line-oriented dotenv bytes, not an adjacent file, inline string or Base64 content. The containing manifest determines ordering and override precedence. Keep deployment-instance values and credentials out of reusable documents; EnvFile is not a secret reference. Its compatibility-corpus example is shape-only: the current catalogue declares no EnvFile deployment.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
payload	Payload	Required	The shared Payload reference whose verified dotenv bytes are passed to Docker or Compose.

Relationships

- From DockerCompose: containment via envFiles; diagram multiplicity 0..*.
- From DockerImage: containment via envFiles; diagram multiplicity 0..*.
- To Payload: containment via payload; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "payload": {
    "url": "https://example.org/default.env",
    "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
  }
}
```

EnvironmentVariable

Documents an environment option accepted by a service.

The declaration explains the option; deployment inputs provide the applied value. A reusable default can be used when there is no override, but a sensitive option must not embed a default. Marking a value sensitive is not encryption or a substitute for secret provisioning.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
defaultValue	string	Conditional / optional	Reusable non-sensitive text available when no instance override is supplied.
description	string	Conditional / optional	An explanation of what the option controls.
name	string	Required	The environment variable name consumed by the workload. minimum length: 1.
required	boolean	Conditional / optional	Whether the deployer must supply a value when the declaration provides none.
sensitive	boolean	Conditional / optional	Whether tooling must treat values for this option as sensitive.

Relationships

- From DeploymentConfig: containment via environment; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "MODE",
  "defaultValue": "demo"
}
```

HelmChart

References a Helm chart and preserves its reusable chart-level values.

Keep chart version separate from the represented tool release. The portable model permits an omitted repository, but the current dotx executor needs repository plus chart name, a complete OCI chart reference, or an explicit --registry / DOTX_REGISTRY input. It does not resolve pre-registered Helm aliases from the operator environment. Values are ordered, with later entries overriding earlier ones. Release name and namespace are instance concerns, while service-facing configuration belongs on DeploymentService.config.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`chart`	string	Required	The chart name or self-contained chart reference. minimum length: 1.
`repository`	string	Conditional / optional	The chart repository URL when the chart reference needs one. pattern restrictions (see schema).
`type`	DeploymentManifestType	Required	The discriminator selecting a Helm chart deployment. exact value "HELM_CHART".
`values`	array of HelmValues	Conditional / optional	Ordered Helm values documents merged for chart rendering and installation.
`version`	Version	Required	The chart version selector, not the application's software version. pattern restrictions (see schema).

Relationships

- From DeploymentManifest: inheritance.
- To Version: containment via version; diagram multiplicity 1.
- To HelmValues: containment via values; diagram multiplicity 0..*.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "type": "HELM_CHART",
  "repository": "https://prometheus-community.github.io/helm-charts",
  "chart": "prometheus",
```

```
  "version": "29.38.2",
  "values": [
    {
      "format": "YAML",
      "payload": {
        "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/helm-chart-values-0.yaml",
        "sha256": "a062a29a63ec5a729c1c6a7ebff834dd446f2041f970ca7fd1a3e7e6723b5f6b"
      }
    }
  ]
}
```

HelmValues

References one pinned values document for a Helm chart's template configuration.

Use this for chart-level settings such as resource configuration or persistence, rather than inventing extra portable model fields. Its shared Payload is an HTTPS URL and SHA-256 pin, not embedded YAML/JSON or Base64. The downloaded document format must be supported by Helm values validation. The containing chart controls merge order; supported environment, file and mount options still belong in service configuration.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`format`	DataFormat	Required	The downloaded Helm values document must use YAML or JSON; other DataFormat values fail owner validation. schema enum: "YAML", "JSON", "PROTOBUF", "GRAPHQL", "N_TRIPLES".
`payload`	Payload	Required	The shared Payload reference whose verified bytes form one values-file input.

Relationships

- From HelmChart: containment via values; diagram multiplicity 0..*.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To Payload: containment via payload; diagram multiplicity 1.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "format": "YAML",
  "payload": {
    "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/helm-chart-values-0.yaml",
```

```
    "sha256": "a062a29a63ec5a729c1c6a7ebff834dd446f2041f970ca7fd1a3e7e6723b5f6b"
  }
}
```

KubernetesManifest

References the exact Kubernetes resource document used by a deployment.

The shared Payload identifies the complete YAML or JSON bytes by HTTPS URL and SHA-256; there is no inline, Base64 or adjacent-manifest fallback. Native resource syntax, tool readiness and live rollout are executor concerns beyond the domain's format checks. Do not confuse a logical service with the exact workload that an instance must select.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`format`	DataFormat	Required	The downloaded Kubernetes documents must use YAML or JSON; other DataFormat values fail owner validation. schema enum: "YAML", "JSON", "PROTOBUF", "GRAPHQL", "N_TRIPLES".
`payload`	Payload	Required	The shared Payload reference whose verified resource documents are applied by the adapter.
`type`	DeploymentManifestType	Required	The discriminator selecting the Kubernetes manifest deployment. exact value "KUBERNETES_MANIFEST".

Relationships

- From DeploymentManifest: inheritance.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To Payload: containment via payload; diagram multiplicity 1.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "type": "KUBERNETES_MANIFEST",
  "format": "YAML",
  "payload": {
    "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/configuration-kubernetes-manifest.yaml",
    "sha256": "ab0749b1a3e9ab748f7a531c69b90d45161ff42eac0688edee2812533be26174"
  }
}
```

OciLinuxNamespace

Identifies a Linux namespace type in OCI execution requirements.

In hostNamespaces, this value requests sharing that namespace with the host, not creating a new isolated namespace. The serialized spelling follows OCI vocabulary. A model value is not permission to grant access, nor a guarantee of support by every adapter.

Java authority | Generated JSON Schema

Exact JSON values: `pid`, `network`, `mount`, `ipc`, `uts`, `user`, `cgroup`, `time`.

JSON representation: string; schema enum: "pid", "network", "mount", "ipc", "uts", "user", "cgroup", "time".

Relationships

- From OciRuntimeGrant: typed association via hostNamespaces; diagram multiplicity 0..*.
- From OciRuntimeRequirements: typed association via hostNamespaces; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"pid"
```

OciRuntimeRequirements

Declares additional Linux-container execution requirements using OCI vocabulary.

These are requests, not grants or unrestricted privileged mode. Compositions must explicitly authorize the declared set, and an adapter must fail closed when it cannot materialize the request. Duplicate declarations are rejected rather than silently removed during collection conversion.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`capabilities`	array of string	Required	Additional Linux process capabilities needed by the workload. distinct items.
`hostNamespaces`	array of OciLinuxNamespace	Required	Namespace types that the workload needs to share with the host instead of isolating. distinct items.

Relationships

- From DeploymentConfig: containment via ociRuntime; diagram multiplicity 0..1.
- To OciLinuxNamespace: typed association via hostNamespaces; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "capabilities": [
    "CAP_NET_RAW",
    "CAP_NET_ADMIN"
  ],
  "hostNamespaces": [
    "network",
    "pid"
  ]
}
```

Platform

Identifies the infrastructure targeted by a runtime, independently of its deployment mechanism.

Keep the platform version separate from the software release and chart version. Java construction defaults the engine to Kubernetes and the platform version to latest; those defaults do not waive required raw JSON properties. A platform declaration does not install the infrastructure or prove that its tools are available; the deployment environment resolves version selectors and checks readiness.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`engine`	PlatformEngine	Required	The infrastructure engine that hosts the workload. schema enum: "DOCKER", "KUBERNETES", "VAGRANT", "WEBASSEMBLY".
`version`	Version	Required	The platform or runtime version selector, not the deployed product version. pattern restrictions (see schema).

Relationships

- From Runtime: containment via platform; diagram multiplicity 1.
- To Version: containment via version; diagram multiplicity 1.
- To PlatformEngine: typed association via engine; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "engine": "DOCKER",
  "version": "latest"
}
```

PlatformEngine

Names where a workload runs.

Use this infrastructure vocabulary on Platform. It is different from the manifest mechanism: multiple deployment mechanisms can target the same engine. An engine value does not promise that the current executor supports every operation on that platform.

Java authority | Generated JSON Schema

Exact JSON values: `DOCKER`, `KUBERNETES`, `VAGRANT`, `WEBASSEMBLY`.

JSON representation: string; schema enum: "DOCKER", "KUBERNETES", "VAGRANT", "WEBASSEMBLY".

Relationships

- From DeploymentManifestType: typed association via platform; diagram multiplicity 0..1.
- From Platform: typed association via engine; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"DOCKER"
```

Runtime

Groups the deployment alternatives for one infrastructure platform inside a DOT.

Choose a deployment from this group rather than treating every entry as a workload to start. Deployment IDs must be distinct within the runtime, and every manifest must target the runtime platform. This declaration is not a running instance or a readiness report.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`platform`	Platform	Required	The infrastructure on which these deployment alternatives can run.
`deployments`	array of Deployment	Required	Named ways to deploy this DOT on the selected platform. minimum items: 1.

Relationships

- From Dot: containment via runtimes; diagram multiplicity 0..*.
- To Deployment: containment via deployments; diagram multiplicity 1..*.
- To Platform: containment via platform; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
```

```
  "platform": {
    "engine": "DOCKER",
    "version": "latest"
  },
  "deployments": [
    {
      "id": "image",
      "manifest": {
        "type": "DOCKER_IMAGE",
        "image": "docker.io/example/api:1.0.0"
      },
      "services": []
    }
  ]
}
```

ServicePort

Names a service endpoint port so interfaces can bind to it without copying runtime addresses.

Port names must be distinct within the service and must match AnchorBinding.port exactly. A protocol label describes the endpoint; it does not itself supply an API Contract or authorize a probe. Host forwarding and externally reachable addresses are deployment-instance concerns.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`name`	string	Required	The logical port name used by bindings and references. minimum length: 1.
`protocol`	string	Required	The application or transport protocol label for this endpoint. minimum length: 1.
`port`	integer	Required	The numeric port exposed by the service inside its deployment. minimum: 1; maximum: 65535.

Relationships

- From DeploymentService: containment via ports; diagram multiplicity 0..*.
- From AnchorBinding: ID/name reference via port.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "http",
  "protocol": "TCP",
  "port": 8080
}
```

Vagrantfile

References the complete Vagrantfile used to provision a virtual-machine deployment.

The shared Payload pins downloaded Ruby DSL bytes; it is not embedded code, Base64, a separately structured box declaration or an adjacent-file lookup. Review provisioning, mounts and networking before execution. Service configuration and instance overrides remain separate; verifying the document does not run or certify the virtual machine.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`payload`	Payload	Required	The shared Payload reference whose verified program bytes are supplied to Vagrant.
`type`	DeploymentManifestType	Required	The discriminator selecting the Vagrant deployment. exact value "VAGRANTFILE".

Relationships

- From DeploymentManifest: inheritance.
- To Payload: containment via payload; diagram multiplicity 1.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "type": "VAGRANTFILE",
  "payload": {
    "url":
"https://gist.githubusercontent.com/MaurizioCasciano/89e8ff4a9ee095a588c919f767b7daf2/
raw/13ab666ed5afa076c856d36ea0cc49110b043cc2/configuration-vagrantfile.rb",
    "sha256": "91b0f9eb3bd586c72b14bbd1d02e36054adcc6cd4ad2357416792a2c8beb250c"
  }
}
```

VolumeMount

Declares a logical storage mount expected by a service.

Keep the logical name and workload destination separate from a host bind path, named volume or persistent-volume claim. The instance supplies the concrete source and authorization. A declared read-only expectation must be enforceable by the selected adapter; it is not a storage allocation or a cleanup policy.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`description`	string	Conditional / optional	An explanation of the expected contents or use of the mount.

JSON field	Value	Base presence	Meaning and constraints
`name`	string	Required	The logical identifier used to bind storage to this mount. minimum length: 1.
`path`	string	Required	The destination path inside the workload. minimum length: 1.
`readOnly`	boolean	Conditional / optional	Whether the workload expects read-only access to this mount.
`required`	boolean	Conditional / optional	Whether the deployer must provide a concrete storage source.

Relationships

- From DeploymentConfig: containment via volumes; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "data",
  "path": "/var/lib/example",
  "readOnly": false
}
```

WebAssemblyExecution

Selects the WASI execution world expected by a WebAssembly artifact.

Match the artifact's implemented world to the intended command or HTTP service behavior. Command arguments and HTTP service ports have different executor rules. This selector is not a generic promise to run every WebAssembly component or provide unrestricted operating-system access.

Java authority | Generated JSON Schema

Exact JSON values: `WASI_CLI`, `WASI_HTTP`.

JSON representation: string; schema enum: "WASI_CLI", "WASI_HTTP".

Relationships

- From WebAssemblyModule: typed association via execution; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"WASI_CLI"
```

WebAssemblyModule

References exact WebAssembly bytes and the WASI execution world used to run them.

Keep the module location and checksum together so the adapter can verify bytes before execution. The standard adapter supports local/file and HTTPS locations, and command arguments apply only to CLI execution. Service-declared configuration, explicit directory preopens and adapter support determine available resources; choosing a world does not grant arbitrary host access.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`arguments`	array of string	Required	Arguments passed to the CLI command after the module path.
`execution`	WebAssemblyExecution	Required	The WASI world used to invoke the artifact. schema enum: "WASI_CLI", "WASI_HTTP".
`module`	string	Required	The location from which the adapter obtains the module or component. minimum length: 1; pattern restrictions (see schema).
`sha256`	string	Required	The lower-case SHA-256 pin of the exact artifact bytes. minimum length: 1; pattern restrictions (see schema).
`type`	DeploymentManifestType	Required	The discriminator selecting the WebAssembly deployment. exact value "WASM_MODULE".

Relationships

- From DeploymentManifest: inheritance.
- To WebAssemblyExecution: typed association via execution; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "type": "WASM_MODULE",
  "module": "module.wasm",
  "execution": "WASI_CLI",
  "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "arguments": []
}
```

Compositions and connections

How to read this reference explains full objects versus ID-only views, conditional fields and validation boundaries.

AnchorReference

Identify a logical Anchor on one composition member.

The Anchor ID belongs to the referenced DOT, while the instance name belongs to this composition. Neither field is an address, a service name or a Contract operation selector; the executor resolves

those relationships from the selected definition.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`instance`	string	Required	Name of the composition member containing the Anchor. minimum length: 1.
`anchorId`	string	Required	Exact DOT-local Anchor key on that member. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema).

Relationships

- From ExposedAnchor: containment via target; diagram multiplicity 1.
- From Wire: containment via from; diagram multiplicity 1.
- From Wire: containment via to; diagram multiplicity 1.
- To DotInstance: ID/name reference via instance.
- To Anchor: ID/name reference via anchorId.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "instance": "metrics",
  "anchorId": "http-api"
}
```

CdiDeviceBinding

Assign an installed CDI device to a logical device requirement of a deployment.

Bind the declared requirement to a concrete device of the same CDI kind. This does not install a CDI specification, allocate hardware or authorize an arbitrary host device path; availability and supported adapter behavior remain deployment concerns.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`requirement`	string	Required	Logical device-requirement name declared by the selected service configuration. minimum length: 1.
`device`	string	Required	Fully qualified CDI device identity chosen by the operator. maximum length: 381; pattern restrictions (see schema).

Relationships

- From DeploymentInputs: containment via cdiDevices; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "requirement": "gpu",
  "device": "vendor.example/gpu=0"
}
```

ComposeWorkloadBinding

Select the Compose service implementing one logical DOT service.

Use the service key in the declared Compose manifest, not a generated container name or the logical DOT service ID by assumption. The executor verifies that the mapping is valid for the selected deployment.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`service`	string	Required	Compose manifest service key selected for this logical service. minimum length: 1.

Relationships

- From ServiceWorkloadBinding: containment via compose; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "service": "api"
}
```

Composition

Describe one reusable use case by selecting DOT instances and connecting their logical interfaces.

This is a separate aggregate, not a DOT containing other DOT definitions. Instance references must name declared members and explicit readiness dependencies must be acyclic; interface wires may form cycles. Catalogue resolution, reviewed locks, credentials, deployment ownership and live acceptance remain separate executor concerns.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
apiVersion	string	Required	Identifies the portable composition input profile understood by the reader. exact value "dot.mare6g.eu/v1"; pattern restrictions (see schema).
kind	string	Required	Distinguishes a composition declaration from other document kinds. exact value "Composition"; pattern restrictions (see schema).
id	string	Required	Stable identity used to associate lifecycle ownership with this composition, independent of its display name. pattern restrictions (see schema).
name	string	Required	Human-readable name for the use case, not a replacement for its ownership identity. minimum length: 1.
version	string	Required	Version label for this use-case declaration; it is not the version of every participating tool. minimum length: 1.
dots	array of DotInstance	Required	Named instances selecting canonical DOT definitions and their deployments. minimum items: 1.
contracts	array of ContractInput	Conditional / optional	Explicit interface-document inputs resolved from authoritative owners, without transferring Contract ownership.
wires	array of Wire	Conditional / optional	Logical connections from provided Anchors to required Anchors.
exposedAnchors	array of ExposedAnchor	Conditional / optional	Names for selected member Anchors at the composition boundary; these do not themselves publish a network listener.

Relationships

- To ContractInput: containment via contracts; diagram multiplicity 0..*.
- To DotInstance: containment via dots; diagram multiplicity 1..*.
- To ExposedAnchor: containment via exposedAnchors; diagram multiplicity 0..*.
- To Wire: containment via wires; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "apiVersion": "dot.mare6g.eu/v1",
  "kind": "Composition",
  "id": "00000000-0000-4000-8000-000000000001",
  "name": "example",
```

```
  "version": "v1",
  "dots": [
    {
      "name": "metrics",
      "source": "repository://Example@1.0.0",
      "dotId": "Example@1.0.0",
      "dotVersion": "1.0.0"
    }
  ]
}
```

ContractInput

Make an authoritative Contract document available to composition resolution.

Reference the originating DOT instead of copying its Contract declaration or payload into a consumer. The owner supplies the pinned URL and digest. Historical wrapper sources remain a compatibility path, not a fallback when an owning declaration is missing.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`id`	string	Required	Exact originating Contract identity used by the selected interfaces. minimum length: 1; pattern restrictions (see schema).
`source`	string	Required	Source identifying the owning DOT whose root declaration supplies the Contract metadata and verified payload. minimum length: 1.

Relationships

- From Composition: containment via contracts; diagram multiplicity 0..*.
- To Contract: ID/name reference via id.

Example derived from the canonical Prometheus definition (not a second product)

```
{
  "id": "Prometheus@3.15.1::OpenAPI::HTTP",
  "source": "repository://Prometheus@3.15.1"
}
```

DeploymentConfigFile

Bind a local configuration file to a service-declared file destination.

The source is deployment input, not another file the executor may inspect for deployment instructions. The target must match the selected service configuration; source existence, path resolution and unambiguous service selection are executor checks.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`service`	string	Conditional / optional	Logical deployment-service ID receiving the file. minimum length: 1.
`source`	string	Required	Local input file to read, resolved in the composition input context. minimum length: 1.
`target`	string	Required	File path declared by the service where this configuration is supplied. minimum length: 1.

Relationships

- From DeploymentInputs: containment via configFiles; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "service": "api",
  "source": "inputs/config.yaml",
  "target": "/etc/example/config.yaml"
}
```

DeploymentEnvironment

Provide one declared environment setting to a selected deployment service.

Choose either a literal value or a secret reference, never both. An empty literal remains a deliberate value. Sensitive settings use references rather than authored secrets; omitted service selection is usable only when the executor can resolve it unambiguously.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`service`	string	Conditional / optional	Logical deployment-service ID receiving this setting. minimum length: 1.
`name`	string	Required	Environment-variable name declared by that service. minimum length: 1.
`value`	string	Conditional / optional	Literal configuration text for a non-sensitive setting.
`secretRef`	SecretReference	Conditional / optional	Reference to the private value supplied through an executor-supported secret store.

Relationships

- To SecretReference: containment via secretRef; diagram multiplicity 0..1.
- From DeploymentInputs: containment via environment; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "MODE",
  "value": ""
}
```

DeploymentInputs

Supply deployment-instance choices without changing the reusable DOT definition.

Inputs target configuration declared by services of the selected deployment. They do not introduce another manifest or imply permission to use undeclared environment settings, workloads, host resources or hardware. Resolution and adapter support are checked by the executor.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`environment`	array of DeploymentEnvironment	Conditional / optional	Service-scoped environment values or references to private values.
`configFiles`	array of DeploymentConfigFile	Conditional / optional	Local configuration files bound to declared service file destinations.
`volumes`	array of DeploymentVolumeSource	Conditional / optional	Explicit host storage sources bound to declared logical volume requirements.
`workloads`	array of ServiceWorkloadBinding	Conditional / optional	Mappings from logical service IDs to actual Compose or Kubernetes workloads.
`ociRuntime`	OciRuntimeGrant	Conditional / optional	Explicit authorization for the declared Linux capability and host-namespace requirements. conditional constraints (see schema).
`cdiDevices`	array of CdiDeviceBinding	Conditional / optional	Concrete CDI device assignments for declared logical device requirements. distinct items.

Relationships

- To CdiDeviceBinding: containment via cdiDevices; diagram multiplicity 0..*.
- To DeploymentConfigFile: containment via configFiles; diagram multiplicity 0..*.
- To DeploymentEnvironment: containment via environment; diagram multiplicity 0..*.
- To DeploymentVolumeSource: containment via volumes; diagram multiplicity 0..*.
- To OciRuntimeGrant: containment via ociRuntime; diagram multiplicity 0..1.

- To ServiceWorkloadBinding: containment via workloads; diagram multiplicity 0..*.
- From DotInstance: containment via deploymentInputs; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "environment": [
    {
      "service": "api",
      "name": "MODE",
      "value": "demo"
    }
  ]
}
```

DeploymentVolumeSource

Bind an explicit host storage source to a declared service volume.

This records an operator grant, not a request to discover or create arbitrary host storage. The current executor requires a supported direct-container adapter and an absolute bind source; read-only intent comes from the declared volume requirement rather than an invented field here.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
service	string	Conditional / optional	Logical deployment-service ID whose volume is being bound. minimum length: 1.
name	string	Required	Logical volume name declared by that service, not the host path. minimum length: 1.
type	string	Required	Storage-source mechanism interpreted by the supported runtime adapter. exact value "bind"; pattern restrictions (see schema).
source	string	Required	Explicit host location supplied for the declared volume. minimum length: 1.

Relationships

- From DeploymentInputs: containment via volumes; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "service": "api",
  "name": "data",
  "type": "bind",
  "source": "./data"
```

```
}
```

DotInstance

Select one canonical DOT release for a named role in a composition.

Use the instance name in graph references and the DOT identity for catalogue resolution. A qualified source must agree with the selected DOT metadata. Choose a deployment ID rather than a manifest-type label; dependencies express readiness order, not interface compatibility or application data flow.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`name`	string	Required	Composition-local name used by wires, dependencies and other instance references. minimum length: 1.
`source`	string	Required	Location or qualified catalogue reference from which the canonical definition is resolved. minimum length: 1.
`dotId`	string	Required	Exact identity expected in the resolved DOT definition. minimum length: 1; pattern restrictions (see schema).
`dotVersion`	string	Required	Exact component version expected in that definition, checked independently of the source location. minimum length: 1.
`deployment`	string	Conditional / optional	ID of the deployment alternative to use from the selected DOT. minimum length: 1.
`dependsOn`	array of string	Conditional / optional	Other instance names whose readiness is a prerequisite for this instance. distinct items.
`deploymentInputs`	DeploymentInputs	Conditional / optional	Configuration, workload selection and explicit runtime grants for this deployment instance.

Relationships

- From Composition: containment via dots; diagram multiplicity 1..*.
- To DeploymentInputs: containment via deploymentInputs; diagram multiplicity 0..1.
- To Dot: ID/name reference via dotId / source.
- From AnchorReference: ID/name reference via instance.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "metrics",
  "source": "repository://Example@1.0.0",
```

```
  "dotId": "Example@1.0.0",
  "dotVersion": "1.0.0",
  "dependsOn": [
    "identity"
  ]
}
```

ExposedAnchor

Give a member Anchor a stable name at the composition boundary.

Exposure names an existing logical interface; it does not create an API, a public listener or an authorization policy. Forwarding, reachable addresses and live checks are explicit executor operations.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`name`	string	Required	Composition-local boundary name used to refer to this exposed interface. minimum length: 1.
`target`	AnchorReference	Required	Member Anchor selected as the boundary interface.

Relationships

- From Composition: containment via exposedAnchors; diagram multiplicity 0..*.
- To AnchorReference: containment via target; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "api",
  "target": {
    "instance": "metrics",
    "anchorId": "http-api"
  }
}
```

KubernetesWorkloadBinding

Select the Kubernetes workload implementing one logical DOT service, including Helm-rendered workloads.

Labels must identify the intended workload unambiguously within the deployment scope. Select a container when needed rather than relying on order. This is not a grant to adopt unrelated resources or a replacement for lifecycle ownership checks.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`matchLabels`	map of string	Required	Label key/value pairs used to select the workload. minimum keys: 1.
`container`	string	Conditional / optional	Name of the container within the selected workload to which service configuration applies. minimum length: 1.

Relationships

- From ServiceWorkloadBinding: containment via kubernetes; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "matchLabels": {
    "app": "example"
  },
  "container": "api"
}
```

Materializer

Turn a resolved wire value or local input file into declared consumer configuration.

A file destination can copy a source file or inject a resolved value at a JSON Pointer; injection couples valueFrom with that pointer. An environment destination names a declared variable and has no source file. URI templates transform the selected value, not arbitrary runtime state. These actions configure a connection; they do not translate protocols.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`type`	string	Required	Destination mechanism used to supply configuration to the consumer. schema enum: "CONFIG_FILE", "ENVIRONMENT_VARIABLE".
`source`	string	Conditional / optional	Local source file for a file-based materialization. minimum length: 1.
`valueFrom`	string	Conditional / optional	Which resolved endpoint, operation or channel value to use for injection. schema enum: "ENDPOINT_URL", "OPERATION_URL", "ENDPOINT_HOST", "ENDPOINT_HOST_PORT", "ENDPOINT_PORT", "CHANNEL_ADDRESS".
`uriTemplate`	string	Conditional / optional	URI template applied to the selected value through its value variable.
`target`	MaterializerTarget	Required	Exact instance, service and declared configuration destination receiving the result.

Relationships

- To MaterializerTarget: containment via target; diagram multiplicity 1.
- To MaterializerType: typed association via type; diagram multiplicity 1.
- To MaterializerValueFrom: typed association via valueFrom; diagram multiplicity 0..1.
- From Wire: containment via materializer; diagram multiplicity 0..1.
- From Wire: containment via materializers; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "type": "ENVIRONMENT_VARIABLE",
  "valueFrom": "ENDPOINT_URL",
  "target": {
    "instance": "client",
    "service": "api",
    "environmentVariable": "API_URL"
  }
}
```

MaterializerTarget

Identify the declared consumer configuration location written by a materializer.

Select fields matching the materializer destination rather than mixing file and environment targets. File paths and variable names must already be declared by the target service. The JSON Pointer addresses content inside the source configuration, not a filesystem path or a schema field.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`instance`	string	Required	Composition member receiving the materialized configuration. minimum length: 1.
`service`	string	Required	Logical deployment service receiving the configuration. minimum length: 1.
`path`	string	Conditional / optional	Declared service file destination used for file materialization. minimum length: 1.
`jsonPointer`	string	Conditional / optional	Location in structured configuration where a resolved value is injected. pattern restrictions (see schema).
`environmentVariable`	string	Conditional / optional	Declared service environment-variable name receiving the resolved value. minimum length: 1.

Relationships

- From Materializer: containment via target; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "instance": "client",
  "service": "api",
  "path": "config.json",
  "jsonPointer": "/routes/0/uri"
}
```

MaterializerType

Choose the consumer configuration destination for a materializer.

The choice determines which source and target fields make sense together. It does not select a deployment mechanism or grant permission to modify undeclared consumer settings. Use the exact schema spelling.

Java authority | Generated JSON Schema

Exact JSON values: `CONFIG_FILE`, `ENVIRONMENT_VARIABLE`.

JSON representation: string; schema enum: "CONFIG_FILE", "ENVIRONMENT_VARIABLE".

Relationships

- From Materializer: typed association via type; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"CONFIG_FILE"
```

MaterializerValueFrom

Choose which resolved connection value a materializer uses.

Endpoint, operation and channel values have different meanings; select the one expected by the consumer. This is not a general expression language, a secret lookup or a source of live runtime state. The schema owns the supported choices.

Java authority | Generated JSON Schema

Exact JSON values: `ENDPOINT_URL`, `OPERATION_URL`, `ENDPOINT_HOST`, `ENDPOINT_HOST_PORT`, `ENDPOINT_PORT`, `CHANNEL_ADDRESS`.

JSON representation: string; schema enum: "ENDPOINT_URL", "OPERATION_URL", "ENDPOINT_HOST", "ENDPOINT_HOST_PORT", "ENDPOINT_PORT", "CHANNEL_ADDRESS".

Relationships

- From Materializer: typed association via valueFrom; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"ENDPOINT_URL"
```

NetworkBinding

Declare an existing gateway through which a wire crosses runtime boundaries.

The operator supplies reachability; the executor does not provision the gateway, change host routing or turn a logical wire into connectivity. The declaration is separate from active reachability and Contract checks.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`adapter`	string	Required	Cross-runtime connection strategy understood by the executor. exact value "EXTERNAL_GATEWAY"; pattern restrictions (see schema).
`host`	string	Required	Gateway host reachable from the consuming runtime, without a URL scheme or path. minimum length: 1; maximum length: 253; pattern restrictions (see schema).
`port`	integer	Required	Gateway listener port corresponding to the selected endpoint. minimum: 1; maximum: 65535.

Relationships

- From Wire: containment via network; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "adapter": "EXTERNAL_GATEWAY",
  "host": "gateway.example.org",
  "port": 443
}
```

OciRuntimeGrant

Authorize the Linux runtime privileges explicitly declared by a selected deployment.

A declaration of elevated requirements is not permission to grant them. The executor checks the exact grant against the DOT requirements and rejects unsupported adapters or mismatches; this is not an unrestricted privileged-container switch.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`capabilities`	array of string	Conditional / optional	Linux capabilities the operator explicitly permits for the selected workload. distinct items.
`hostNamespaces`	array of string	Conditional / optional	Host namespaces whose inheritance the operator explicitly permits. distinct items.

Relationships

- From DeploymentInputs: containment via ociRuntime; diagram multiplicity 0..1.
- To OciLinuxNamespace: typed association via hostNamespaces; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "capabilities": [
    "CAP_NET_RAW"
  ]
}
```

OperationParameterBinding

Supply one named operation or channel parameter on a wire.

The containing map supplies the parameter name; this object supplies its value. The executor checks it against the selected Contract operation. Do not use operation parameters as a substitute for secret references or undeclared deployment configuration.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`value`	string	Required	Text used to bind the named parameter while resolving the operation or channel. minimum length: 1.

Relationships

- From Wire: containment via parameters; diagram multiplicity 0..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "value": "demo"
}
```

SecretReference

Identify a private value without storing that value in the composition.

A reference is not the credential itself and does not imply that every store or adapter is supported. The executor applies store-specific lookup and native-reference rules; keep resolved values out of authored documents, locks and public evidence.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`store`	string	Required	Secret-store mechanism used to interpret the reference. minimum length: 1.
`name`	string	Required	Secret object or private input name within that store. minimum length: 1.
`key`	string	Conditional / optional	Entry within the named secret where the selected store needs one.
`version`	string	Conditional / optional	Store-specific secret revision selector, not a DOT or composition version.

Relationships

- From AsyncAPIProbeAuthentication: containment via passwordSecretRef; diagram multiplicity 1.
- From DeploymentEnvironment: containment via secretRef; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "store": "env",
  "name": "DEMO_PASSWORD"
}
```

SecurityBinding

Explain how one required Contract security scheme is supplied by a consuming service.

The executor checks the scheme against the selected operation and the mechanism against its supported security profile. Credential parameters refer to declared configuration on the consuming instance, not a new identity provider or literal token. Declared scopes do not by themselves prove that access was granted.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`scheme`	string	Required	Security-scheme name declared by the selected Contract operation. minimum length: 1.

JSON field	Value	Base presence	Meaning and constraints
`mechanism`	string	Required	Executor-supported authentication mechanism used to satisfy that scheme. minimum length: 1.
`participant`	ServiceReference	Required	Consuming instance and logical service whose declared inputs supply the credentials.
`parameters`	map of SecurityParameterBinding	Required	Mechanism-specific parameter names mapped to declared service environment settings. minimum keys: 1.
`scopes`	array of string	Conditional / optional	Requested scope names used when satisfying the Contract security requirement. distinct items.

Relationships

- To SecurityParameterBinding: containment via parameters; diagram multiplicity 1..*.
- To ServiceReference: containment via participant; diagram multiplicity 1.
- From WireSecurity: containment via bindings; diagram multiplicity 1..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "scheme": "bearer",
  "mechanism": "HTTP_BEARER",
  "participant": {
    "instance": "client",
    "service": "api"
  },
  "parameters": {
    "token": {
      "environmentVariable": "DEMO_TOKEN"
    }
  },
  "scopes": [
    "read"
  ]
}
```

SecurityParameterBinding

Connect one authentication parameter to a declared service environment setting.

The containing security binding determines what the parameter means. This object identifies where its value is configured; sensitive values must still be supplied through the proper private input path rather than embedded here.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`environmentVariable`	string	Required	Declared variable on the participant service supplying this authentication parameter. minimum length: 1.

Relationships

- From SecurityBinding: containment via parameters; diagram multiplicity 1..*.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "environmentVariable": "DEMO_TOKEN"
}
```

ServicePortReference

Select a declared service port used as a concrete wire endpoint.

The port field is a named service-port key, not a numeric port or URL. Endpoint selection may identify broker transport separately from the logical message producer; the executor checks the selected endpoint against the wire and deployment.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`instance`	string	Required	Composition member supplying the endpoint service. minimum length: 1.
`service`	string	Required	Logical deployment service supplying the endpoint. minimum length: 1.
`port`	string	Required	Declared port name on that service. minimum length: 1.

Relationships

- From Wire: containment via endpoint; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "instance": "metrics",
  "service": "api",
  "port": "http"
}
```

ServiceReference

Identify a logical service on a composition member.

Service identities come from the selected deployment, not from runtime container or Pod names. Java checks graph membership; the executor additionally resolves the service against the chosen DOT deployment.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`instance`	string	Required	Composition member containing the service. minimum length: 1.
`service`	string	Required	Logical service ID within that member deployment. minimum length: 1.

Relationships

- From SecurityBinding: containment via participant; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "instance": "metrics",
  "service": "api"
}
```

ServiceWorkloadBinding

Map a logical deployment service to the workload that actually materializes it.

Select one workload mechanism for the deployment in use. The outer service is a DOT service ID; the nested selector names runtime objects. This distinction is essential for multi-service manifests and prevents the executor from guessing or operating on an unrelated workload.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`service`	string	Required	Logical service ID in the selected deployment. minimum length: 1.
`compose`	ComposeWorkloadBinding	Conditional / optional	Selector for the Compose service implementing that logical service.
`kubernetes`	KubernetesWorkloadBinding	Conditional / optional	Selector for the Kubernetes workload and, where necessary, its container.

Relationships

- From DeploymentInputs: containment via workloads; diagram multiplicity 0..*.

- To ComposeWorkloadBinding: containment via compose; diagram multiplicity 0..1.
- To KubernetesWorkloadBinding: containment via kubernetes; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "service": "api",
  "compose": {
    "service": "server"
  }
}
```

Wire

Connect a provided Anchor to a required Anchor and declare how the consumer receives its connection details.

The arrow expresses logical provision, not necessarily packet direction. Parameter, endpoint, security and materializer choices must agree with the resolved interfaces. Use the singular or plural materializer form, not both. A compatible declaration or configured connection is not proof of network reachability or a successful live operation.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`name`	string	Required	Composition-local name used to identify this connection and select its probe. minimum length: 1.
`from`	AnchorReference	Required	Provider-side Anchor reference.
`to`	AnchorReference	Required	Consumer-side Anchor reference.
`parameters`	map of OperationParameterBinding	Conditional / optional	Named values used to bind parameters of the selected operation or channel. minimum keys: 1.
`endpoint`	ServicePortReference	Conditional / optional	Explicit service-port endpoint when the wire needs one instead of the inferred binding.
`network`	NetworkBinding	Conditional / optional	Externally supplied gateway address for a cross-runtime connection.
`materializer`	Materializer	Conditional / optional	Single consumer-configuration action for this wire. conditional constraints (see schema).
`materializers`	array of Materializer	Conditional / optional	Several consumer-configuration actions derived from this same wire. minimum items: 1.

JSON field	Value	Base presence	Meaning and constraints
`security`	WireSecurity	Conditional / optional	Bindings connecting Contract security requirements to declared consumer credential inputs.
`probe`	WireProbe	Conditional / optional	Optional protocol-specific fixture for a supported active Contract check. minimum keys: 1.

Relationships

- From Composition: containment via wires; diagram multiplicity 0..*.
- To AnchorReference: containment via from; diagram multiplicity 1.
- To AnchorReference: containment via to; diagram multiplicity 1.
- To Materializer: containment via materializer; diagram multiplicity 0..1.
- To Materializer: containment via materializers; diagram multiplicity 0..*.
- To NetworkBinding: containment via network; diagram multiplicity 0..1.
- To OperationParameterBinding: containment via parameters; diagram multiplicity 0..*.
- To ServicePortReference: containment via endpoint; diagram multiplicity 0..1.
- To WireProbe: containment via probe; diagram multiplicity 0..1.
- To WireSecurity: containment via security; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "name": "api",
  "from": {
    "instance": "provider",
    "anchorId": "http-api"
  },
  "to": {
    "instance": "consumer",
    "anchorId": "http-api"
  }
}
```

WireSecurity

Bind security requirements of the selected Contract operation to consumer inputs.

Bindings must satisfy a supported complete security requirement alternative, not merely name a familiar authentication technology. Resolution checks declarations; successful authentication, authorization and policy behavior require separate live evidence.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`bindings`	array of SecurityBinding	Required	Security-scheme bindings participating in this wire authentication configuration. minimum items: 1.

Relationships

- To SecurityBinding: containment via bindings; diagram multiplicity 1..*.
- From Wire: containment via security; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "bindings": [
    {
      "scheme": "bearer",
      "mechanism": "HTTP_BEARER",
      "participant": {
        "instance": "client",
        "service": "api"
      },
      "parameters": {
        "token": {
          "environmentVariable": "DEMO_TOKEN"
        }
      }
    }
  ]
}
```

Probes and assertions

How to read this reference explains full objects versus ID-only views, conditional fields and validation boundaries.

AsyncAPIProbe

Describe an active broker delivery check tied to the selected AsyncAPI operation and message.

The executor verifies the message against the Contract and injects fresh correlation at the selected pointer before testing delivery. Match the transport to the resolved broker bindings; stale or unrelated messages are not successful acceptance. This does not establish broker durability, production authorization or end-to-end business processing.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
protocol	string	Required	Broker transport used to execute this delivery fixture. schema enum: "MQTT", "NATS", "KAFKA", "AMQP".
message	object	Required	JSON message template whose application content is checked against the selected Contract message.
correlationPointer	string	Required	Location within the message where the fresh probe correlation value is inserted and later checked. pattern restrictions (see schema).
qos	integer	Conditional / optional	MQTT request quality accepts 0, 1 or 2 at resolution; omit or use zero for other transports. Actual delivery still needs a live probe. minimum: 0; maximum: 2.
authentication	AsyncAPIProbeAuthentication	Conditional / optional	Broker login configuration for a supported authenticated probe.

Relationships

- To AsyncAPIProbeAuthentication: containment via authentication; diagram multiplicity 0..1.
- To AsyncAPIProtocol: typed association via protocol; diagram multiplicity 1.
- From WireProbe: containment via asyncapi; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "protocol": "MQTT",
  "message": {
    "id": "demo"
  },
  "correlationPointer": "/id",
  "qos": 1
}
```

AsyncAPIProbeAuthentication

Provide broker login identity and a reference to its private password for an active delivery check.

The current AMQP 0-9-1 probe requires this authentication object, with a username and passwordSecretRef. Current MQTT, NATS and Kafka probes reject this object; their supported probe profiles are unauthenticated. This is a dotx probe limitation, not a claim that those brokers lack authentication. The password stays in an executor-supported secret source rather than in the fixture, and this object does not configure the broker itself.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`username`	string	Required	Login identity presented by the probe client. minimum length: 1.
`passwordSecretRef`	SecretReference	Required	Reference used to obtain the private broker password at execution time.

Relationships

- To SecretReference: containment via passwordSecretRef; diagram multiplicity 1.
- From AsyncAPIProbe: containment via authentication; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "username": "demo",
  "passwordSecretRef": {
    "store": "env",
    "name": "DEMO_PASSWORD"
  }
}
```

AsyncAPIProtocol

Select the broker transport used by an AsyncAPI active probe.

This choice is separate from message ownership and must agree with the resolved transport binding and supported adapter. It does not declare that every feature of that broker protocol is implemented. Use the exact values emitted by the schema.

Java authority | Generated JSON Schema

Exact JSON values: `MQTT`, `NATS`, `KAFKA`, `AMQP`.

JSON representation: string; schema enum: "MQTT", "NATS", "KAFKA", "AMQP".

Relationships

- From AsyncAPIProbe: typed association via protocol; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"MQTT"
```

GRPCProbe

Describe a reflection-based RPC check with a finite request and response sequence.

Choose the single-request or request-sequence form, not both, to match the selected RPC shape. The current profile requires server reflection and the selected Contract method must declare

MethodOptions.idempotency_level = NO_SIDE_EFFECTS. Reflection alone does not satisfy that safety requirement. Every streaming RPC needs an explicit positive responseCount; unary probes default to one response. Finite stream completion, response count and content assertions are separate checks; declaring a fixture is not proof that a server implements the RPC.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`plaintext`	boolean	Conditional / optional	Whether the probe uses an unencrypted connection instead of its TLS transport.
`serverReflection`	boolean	Required	Declaration that server reflection is used to obtain the executable RPC description. exact value true.
`request`	object	Conditional / optional	Single JSON request message for an RPC with a non-streaming request side.
`requests`	array of object	Conditional / optional	Ordered JSON messages supplied to a client-streaming request side. minimum items: 1.
`responseCount`	integer	Conditional / optional	Expected number of responses after the finite RPC completes; explicitly positive for every streaming RPC. minimum: 1.
`responseMatch`	string	Conditional / optional	Which received responses must satisfy the content expectation. schema enum: "ALL", "ANY", "FIRST", "LAST".
`expect`	JSONExpectation	Conditional / optional	JSON value assertion evaluated against the selected response messages.

Relationships

- To GRPCResponseMatch: typed association via responseMatch; diagram multiplicity 0..1.
- To JSONExpectation: containment via expect; diagram multiplicity 0..1.
- From WireProbe: containment via grpc; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "serverReflection": true,
  "request": {}
}
```

GRPCResponseMatch

Choose which responses in a finite RPC result must match an expectation.

Response selection does not replace the expected response count or the RPC completion check. Choose a policy appropriate to the operation instead of treating one matching streaming message

as proof of the entire exchange. The schema owns the policy names.

Java authority | Generated JSON Schema

Exact JSON values: `ALL`, `ANY`, `FIRST`, `LAST`.

JSON representation: string; schema enum: "ALL", "ANY", "FIRST", "LAST".

Relationships

- From GRPCProbe: typed association via responseMatch; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"ALL"
```

GraphQLOperation

Supply a supporting GraphQL operation within a probe sequence.

Use this for the verification, trigger or cleanup role selected by GraphQLProbe. The executor checks the document against the same verified schema and the operation kind required for that role; an operation name is not a substitute for the document.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`document`	string	Required	Executable GraphQL document for the supporting operation. minimum length: 1.
`operationName`	string	Conditional / optional	Name selecting the intended operation within that document. pattern restrictions (see schema).
`variables`	object	Conditional / optional	Named GraphQL inputs for this supporting operation, separate from the main probe inputs.

Relationships

- From GraphQLProbe: containment via cleanup; diagram multiplicity 0..1.
- From GraphQLProbe: containment via trigger; diagram multiplicity 0..1.
- From GraphQLProbe: containment via verify; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "document": "query Status { status }",
  "operationName": "Status",
  "variables": {}
}
```

GraphQLProbe

Describe a GraphQL query, mutation or subscription check against the selected interface.

The executable document must agree with the verified schema and Anchor-selected operation. Queries use HTTP (the default), may declare expect, and cannot declare verify, trigger or cleanup. Mutations require HTTP, expect, a verify query and a cleanup mutation; trigger is forbidden. Subscriptions require explicit SSE transport, expect, a trigger mutation and a cleanup mutation; verify is forbidden. These are bounded acceptance fixtures, not a general GraphQL client or a guarantee of application rollback.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`path`	string	Required	Absolute HTTP endpoint path on the resolved service, starting with a slash; not a full URL. pattern restrictions (see schema).
`document`	string	Required	Executable GraphQL document used for the main check. minimum length: 1.
`operationName`	string	Conditional / optional	Operation name selecting the intended operation within that document. pattern restrictions (see schema).
`variables`	object	Conditional / optional	Named GraphQL input values for the main operation.
`transport`	string	Conditional / optional	HTTP is the default for queries and mutations; subscriptions must explicitly select SSE. schema enum: "HTTP", "SSE".
`expect`	JSONExpectation	Conditional / optional	JSON result assertion; optional for a query and required for a mutation or subscription.
`verify`	GraphQLOperation	Conditional / optional	Follow-up query used to verify the effect of a mutation.
`trigger`	GraphQLOperation	Conditional / optional	Mutation used to produce an event for a subscription check.
`cleanup`	GraphQLOperation	Conditional / optional	Mutation intended to remove the test effect after the supported probe sequence.

Relationships

- To GraphQLOperation: containment via cleanup; diagram multiplicity 0..1.
- To GraphQLOperation: containment via trigger; diagram multiplicity 0..1.
- To GraphQLOperation: containment via verify; diagram multiplicity 0..1.
- To GraphQLTransport: typed association via transport; diagram multiplicity 0..1.
- To JSONExpectation: containment via expect; diagram multiplicity 0..1.

- From WireProbe: containment via graphql; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "path": "/graphql",
  "document": "{ status }",
  "transport": "HTTP"
}
```

GraphQLTransport

Choose how the GraphQL probe operation is delivered.

Transport and operation kind are separate choices and must form an executor-supported combination. Selecting a streaming transport does not make a query a subscription or grant support for arbitrary GraphQL protocols. Use the exact schema spelling.

Java authority | Generated JSON Schema

Exact JSON values: `HTTP`, `SSE`.

JSON representation: string; schema enum: "HTTP", "SSE".

Relationships

- From GraphQLProbe: typed association via transport; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
"HTTP"
```

JSONExpectation

Assert that a selected part of a JSON result equals an expected value.

The pointer selects content, not a URL or query expression; an empty pointer selects the whole document. Explicit null is an expected value and must not be confused with an omitted equals field. Protocol-specific execution determines which result document is checked.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`pointer`	string	Required	JSON Pointer locating the value to compare within the result document. pattern restrictions (see schema).
`equals`	JSON value	Required	Expected JSON value, including an explicit null when that is the intended assertion.

Relationships

- From GRPCProbe: containment via expect; diagram multiplicity 0..1.
- From GraphQLProbe: containment via expect; diagram multiplicity 0..1.
- From OpenAPIProbe: containment via expect; diagram multiplicity 1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "pointer": "/status",
  "equals": "ready"
}
```

OpenAPIProbe

Add a JSON response-value assertion to a supported OpenAPI wire probe.

The operation comes from the wire Anchor and verified Contract, not a second handwritten URL or operation definition. When this assertion block is present, expect is required and the selected operation must be a safe GET or OPTIONS operation; HEAD cannot carry this JSON assertion. A successful HTTP status alone is different from a matched response expectation.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`expect`	JSONExpectation	Required	Result-value assertion applied in addition to the native Contract response checks.

Relationships

- To JSONExpectation: containment via expect; diagram multiplicity 1.
- From WireProbe: containment via openapi; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "expect": {
    "pointer": "/status",
    "equals": "ready"
  }
}
```

WireProbe

Choose a protocol-specific active-check fixture for a wire.

Select one protocol fixture matching the resolved Contract standard. Portable fixture validity does not promise that an executor supports every deployment, operation or transport combination.

Active checks may call APIs or publish messages and must be deliberately selected; they are not passive topology inspection.

Java authority | Generated JSON Schema

JSON field	Value	Base presence	Meaning and constraints
`openapi`	OpenAPIProbe	Conditional / optional	Response-value expectation for a supported OpenAPI operation.
`graphql`	GraphQLProbe	Conditional / optional	GraphQL operation, transport and any verification or cleanup operations.
`grpc`	GRPCProbe	Conditional / optional	Reflection-based RPC request and finite-response assertions. conditional constraints (see schema).
`asyncapi`	AsyncAPIProbe	Conditional / optional	Broker delivery fixture with message correlation and optional authentication.

Relationships

- To AsyncAPIProbe: containment via asyncapi; diagram multiplicity 0..1.
- To GRPCProbe: containment via grpc; diagram multiplicity 0..1.
- To GraphQLProbe: containment via graphql; diagram multiplicity 0..1.
- To OpenAPIProbe: containment via openapi; diagram multiplicity 0..1.
- From Wire: containment via probe; diagram multiplicity 0..1.

Shape example from the reviewed Java compatibility corpus (not a runnable deployment)

```
{
  "openapi": {
    "expect": {
      "pointer": "/status",
      "equals": "ready"
    }
  }
}
```

DOT-CLI

CLI overview

`dotx` is the Go command-line application provided by the independent DOT-CLI repository. It operates on DOT definitions and compositions; it is not a catalogue server or a replacement for Docker, Kubernetes, Helm, Vagrant or Wasmtime.

Start with a working DOT

Follow Getting started for checksummed Linux-amd64 installation and the Prometheus Docker Compose quickstart: validate, deploy, verify an API response, inspect status and clean up. Running the released binary does not require a Java or Go development toolchain.

Use the guided demo to see a topology and a multi-DOT composition. The MARE PoC walkthroughs include a presenter pause for inspecting verified live results before cleanup.

Find the right command

Task	Command family
Check a definition, bundle or complete catalogue	`dotx validate`
Read a DOT's declared interfaces and deployment choices	`dotx inspect`
Discover and inspect catalogue entries	`dotx repository`
Author a DOT; create a separate editing candidate in 0.4.0	`dotx new`; current source: `dotx edit`
Operate one selected DOT deployment	`dotx deploy`, `status`, `start`, `stop`, `destroy`, `ps`
Validate, lock, inspect and display a multi-DOT graph	`dotx compose validate`, `lock`, `inspect`, `topology`
Operate and verify a composition	`dotx compose apply`, `status`, `probe`, `destroy`
Relate a locked graph to observed runtimes and historical probes	`dotx compose topology --status` (use the applied run's cache/configuration)
Refresh or check composition README diagrams and Mermaid files	`dotx compose topology Compositions --write` / `--check`
Check deployment tools and runtime prerequisites	`dotx tool`

Ask the installed binary for its exact options:

```
dotx version
dotx version --verbose
dotx help
dotx compose --help
dotx compose inspect --help
```

`version --verbose` adds the source revision, modified-worktree status, Go version and OS/architecture without contacting a server. Missing build metadata is shown as `unknown`. Plain `version` remains a single line; the version number alone does not identify the exact source build used for a rehearsal.

Current source keeps mounted literal/default configuration files separate for each execution, including Docker image deployments and previews. Docker image replacement verifies the existing container's selected DOT/deployment/runtime ownership and removes its exact ID; a foreign collision stops the command. Kubernetes literal environment overrides replace an earlier `valueFrom` source on the selected container. These fixes require a matching source build; they do not change the already published binary or establish live API acceptance. Compose path-based lifecycle also retains verified dotenv inputs needed for native interpolation and profile selection. Its live preflight uses Compose's `config --variables` and `config --profiles` support; dry-runs remain previews.



0.4.0 uses raw HTTPS URL and SHA-256 references for deployment manifests, Helm values, environment files and optional non-sensitive configuration defaults, as Contracts already do. Inline text and Base64 in those file fields are rejected. Use the matching 0.4.0 CLI, generated schema and catalogue; historical v0.3.0 binary and its pinned catalogue retain their original format.

Validate the catalogue online once to prepare the verified document cache, then use `dotx validate <catalogue> --offline` to check that the required bytes are available without downloads. A missing or corrupt entry fails closed. Validation does not start a workload. See Payload for the shared model and its integrity/hosting boundaries.

Current-source repository validation applies reusable publication checks to external/internal definitions with owned Contracts or explicitly implemented provided interfaces. Consumed dependencies alone cannot qualify an exposed API. `--profile interoperable` additionally requires Contract coverage for every Anchor and exposed service port. Runtime-only, client-only and incomplete definitions remain visible as gaps in the catalogue readiness report; base validation is not a blanket requirement that every definition is interface-complete. These declaration checks do not establish live producer behavior.

Composition subcommand help includes only that operation's flags, an example, a suggested next step and its safety boundary. For example, `compose inspect` resolves declared inputs without deployment, `apply --dry-run` previews commands, and `status` reads recorded runtime state. The complete production profile remains available through `dotx compose --help`.

Maintain composition topology documentation

From a standalone catalogue checkout, the selected release or matching source CLI can update every composition's existing README topology block and adjacent `topology.mmd`:

```
dotx compose topology Compositions --write
dotx compose topology Compositions --check
```

From DOT-Design, use `DOT-Repository/Compositions` instead. A single composition file is also accepted. Both modes require verified locks; no lock is repaired automatically. The command checks every graph and destination before writing, preserves authored text outside the marked block, and rejects symlinked files and malformed markers. `--check` writes nothing and exits nonzero on drift. Individual files are replaced atomically, not as a multi-file transaction; resolve any I/O failure and rerun. Display options such as `--format`, `--color` and `--status` cannot be combined with documentation modes.

Only `dotx` is needed: the former Bash generator and its jq/awk dependencies are not required. The optional `task compositions:topology:generate` and `:check` wrappers remain available in both checkouts. A generated diagram is a locked resolved composition view, not proof of live interoperability.

Find a DOT by its readable release coordinate

The current source-built CLI discovers the catalogue from your working directory. At each level, starting in the current directory and walking toward the filesystem root, it checks for `DOTs/` first, then `DOT-Repository/DOTs/`. Commands therefore work from DOT-Design, DOT-CLI, a standalone catalogue checkout, or a directory inside one of its DOTs/compositions. A renamed catalogue is recognized by its `DOTs/` directory, not by its Git remote or directory name.

```
dotx repository list --skill Metrics-Scraping
dotx repository list --verbose
dotx repository inspect Prometheus@3.15.1 --json
dotx repository resolve Prometheus@3.15.1
```

The default catalogue table starts with names, versions and scopes, followed by up to three Skill names. Every matching DOT remains visible; `+N more` indicates additional Skills. `--verbose` shows IDs, groups and categories. `--json` retains the complete structured report and takes precedence over `--verbose`. Listing definitions does not prove live readiness.

`inspect` and `resolve` accept an exact, case-sensitive DOT `name@version`, as well as the authoritative ID/version selector. The version is exact: no fuzzy name matching, automatic latest-version choice or identity migration is performed. The last `@` separates the version; quote names containing spaces. Ambiguous matches fail, and ID matches take precedence over coincidentally matching names. JSON output reports the authoritative case-sensitive ID and definition.

Explicit `--repository <path-or-git-url>` overrides discovery. If the selected catalogue is malformed, unreadable or missing, the command fails rather than switching to a different catalogue. Without a

discoverable catalogue, provide that flag explicitly. Composition sources remain `repository://<id>@<version>`; display-name lookup does not replace source identities, locks or runtime ownership checks. No separate catalogue inventory is required.

Author a runtime-only scaffold

To connect existing DOTs without assembling identifiers manually, follow Create a composition from a reviewed example. That flow verifies a template's lock and creates a fresh ownership identity; it does not create new product definitions or deploy workloads.

Native authoring writes `dot.json` without Git, Jsonnet or template downloads. From the DOT-Design checkout:

```
dotx new --from docker-image prom/prometheus:v3.15.1 --name Prometheus --version
3.15.1 --skill Metrics-Scraping --port http:9090 --output ./my-dot/dot.json
dotx validate ./my-dot --schema DOT-Repository/Schema/schema.json
dotx inspect ./my-dot
# Alternatively: dotx new --interactive
```

This does not infer Contracts or Anchors and does not prove interoperability or live readiness. Reuse complete catalogue bundles where available; native authoring is a starting point for a genuinely new DOT. Add authoritative Contracts and explicit Anchors/bindings before integration. The positional `dotx new <name>` mode remains a template-based project workflow requiring Git, Jsonnet and network access. See `dotx new --help` for both modes.

Native authoring derives `<name>@<version>` as the DOT ID and requires at least one explicit `--skill <id>` (repeat the flag for multiple capabilities). Interactive authoring asks for a Skill when none was supplied. Declare the capability's meaning in the selected catalogue's `Skills/`; do not infer capabilities, interoperability or live readiness from the image alone. The actual tool release must be supplied explicitly; no `0.0.0` default is invented and Helm chart versions do not determine application versions.

For an owned OpenAPI interface, native authoring can also verify a generated-model Contract declaration, select operations and generate Anchor IDs and named-port bindings. See the executable interface-authoring exercise. Published v0.3.0 requires explicit authoring for other standards and dependency references. The 0.4.0 release adds guided imports for the supported Contract standards and separate editing candidates; the authoring guide states their supported profiles and review boundaries.

Inspect deployment choices

From the DOT-Design checkout, use the current source-built CLI:

```
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT/3.15.1
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT/3.15.1 --verbose
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT/3.15.1 --json
```

The default overview lists every deployment's ID, platform, manifest type, service count and required inputs, followed by short Contract, Anchor and Skill previews. It never automatically chooses a deployment. `INPUTS*` counts required environment/config-file declarations without defaults for each service; it is not a readiness result. Storage and host/device declarations are flagged separately so you can review persistence and permissions before deploying.

Use `--verbose` for IDs, all interfaces/selectors, service bindings and detailed configuration declarations. `--json` preserves the complete structured report and overrides `--verbose` when combined. Invalid sensitive defaults are rejected in every mode; provide secrets through deployment-instance inputs instead. Inspection does not start workloads, test runtime availability or prove live API acceptance. It does not replace `dotx validate`.

Root Contract metadata comes directly from complete `dot.json` declarations; ID-only roots fail rather than importing metadata from local files. JSON keeps the existing report structure, IDs and ordering. Inspection never downloads or verifies payload bytes; use `dotx validate` for that separate check. Validation caches digest-verified bytes outside the DOT. Warm it online once before `--offline`; a cold cache fails without requests or writes. Deployment support uses the same capability list as deploy/lifecycle, including WebAssembly.

Human output provides a validation command and a dry-run template. Choose an exact deployment ID from the displayed choices and replace `<deployment-id>`; inspection never selects one automatically. Remote Git targets instead require a persistent checkout for these next steps, since their inspection clone is temporary.

Understand validation failures

The current source CLI reports a diagnostic code, the affected file, an RFC 6901 JSON Pointer, a short explanation and a repair hint. Schema diagnostics use the selected generated schema to show expected types and exact enum spellings; there is no separate hand-maintained model definition. Input values, manifest contents and raw download errors are withheld to protect credentials. An empty pointer means a document-root or file-level failure. Some catalogue metadata failures identify a directory rather than an individual file.

In 0.4.0, every DOT reader checks the embedded, Java-generated schema before converting JSON into Go values. This includes inspection, catalogue discovery, composition resolution and runtime commands. An explicit `--schema` can add constraints but cannot waive required model fields, invalid nulls or retired configuration keys. Keep the CLI, generated schema and catalogue revisions aligned. This shared raw-input stage neither fetches payloads nor mutates workloads. `dotx validate` still verifies declared documents; that verification and live API acceptance are separate stages.

For editors, scripts and CI, request one JSON report on stdout, including failures:

```
dotx validate DOT-Repository --offline --json
```

Run this example from a DOT-Design checkout with its pinned catalogue and source-built CLI; from a standalone catalogue checkout use `.` instead of `DOT-Repository`. The report has `formatVersion: 1`, `valid`, `target`, `kind`, completed `dots`/`compositions` counts, and `diagnostics` containing `code`, `file`, `pointer`, `message`, `hint` and optional `keyword`/`expected` fields. Exit codes remain `0` (valid), `1` (invalid or

unavailable), and `2` (usage error). Normal JSON-mode validation failures leave stderr empty. Validation stops at the first failing definition or phase, although a schema failure can report multiple constraints. It never repairs inputs or rewrites reviewed locks automatically.

The same source CLI also reports failures from `dotx compose validate <composition.yaml> --json` and `dotx compose inspect <composition.yaml> --json` as one secret-safe JSON document on stdout, including usage errors. These failure reports have `formatVersion: 1`, `valid: false`, `kind: composition`, `target`, shared `diagnostics` and `evidence: composition-validation-not-live-acceptance`. Normal JSON failures leave stderr empty; output-write failures use stderr. Schema diagnostics retain exact field pointers, while resolution failures without finer context identify the composition file. Raw parser, document and transport values are withheld.

Successful composition validation still returns its existing `valid`, `compositionId` and `unknownWires` summary, and successful inspection still returns resolved execution details. Check the exit code and result, not merely whether stdout contains JSON. These failure-report guarantees do not extend to runtime `apply`, `upgrade`, `probe` or `status` commands. No lock is regenerated automatically, and an unknown wire remains an explicit limitation rather than live acceptance.

Recover from a composition runtime failure

The 0.4.0 release also gives `compose apply`, `upgrade`, `probe`, `status` and `destroy` secret-safe diagnostic codes and recovery hints on stderr. This is **not** a new runtime JSON failure envelope: successful JSON shapes and existing exit-code meanings are preserved. Raw tool, parser, transport and credential values are withheld. A diagnostic identifies an instance or wire when the failing boundary supplies that context; generic failures identify the composition instead.

Code	What to do next
`COMPOSITION_INSTANCE_DEPLOY` / `COMPOSITION_INSTANCE_CLEANUP`	Inspect the named instance and recorded status. Use the original CLI and run identity for cleanup; never remove unrelated resources.
`COMPOSITION_WIRE_PROBE`	Review the named wire's operation, endpoint, authentication and probe inputs. A failed probe is not producer acceptance.
`COMPOSITION_BUSY` / `COMPOSITION_RESUME_STATE`	Check the active operation and recorded state. Do not bypass locks, adopt another run or regenerate a composition lock to conceal a mismatch.
`COMPOSITION_UPGRADE_STORAGE` / `COMPOSITION_UPGRADE_RESTORED`	Review storage safety or the restored previous deployment before retrying. Restoration is not a successful upgrade.

Follow the emitted hint and run `dotx compose status <composition.yaml>` with the same cache/configuration as the applied run. No diagnostic automatically repairs locks, changes ownership or grants permissions. These improvements do not claim an equivalent failure-report format for top-level single-DOT runtime commands.

For container-backed probes, prepare runner images explicitly before live testing:

```
dotx compose probe composition.yaml --locked --images
```

This read-only command emits the selected pinned helper images as JSON. It does not pull images, access lifecycle state, load credentials or execute a Contract operation. A caller preparing a Docker runner must cache the exact image pin on the host daemon; a Kubernetes runner needs its node cache, which is distinct. The four-Contract demo's preparation task performs the appropriate verified pulls.

Failure-only runtime diagnostics separate image checks, runner startup and operation execution where evidence permits. An unresolved startup phase is not reported as proof that a request reached the producer. Diagnostics retain static codes, timings and recovery guidance, not raw tool stderr, credential-bearing arguments or response payloads. Existing successful probe output and exit-code meanings remain unchanged.

For reusable catalogue examples and explicit deployment selection, see Deployments and examples. For Anchor selectors, Wires, native Secret references, locks and lifecycle safety, use the composition guide.

Validation and resolution are not live acceptance. Active probes run only for supported, explicitly configured Contract operations and transports. The walkthroughs state which endpoints are actually exercised and distinguish real producers, source-adapted integrations and synthetic fixtures.

The `0.4.0` release also supports the bounded native SPARQL query and metrics probes. SPARQL uses the fixed read-only `ASK {}` request through the selected service binding. Metrics stay under OpenAPI and validate the declared successful OpenMetrics 1.0.0 or Prometheus text 0.0.4 representation with official parsers. OpenMetrics requires EOF; Prometheus text has no EOF marker, so syntactically complete truncation cannot be detected. These checks do not imply arbitrary queries, updates, metric semantics, authorization or production acceptance. They are absent from the published v0.3.0 binary.

Current-source Kubernetes/Helm probes read retained logs only after checking the probe container's completion and exit code. This avoids losing output emitted before an attach stream starts. Each probe executes once, with no hidden retry of an API operation or delivery. Cleanup uses the Pod's create-time UID and a bounded deletion wait; replacement Pods are not adopted, and failed cleanup cannot produce successful acceptance. This hardening requires the matching source binary, not the original v0.3.0 release.

Ctrl-C or SIGTERM cancels the current Kubernetes probe, completes bounded UID-safe cleanup and stops subsequent selected operations. Recorded evidence reports `CANCELLED` (or `RUNNER_CLEANUP_FAILED` if cleanup fails); the command returns 130 for Ctrl-C or 143 for SIGTERM. Allow cleanup to finish before closing the terminal; force-killing the process cannot guarantee cleanup.

Versions and model authority

See Adopting DOT for compatible artifact sets, editor schema associations and the current-source, deployment-aware `dotx tool check --dot <bundle> --deployment <exact-id>` command. A successful prerequisite report is not live API acceptance.

DOT-Design and its pinned DOT-CLI source versions are kept aligned. Release installation

instructions use the published binary; development demos and the diagnostics above may require the exact source revision pinned by the DOT-Design checkout, even when its version label matches a release. See Development and testing for source installation, Bash completion, local CI-equivalent Tasks and prerequisites.

All portable models belong to DOT-Domain. The CLI consumes its generated JSON Schemas and Go bindings; it does not define a competing DOT or Composition input model. Resolved execution details, lock fingerprints and private runtime state are executor concerns. DOT definitions and reusable composition inputs live in DOT-Repository.

Installation and first DOT

Zero to Hero: hands-on tutorial

Compatibility, editor setup and preflight

Compatibility and migration policy

Author a DOT and its interfaces

Start by reusing a complete catalogue DOT. Author a new definition only for a genuinely new component or a local exercise; do not publish another root for an existing product. Add deployment alternatives to its canonical bundle instead.

These commands require the **selected release CLI `v0.4.0` or its matching source build**, `jq`, catalogue access and network access to the pinned raw Contract. The released flow below covers owned OpenAPI interfaces; the guided editor described later is included in 0.4.0. Current-source authoring summaries name the saved file, distinguish declaration validation from live acceptance, and confirm unchanged runtime state. Follow-up commands use the actual quoted path (including an explicit `./` prefix for a relative filename starting with `-`); validation retains repository/offline selection while `inspect` reads the local definition without those flags. Follow the compatible-artifact installation routes. No Java, Jsonnet, Docker daemon or Kubernetes cluster is needed for the commands below; the last command renders a deployment without starting it.

Confirm the actual component release

`--version` is required for noninteractive authoring. Interactive mode asks for the actual tool release without inventing `0.0.0`. Image packaging tags and Helm chart versions are not reliable substitutes: confirm the tool release yourself. Explicit source/test versions remain available for unreleased components.

Declare at least one meaningful `--skill`; repeat it for multiple capabilities. Use an existing catalogue capability where appropriate and document a genuinely new capability in `Skills/`. Creation does not infer Skills from an image.

Rehearse with the existing Prometheus Contract

This temporary exercise reuses Prometheus's existing authoritative declaration; it creates no second catalogue entry and copies no dependency interface. From DOT-Design, prepare an empty exercise directory:

```
# Run from DOT-Design; set DOT_REPOSITORY=$PWD from a standalone catalogue.
DOT_REPOSITORY="${DOT_REPOSITORY:-$PWD/DOT-Repository}"
AUTHORING_DIR="$(mktemp -d -t dotx-authoring.XXXXXXXX)"
PROMETHEUS_JSON="$(dotx repository resolve Prometheus@3.15.1 --repository
"$DOT_REPOSITORY")"
jq '.contracts[0]' "$PROMETHEUS_JSON" \
  >"$AUTHORING_DIR/contract.json"
```

`contract.json` is one complete `Contract` in the generated DOT-Domain model, not a raw OpenAPI document or catalogue wrapper. It contains the owned ID, name, standard version, standard, format and HTTPS `payload.url`/`sha256`. For your own tool, provide its own declaration with a release-matched official specification where available. Never relabel another component's Contract.

```
dotx new --from docker-image prom/prometheus:v3.15.1 \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract "$AUTHORING_DIR/contract.json" \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' --bind http \
  --output "$AUTHORING_DIR/bundle/dot.json"
```

Expected: an owned-interface verification message (`Owned OpenAPI interface verified` in v0.3.0; `Owned interface verified` in current source). The command validates the declaration against the generated schema, checks exact ownership, downloads and verifies the payload using the ordinary bounded HTTPS/SHA-256 verifier, and resolves the selected operation using the composition resolver. It writes the complete Contract, the explicit Anchor ID, its service Contract reference and the named-port binding into `dot.json` only after verification succeeds.

`build-information` is the stable DOT-local wire identity, not a generated UUID or a display-name alias. `--operation <anchor-id>=<selector>` uses that exact key for both the Anchor and its service binding; `name` initially has the same value but may subsequently change independently. Use kebab-case keys. The selector remains separate from the Contract ID. Repeat `--operation` for additional provided operations on the same declared TCP port. `--bind` requires an exact service port name and never guesses a listener. Every explicit `--port` must use `[name:]<1-65535>[/protocol]`, for example `http:9090/TCP`. An invalid entry rejects the complete draft; it is never silently discarded.

For a menu of operations and port choices, use `--interactive --contract "$AUTHORING_DIR/contract.json"` with the same source/name/version/Skill/port arguments and a new output path. Without an imported Contract, interactive mode offers an optional declaration-file prompt; leave it empty for a runtime-only scaffold. Do not overwrite this exercise's output unintentionally.

Validate, inspect and render

```
dotx validate "$AUTHORING_DIR/bundle" \
  --schema "$DOT_REPOSITORY/Schema/schema.json" --profile reusable
dotx inspect "$AUTHORING_DIR/bundle"
dotx deploy "$AUTHORING_DIR/bundle" --deployment docker-image --dry-run
```

Expected: the reusable profile passes, inspection shows one owned Contract and one provided Anchor, and dry-run renders the Docker command. This checks the declared interface and binding, **not** a running API. The flow does not establish that a URL is vendor-owned or that its release metadata is truthful; review provenance. For live acceptance, follow the source-qualified guided demo. The released quickstart uses its own pinned artifact set; do not replace only one component of a qualified pair with an unrelated binary or catalogue revision.

The published v0.3.0 guided interface profile covers **owned OpenAPI provided operations** only. AsyncAPI, GraphQL, Protobuf, external dependency references and provider-neutral requirements remain supported by the domain and composition commands, but require explicit authoring. No protocol metadata is duplicated from the Contract and no compatibility or live-readiness claim is inferred.

Import directly from the official raw URL

Alternatively, the same native authoring command can construct the generated Contract declaration from a raw HTTPS URL. Run this after the previous exercise so both temporary outputs can be inspected and cleaned up together:

```
# Alternative: supply the official raw URL without hand-writing contract.json.
# This exercise creates no second catalogue entry.
AUTHORING_SPEC_BASE=https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1
dotx new --from docker-image prom/prometheus:v3.15.1 \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract-url "$AUTHORING_SPEC_BASE/web/api/v1/testdata/openapi_3.1_golden.yaml" \
  --contract-name HTTP --contract-version 3.1.0 --contract-format YAML \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' --bind http \
  --output "$AUTHORING_DIR/url-bundle/dot.json"
dotx validate "$AUTHORING_DIR/url-bundle" \
  --schema "$DOT_REPOSITORY/Schema/schema.json" --profile reusable
jq '.contracts[0]' "$AUTHORING_DIR/url-bundle/dot.json"
```

Expected: the resulting ID is `Prometheus@3.15.1::OpenAPI::HTTP`, with the official URL and a 64-character lower-case SHA-256 in `payload`. No raw document is embedded. `--contract-version 3.1.0` matches the document's `openapi: 3.1.0`, not its `info.version` or the tool's `--version 3.15.1`. The upstream document's `info.version: 0.0.1-undefined` remains unchanged inside the pinned payload; it is not copied into DOT Contract metadata. A standard-version mismatch fails before publication. Do not infer versions from a filename or rewrite upstream bytes.

To require bytes already approved by your team, add `--contract-sha256 <trusted-digest>`. A mismatch fails without replacing output. Without that option, the command pins the bytes retrieved on first use: review the URL, upstream provenance, release match, license and resulting digest before committing. Computing a hash does not prove that the URL is vendor-owned or that its API matches a deployed producer.

The downloader reuses validation's HTTPS, public-address, redirect, timeout, size and credential-scoping controls. Mutable GitHub branch URLs are refused; use an official release tag or full commit. Transport failures withhold URL and credential details. `--contract` and `--contract-url` are mutually exclusive. For a derived specification hosted on GitHub Gist, use the raw `gist.githubusercontent.com` URL containing the full 40-character revision and filename, without query parameters, plus a trusted digest. Secret Gists are readable without authentication by anyone with the URL; GitHub tokens are not sent to them. Preserve source/license provenance and do not label a derived projection vendor-official merely because it is hosted externally. The published v0.3.0 URL importer requires explicit `--contract-name`, `--contract-version` and `--contract-format JSON|YAML`; it does not guess a standard from the URL.

Interactive authoring accepts either a declaration filename or an HTTPS URL, asks for missing metadata and offers the same verified operation/port menus. The digest selected during that prompt must still match at publication.

Recover from errors and clean up

- Missing release: supply `--version`; no output is created.
- Ownership mismatch: use the actual originating product/release, or an external Anchor reference for a dependency. Do not rename a dependency into ownership.
- Download or digest failure: check access, the official raw URL and expected bytes. Never change a hash solely to bypass a failure.
- Invalid selector: choose an operation from the verified specification. Encode OpenAPI selectors as URI-fragment JSON Pointers, including `~1` for `/`.
- Binding failure: declare `--port http:9090` and use `--bind http`.
- Existing output: choose a new path. `--force` permits replacement only after verification. Complete bytes are staged and published atomically; a concurrently created output is not overwritten without `--force`. Forced replacement preserves existing Unix file mode bits and does not follow a destination symlink into another file. Staging failures preserve the old file.

Non-overwriting publication requires hard-link support in the output filesystem; unsupported filesystems fail closed rather than falling back to truncation.

`new` creates a fresh definition; it is not an incremental editor or migration command. `--force` replaces the whole scaffold using the supplied Anchor IDs; it does not preserve interfaces omitted from the new command. For an existing integrated DOT, edit its definition while retaining stable Anchor identities and review affected composition locks instead of regenerating the scaffold.

```
# Only the files created by this exercise; unexpected extra files are retained.
rm -- "$AUTHORING_DIR/bundle/dot.json" "$AUTHORING_DIR/contract.json"
if [[ -d "$AUTHORING_DIR/url-bundle" ]]; then
```

```
  rm -- "$AUTHORING_DIR/url-bundle/dot.json"
  rmdir -- "$AUTHORING_DIR/url-bundle"
fi
rmdir -- "$AUTHORING_DIR/bundle" "$AUTHORING_DIR"
```

Only exercise files are removed. No containers, networks, volumes or cluster resources were created. Developers can run the same documented commands and ownership-negative control with `task testing:authoring` from DOT-Design.

Guided authoring in 0.4.0

Use the 0.4.0 CLI and its matching catalogue. Guided authoring reuses the existing model authority and supported profiles; it adds no new runtime. 0.4.0 includes the approved file-payload input-format change. Record `dotx version --verbose` and the exact catalogue revision. Historical v0.3.0 binaries retain their original behavior.

`dotx new --interactive` offers verified operations for the following standards. The SPARQL row is supported in **0.4.0**; the published v0.3.0 binary is unchanged:

Standard	Raw format	Supported selector/profile
OpenAPI	JSON or YAML	Operation JSON Pointer, including supported additional operations
AsyncAPI	JSON or YAML	Version-3 send/receive operation; explicit broker endpoint in the composition
Protobuf	PROTOBUF	Fully qualified service method in a standalone specification
GraphQL	GRAPHQL	Declared query, mutation or subscription root field
SPARQL	N_TRIPLES	Exact W3C Service Description `sd:endpoint` IRI; SPARQL 1.1 Query with SPARQL Results JSON

New, edit and composition authoring use the same native Huh forms for text, editable defaults, scrolling choices and confirmations. Enter accepts a prefilled value; Ctrl-U clears the field. Candidate publication offers **Write draft** / **Cancel**, defaulting to **Cancel**. For `new --interactive`, the summary lists the validated candidate's identity, Skills, interfaces, deployments, ports and bindings, plus its byte-level SHA-256. Confirmation publishes that same prepared snapshot; Contract retrieval and candidate generation are not repeated after review. EOF or an incomplete form cannot authorize publication. Defaults containing control characters use an explained escaped line fallback when a single-line form cannot preserve their bytes losslessly.

In single-choice menus, type initial letters to filter immediately: `Pro` finds Prometheus, `Key` finds Keycloak. Search is case-insensitive and also matches text within a choice. Use □/□ to browse, Backspace to edit the query, Enter to select and Ctrl-C to cancel. No `/` prefix is needed. Checkbox menus use Space to toggle, `/` to filter and Esc to finish filtering before toggling.

Set `DOTX_NO_TUI=1` or `ACCESSIBLE=1` for non-redrawing line menus. They also apply when output is

redirected or the terminal is small or `TERM=dumb`: numbered menus use `/text`, `n`/`p` and `q`; configuration checkboxes use comma-separated names, `*` for all and empty for none. Confirmation remains an explicit text prompt, with no as the default. The browser uses the ordinary composition parsers and operation resolver; it does not create another compatibility engine. An operation menu is not evidence that every transport or feature in the standard is executable. Imported Protobuf dependencies that cannot be resolved by the existing standalone/well-known-import profile fail with no draft publication.

For URL imports, choose the exact canonical `--contract-standard`, name and raw format. Current source derives an omitted standard version from the verified payload's native marker and omits it for GraphQL SDL. Tool `--version` remains the actual component release. A supplied standard version must still match. Use `--offline` only with the trusted digest and already verified cached bytes; a cold or corrupt cache is not repaired or bypassed.

For SPARQL, `--contract-standard SPARQL --contract-format N_TRIPLES` selects the query-only profile; `--contract-version 1.1` describes the standard, never the product release. The selector is the exact absolute `sd:endpoint` IRI from the pinned Service Description, not its payload URL or a blank-node identifier. The description must advertise `sd:SPARQL11Query` and `http://www.w3.org/ns/formats/SPARQL_Results_JSON`. The endpoint's path is used with the selected deployment service binding; the descriptor's host is not a runtime destination. Creation and editing retain the ordinary ownership, digest, selector and binding checks. No datasets, arbitrary query fixtures, updates or authentication guarantees are inferred.

For metrics, choose `OpenAPI`, its actual OpenAPI version, and an operation whose successful response declares OpenMetrics 1.0.0 or Prometheus text 0.0.4. `OpenMetrics` is not a Contract standard or a file format choice. Preserve the producer's actual negotiated format and label source-derived OpenAPI projections as derived. The exact response declaration controls native probe validation; a metrics media type on an error/default-only response does not activate it. See native profile limits and active probe checks.

AsyncAPI operations materialize a send/receive participant, not a local broker listener. Do not pass `--bind`; choose the broker's actual service/port when building the composition. Existing MQTT, NATS, Kafka and AMQP acceptance profiles remain authoritative; declarations do not provision brokers or prove delivery.

Improve an existing DOT safely

From the matching **DOT-Repository checkout root**:

```
mkdir -p drafts
PROMETHEUS_JSON="$(dotx repository resolve Prometheus@3.15.1 --repository .)"
dotx edit "$PROMETHEUS_JSON" --interactive \
  --output drafts/prometheus.candidate.json --repository . --offline
```

Choose Skills, add an owned Contract, update its URL, add/change an Anchor operation or rebind an owned Anchor to a named service/port. IDs are retained for existing Anchors; no unaffected ID is regenerated. Selecting an AsyncAPI send/receive operation derives the correct role. Runtime-only definitions can gain their first interface; incomplete interoperability cannot be saved as a reusable

candidate.

Expected: a semantic change summary followed by explicit confirmation. The complete candidate is checked against the generated model and ordinary Contract, selector and per-deployment binding rules before writing private, complete bytes. The original, deployment manifests, configuration, unrelated fields, locks and runtime state stay untouched. The editor refuses an existing output, a symlinked output parent, a changed original or a second catalogue `dot.json`. Cancellation, EOF, verification and publication errors leave no partial candidate.

The completion summary prints commands for reviewing the diff, validating with the selected local catalogue and offline mode, and inspecting the candidate. `dotx validate <candidate.json> --repository <local-catalogue> --offline` also works outside the catalogue tree: it uses that checkout's generated schema and exact external Contract origins. It neither downloads missing bytes nor repairs corrupt caches. `dotx inspect <candidate.json>` displays the declaration; it does not accept repository/offline flags or prove a live API response.

For `dotx new`, the summary uses a catalogue discovered from the working directory when available. Its `--repo`/`--repository` option identifies a Helm chart repository, not a DOT catalogue. If no catalogue is discoverable, offline revalidation needs an explicitly selected local schema or catalogue; the summary explains this rather than printing an unusable command.

Review the diff and validate the candidate. **Draft creation is not promotion**: explicitly decide whether to replace the canonical file, review dependent composition locks for the intended interface change and rehearse affected APIs. Do not publish a candidate as another product root. Delete only the candidate file when abandoning the exercise; no runtime cleanup is necessary.

Create a composition

A DOT describes one component. A Composition chooses concrete deployments of several DOTs and connects their Anchors through Wires. Start from a catalogue example whose README describes your intended use case and its acceptance limits. The selected release `v0.4.0` and its matching source CLI provide template-based `dotx compose new`. Selecting DOTs with `--from-dots` is included in DOT-CLI 0.4.0. Use a matching CLI and pinned catalogue.

Choose, preview and create

From your pinned **DOT-Repository checkout**, warm its verified Contract cache once:

```
dotx validate .
dotx compose new
```

0.4.0 uses native forms: arrows browse, typing filters, Enter selects, and Ctrl-C cancels. Choose a template, accept or edit its suggested name/output, review the preview, then explicitly choose **Write draft**. Ctrl-U clears a text field before replacement. The published v0.3.0 flow uses numbered prompts and requires an unused output path in an existing directory. Neither cancellation nor EOF publishes a file. Noninteractive creation requires all choices and `--yes`; no CI job needs to wait for input.

For a repeatable exercise using the existing Node Exporter/Prometheus graph:

```
dotx compose new \
  --from Compositions/node-exporter-prometheus-monitoring/composition.yaml \
  --name my-monitoring --output my-monitoring.yaml --offline --yes
dotx compose lock my-monitoring.yaml --offline
dotx compose inspect my-monitoring.yaml --offline --locked
dotx compose topology my-monitoring.yaml --locked
```

Expected: a preview of two DOTs and one wire, a fresh ownership UUID, and a new YAML file with the same exact tool releases, deployment choices, Contract and Anchor IDs. The reviewed template and its lock are unchanged. The new lock is created only by your explicit `lock` command; review it before committing. The Composition graph version is retained; it is not another tool release.

No workloads are started. Offline creation needs a warm verified Contract cache; without `--offline`, bounded downloads and cache writes may occur even if no draft is published. Printed follow-up commands retain `--offline` when selected.

▼ *How draft validation works*

The command verifies its input byte snapshot against the template's adjacent lock, applies changes to the Java-generated Composition model, preserves ordinary YAML shape/comments, then checks the rendered values exactly and validates the candidate with the same generated schema and resolver. YAML merge keys are expanded by the YAML parser where necessary: inherited IDs, names and file sources must receive the intended new values too. Authoring uses a fresh model of the verified input bytes, separate from inferred execution services and probes. It does not infer compatibility, invent wires, copy product definitions, download templates from Git, change secret values or start a runtime.

Review shared inputs before customization

Filesystem sources are rebased to the template's existing inputs, **not copied**. This avoids silent configuration duplication, but editing a referenced file would affect both compositions. Copy an input to your intended directory and change the new composition's `source` before customizing it. Repository DOT and Contract sources, absolute host bind grants and Secret references retain their identities. Host bind paths are not rebased as if they were relative configuration documents.

A fresh UUID prevents lifecycle-state adoption; it does **not** isolate fixed ports, application/workload names, named storage or externally owned gateways. Review these, Kubernetes namespaces and credentials before any real deployment. Existing collision, ownership, configuration and probe safety checks still apply. A checked resolved composition or generated topology is not live producer acceptance, security hardening or independent adopter acceptance. The template README's limitations continue to apply; creation does not qualify an untested producer.

Recover and clean up

Creator failures now include a stable diagnostic code and a specific recovery hint, without printing raw parser, filesystem or download errors:

- `CONTRACT_CACHE_MISS`: warm the pinned owner/catalogue online, then retry offline.
- `COMPOSITION_NEW_TEMPLATE_SCHEMA`: validate the template against the generated schema.
- `COMPOSITION_NEW_TEMPLATE_LOCK`: inspect and explicitly review an intentional template-lock update.
- `COMPOSITION_NEW_OUTPUT_PARENT`: choose an existing real directory without directory symlinks.
- `COMPOSITION_NEW_OUTPUT_LOCK` or `COMPOSITION_NEW_PUBLISH`: choose an unused destination; there is no force mode.
- `COMPOSITION_NEW_RESOLUTION`: validate exact releases, Contract origins and deployment choices using the same repository/offline options.

Failure never automatically repairs inputs or locks, starts a runtime, or publishes a partial draft. Diagnostic wording is executor output, not a new model.

- Missing or stale template lock: inspect the original inputs and lock diff. Fix an intentional change in the template separately; creation never repairs it.
- Existing output or adjacent lock: choose a new path. There is no `--force`.
- Missing or symlinked output parent: create/select a real directory first.
- Cold offline cache: run online validation of the pinned catalogue first. Digest mismatches require investigation, not changing a pin to bypass checks.
- Need diagnostic detail: run `dotx compose validate <template> --locked --json`. Creator failure messages withhold raw parser values, URLs and credentials.

Complete private bytes are published without overwriting a concurrently created file. Non-overwriting publication needs hard-link support; unsupported filesystems fail closed. No lifecycle state, workloads, networks or volumes are created. Remove only the exercise's `my-monitoring.yaml` and `my-monitoring.lock.json` when finished. For a reusable catalogue contribution, choose its final directory before creating it, add a README, review its lock and use `dotx compose topology <composition.yaml> --write` to generate its topology docs.

Current-source alternative: build from selected DOTs

The selection table shows the exact DOT release, catalogue origin and declared interfaces. The following deployment table groups that DOT's choices by engine, type and profile. Origin comes from the same ownership/evidence paths used by catalogue validation. `testing` can include fixtures or planning definitions; `Contracts` and `Anchors` describe declarations, not live compatibility or production readiness. No entries are silently hidden.

After publication, the summary names the saved file, declaration-validation boundary and unchanged runtime state. It prints copyable commands to inspect the draft, explicitly create its lock and inspect it locked, retaining the selected repository and offline mode. None of these actions runs automatically. Review shared inputs and the newly created lock before any deployment.

Use DOT-CLI 0.4.0 and its matching catalogue. From the DOT-Repository checkout, with verified cached Contracts:

```
dotx compose new --from-dots --repository . --offline --name my-monitoring
```

Omit `--name` to open a native Huh **Composition name** field prefilled with a random, readable suggestion such as `calm-otter`. Enter accepts it; Ctrl-U clears the field so you can type a replacement. Names, paths, selections and confirmations all use the shared form interface, not just the choice menus.

The adjective–animal suggestion varies each run; compound animal names are hyphenated too (for example `calm-guinea-pig`). Ctrl-C cancels a native form; the line fallback also supports `q`. EOF or an incomplete form aborts without publishing or accepting defaults. The same field is used by current-source template creation. Existing catalogue folder/file names are skipped, but a suggestion does not reserve a destination: final publication checks still reject concurrent collisions. An explicit `--name` is retained exactly and skips the prompt; noninteractive `--yes` still requires it. This friendly label is separate from the fresh ownership UUID and is not a secret or security identifier.

Select exact releases and explicit deployments. The native **Instance name** field is prefilled with a lowercase, hyphenated label derived from the selected tool's name, such as `prometheus`.

Press Enter to use it, or Ctrl-U and type a unique label such as `metrics-primary`. Suggestions skip aliases already used in this composition: `prometheus`, then `prometheus-2`, `prometheus-3`, and so on. Explicit duplicate names are rejected; custom names are never silently rewritten. Cancel or EOF never accepts an unanswered default. This label does not change the selected DOT's identity or tool release, and does not isolate fixed workload names, ports or storage. Existing compositions and locks are not renamed or regenerated.

Deployment choices put the platform first, followed by the manifest type and the exact profile ID:

```
ENGINE      | DEPLOYMENT TYPE  | PROFILE
DOCKER      | DOCKER_COMPOSE   | docker-compose
DOCKER      | DOCKER_IMAGE     | docker
KUBERNETES  | HELM_CHART       | helm-chart
```

Options are grouped by engine, then manifest type, retaining the declared profile order within each group. Only the selected DOT's deployments are listed; the menu is not filtered by locally installed tools because authoring does not deploy. The profile ID distinguishes alternatives with the same engine and type. Headers and columns stay aligned while filtering, in both scrolling and numbered menus. Untrusted display text is escaped rather than interpreted by the terminal.

The native scrolling selector lets you browse **all** catalogue DOTs with □/□, type initial letters such as `Pro` or `Key` to filter immediately, select with Enter and cancel with Ctrl-C. Backspace edits the case-insensitive substring query; no `/` prefix is needed. New and edit use this same TUI. Deployment, Anchor and connection choices use the same UI. No DOT, Contract or Anchor ID needs to be constructed manually.

For a screen reader or the numbered interface, set `DOTX_NO_TUI=1` or `ACCESSIBLE=1` before the command. Redirected output, small and `TERM=dumb` terminals also use this non-redrawing fallback: `/text` filters, `n`/`p` pages, a displayed number selects and `q` cancels. The header states the visible range

and total; the first 12 rows are not the whole catalogue. The UI does not change validation or perform any deployment. Input must still be a terminal; piped input is rejected. The native Go interface uses Bubble Tea through Huh form components. Views share navigation and cancellation behavior without owning model validation, deployment or cleanup logic. Draft publication uses **Write draft** / **Cancel** buttons, initially selecting **Cancel**: Enter alone does not authorize a write. Existing defaults containing control characters use an explained escaped line fallback when a single-line form cannot preserve them losslessly. Original bytes remain authoritative. If you omit `--output`, press Enter to accept `Compositions/my-monitoring/composition.yaml` inside the selected catalogue. From DOT-Design, `--repository DOT-Repository` instead suggests `DOT-Repository/Compositions/my-monitoring/composition.yaml`. This is a draft's storage location, not a deployment destination. The same default applies to the current-source interactive template flow. Missing default folders are created only after validation and explicit write confirmation; cancellation creates no folders. A composition name used as a folder must be one path component, not `.`/`..` or a path; other valid display names can use an explicit output path.

A relative `--repository` path is relative to your current working directory, not the draft's directory. The current-source resolver retains an absolute execution location, so a draft saved elsewhere still deploys the selected catalogue bundle. Portable sources and lock identities remain `repository://` references; no machine-local path is written into the authored YAML or lock.

To save elsewhere, enter another path at the prompt or supply `--output`. An explicit output's parent must already exist. Existing files, adjacent locks and directory symlinks are refused; there is no overwrite mode. Publication failure removes only still-identical empty folders created by this operation, never pre-existing folders or concurrent contents. Creating the catalogue draft does not add a README, topology artifacts or a lock; review and complete those before contributing the composition. For the real monitoring example choose Node Exporter and Prometheus, then connect `node-metrics` to `scrape-target`. Select a config-file materializer deriving `ENDPOINT_HOST_PORT`, targeting Prometheus's declared configuration file and the existing scrape-target JSON Pointer. Input paths are relative to the new YAML; review the existing monitoring README rather than inventing a producer interface.

The menu labels connections accepted, incompatible or UNKNOWN using the ordinary compatibility authority. UNKNOWN is not verified; required authentication must be explicitly bound and checked. Required operation parameters and service configuration are separate from identifiers. Enter native Secret **references**, never their values. A reviewed optional `DeploymentInputs` YAML/JSON fragment uses the exact generated model for advanced workload selectors, volumes and OCI/CDI grants; paths resolve beside the new composition, not beside the fragment. Missing, ambiguous, unsupported or unsafe inputs fail closed without output.

Brokered AsyncAPI send/receive pairs require a separately selected broker service/port, including existing MQTT, NATS, Kafka and AMQP profiles. This endpoint is not another Anchor wire. Cross-runtime pairs require an explicit, already provisioned `EXTERNAL_GATEWAY` host/port. The operator owns that gateway and its cleanup; the CLI neither creates networking infrastructure nor uses host-network fallbacks. The complete graph still passes the existing resolver's constraints.

Expected: exact deployment choices and a topology preview, then explicit write confirmation. A fresh ownership UUID and generated-model round-trip check precede non-overwriting publication. No lock is created, no existing state is adopted and nothing is deployed. `--from-dots` cannot be combined with `--from` or `--yes`, and requires a terminal. A complete validated draft is not live API

acceptance.

After reviewing the YAML and every referenced input, explicitly create and review its lock using the same catalogue/offline options, then follow the real monitoring or authenticated secure-API acceptance route. Declaration validation does not qualify a catalogue gap, producer double or fixture for production use.

Deployments and examples

Compositions and lifecycle

Topology and guided demo

Development and testing

DOT-Repository

Catalogue overview

Find a tool, choose its deployment and connect it to others. DOT-Repository is the version-controlled catalogue used by `dotx`. It holds reusable DOT definitions, capability references, generated schemas and reviewed compositions. It can be cloned independently of the DOT-Design development workspace.

New to the ecosystem? Start with DOT-Design 101, then follow Zero to Hero to use and create real DOTs. For installation and repository access, use Getting started.

Find and inspect a DOT

Use one matching CLI/catalogue revision set. Released walkthroughs use the catalogue recorded in the CLI release manifest. For current-source work, use one DOT-Design checkout's pinned CLI and catalogue; a version label alone does not identify later changes. See compatibility and access guidance.

With that CLI installed, run from the **DOT-Repository checkout root**:

```
dotx repository list --repository .
dotx repository inspect Prometheus@3.15.1 --repository .
dotx repository resolve Prometheus@3.15.1 --repository .
```

Expected: the discovered catalogue, details of that exact Prometheus release and the path to its `dot.json`. Inspection helps you choose an explicit deployment ID; it does not start a service. These commands create no workloads, so no runtime cleanup is needed.

Read the selected release's adjacent README for prerequisites, supported deployment choices, interface limitations and the kind of evidence available. Use the Prometheus quickstart for an actual deployment, API check and cleanup. Many catalogue examples are previews rather than complete live walkthroughs.

Understand the layout

```
DOT-Repository/
  DOTs/
    external/<product>/<tool-release>/dot.json
    internal/<product>/<tool-release>/dot.json
    testing/...
  Skills/
  Compositions/
  Schema/
```

Directory	What belongs there
`DOTs/external/`	Tools owned outside MARE. External ownership does not establish readiness.
`DOTs/internal/`	Components owned or developed within MARE.
`DOTs/testing/`	Clearly identified synthetic, source-adapted, planning or runtime-test definitions.
`Skills/`	Capability definitions referenced by the DOTs' textual Skill IDs.
`Compositions/`	Reusable use-case graphs with reviewed locks, README guidance and non-secret inputs.
`Schema/`	JSON Schemas generated from DOT-Domain. Catalogue authors do not define a second model here.

The `DOTs/` filesystem is the index: definitions are discovered recursively. Current catalogue publication uses one product folder with exact release leaves. Each release has one canonical `dot.json` containing all its supported runtime/deployment alternatives. There is no mutable `latest` copy. The released tutorial's pinned catalogue may retain the historical layout; identity-based CLI lookups avoid depending on either directory spelling. See products and supported releases.

Validate declarations

From the same **DOT-Repository checkout root**:

```
dotx validate .
```

This discovers the DOT definitions and reusable compositions, checks their applicable declaration rules and references, verifies declared URL/hash documents, and validates the required reviewed composition locks. Downloads may be needed. `--offline` requires an already verified cache; missing or corrupt bytes fail rather than silently changing the selected inputs.

Validation starts no workloads. It is separate from a successful deployment, live API acceptance and production readiness:

Evidence	Meaning
Base/reusable validation	The selected declaration rules and applicable interface bindings pass.
Strict interface profile	`--profile interoperable` checks declared Contract coverage for Anchors and exposed service ports. Catalogue entries can still have explicit interface gaps.
Live acceptance	The recorded producer, deployment and operations passed the exercised runtime assertions, with a recorded cleanup result. It does not qualify every alternative or API.

Do not add placeholder Contracts to make a readiness result green. Prefer release-matched official specifications. Keep source-derived Contracts, explicit interface gaps, runtime-only components and

test fixtures clearly identified. See readiness and integration limits.

Reuse and contribute

- **Use a DOT:** inspect its exact release and follow its supported quickstart.
- **Improve a definition:** keep the product identity, interfaces and deployment alternatives together. Follow safe DOT authoring and editing.
- **Add a use case:** keep scenario-specific configuration outside product definitions. Use composition authoring, then review its topology and lock before any deployment.
- **Operate a composition:** follow its own prerequisites, live assertions and ownership-safe cleanup. The composition walkthroughs explain the qualified paths and remaining boundaries.

Open the catalogue on GitHub.

Products and tool releases

DOT examples and deployments

Create or improve a DOT

Create a composition

Reusable composition walkthroughs

Interface coverage and readiness